DokTool

DokTool — the learning book

Every application, module, screen and button of the DokTool code, explained so you can read it as if you had written it.

Edition of 26 October 2026 · branch feature/medicalcul-and-pregnancy-import

Contents

1. Welcome — how to read this book
   - 1.1 How the book is organised
   - 1.2 What DokTool is
   - 1.3 Who uses it
   - 1.4 The rules every line of code respects
   - 1.5 Words you will meet everywhere
2. The repository — where everything lives
   - 2.1 The top-level folders
   - 2.2 How the web app and the mobile app are separated
   - 2.3 How the three applications talk
   - 2.4 Inside apps/api
   - 2.5 Tools you need
   - 2.6 Running everything on your machine
   - 2.7 The one command that checks everything
3. The architecture in one chapter
   - 3.1 The life of one request
   - 3.2 Three kinds of identity
   - 3.3 The access decision
   - 3.4 One error format for everything
   - 3.5 Files: R2, signed URLs and upload intents
   - 3.6 Background jobs
   - 3.7 Messages to people
   - 3.8 Offline
   - 3.9 How modules are allowed to depend on each other
4. How the API is built — NestJS, using DokTool's own code
   - 4.1 Start-up: main.ts and app.setup.ts
   - 4.2 The root module
   - 4.3 A module: notifications
   - 4.4 A controller: endpoints and guards
   - 4.5 DTOs: the contract
   - 4.6 A service and the database
   - 4.7 Guards and decorators you will see everywhere
   - 4.8 Refusing: DomainError
   - 4.9 How the API is tested
5. Accounts: sign-up, sign-in, sessions, recovery, deletion
   - 5.1 One field for e-mail or phone
   - 5.2 Sign-up in two steps
   - 5.3 One-time codes
   - 5.4 Sign-in, lockout and sessions
   - 5.5 Changing contacts and password
   - 5.6 Assisted recovery: « J'ai perdu l'accès aux deux »
   - 5.7 Deleting an account
   - 5.8 The numbers, in one place
6. Access: who may open what
   - 6.1 The inputs
   - 6.2 The policy table
   - 6.3 Placing a person in one tier
   - 6.4 Re-verification never interrupts access
   - 6.5 From tier to profile, and from profile to items
   - 6.6 Re-checking on every endpoint
   - 6.7 Viewers: user or demo
   - 6.8 The demo
   - 6.9 What the apps do with the profile
7. Content: reference PDFs, calculators, pregnancy, search, offline
   - 7.1 Reference library: ordonnances and certificates
   - 7.2 StorageService: one door to files
   - 7.3 Calculators: a formula as data
   - 7.4 Grossesse · Allaitement
   - 7.5 Search
   - 7.6 Favorites and history
   - 7.7 The offline manifest
8. Verification, payments, finance, analytics, audit, admin security
   - 8.1 Verification
   - 8.2 Billing: plans, checkout, gateways
   - 8.3 Finance
   - 8.4 Analytics without spying
   - 8.5 The audit log
   - 8.6 Admin security: password, TOTP, grants
   - 8.7 The admin module
9. How Admin Web is built — Next.js, React and the console kit
   - 9.1 The layers
   - 9.2 One origin: the API behind Admin Web
   - 9.3 Calling the API
   - 9.4 Server state with TanStack Query
   - 9.5 Who sees what: grants in the UI

9.6 The console kit (src/ui)
9.7 Forms
9.8 Signing in
9.9 Languages and formatting

10. Admin Web, screen by screen

10.1 Vue d'ensemble
10.2 Utilisateurs
10.3 Vérifications
10.4 Récupération
10.5 Bibliothèque (ordonnances, certificats)
10.6 Calculateurs
10.7 Grossesse · Allaitement
10.8 Abonnements, Transactions, Finance
10.9 Analytique and Journal d'audit
10.10 Permissions and Paramètres

11. How the mobile app is built — Flutter, Riverpod, go_router, drift

11.1 The folders
11.2 Start-up
11.3 Riverpod in five minutes
11.4 The session
11.5 Routes and redirects
11.6 Talking to the API
11.7 The local database (drift)
11.8 The widget kit and tokens
11.9 Texts
11.10 Tests

12. The mobile app, screen by screen

12.1 Welcome and onboarding (features/onboarding)
12.2 The shell and Home
12.3 Bibliothèque (features/library)
12.4 Recherche (features/search)
12.5 Reading a reference PDF (features/reference)
12.6 A calculator (features/calculators)
12.7 Grossesse · Allaitement (features/pregnancy)
12.8 Activité et favoris (features/activity)
12.9 Profil, Paramètres, Sécurité (features/profile, settings)
12.10 Vérification (features/verification)
12.11 Abonnement (features/subscription)
12.12 Accès perdu ? (features/recovery)
12.13 Données et hors ligne (features/offline)
12.14 Notifications

13. Eight journeys through the whole system

13.1 A doctor joins, gets verified and subscribes
13.2 A visitor tries the demo, then creates an account
13.3 Opening a prescription model
13.4 Using a calculator, offline
13.5 Authoring and publishing a calculator (two admins)
13.6 Pregnancy profiles from a spreadsheet
13.7 Lost both e-mail and phone
13.8 A subscription reaches its end

14. Working on the code: tests, CI and recipes

14.1 Before every commit
14.2 Recipe: add an API endpoint
14.3 Recipe: add a column or a table
14.4 Recipe: add or change a text
14.5 Recipe: add a mobile screen
14.6 Recipe: add an Admin Web section
14.7 Adding a calculator is not a code change
14.8 The checklist that keeps DokTool safe
14.9 What the October 2026 audit changed
14.10 Rebuilding this book

Appendix A — Every file, function and button

1. Welcome — how to read this book

This book walks you through the whole DokTool code base, from the first line the server runs to the last button of the mobile app. Its goal is simple: after reading it, you should be able to open any file and say what it does, why it is there and what happens when someone presses a button — as if you had written it yourself.

1.1 How the book is organised

- **Part 1 — Foundations (chapters 1–3).** What DokTool is, how the repository is laid out, and how the three applications talk to each other.
- **Part 2 — The API (chapters 4–8).** The NestJS server: how a request is handled, then every module — accounts, access, content, calculators, pregnancy, verification, payments, finance, analytics and the admin side.
- **Part 3 — Admin Web (chapters 9–10).** The Next.js console the team uses: its building blocks, then every screen and button.
- **Part 4 — The mobile app (chapters 11–12).** The Flutter app: its foundations, then every screen and button.
- **Part 5 — Journeys (chapter 13).** Eight complete stories followed through all three applications, step by step.
- **Part 6 — Working on the code (chapter 14).** Tests, CI, and recipes for the changes you will make most often.
- **Appendix A.** Generated from the code: every file, every documented function and every commented button, in the order of the file. It is the reference you keep open next to the editor.

> **How to study.** Read a chapter, then open the files it names in your editor and read them top to bottom. Every function and every button now has a short comment explaining it (that was part of the October 2026 audit). Code shown in this book is not retyped: it is pulled from the files when the book is built, with its real line numbers, so what you read here is exactly what is in the repository.

1.2 What DokTool is

DokTool is a **mobile clinical utility** for doctors, residents and medical students in Algeria. It is deliberately small and fast. It is **not** a patient record, a prescription generator, a drug database, a chatbot or a marketing platform. It offers four things:

Resource	What the doctor does with it
Ordonnances (prescription models)	Searches, finds a reference model and reads it as a PDF. Read-only: no editing, no patient field, no printing, no sharing.
Certificats (certificate models)	The same, for medical certificates.
Calculateurs (clinical calculators)	Picks a score or formula, enters validated values, gets a result with its interpretation band and its published source.

Grossesse · Allaitement (pregnancy and lactation)	Looks up a substance and reads, per trimester and for breastfeeding, the conduct and the evidence level — always separate, never a "safe / unsafe" verdict.

The product summary in `CLAUDE.md` fits in three lines, and the whole code base serves them:

```
References    SEARCH → FIND REFERENCE MODEL → VIEW PDF → USE AS CLINICAL REFERENCE
Calculators   SELECT TOOL → ENTER VALIDATED INPUTS → CALCULATE → REVIEW RESULT + SOURCE → DONE
Prospects     DEMO → EXPLORE → UNDERSTAND → SUBSCRIBE
```

1.3 Who uses it

Person	Application	What they can do
Doctor or resident, verified and subscribed (« Pro »)	Mobile	Everything: every model, calculator and profile, offline copies.
Doctor or resident not yet subscribed	Mobile	A real account with the demo sample set. May buy once their professional status is verified.
Medical student, verified	Mobile	Free: the calculators; models and profiles are visible as titles only (locked), except the demo samples. No purchase.
Demo visitor	Mobile	No sign-up, no contact data, no time limit: the demo sample items only. The demo is an *access context*, not a kind of person.
Admins (Super Admin, editors, reviewers, finance…)	Admin Web	Upload models, author and review calculators and profiles, verify professionals, manage plans, read finance, analytics and the audit log — each according to their *grants*.

1.4 The rules every line of code respects

These rules come from the Product Owner's decisions (`docs/adr/ADR-006-product-owner-decisions.md`) and the design system. Whenever you wonder "why is this code so careful here?", the answer is usually one of them.

1. The backend decides access. The server computes an *AccessProfile* (tier + the state of every feature) and re-checks it on every endpoint. The app only draws what it receives; it never says `if (student)`.

2. Locked is not hidden. A locked item stays visible with its lock and the reason. Only *deferred* modules (drug database, prescription generation…) are never shown at all.

3. No patient data, anywhere. Calculator inputs never leave the phone; analytics carry no free text, no search words and no input values.

4. Reference PDFs are read-only. No editor, no export, no print, no share, no download button. Files live in private storage and are reached only through short-lived signed URLs.

5. Clinical content is never invented. A calculator formula comes from a cited source, is reviewed by someone other than its author, has test cases, and is immutable once published. AI never writes or changes a formula or a profile.

6. Calm money. Subscriptions show what is included, the price, the duration and « non remboursable », with no red, no countdown and no refund workflow.

7. Bilingual from one catalogue. French and English for every text, from `packages/i18n`. No Arabic or right-to-left inside the app itself.

1.5 Words you will meet everywhere

Word in the code	Meaning
`Tier`	Where a person lands: `DEMO · STUDENT · PROFESSIONAL_UNPAID · PROFESSIONAL · RESTRICTED`.
`FeatureKey`	A capability the policy decides: `search`, `reference.view`, `calculator.use`, `offline.sync`, `subscription.purchase` …
`LockReason` / `UnlockPath`	Why something is locked (`SUBSCRIPTION_REQUIRED`, `VERIFICATION_PENDING` …) and the one screen that can unlock it.
`demoSample`	A flag on content: this item is part of the free sample set (demo, unpaid professionals, students).
`Viewer`	Whoever is looking at content: a signed-in user *or* a demo session.
`Grant`	An admin permission, like `calculators.publish` or `finance.read`.
`Version` (calculator, profile, plan)	Clinical content and prices are versioned: a published version never changes; a change is a new version.
`DomainError`	The one way the API refuses something: a stable `code` the apps translate.

2. The repository — where everything lives

DokTool is one Git repository (a *monorepo*) holding three applications and four shared packages. The web console and the mobile app are already completely separate: they live in different folders, use different languages and frameworks, and never import each other's code. What they share is a **contract** with the server, generated automatically.

2.1 The top-level folders

```
doktool/
├── apps/
│   ├── api/       ← the server (NestJS, TypeScript). The ONLY part that touches the database.
│   │   ├── src/           code: main.ts, app.module.ts, common/, config/, infrastructure/, modules/,
│   │   ├── prisma/        schema.prisma (47 tables) + migrations/ (hand-written SQL)
│   │   └── test/          unit/ (pure logic) and integration/ (real HTTP + real PostgreSQL)
│   ├── admin/     ← Admin Web (Next.js, React, TypeScript). The team's browser console.
│   │   ├── src/app/       the pages (one folder = one URL)
│   │   ├── src/features/  the screens, one folder per console section
│   │   ├── src/ui/        the shared components (Button, DataTable, Dialog, Drawer…)
│   │   └── src/api/       the typed API client (schema.d.ts is generated)
│   └── mobile/    ← the app (Flutter, Dart) for iOS and Android.
│       ├── lib/src/core/      networking, session, local database, secure storage
│       ├── lib/src/features/  one folder per feature (onboarding, library, calculators…)
│       ├── lib/src/ui/        the shared widgets (DtButton, DtAlert, ContentRow…)
│       └── test/              widget and unit tests
├── packages/
│   ├── api-contract/        openapi.json — the contract both clients are generated from
│   ├── design-tokens/       colours, spacing, radii… → CSS, Tailwind, TypeScript and Dart
│   ├── i18n/                messages.fr.json + messages.en.json → Flutter ARB + admin catalogue
│   └── calculator-vectors/  test cases both calculator engines must pass
├── design/        the design system: specifications, component cards, tokens.json
└── docs/          decisions (adr/), audits, operations guides, this book
```

2.2 How the web app and the mobile app are separated

The question "where is the web code, where is the mobile code?" has a clean answer:

	Admin Web	Mobile app
Folder	`apps/admin`	`apps/mobile`
Language	TypeScript + React (TSX)	Dart
Framework	Next.js (App Router)	Flutter
Who uses it	The DokTool team, on a computer (≥ 1024 px)	Doctors, residents, students, demo visitors, on a phone
API it calls	`/admin/v1/*` with a session cookie + 2FA	`/v1/*` with a bearer token
Generated client	`apps/admin/src/api/schema.d.ts`	`apps/mobile/lib/src/core/api/generated/`
Package manager	npm (workspace)	`flutter pub`

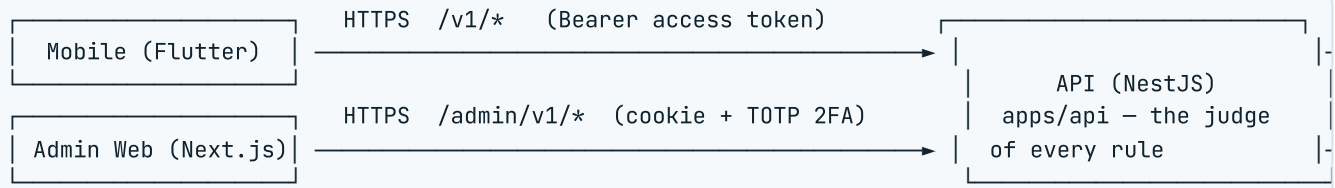
Tests	Vitest (`apps/admin/src/test`)	`flutter test` (`apps/mobile/test`)

The only things they have in common come from `packages/`: the same words (i18n), the same colours (design tokens) and the same API contract. None of these is imported as source code; each is **generated** into the format the app needs.

```
design/system/tokens.json ──generate──► tokens.css + tailwind preset (admin)   +  doktool_tokens.dar
packages/i18n/messages.*.json ─build──► admin catalogue (TS)                  +  lib/l10n/*.arb (mobi
apps/api (decorators) ──openapi──► packages/api-contract/openapi.json ──► schema.d.ts (admin) + Dart
```

Never edit a generated file by hand. `schema.d.ts`, everything under `core/api/generated/`, `doktool_tokens.dart` and the ARB files are rewritten by their generators. CI compares them with a fresh generation and fails on any difference ("drift").

2.3 How the three applications talk



2.4 Inside apps/api

- `src/main.ts` — starts the server. `src/app.setup.ts` — HTTP settings shared with the tests. `src/app.module.ts` — assembles everything.
- `src/config/` — reads and validates the environment variables (`env.ts`).
- `src/common/` — small shared helpers: the error envelope, client info, CSV and search text.
- `src/infrastructure/` — technical services every module may use: the database client, cryptography, background jobs, rate limits, e-mail and SMS, token verification.
- `src/modules/` — the **20 domain modules**, one per business area. Each owns its tables and exposes only what its `index.ts` exports.
- `src/scripts/` — command-line tools: create an admin, generate the OpenAPI contract, seed demo data, Medicalcul extraction and import.
- `prisma/schema.prisma` — every table, grouped by owning module.

The list of modules is one file:

apps/api/src/modules/index.ts

```
23  export const DOMAIN_MODULES = [
24    AuthModule,
25    AdminAuthModule,
26    UsersModule,
27    VerificationModule,
28    AccessModule,
29    BillingModule,
30    FinanceModule,
31    ReferenceLibraryModule,
32    ClinicalReviewModule,
33    CalculatorsModule,
```

```
  PregnancyModule,
  SearchModule,
  ActivityModule,
  OfflineModule,
  NotificationsModule,
  AnalyticsModule,
  AuditModule,
  DemoModule,
  StorageModule,
  AdminModule,
] as const;
```

2.5 Tools you need

Tool	Used for
Node.js ≥ 24 + npm	The API, Admin Web and the packages (one npm *workspace* at the root).
PostgreSQL ≥ 15	The database (one for development, one dedicated to tests).
Flutter (stable)	The mobile app; Android SDK + emulator, and Xcode on a Mac for iOS.
Chrome	Building the PDF of this book.

2.6 Running everything on your machine

From the repository root, once:

```
npm install                                        # installs API, Admin Web and packages
cp apps/api/.env.example apps/api/.env             # then fill DATABASE_URL and fresh 32+ char secrets
cp apps/admin/.env.example apps/admin/.env
npm run prisma:migrate:deploy -w @doktool/api    # creates the tables
npm run admin:create -w @doktool/api -- --email you@example.dz --name "Your Name"
```

Then, in three terminals:

```
npm run start:dev -w @doktool/api                # API on http://localhost:3000 (Swagger at /docs)
npm run dev -w @doktool/admin                    # Admin Web on http://localhost:3001

cd apps/mobile
flutter pub get
dart run build_runner build --delete-conflicting-outputs
flutter gen-l10n
flutter run --dart-define=API_BASE_URL=http://localhost:3000
```

Sample data. `npm run demo:seed -w @doktool/api` fills a database whose name ends in `_demo` with demonstration accounts and content (it refuses any other database). The passwords it prints are shown once.

2.7 The one command that checks everything

```
DATABASE_URL=postgresql://…/doktool_test npm run ci
cd apps/mobile && flutter analyze && flutter test
```

`npm run ci` runs, in order: Prettier formatting, ESLint, TypeScript type checks, the i18n parity check (every key in both languages), the design-token checks (no raw colour, radius, blur or duration in the code), the OpenAPI drift check, the admin client drift check, then every test of every workspace. At the

end of the October 2026 audit it was green: 613 API tests, 30 Admin Web tests, the package tests, and 167 Flutter tests.

3. The architecture in one chapter

Before reading modules one by one, it helps to follow one request from the phone to the database and back, and to know the handful of mechanisms every feature reuses: tokens, the access decision, the error envelope, file storage, background jobs, messages and offline copies.

3.1 The life of one request

Take the doctor opening a prescription model on the phone. This is what happens, file by file:

```
PHONE                                        API                                                    DATABASE
reference_viewer_screen.dart
  └ openedReferenceProvider
     └ ReferenceFiles.download(id)
        └ generated client  GET /v1/reference-documents/{id}/file-url
           └ AuthInterceptor adds "Authorization: Bearer <access token>"
                                  ──────────►  ViewerAuthGuard: token valid? session live?
                                               ReferenceLibraryController.fileUrl()
                                                 └ ReferenceDocumentsService.fileUrl()   reads the pu
                                                 └ AccessService.decideItems()           policy evalu
                                                 └ StorageService.getSignedReadUrl()     10-minute si
                                  ◄──────────  { url, sha256, … }  (or { error: { code: "FEATURE_LOCK
        └ dio GET signed url (no token) ──────────────────────────────────────────────────────────────► R2 b
        └ SHA-256 of the bytes == published sha256 ?  else refuse to show
  └ pdfrx draws the pages (no download, print or share button)
```

Three ideas are visible in this one example and repeat everywhere:

1. **The guard proves who is calling** (a user token, a demo token or an admin cookie).
2. **The access policy decides what they may do**, again, on this exact request.
3. **Files never pass through the API**: it hands out a short signed URL, and the client checks the file's fingerprint.

3.2 Three kinds of identity

Who	Proof	Lifetime	Where in the code
A signed-in user (mobile)	A JWT access token in `Authorization: Bearer`, plus a rotating refresh token kept in the phone's Keychain/Keystore	Access 15 min; refresh rotates on every use and is bound to the device's installation id	`infrastructure/security/user-tokens.ts`, `modules/auth/sessions.service.ts`, mobile `core/auth/`
A demo visitor (mobile)	A demo access token + a renewal token	Access 1 hour, renewed silently	`infrastructure/security/demo-tokens.ts`, `modules/demo/`
An admin (Admin Web)	An HTTP-only session cookie obtained after password *and* a TOTP code	Idle and absolute limits	`modules/admin-auth/`

The user guard is short enough to read whole. Notice that it only proves identity — it does not decide access:

apps/api/src/infrastructure/security/user-auth.guard.ts

```
/** Requires a valid user access token (`Authorization: Bearer …`). Admin cookies are ignored. */
@Injectable()
export class UserAuthGuard implements CanActivate {
  constructor(private readonly tokens: UserTokens) {}

  /**
   * Runs before the route: extracts the bearer token, verifies it (including that its session is
   * still live) and stores the principal on the request.
   */
  async canActivate(context: ExecutionContext): Promise<boolean> {
    const request = context.switchToHttp().getRequest<AuthenticatedRequest>();
    const header = request.get('authorization') ?? '';
    const match = /^Bearer ([A-Za-z0-9._~+/=-]+)$/.exec(header);
    if (!match?.[1]) throw new DomainError('UNAUTHENTICATED', 'Missing bearer token.');
    request.user = await this.tokens.verify(match[1]);
    return true;
  }
}
```

And the token check also asks the database whether the session is still live, so a sign-out, a password change or a suspension takes effect on the very next request rather than when the 15-minute token expires:

apps/api/src/infrastructure/security/user-tokens.ts

```
/**
 * Signs and verifies user access tokens (HS256, issuer `doktool-api`, audience `doktool-user`).
 * Lives outside the auth module so any module can guard its routes without importing auth.
 *
 * A token is accepted only while its session is live (ADR-023): signing a device out, changing
 * the password, a recovery, a suspension or a deletion take effect on the next request rather
 * than when the 15-minute token expires. This is a read-only, primary-key lookup of the auth
 * module's `user_sessions` — the one place outside auth that reads it (ADR-001 exception).
 */
@Injectable()
export class UserTokens {
  private readonly key: Uint8Array;

  constructor(
    @Inject(ENV) env: Env,
    private readonly prisma: PrismaService,
  ) {
    this.key = new TextEncoder().encode(env.USER_JWT_SECRET);
  }

  /** Signs a 15-minute access token for a user session (subject = user id, `sid` = session id). */
  async issue(principal: UserPrincipal): Promise<{ token: string; expiresAt: Date }> {
    const issuedAt = Math.floor(Date.now() / 1000);
    const expiresAt = new Date((issuedAt + ACCESS_TOKEN_TTL_SECONDS) * 1000);
    const token = await new SignJWT({ sid: principal.sessionId })
      .setProtectedHeader({ alg: 'HS256', typ: 'JWT' })
      .setIssuer(TOKEN_ISSUER)
      .setAudience(USER_AUDIENCE)
      .setSubject(principal.userId)
      .setJti(randomUUID())
      .setIssuedAt(issuedAt)
      .setExpirationTime(Math.floor(expiresAt.getTime() / 1000))
```

Refresh tokens are single-use. Using one twice, or from another device, revokes the whole session — this is how a stolen refresh token is detected:

apps/api/src/modules/auth/sessions.service.ts

```ts
  /**
   * Rotates a refresh token. A reused token, a token from another device, an expired or revoked
   * session, or a suspended account all revoke the session and answer SESSION_INVALID.
   */
  async refresh(refreshToken: string, deviceId: string, client: ClientInfo): Promise<AuthTokens> {
    const outcome = await this.prisma.$transaction(async (tx): Promise<RefreshOutcome> => {
      const [row] = await tx.$queryRaw<RefreshRow[]>`
        SELECT rt.id AS token_id, rt.session_id, rt.expires_at AS token_expires_at, rt.used_at,
               s.user_id, s.device_id, s.revoked_at, s.expires_at AS session_expires_at
        FROM refresh_tokens rt JOIN user_sessions s ON s.id = rt.session_id
        WHERE rt.token_hash = ${sha256Hex(refreshToken)}
        FOR UPDATE OF rt, s`;
      if (!row) return { kind: 'rejected', message: 'Unknown refresh token.' };

      const revoke = async (
        reason: SessionRevocationReason,
        message: string,
      ): Promise<RefreshOutcome> => {
        if (!row.revoked_at) {
          await tx.userSession.update({
            where: { id: row.session_id },
            data: { revokedAt: new Date(), revokedReason: reason },
          });
        }
        return { kind: 'rejected', message };
      };

      const now = new Date();
      if (row.revoked_at) return { kind: 'rejected', message: 'Session revoked.' };
      if (row.used_at) return revoke('REFRESH_REUSE', 'Refresh token reused; session revoked.');
      if (row.device_id !== deviceId)
        return revoke('DEVICE_MISMATCH', 'Refresh token used from another device.');
      if (row.token_expires_at <= now || row.session_expires_at <= now) {
        return { kind: 'rejected', message: 'Session expired.' };
      }
      if (!(await this.users.isActive(row.user_id))) {
        return revoke('ACCOUNT_SUSPENDED', 'Account is not active.');
      }

      const next = randomToken();
      const expiresAt = new Date(
        Math.min(now.getTime() + REFRESH_TOKEN_TTL_DAYS * DAY_MS, row.session_expires_at.getTime()),
      );
      await tx.refreshToken.update({ where: { id: row.token_id }, data: { usedAt: now } });
```

3.3 The access decision

Access is computed, never stored. `AccessService.profileFor(viewer)` gathers three facts — the account (status, professional type), the verification cases and the current subscription — and gives them to a **pure function**, `evaluateAccess`, together with the policy table. Chapter 6 reads that table line by line. What matters here: every content endpoint calls it, and the mobile app receives the result from `GET /v1/me/access` only to *draw* locks.

apps/api/src/modules/access/access.service.ts

```ts
/**
 * The only place access is decided (ADR-004). Gathers the inputs from the owning modules and
 * evaluates the policy for each request; nothing is cached or stored.
 */
```

```ts
@Injectable()
export class AccessService {
  constructor(
    private readonly users: UsersService,
    private readonly verification: VerificationStatusService,
    private readonly subscriptions: SubscriptionStatusService,
  ) {}

  /** The access profile of the caller (user or demo), computed fresh on every call. */
  async profileFor(viewer: Viewer): Promise<AccessProfile> {
    return evaluateAccess(await this.inputsFor(viewer));
  }

  /** Admin Web « Accès »: the profile of one account and its inputs (read-only). */
  async explainForUser(userId: string): Promise<{ profile: AccessProfile; inputs: AccessInputs }> {
    const inputs = await this.inputsFor({ accessContext: 'AUTHENTICATED', userId, sessionId: '' });
    return { profile: evaluateAccess(inputs), inputs };
  }

  /** Decides many items at once for list and search endpoints. */
  async decideItems<T extends AccessibleItem>(
    viewer: Viewer,
    items: readonly T[],
  ): Promise<{ profile: AccessProfile; decisions: ItemAccess[] }> {
    const profile = await this.profileFor(viewer);
    return { profile, decisions: items.map((item) => decideItem(profile, item)) };
  }

  /**
   * Collects the account facts, verification cases and subscription in parallel; a deleted account
   * is treated as signed out.
   */
  private async inputsFor(viewer: Viewer): Promise<AccessInputs> {
    if (viewer.accessContext === 'DEMO') {
      return { accessContext: 'DEMO', demoContext: viewer.demoContext };
    }
    const [account, verification, subscription] = await Promise.all([
      this.users.getAccessFacts(viewer.userId),
      this.verification.accessSnapshot(viewer.userId),
      this.subscriptions.current(viewer.userId),
    ]);
    if (!account || account.status === 'DELETED') {
      throw new DomainError('UNAUTHENTICATED', 'Account not found.');
    }
    return { accessContext: 'AUTHENTICATED', account, verification, subscription };
  }
}
```

3.4 One error format for everything

Every refusal is a `DomainError` with a stable code (`FEATURE_LOCKED`, `OTP_INVALID`, `SESSION_INVALID`…). The global `ErrorFilter` turns any exception into the same JSON:

```
HTTP 403
{ "error": { "code": "FEATURE_LOCKED", "message": "for developers only", "details": { "lockReason": '
```

The apps never show `message`; they translate `code` with their own catalogue (mobile `ApiError` + `commonErrorText`, admin `useErrorText`).

apps/api/src/common/errors/error.filter.ts

```ts
/** Renders every failure as `{ error: { code, message, details } }`. */
@Catch()
export class ErrorFilter implements ExceptionFilter {
```

```
  private readonly logger = new Logger(ErrorFilter.name);

  /** Called by Nest for any exception thrown while handling a request; writes the JSON error. */
  catch(exception: unknown, host: ArgumentsHost): void {
    const response = host.switchToHttp().getResponse<Response>();
    const [status, body] = this.toBody(exception);
    response.status(status).json(body);
  }

  /**
   * Maps an exception to [HTTP status, body]: DomainError as is, validation errors as
```

3.5 Files: R2, signed URLs and upload intents

All files (reference PDFs, profile photos, verification documents, recovery photos) are in Cloudflare R2, reached only through `StorageService`. Two buckets exist: `doktool-content` (reference PDFs, photos) and `doktool-private` (identity documents). Nothing is public. Uploads follow the same three steps everywhere:

1. The client declares the file (purpose, size, type, SHA-256). The API records an **upload intent** and answers with a signed PUT URL valid 15 minutes.
2. The client sends the bytes straight to R2.
3. The client confirms; the API checks size, type and the first bytes' signature (a PDF must start with `%PDF`), then attaches the file to its record.

apps/api/src/modules/storage/storage.policy.ts

```
/** Signed read URL lifetimes, in seconds. */
export const SIGNED_READ_TTL = {
  referenceDocument: 10 * 60,
  profileImage: 60 * 60,
  verificationDocument: 60,
  recoveryDocument: 60,
} as const;

/** A direct upload must happen within this delay. */
export const UPLOAD_URL_TTL_SECONDS = 15 * 60;
```

3.6 Background jobs

Work that must happen later or regularly runs in **pg-boss**, a job queue stored in PostgreSQL itself (no Redis, ADR-007). Each module registers its own schedules when it starts:

Job	Schedule	What it does
`reference-library` sweep	every minute	Processes uploaded PDFs: page count, checksum, file status.
`auth.account-recovery`	every 10 min	Applies approved recoveries once their 24-hour hold is over.
`billing.reconcile-checkouts`	every 10 min	Asks the gateway about payments still pending.
`billing.advance-subscriptions`	daily 02:30	Reminders, grace period, end of subscriptions.

`verification.expire-and-remind`	daily 06:17	Expiry reminders and expirations of verifications.
`verification.purge-documents`, `storage.purge-expired-uploads`, `auth.purge-expired`, `users.purge-unconfirmed`, `notifications.purge-old`…	hourly / daily	Delete what the retention rules say must not be kept.

apps/api/src/modules/notifications/notifications.module.ts

```ts
export class NotificationsModule implements OnModuleInit {
  constructor(
    private readonly jobs: JobsService,
    private readonly notifications: NotificationsService,
  ) {}

  /** Schedules the daily deletion of notifications older than the retention period. */
  onModuleInit(): void {
    this.jobs.register('notifications.purge-old', '41 3 * * *', async () => {
      await this.notifications.purgeOld();
    });
  }
}
```

3.7 Messages to people

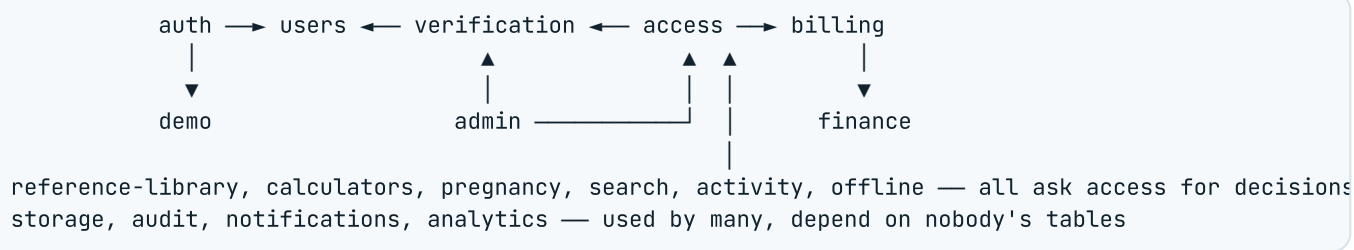
Codes and security notices go out by e-mail (SMTP) or SMS (Infobip or Twilio, with a fallback, ADR-022) through `MessageDelivery`. Their texts come from server catalogue keys (`server.security.…`) in the account's language. In-app notifications are rows of the `notifications` table holding a catalogue key and parameters — never free text.

3.8 Offline

Every real account (students included) can keep, on the phone, the reference PDFs it may open, the published calculator versions and the pregnancy profiles it opened. The API publishes an **offline manifest** (`GET /v1/offline/manifest`): what this person may keep, with a version and a SHA-256 for each item. The app's `OfflineSyncController` downloads what is missing or changed, deletes the rest, and always shows the data version and whether what you see is live or cached.

3.9 How modules are allowed to depend on each other

Modular monolith (ADR-001). One server, but each module owns its tables. Another module may use it only through what its `index.ts` exports — usually a service with a few read methods. That is why, for example, `AccessService` asks `UsersService.getAccessFacts()` instead of reading the users table itself. A unit test checks that every Prisma model sits in the section of the module that owns it.



4. How the API is built — NestJS, using DokTool's own code

NestJS is a framework that organises a Node.js server into **modules**, each made of **controllers** (HTTP endpoints), **services** (the logic) and **DTOs** (the shapes of what comes in and goes out). Nest builds every object and hands it what it needs through its constructor — *dependency injection*. This chapter teaches those pieces with the smallest complete module of DokTool: notifications.

4.1 Start-up: main.ts and app.setup.ts

The server starts in `main.ts`. It reads the environment once, builds the application from `AppModule`, applies the shared HTTP settings and listens:

apps/api/src/main.ts

```ts
import 'reflect-metadata';
import { Logger } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module.js';
import { configureApp } from './app.setup.js';
import { loadEnv } from './config/env.js';

// Server entry point: read and check the environment, build the Nest application, apply the shared
// HTTP settings, then listen on PORT.
const env = loadEnv(process.env);
const app = await NestFactory.create(AppModule.forRoot(env));
configureApp(app, env);
await app.listen(env.PORT);
new Logger('Bootstrap').log(`DokTool API listening on port ${env.PORT} (${env.NODE_ENV})`);
```

`loadEnv` (in `config/env.ts`) validates every environment variable with a Zod schema: a missing secret or a malformed URL stops the server at start-up instead of failing later. `configureApp` is shared with the integration tests, so tests run the exact same HTTP stack as production:

apps/api/src/app.setup.ts

```ts
/** HTTP settings shared by the server entry point and the end-to-end tests. */
export function configureApp(app: INestApplication, env: Env): void {
  const http = app.getHttpAdapter().getInstance() as Express;
  http.disable('x-powered-by');
  // Client IPs (rate limits, audit) come from X-Forwarded-For only behind known proxies.
  http.set('trust proxy', env.TRUST_PROXY_HOPS);

  app.use(helmet());
  app.use(cookieParser());
  // Gateway webhooks are signed over their exact bytes (ADR-021): keep them raw, and small.
  app.use(PAYMENT_WEBHOOKS_PATH, raw({ type: () => true, limit: '64kb' }));
  if (env.STORAGE_DRIVER === 'local') {
    // Local signed uploads carry raw file bytes (ADR-011); R2 uploads never reach the API.
    app.use(LOCAL_OBJECTS_PATH, raw({ type: () => true, limit: '21mb' }));
    // Admin Web uploads PDFs and shows verification documents from its own origin, as it does
    // against R2 (whose bucket CORS allows ADMIN_WEB_ORIGIN, docs/operations/r2-cors.md).
    app.use(LOCAL_OBJECTS_PATH, (request: Request, response: Response, next: NextFunction) => {
      response.setHeader('Cross-Origin-Resource-Policy', 'cross-origin');
      if (request.get('origin') === env.ADMIN_WEB_ORIGIN) {
        response.setHeader('Access-Control-Allow-Origin', env.ADMIN_WEB_ORIGIN);
        response.setHeader('Vary', 'Origin');
        response.setHeader('Access-Control-Allow-Methods', 'GET, PUT');
```

```
        response.setHeader('Access-Control-Allow-Headers', 'Content-Type');
        response.setHeader('Access-Control-Max-Age', '600');
      }
      if (request.method === 'OPTIONS') {
        response.status(204).end();
        return;
      }
      next();
    });
  }
  app.useGlobalPipes(
    new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true }),
  );
  app.useGlobalFilters(new ErrorFilter());
  app.enableShutdownHooks();

  // Interactive API docs for developers only; the published contract is packages/api-contract.
  if (env.NODE_ENV !== 'production') {
    SwaggerModule.setup('docs', app, () => buildOpenApiDocument(app));
  }
}
```

Read it line by line: `helmet()` adds security headers; `cookieParser()` reads the admin session cookie; payment webhooks keep their raw bytes (their signature is computed over them); the global `ValidationPipe` rejects any field a DTO does not declare (`forbidNonWhitelisted`) and converts types (`transform`); the global `ErrorFilter` gives every error the same shape; Swagger is served at `/docs` outside production.

4.2 The root module

apps/api/src/app.module.ts

```
/**
 * Root module: the infrastructure modules (config, database, jobs, crypto, rate limits, messaging,
 * security, health) plus every domain module listed in modules/index.ts.
 */
@Module({})
export class AppModule {
  /** Builds the root module for one validated environment (the tests pass their own). */
  static forRoot(env: Env): DynamicModule {
    return {
      module: AppModule,
      imports: [
        ConfigModule.forRoot(env),
        DatabaseModule,
        JobsModule,
        CryptoModule,
        RateLimitModule,
        MessagingModule,
        SecurityModule,
        HealthModule,
        ...DOMAIN_MODULES,
      ],
    };
  }
}
```

The infrastructure modules come first (configuration, database, jobs, cryptography, rate limits, messaging, token security, health), then the 20 domain modules. Most infrastructure modules are *global*: their services can be injected anywhere without importing the module.

4.3 A module: notifications

A module declares its controllers, its providers (services) and what it exports to other modules. This one also schedules a daily clean-up job when it starts:

apps/api/src/modules/notifications/notifications.module.ts

```ts
import { Module, type OnModuleInit } from '@nestjs/common';
import { JobsService } from '../../infrastructure/jobs/jobs.service.js';
import { NotificationsController } from './notifications.controller.js';
import { NotificationsService } from './notifications.service.js';

/**
 * In-app notifications rendered from catalogue keys.
 *
 * Owns: `notifications`.
 * Other modules reach it only through ./index.ts.
 */
@Module({
  controllers: [NotificationsController],
  providers: [NotificationsService],
  exports: [NotificationsService],
})
export class NotificationsModule implements OnModuleInit {
  constructor(
    private readonly jobs: JobsService,
    private readonly notifications: NotificationsService,
  ) {}

  /** Schedules the daily deletion of notifications older than the retention period. */
  onModuleInit(): void {
    this.jobs.register('notifications.purge-old', '41 3 * * *', async () => {
      await this.notifications.purgeOld();
    });
  }
}
```

Other modules must import only from `index.ts` — the module's public door:

apps/api/src/modules/notifications/index.ts

```ts
// Public entry point of the notifications module (ADR-001). Other modules import from here only.
export { NotificationsModule } from './notifications.module.js';
export {
  NotificationsService,
  type NewNotification,
  type NotificationParams,
} from './notifications.service.js';
```

4.4 A controller: endpoints and guards

A controller maps URLs to methods. Decorators (the `@…` lines) tell Nest the HTTP verb, the guard to run first, and tell Swagger how to document the endpoint:

apps/api/src/modules/notifications/notifications.controller.ts

```ts
@ApiTags('notifications')
```

- `@Controller(`${USER_API_PREFIX}/me/notifications`)` — every route here starts with `/v1/me/notifications`.
- `@UseGuards(UserAuthGuard)` — a valid user token is required (a demo token is not enough: demos have no notifications).

- `@CurrentUser() user` — injects the principal the guard stored on the request (`userId`, `sessionId`).
- `@Query() query: NotificationListQuery` — the query string, validated against the DTO class.
- `@Param('id', new ParseUUIDPipe())` — the URL id, refused unless it is a UUID.
- `@HttpCode(HttpStatus.NO_CONTENT)` — answers 204 with no body.

Controllers stay thin. In DokTool a controller only translates HTTP into a service call and the service's result into a DTO. Every rule lives in services, so it can be tested and reused.

4.5 DTOs: the contract

A DTO is a class with decorated fields. `class-validator` decorators (`@IsOptional`, `@IsISO8601` …) check incoming data; `@ApiProperty` describes the field in the OpenAPI contract, from which the mobile and admin clients are generated. Changing a DTO therefore changes both apps' generated code.

apps/api/src/modules/notifications/notifications.dto.ts

```
export class NotificationDto {
  @ApiProperty({ format: 'uuid' })
  id!: string;

  @ApiProperty({ example: 'verification.decided' })
  type!: string;

  @ApiProperty({
    description: 'Catalogue key the app renders.',
    example: 'mobile.notifications.verificationApproved',
  })
  messageKey!: string;

  @ApiProperty({
    type: 'object',
    additionalProperties: { type: 'string' },
    description: 'Values of the message placeholders, as strings (numbers in decimal).',
  })
  params!: Record<string, string>;

  @ApiPropertyOptional({ format: 'date-time', nullable: true, type: String })
  readAt!: string | null;

  @ApiProperty({ format: 'date-time' })
  createdAt!: string;

  static from(record: NotificationRecord): NotificationDto {
    return {
      id: record.id,
      type: record.type,
      messageKey: record.messageKey,
      params: Object.fromEntries(
        Object.entries(record.params).map(([key, value]) => [key, String(value)]),
      ),
      readAt: record.readAt?.toISOString() ?? null,
      createdAt: record.createdAt.toISOString(),
    };
  }
}
```

The static `from()` turns an internal record into exactly what the client may see (dates as ISO text, parameters as strings).

4.6 A service and the database

Services are `@Injectable()` classes. The constructor lists what they need; Nest provides it. Here the only dependency is `PrismaService`, the single database client:

apps/api/src/modules/notifications/notifications.service.ts

```ts
/** In-app notifications, rendered by the clients from catalogue keys (section 19). */
@Injectable()
export class NotificationsService {
  constructor(private readonly prisma: PrismaService) {}

  /** Account deletion: the person's notifications go with the account. */
  async eraseUser(userId: string): Promise<void> {
    await this.prisma.notification.deleteMany({ where: { userId } });
  }

  /**
   * Creates a notification; its message key must exist in the catalogue (checked so the app can
   * always render it).
   */
  async notify(input: NewNotification): Promise<void> {
    if (!input.messageKey.startsWith('mobile.notifications.') || !(input.messageKey in CATALOGUE)) {
      throw new Error(`Unknown notification message key "${input.messageKey}"`);
    }
    await this.prisma.notification.create({
      data: {
        userId: input.userId,
        type: input.type,
        messageKey: input.messageKey,
        params: input.params ?? {},
      },
    });
  }

  /** True when a notification of this type has all of `match` among its parameters (idempotent
reminders). */
  async exists(userId: string, type: string, match: NotificationParams): Promise<boolean> {
    const count = await this.prisma.notification.count({
      where: {
        userId,
        type,
        AND: Object.entries(match).map(([key, value]) => ({
          params: { path: [key], equals: value },
        })),
      },
    });
    return count > 0;
  }
```

Prisma turns the models of `prisma/schema.prisma` into a typed client: `this.prisma.notification.findMany(…)` is checked by the compiler against the table's columns. The model behind this service:

apps/api/prisma/schema.prisma

```
/// Rendered in the user's language from catalogue keys.
model Notification {
  id         String    @id @default(uuid(7)) @db.Uuid
  userId     String    @map("user_id") @db.Uuid
  user       User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  type       String
  messageKey String    @map("message_key")
  params     Json      @default("{}")
  readAt     DateTime? @map("read_at") @db.Timestamptz(3)
```

```
  createdAt  DateTime  @default(now()) @map("created_at") @db.Timestamptz(3)

  @@index([userId, readAt, createdAt(sort: Desc)])
  @@map("notifications")
}
```

Note the conventions used in the whole schema: UUID v7 identifiers, `snake_case` column names mapped from `camelCase` fields, timestamps with time zone, and an index shaped exactly like the list query (user, unread, newest first).

apps/api/src/infrastructure/database/prisma.service.ts

```ts
/**
 * The single database client. Connects lazily on the first query, so booting the app (and
 * generating the OpenAPI document) never needs a reachable database.
 *
 * Table ownership (ADR-001): a domain module only writes the tables its section of
 * prisma/schema.prisma assigns to it.
 */
@Injectable()
export class PrismaService extends PrismaClient implements OnModuleDestroy {
  constructor(@Inject(ENV) env: Env) {
    // Every session runs in UTC: the adapter sends dates without an offset, and a server whose
    // time zone is not UTC would otherwise store them shifted by its offset.
    super({
      adapter: new PrismaPg({ connectionString: env.DATABASE_URL, options: '-c TimeZone=UTC' }),
    });
  }

  /** True when the database answers a trivial query. */
  async isReachable(): Promise<boolean> {
    try {
      await this.$queryRaw`SELECT 1`;
      return true;
    } catch {
      return false;
    }
  }

  /** Closes the database connections when the application shuts down. */
  async onModuleDestroy(): Promise<void> {
    await this.$disconnect();
  }
}
```

4.7 Guards and decorators you will see everywhere

Decorator / guard	Where	What it does
`@UseGuards(UserAuthGuard)` + `@CurrentUser()`	user-only routes	Valid user token; the principal is `{ userId, sessionId }`.
`@UseGuards(ViewerAuthGuard)` + `@CurrentViewer()`	content routes	A user token *or* a demo token; the viewer says which.
`@RequireFeature('offline.sync')`	feature routes	Viewer guard + the access policy: refuses with `FEATURE_LOCKED` and the unlock path.
`@RequireGrants('finance.read')` + `@CurrentAdmin()`	admin routes	Admin session cookie, origin check, and the listed grants.

apps/api/src/modules/access/require-feature.ts

```ts
/** Re-evaluates the policy on every request and refuses a LOCKED feature (L34: every endpoint re-checks). */
@Injectable()
export class FeatureGuard implements CanActivate {
  constructor(
    private readonly access: AccessService,
    private readonly reflector: Reflector,
  ) {}

  /**
   * Reads the feature named by @RequireFeature on the route, computes the caller's profile and
   * refuses with FEATURE_LOCKED (and the unlock path) when that feature is locked.
   */
  async canActivate(context: ExecutionContext): Promise<boolean> {
    const key = this.reflector.getAllAndOverride<FeatureKey | undefined>(REQUIRED_FEATURE, [
      context.getHandler(),
      context.getClass(),
    ]);
    if (!key) return true;
    const request = context.switchToHttp().getRequest<Request & { viewer?: Viewer }>();
    if (!request.viewer) throw new DomainError('UNAUTHENTICATED', 'Viewer required.');
    const feature = featureOf(await this.access.profileFor(request.viewer), key);
    if (feature.state === 'LOCKED') {
      throw new DomainError('FEATURE_LOCKED', `Feature ${key} is locked.`, {
        lockReason: feature.lockReason ?? undefined,
        unlockPath: feature.unlockPath,
      });
    }
    return true;
  }
}
```

apps/api/src/modules/access/require-feature.ts

```ts
/** Guards a route with a user or demo token and the given feature being AVAILABLE. */
export function RequireFeature(key: FeatureKey) {
  return applyDecorators(
    SetMetadata(REQUIRED_FEATURE, key),
    UseGuards(ViewerAuthGuard, FeatureGuard),
    ApiBearerAuth('user'),
    ApiBearerAuth('demo'),
  );
}
```

4.8 Refusing: DomainError

Services never build HTTP responses. They throw `new DomainError('CODE', 'developer message', details?)`; the code decides the HTTP status:

apps/api/src/common/errors/domain-error.ts

```ts
/**
 * Stable error codes of the API contract. Clients translate them with their own catalogue; the
 * `message` field is for developers and logs only.
 */
export const ErrorCode = {
  VALIDATION_FAILED: HttpStatus.BAD_REQUEST,
  UNAUTHENTICATED: HttpStatus.UNAUTHORIZED,
  INVALID_CREDENTIALS: HttpStatus.UNAUTHORIZED,
  SESSION_INVALID: HttpStatus.UNAUTHORIZED,
  FORBIDDEN: HttpStatus.FORBIDDEN,
  ACCOUNT_SUSPENDED: HttpStatus.FORBIDDEN,
  IDENTIFIER_NOT_CONFIRMED: HttpStatus.FORBIDDEN,
  ORIGIN_NOT_ALLOWED: HttpStatus.FORBIDDEN,
  FEATURE_LOCKED: HttpStatus.FORBIDDEN,
```

```
  NOT_FOUND: HttpStatus.NOT_FOUND,
  ACCOUNT_EXISTS: HttpStatus.CONFLICT,
  PROFILE_ALREADY_SET: HttpStatus.CONFLICT,
  INVALID_STATE: HttpStatus.CONFLICT,
  VERSION_CONFLICT: HttpStatus.CONFLICT,
  CONTENT_UNAVAILABLE: HttpStatus.GONE,
  OTP_INVALID: HttpStatus.UNPROCESSABLE_ENTITY,
  OTP_EXPIRED: HttpStatus.GONE,
  TOTP_INVALID: HttpStatus.UNPROCESSABLE_ENTITY,
  UPLOAD_INVALID: HttpStatus.UNPROCESSABLE_ENTITY,
  UPLOAD_EXPIRED: HttpStatus.GONE,
  CHALLENGE_INVALID: HttpStatus.UNAUTHORIZED,
  ACCOUNT_LOCKED: HttpStatus.LOCKED,
  OTP_LOCKED: HttpStatus.LOCKED,
  OTP_RESEND_TOO_SOON: HttpStatus.TOO_MANY_REQUESTS,
  OTP_SEND_LIMIT: HttpStatus.TOO_MANY_REQUESTS,
  RATE_LIMITED: HttpStatus.TOO_MANY_REQUESTS,
  CHANNEL_UNAVAILABLE: HttpStatus.SERVICE_UNAVAILABLE,
} as const;
```

4.9 How the API is tested

- **Unit tests** (`apps/api/test/unit`): pure logic without a database — the access policy matrix, the calculator engine, CSV cells, payment gateway parsing, search ranking.
- **Integration tests** (`apps/api/test/integration/*.e2e.spec.ts`): the real application (`configureApp` included) against a real PostgreSQL test database, driven over HTTP with `supertest`. Helpers in `actors.ts` create a doctor, a student, an admin with given grants..., and `content.ts` publishes test content.

```
DATABASE_URL="postgresql://…/doktool_test" npx vitest run --project unit
DATABASE_URL="postgresql://…/doktool_test" npx vitest run --project integration test/integration/acti
```

5. Accounts: sign-up, sign-in, sessions, recovery, deletion

Two modules hold accounts. `users` owns the account rows (names, contacts, professional profile, preferences). `auth` owns everything that proves who someone is: codes, passwords, sessions, refresh tokens, contact changes, recovery and deletion (ADR-009, ADR-023).

5.1 One field for e-mail or phone

People sign up with an e-mail *or* an Algerian mobile number, typed in the same field. Everything starts by normalising it:

apps/api/src/modules/users/identifier.ts

```ts
/**
 * Parses what a person typed as e-mail or phone.
 * E-mails are lower-cased; phones become E.164, with Algerian local mobiles mapped to +213.
 * Returns null when the input is neither.
 */
export function parseIdentifier(raw: string): Identifier | null {
  const input = raw.trim();
  if (input.includes('@')) {
    const email = input.toLowerCase();
    return email.length <= 254 && EMAIL.test(email) ? { channel: 'EMAIL', value: email } : null;
  }
  let phone = input.replace(/[\s.\-()]/g, '');
  if (phone.startsWith('00')) phone = `+${phone.slice(2)}`;
  else if (ALGERIAN_LOCAL_MOBILE.test(phone)) phone = `+213${phone.slice(1)}`;
  return E164.test(phone) ? { channel: 'SMS', value: phone } : null;
}
```

E-mails are lower-cased; `0555 12 34 56` becomes `+213555123456` (E.164). The `channel` (`EMAIL` or `SMS`) then decides how codes are sent.

5.2 Sign-up in two steps

```
App                                  API                                          DB / providers
SignUpScreen « Créer mon compte »
  POST /v1/auth/sign-up  {firstName, lastName, identifier, password, wilayaCode, termsVersion, langua
                                     AuthService.signUp
                                       parse identifier, password ≠ identifier, terms version currer
                                       UsersService: create (or restart) an UNCONFIRMED account
                                       OtpService.issue(SIGN_UP)  ──────────────► code by e-mail / SM
                       ◄— { challengeId, destinationHint "a•••@x.dz", resendAvailableAt, expiresAt }
OtpScreen (6 boxes)
  POST /v1/auth/sign-up/confirm {challengeId, code, deviceId}
                                     OtpService.consume → contact confirmed
                                     DemoService.markConverted(deviceId)   (if this phone ran a demo
                                     SessionsService.open → access + refresh tokens
                       ◄— AuthTokens
RoleSelectScreen « Qui êtes-vous ? » → PUT professional profile → verification
```

apps/api/src/modules/auth/auth.service.ts

```ts
  /**
   * Creates (or restarts) an unconfirmed account and sends a 6-digit code to the e-mail or phone.
   */
  async signUp(input: SignUpInput, client: ClientInfo): Promise<OtpChallengeView> {
```

```
    await this.limiter.hit(RATE_LIMITS.signUpPerIp, clientKey(client));
    const identifier = requireIdentifier(input.identifier);
    assertPasswordAllowed(input.password, identifier);
    if (input.termsVersion !== this.env.TERMS_VERSION) {
      throw new DomainError(
        'VALIDATION_FAILED',
        'The terms have changed; show the current version.',
        {
          fields: { termsVersion: [`Current version is ${this.env.TERMS_VERSION}.`] },
        },
      );
    }
    if (!this.delivery.canReach(identifier.channel, identifier.value)) {
      throw new DomainError(
        'CHANNEL_UNAVAILABLE',
        `${identifier.channel} confirmation is not available for this identifier.`,
      );
    }
    const { userId } = await this.users.createOrRestartPendingAccount({
      identifier,
      passwordHash: await this.hasher.hash(input.password),
      firstName: input.firstName.trim(),
      lastName: input.lastName.trim(),
      wilayaCode: input.wilayaCode,
      termsVersion: input.termsVersion,
      language: input.language,
    });
    return this.otp.issue('SIGN_UP', identifier, userId, input.language);
  }
```

An unconfirmed account that is never confirmed is deleted by the `users.purge-unconfirmed` job. Signing up again with the same identifier restarts it instead of failing — someone who lost the first code is not stuck.

apps/api/src/modules/auth/auth.service.ts

```
  /**
   * Checks the sign-up code, marks the contact confirmed and signs the new account in on this
   * device.
   */
  async confirmSignUp(
    challengeId: string,
    code: string,
    device: DeviceInput,
    client: ClientInfo,
  ): Promise<AuthTokens> {
    await this.limiter.hit(RATE_LIMITS.otpVerifyPerIp, clientKey(client));
    const { userId, channel } = await this.otp.consume(challengeId, 'SIGN_UP', code);
    await this.users.markIdentifierConfirmed(userId, channel);
    await this.users.clearSignInFailures(userId);
    // A demo tried on this device became an account (analytics « Démo » funnel).
    await this.demo.markConverted(device.id, userId);
    return this.sessions.open(userId, device, client);
  }
```

5.3 One-time codes

`OtpService` serves four purposes: sign-up, password reset, contact change and account recovery. Its rules: 6 digits; the code itself is never stored (only an HMAC bound to the challenge); 60 seconds between sends and at most 5 sends; 3 wrong codes lock the challenge for 10 minutes; a challenge can only be consumed for its own purpose.

apps/api/src/modules/auth/otp.service.ts

```ts
  /**
   * Checks a code: right purpose, not used, not expired, not locked. A right code consumes the
   * challenge once; a wrong one counts towards the lock.
   */
  private async verify(challengeId: string, purpose: OtpPurpose, code: string) {
    const challenge = await this.prisma.otpChallenge.findUnique({ where: { id: challengeId } });
    const now = new Date();
    if (
      !challenge ||
      challenge.purpose !== purpose ||
      challenge.consumedAt ||
      challenge.expiresAt <= now
    ) {
      throw new DomainError('OTP_EXPIRED', 'This code has expired. Ask for a new one.');
    }
    if (challenge.lockedUntil && challenge.lockedUntil > now) {
      throw new DomainError('OTP_LOCKED', 'Too many wrong codes.', {
        retryAt: challenge.lockedUntil.toISOString(),
      });
    }

    if (
      delivers(challenge.purpose, challenge.userId) &&
      hexDigestsEqual(challenge.codeHash, this.hash(challenge.id, code))
    ) {
      const { count } = await this.prisma.otpChallenge.updateMany({
        where: { id: challenge.id, consumedAt: null },
        data: { consumedAt: now },
      });
      if (count === 1) return challenge;
      throw new DomainError('OTP_EXPIRED', 'This code has already been used.');
    }

    const [row] = await this.prisma.$queryRaw<
      { failed_attempts: number; locked_until: Date | null }[]
    >`
      UPDATE otp_challenges SET
        failed_attempts = CASE WHEN failed_attempts + 1 >= ${OTP_MAX_FAILED_ATTEMPTS} THEN 0
                               ELSE failed_attempts + 1 END,
        locked_until = CASE WHEN failed_attempts + 1 >= ${OTP_MAX_FAILED_ATTEMPTS}
                            THEN now() + make_interval(mins => ${OTP_LOCK_MINUTES})
                            ELSE locked_until END
      WHERE id = ${challenge.id}::uuid
      RETURNING failed_attempts, locked_until`;
    if (row?.locked_until && row.locked_until > now) {
      throw new DomainError('OTP_LOCKED', 'Too many wrong codes.', {
        retryAt: row.locked_until.toISOString(),
      });
    }
    throw new DomainError('OTP_INVALID', 'Wrong code.', {
      attemptsRemaining:
        OTP_MAX_FAILED_ATTEMPTS - (row?.failed_attempts ?? OTP_MAX_FAILED_ATTEMPTS),
    });
  }
```

No account enumeration. « Mot de passe oublié » always answers with a challenge, even for an identifier with no account — in that case nothing is sent (`delivers()`), but the answer looks identical, so nobody can test which e-mails have an account.

5.4 Sign-in, lockout and sessions

`apps/api/src/modules/auth/auth.service.ts`

```
  /**
   * Password sign-in: rate limit, lockout after repeated failures, suspended and unconfirmed
   * accounts refused. A successful sign-in cancels any recovery someone else requested.
   */
  async signIn(
    rawIdentifier: string,
    password: string,
    device: DeviceInput,
    client: ClientInfo,
  ): Promise<AuthTokens> {
    await this.limiter.hit(RATE_LIMITS.signInPerIp, clientKey(client));
    const identifier = requireIdentifier(rawIdentifier);
    const candidate = await this.users.findSignInCandidate(identifier);
    if (!candidate || candidate.status === 'DELETED') {
      await this.hasher.verifyDecoy(password);
      throw new DomainError('INVALID_CREDENTIALS', 'Wrong identifier or password.');
    }
    if (candidate.lockedUntil && candidate.lockedUntil > new Date()) {
      throw new DomainError('ACCOUNT_LOCKED', 'Too many wrong passwords.', {
        retryAt: candidate.lockedUntil.toISOString(),
      });
    }
    if (!(await this.hasher.verify(candidate.passwordHash, password))) {
      const lockedUntil = await this.users.registerFailedSignIn(candidate.id);
      if (lockedUntil) {
        throw new DomainError('ACCOUNT_LOCKED', 'Too many wrong passwords.', {
          retryAt: lockedUntil.toISOString(),
        });
      }
      throw new DomainError('INVALID_CREDENTIALS', 'Wrong identifier or password.');
    }
    if (candidate.status === 'SUSPENDED') {
      throw new DomainError('ACCOUNT_SUSPENDED', 'This account is suspended.');
    }
    if (!candidate.identifierConfirmed) {
      throw new DomainError(
        'IDENTIFIER_NOT_CONFIRMED',
        'Confirm this identifier first: sign up again to receive a new code.',
      );
    }
    await this.users.clearSignInFailures(candidate.id);
    // The owner has the account: a recovery someone else asked for stops here (ADR-023).
    await this.recovery.cancelForOwner(candidate.id);
    return this.sessions.open(candidate.id, device, client);
  }
```

Read it as a checklist: a rate limit per IP and per identifier; five wrong passwords lock the account for 15 minutes (`ACCOUNT_LOCKED` with `retryAt`); suspended and unconfirmed accounts are refused with their own codes; a success opens a **session** bound to the device's installation id and — important for recovery — cancels any account recovery someone else started.

A session is a row of `user_sessions` with its refresh tokens in `refresh_tokens` (only their SHA-256). The API can list a person's sessions (`GET /v1/me/sessions`) and end one (`DELETE /v1/me/sessions/:sessionId`); the V1 app does not show this list yet, but a password change, a recovery or a suspension use the same revocation. Because `UserTokens.verify` checks the session on every request, the other device is out at its very next call.

5.5 Changing contacts and password

`AccountSecurityService` (Settings › « Sécurité du compte ») follows one principle: **every change asks for the current password**, so a borrowed unlocked phone cannot take over the account; and **every**

change is announced on the other contact, so the owner learns about it even if the attacker controls one channel.

apps/api/src/modules/auth/account-security.service.ts

```
  /** Changes the password; every other device signs out, this one stays signed in. */
  async changePassword(
    userId: string,
    sessionId: string,
    input: { currentPassword: string; newPassword: string },
    client: ClientInfo,
  ): Promise<void> {
    await this.checkPassword(userId, input.currentPassword, 'currentPassword');
    const contacts = (await this.users.contactsOf(userId))!;
    const lowered = input.newPassword.trim().toLowerCase();
    if (lowered === contacts.email || lowered === contacts.phone) {
      throw new DomainError('VALIDATION_FAILED', 'The password cannot be your e-mail or phone.', {
        fields: { newPassword: ['The password cannot be your e-mail or phone.'] },
      });
    }
    await this.users.setPassword(userId, await this.hasher.hash(input.newPassword));
    await this.sessions.revokeOthers(userId, sessionId, 'PASSWORD_CHANGED');
    await this.audit.record({
      actorType: 'USER',
      actorId: userId,
      action: 'user.password_changed',
      entityType: 'User',
      entityId: userId,
      ip: client.ip,
      userAgent: client.userAgent,
    });
    await this.notices.notify(userId, 'passwordChanged');
  }
```

5.6 Assisted recovery: « J'ai perdu l'accès aux deux »

When someone has lost both their e-mail and their phone, a human checks their identity:

apps/api/src/modules/auth/account-recovery.service.ts

```
/**
 * Assisted account recovery (ADR-023): « J'ai perdu l'accès à mon e-mail et à mon téléphone ».
 *
 * 1. The person names the lost contact, their name and a NEW contact, and proves the new contact
 *    with a code. A recovery token (kept by the app only) then allows two photo uploads: an
 *    identity document and a selfie holding it.
 * 2. The account's current contacts are warned; the owner signing in cancels the request.
 * 3. A reviewer (`verification.review`) compares the photos with the account and decides.
 * 4. An approval applies after RECOVERY_HOLD_HOURS: the new contact becomes the only contact, the
 *    password becomes unusable and every device signs out; the person then sets a password with
 *    « Mot de passe oublié » on the new contact. Photos are deleted 30 days after the decision.
 */
@Injectable()
export class AccountRecoveryService {
  private readonly logger = new Logger(AccountRecoveryService.name);

  constructor(
    private readonly prisma: PrismaService,
    private readonly users: UsersService,
    private readonly otp: OtpService,
    private readonly sessions: SessionsService,
    private readonly delivery: MessageDelivery,
    private readonly limiter: RateLimiter,
    private readonly intents: UploadIntentsService,
    private readonly storage: StorageService,
```

```
    private readonly notices: SecurityNoticesService,
    private readonly verification: VerificationStatusService,
    private readonly audit: AuditService,
    private readonly hasher: PasswordHasher,
  ) {}

  // ──────────────────────────────── the person ────────────────────────────────

  /**
   * Step 1: records the lost contact, the name and the new contact, and sends a code to the new
   * contact.
   */
```

```
1 start         lost contact + first/last name + NEW contact      → code sent to the new contact
2 confirm       code proves the new contact                         → recovery token (60 min to finis
3 photos        identity document + selfie → private bucket           (JPEG/PNG, 10 MB, signed upload
4 submit        request goes to Admin Web « Récupération »             answer promised within 72 h
5 review        admin compares photos with the verified identity  → approve (note) or reject (reason)
6 hold 24 h     the owner is told; signing in with the password CANCELS the request
7 apply         job every 10 min: new contact replaces the old ones, password unusable, all devices c
```

The apply step is where the October 2026 audit found a race: the request is now **claimed first** (APPROVED → APPLIED in one conditional update), and only then are the contacts replaced. If the owner cancels at the same instant, the claim fails and nothing changes.

apps/api/src/modules/auth/account-recovery.service.ts

```
  /**
   * Applies an approved request whose hold is over: the new contact replaces the old ones, the
   * password becomes unusable and every device signs out.
   */
  private async apply(request: AccountRecoveryRequest, now: Date): Promise<void> {
    // Claim the request before touching the account: the owner signing in meanwhile cancels it
    // (cancelForOwner), and a cancelled request must never change the account.
    const { count } = await this.prisma.accountRecoveryRequest.updateMany({
      where: { id: request.id, status: 'APPROVED' },
      data: { status: 'APPLIED', appliedAt: now, documentsPurgeAfter: this.purgeDate(now) },
    });
    if (count !== 1) return;
    const identifier = { channel: request.newChannel, value: request.newDestination };
    let replaced: { email: string | null; phone: string | null };
    try {
      replaced = await this.users.replaceContactsForRecovery(
        request.userId!,
        identifier,
        await this.hasher.hash(randomToken()),
      );
    } catch (error) {
      if (error instanceof DomainError && error.code === 'ACCOUNT_EXISTS') {
        // The new contact was taken by another account during the wait: stop, explain.
        await this.prisma.accountRecoveryRequest.update({
          where: { id: request.id },
          data: { status: 'CANCELED', appliedAt: null, documentsPurgeAfter: this.purgeDate(now) },
        });
        await this.notices.toRecoveryContact(
          destinationOf(request),
          'recoveryCanceled',
          request.language,
        );
        return;
      }
      // Anything else: give the request back to the next run, unchanged.
      await this.prisma.accountRecoveryRequest.update({
        where: { id: request.id },
```

```
        data: { status: 'APPROVED', appliedAt: null, documentsPurgeAfter: null },
      });
      throw error;
    }
    await this.sessions.revokeAll(request.userId!, 'ACCOUNT_RECOVERED');
    await this.audit.record({
      actorType: 'SYSTEM',
      actorId: null,
      action: 'account_recovery.applied',
      entityType: 'User',
      entityId: request.userId!,
      after: { requestId: request.id, newChannel: request.newChannel },
    });
    const previous: Destination[] = [
      ...(replaced.email ? [{ channel: 'EMAIL' as const, to: replaced.email }] : []),
      ...(replaced.phone && !replaced.email
        ? [{ channel: 'SMS' as const, to: replaced.phone }]
        : []),
    ].filter((destination) => destination.to !== request.newDestination);
    for (const destination of previous) {
      await this.notices.toRecoveryContact(destination, 'recoveryApplied', request.language);
    }
    await this.notices.toRecoveryContact(
      destinationOf(request),
      'recoveryRestored',
      request.language,
      {
        contact: request.newDestination,
      },
    );
  }
```

5.7 Deleting an account

apps/api/src/modules/auth/account-deletion.service.ts

```
  /**
   * Deletes the account after checking the password: ends subscriptions, erases each module's
   * data, anonymises the account, ends every session and writes an audit entry.
   */
  async delete(userId: string, password: string, client: ClientInfo): Promise<void> {
    await this.limiter.hit(RATE_LIMITS.deleteAccountPerUser, userId);
    const passwordHash = await this.users.passwordHashOf(userId);
    if (!passwordHash) throw new DomainError('UNAUTHENTICATED', 'The account no longer exists.');
    // A form error on the password field, not a 401: the session itself is fine.
    if (!(await this.hasher.verify(passwordHash, password))) {
      throw new DomainError('VALIDATION_FAILED', 'Wrong password.', {
        fields: { password: ['Wrong password.'] },
      });
    }

    const subscriptionsEnded = await this.subscriptions.endForDeletedAccount(userId);
    const documentsDeleted = await this.verification.eraseUser(userId);
    await this.activity.eraseUser(userId);
    await this.notifications.eraseUser(userId);
    await this.pregnancy.eraseUser(userId);
    await this.otp.eraseUser(userId);
    await this.recovery.eraseUser(userId);
    await this.users.eraseAccount(userId);
    await this.sessions.revokeAll(userId, 'ACCOUNT_DELETED');
    await this.audit.record({
      actorType: 'USER',
      actorId: userId,
      action: 'user.account_deleted',
      entityType: 'User',
      entityId: userId,
```

```
      after: { subscriptionsEnded, documentsDeleted },
      ip: client.ip,
      userAgent: client.userAgent,
    });
  }
```

Each module that holds personal data exposes an `eraseUser(userId)` method; deletion calls them all, anonymises the account row (finance and the audit log must keep their history, without a name), revokes every session and writes one audit entry. On the phone, the app then forgets every personal setting it kept (chapter 11).

5.8 The numbers, in one place

apps/api/src/modules/auth/auth.policy.ts

```ts
// User-authentication parameters (docs/adr/ADR-009-authentication.md).
import type { RateLimitRule } from '../../infrastructure/rate-limit/rate-limiter.js';

export const OTP_DIGITS = 6;
export const OTP_TTL_MINUTES = 10;
/** Wrong codes allowed before the challenge locks (spec: 3 tries then 10 minutes). */
export const OTP_MAX_FAILED_ATTEMPTS = 3;
export const OTP_LOCK_MINUTES = 10;
/** Minimum delay between two sends of the same challenge (spec: 60 s). */
export const OTP_RESEND_COOLDOWN_SECONDS = 60;
export const OTP_MAX_SENDS = 5;

export const REFRESH_TOKEN_TTL_DAYS = 30;
export const SESSION_MAX_AGE_DAYS = 180;
/** Revoked or ended sessions stay listable this long before the purge job removes them. */
export const SESSION_RETENTION_DAYS = 30;

export const PASSWORD_MIN_LENGTH = 8;
export const PASSWORD_MAX_LENGTH = 128;

export const RATE_LIMITS = {
  signUpPerIp: { name: 'sign-up:ip', limit: 10, windowSeconds: 3600 },
  signInPerIp: { name: 'sign-in:ip', limit: 20, windowSeconds: 600 },
  refreshPerIp: { name: 'refresh:ip', limit: 120, windowSeconds: 600 },
  otpVerifyPerIp: { name: 'otp-verify:ip', limit: 30, windowSeconds: 600 },
  otpResendPerDestination: { name: 'otp-resend:destination', limit: 5, windowSeconds: 3600 },
  resetPerIdentifier: { name: 'reset:identifier', limit: 5, windowSeconds: 3600 },
  resetPerIp: { name: 'reset:ip', limit: 20, windowSeconds: 3600 },
  deleteAccountPerUser: { name: 'account-delete:user', limit: 5, windowSeconds: 3600 },
  /** Password checks of signed-in changes (contacts, password): a stolen phone can't brute-force. */
  accountChangePerUser: { name: 'account-change:user', limit: 10, windowSeconds: 3600 },
  recoveryPerIp: { name: 'recovery:ip', limit: 5, windowSeconds: 3600 },
  recoveryPerClaimed: { name: 'recovery:claimed', limit: 3, windowSeconds: 24 * 3600 },
  recoveryPerNewContact: { name: 'recovery:contact', limit: 3, windowSeconds: 24 * 3600 },
  recoveryUploadPerRequest: { name: 'recovery:upload', limit: 10, windowSeconds: 3600 },
} as const satisfies Record<string, RateLimitRule>;

export const DAY_MS = 24 * 3600 * 1000;

// Assisted account recovery (ADR-023).
/** The code and the photos must be sent within this delay after the request. */
export const RECOVERY_DOCUMENTS_TTL_MINUTES = 60;
/** An approved recovery applies after this delay; the owner signing in meanwhile cancels it. */
export const RECOVERY_HOLD_HOURS = 24;
/** Recovery photos are deleted this long after the decision (or the cancellation). */
export const RECOVERY_DOCUMENT_RETENTION_DAYS = 30;
/** Requests answered by the team within this delay (shown to the person). */
export const RECOVERY_REVIEW_HOURS = 72;
export const RECOVERY_REJECTION_REASONS = [
```

```
  'PHOTO_UNREADABLE',
  'IDENTITY_MISMATCH',
  'SELFIE_MISMATCH',
  'NO_ACCOUNT',
  'OTHER',
] as const;
export type RecoveryRejectionReason = (typeof RECOVERY_REJECTION_REASONS)[number];
```

6. Access: who may open what

This is the most important chapter for understanding DokTool's behaviour. One table decides what each kind of person can open; one pure function applies it; every endpoint re-checks it; the apps only draw the answer. If you understand `modules/access`, you understand why a screen shows a lock, an explanation or a « Voir les offres » button.

6.1 The inputs

For a demo session, the only input is "this is a demo" (and which profession it previews). For a real account, `AccessService` gathers three facts in parallel, each from the module that owns it:

Fact	Asked from	Contains
Account facts	`UsersService.getAccessFacts`	status (ACTIVE, SUSPENDED, DELETED), professional type (DOCTOR, RESIDENT, STUDENT or not chosen yet)
Verification snapshot	`VerificationStatusService.accessSnapshot`	for IDENTITY, STUDENT_STATUS and PROFESSIONAL_STATUS: status, submission date, expiry, `renewalPending`
Subscription	`SubscriptionStatusService.current`	status (ACTIVE, CANCELED, GRACE…), period end, grace end

6.2 The policy table

The whole V1 policy is data. Read it as a grid: one block per tier; in each, which content items open (`content`) and the state of each feature (`features`):

apps/api/src/modules/access/access-policy.config.ts

```ts
/**
 * The whole V1 policy, one entry per tier (DEMO, STUDENT, PROFESSIONAL_UNPAID, PROFESSIONAL,
 * RESTRICTED).
 */
export const ACCESS_POLICY = {
  version: '2026-10-04.1',
  /** A student whose status case is submitted keeps student access this long (section 27). */
  studentPendingWindowDays: 14,
  /** ACTIVE past its period end still counts as live this long, if billing has not moved it to
GRACE yet. */
  subscriptionGraceFallbackDays: 7,

  tiers: {
    DEMO: {
      content: {
        calculators: demoSamplesOnly('DEMO_LIMIT', 'subscription.plans'),
        referenceDocuments: demoSamplesOnly('DEMO_LIMIT', 'subscription.plans'),
        pregnancyProfiles: demoSamplesOnly('DEMO_LIMIT', 'subscription.plans'),
      },
      features: {
        search: available,
        'reference.view': available,
        'pregnancy.view': available,
        'calculator.use': available,
        // Kept on the device only; discarded with the demo (section 27).
        favorites: available,
```

```
        'activity.history': available,
        'offline.sync': locked('DEMO_LIMIT', 'auth.sign-up'),
        'subscription.purchase': locked('DEMO_LIMIT', 'auth.sign-up'),
        'profile.edit': locked('DEMO_LIMIT', 'auth.sign-up'),
      },
    },
    STUDENT: {
      content: {
        // ADR-006 Q2: the clinical calculators; every V1 calculator is student-tier.
        calculators: { scope: 'STUDENT_TIER', lockReason: 'PROFESSIONAL_ONLY', unlockPath: null },
        // ADR-006 Q2 + D2: titles only, except the demo samples.
        referenceDocuments: demoSamplesOnly('PROFESSIONAL_ONLY', null),
        // ADR-019: like the reference models, names only except the demo samples.
        pregnancyProfiles: demoSamplesOnly('PROFESSIONAL_ONLY', null),
      },
      features: {
        search: available,
        'reference.view': available,
        'pregnancy.view': available,
        'calculator.use': available,
        favorites: available,
        'activity.history': available,
        'offline.sync': available,
        // Students are not a paying audience: explanation only, no upsell.
        'subscription.purchase': locked('PROFESSIONAL_ONLY', null),
        'profile.edit': available,
      },
    },
    PROFESSIONAL_UNPAID: {
      content: {
        // ADR-006 Q1: the demo sample set, with a real account.
        calculators: demoSamplesOnly('SUBSCRIPTION_REQUIRED', 'subscription.plans'),
        referenceDocuments: demoSamplesOnly('SUBSCRIPTION_REQUIRED', 'subscription.plans'),
        pregnancyProfiles: demoSamplesOnly('SUBSCRIPTION_REQUIRED', 'subscription.plans'),
      },
      features: {
        search: available,
        'reference.view': available,
        'pregnancy.view': available,
        'calculator.use': available,
        favorites: available,
        'activity.history': available,
        'offline.sync': locked('SUBSCRIPTION_REQUIRED', 'subscription.plans'),
        'subscription.purchase': { state: 'PURCHASE_AFTER_VERIFICATION' },
        'profile.edit': available,
      },
    },
    PROFESSIONAL: {
      content: {
        calculators: everything,
        referenceDocuments: everything,
        pregnancyProfiles: everything,
      },
      features: {
        search: available,
        'reference.view': available,
        'pregnancy.view': available,
        'calculator.use': available,
        favorites: available,
        'activity.history': available,
        'offline.sync': available,
        // Renewal.
        'subscription.purchase': available,
        'profile.edit': available,
      },
    },
    RESTRICTED: {
```

```
      content: {
        calculators: { scope: 'NONE', fromRestriction: true },
        referenceDocuments: { scope: 'NONE', fromRestriction: true },
        pregnancyProfiles: { scope: 'NONE', fromRestriction: true },
      },
      features: {
        search: available,
        'reference.view': { state: 'RESTRICTION' },
        'pregnancy.view': { state: 'RESTRICTION' },
        'calculator.use': { state: 'RESTRICTION' },
        favorites: { state: 'AVAILABLE', readOnly: true },
        'activity.history': { state: 'AVAILABLE', readOnly: true },
        'offline.sync': { state: 'RESTRICTION' },
        'subscription.purchase': { state: 'RESTRICTION' },
        'profile.edit': available,
      },
    },
  } satisfies Record<Tier, TierSetting>,
} as const;
```

The content rules use three scopes: `ALL` (everything opens), `STUDENT_TIER` (items marked for students, plus samples), `DEMO_SAMPLE` (only items flagged `demoSample`), and `NONE` for a restricted account. A locked item always carries its `lockReason` and, when one exists, its `unlockPath` — the single screen that can help (sign up, see plans, start verification…). Students get explanations without an unlock path: no upsell (Lock L38).

6.3 Placing a person in one tier

`placeInTier` reads like the specification. Follow it from top to bottom with an example in mind — a verified doctor whose subscription ended last week:

apps/api/src/modules/access/access-policy.ts

```
/** Places a person in exactly one tier (section 27 table, ADR-010 §3-4). */
export function placeInTier(inputs: AccessInputs, now: Date, policy: AccessPolicy): Placement {
  if (inputs.accessContext === 'DEMO') return { tier: 'DEMO', restriction: null, purchase: null };

  const { account, verification, subscription } = inputs;
  if (account.status !== 'ACTIVE') return restricted('ACCOUNT_SUSPENDED', null);
  if (account.professionalType === null)
    return restricted('VERIFICATION_REQUIRED', 'onboarding.role');
  if (verification.IDENTITY.status === 'REJECTED') {
    return restricted('VERIFICATION_REQUIRED', 'verification.start');
  }

  if (account.professionalType === 'STUDENT') {
    const student = verification.STUDENT_STATUS;
    if (student.status === 'VERIFIED')
      return { tier: 'STUDENT', restriction: null, purchase: null };
    if (isPendingWithin(student, policy.studentPendingWindowDays, now)) {
      return { tier: 'STUDENT', restriction: null, purchase: null };
    }
    return PENDING.has(student.status)
      ? restricted('VERIFICATION_PENDING', 'verification.status')
      : restricted('VERIFICATION_REQUIRED', 'verification.start');
  }

  const professional = verification.PROFESSIONAL_STATUS;
  if (professional.status === 'REJECTED')
    return restricted('VERIFICATION_REQUIRED', 'verification.start');
  const verifiedOrLapsed = professional.status === 'VERIFIED' || professional.status === 'EXPIRED';
  if (verifiedOrLapsed && isSubscriptionLive(subscription, now, policy)) {
    return { tier: 'PROFESSIONAL', restriction: null, purchase: null };
```

```
  }
  const purchase: Placement['purchase'] =
    professional.status === 'VERIFIED'
      ? 'OPEN'
      : PENDING.has(professional.status) || professional.renewalPending
        ? { lockReason: 'VERIFICATION_PENDING', unlockPath: 'verification.status' }
        : { lockReason: 'VERIFICATION_REQUIRED', unlockPath: 'verification.start' };
  return { tier: 'PROFESSIONAL_UNPAID', restriction: null, purchase };
}
```

1. Not a demo. Account ACTIVE. Professional type chosen (DOCTOR). Identity not rejected.
2. Not a student, so look at `PROFESSIONAL_STATUS`: VERIFIED.
3. `isSubscriptionLive`: ACTIVE subscriptions stay live 7 days past their end while billing moves them to GRACE; GRACE stays live until its end. Last week → still live → **PROFESSIONAL**.
4. After grace: not live. Verified, so `purchase = 'OPEN'` → **PROFESSIONAL_UNPAID** with « Renouveler » available.

apps/api/src/modules/access/access-policy.ts

```
/**
 * True while a subscription gives Pro access: ACTIVE (with a grace fallback), CANCELED until the
 * end of the paid period, GRACE until its end date.
 */
export function isSubscriptionLive(
  subscription: SubscriptionSnapshot | null,
  now: Date,
  policy: AccessPolicy = ACCESS_POLICY,
): boolean {
  if (!subscription) return false;
  const time = now.getTime();
  const periodEnd = subscription.currentPeriodEnd?.getTime() ?? Number.NEGATIVE_INFINITY;
  switch (subscription.status) {
    case 'ACTIVE':
      return time < periodEnd + policy.subscriptionGraceFallbackDays * DAY_MS;
    case 'CANCELED':
      return time < periodEnd;
    case 'GRACE':
      return time < (subscription.graceEndsAt?.getTime() ?? Number.NEGATIVE_INFINITY);
    default:
      return false;
  }
}
```

6.4 Re-verification never interrupts access

A verification expires (professional status after a set period; see chapter 8). Renewing it opens a new case of the same type. Before the October 2026 audit, the newest case — still undecided — replaced the verified one, and a paying doctor fell to PROFESSIONAL_UNPAID the moment they tapped « Renouveler ». `accessSnapshot` now keeps the earlier decision counting while the new case is undecided, and flags `renewalPending` so purchase shows « vérification en cours »:

apps/api/src/modules/verification/verification-status.service.ts

```
  /**
   * What access and checkout rely on (section 27: re-verification never interrupts access). While
   * the latest case of a type is not decided yet, the decision before it keeps counting: a VERIFIED
   * case until its expiry and, for professional status, a lapsed (EXPIRED) one too, since with a
   * live subscription an expiry does not end access before the period end. Without such an earlier
   * decision, the open case itself is reported, as {@link snapshot} does.
   */
  async accessSnapshot(userId: string, now = new Date()): Promise<VerificationSnapshot> {
```

```
    const cases = await this.prisma.verificationCase.findMany({
      where: { userId },
      orderBy: [{ createdAt: 'desc' }, { id: 'desc' }],
      select: { type: true, status: true, submittedAt: true, expiresAt: true },
    });
    const snapshot: VerificationSnapshot = {
      IDENTITY: NOT_STARTED,
      PROFESSIONAL_STATUS: NOT_STARTED,
      STUDENT_STATUS: NOT_STARTED,
    };
    for (const type of TYPES) {
      const ofType = cases.filter((item) => item.type === type);
      const latest = ofType[0];
      if (!latest) continue;
      snapshot[type] = {
        status: latest.status,
        submittedAt: latest.submittedAt,
        expiresAt: latest.expiresAt,
      };
      if (!UNDECIDED.has(latest.status)) continue;
      const previous = ofType.find((item) => !UNDECIDED.has(item.status));
      const stillVerified =
        previous?.status === 'VERIFIED' &&
        (previous.expiresAt === null || previous.expiresAt > now);
      const lapsed =
        type === 'PROFESSIONAL_STATUS' &&
        (previous?.status === 'EXPIRED' || previous?.status === 'VERIFIED');
      if (previous && (stillVerified || lapsed)) {
        snapshot[type] = {
          submittedAt: previous.submittedAt,
          expiresAt: previous.expiresAt,
          status: stillVerified ? 'VERIFIED' : 'EXPIRED',
          renewalPending: true,
        };
      }
    }
    return snapshot;
  }
```

6.5 From tier to profile, and from profile to items

apps/api/src/modules/access/access-policy.ts

```
/**
 * Computes the full AccessProfile: tier, every feature decision and every content rule. Pure: same
 * inputs and time give the same profile.
 */
export function evaluateAccess(
  inputs: AccessInputs,
  now: Date = new Date(),
  policy: AccessPolicy = ACCESS_POLICY,
): AccessProfile {
  const placement = placeInTier(inputs, now, policy);
  const tier = policy.tiers[placement.tier];
  // The content rule of one type; RESTRICTED takes the reason of its restriction.
  const contentRule = (type: ContentType): ContentRule => {
    const rule = tier.content[type];
    if ('fromRestriction' in rule) {
      return {
        scope: 'NONE',
        lockReason: placement.restriction?.lockReason ?? 'VERIFICATION_REQUIRED',
        unlockPath: placement.restriction?.unlockPath ?? null,
      };
    }
    return rule;
  };
```

```
  return {
    accessContext: inputs.accessContext,
    tier: placement.tier,
    features: FEATURE_KEYS.map((key) => decideFeature(key, tier.features[key], placement)),
    content: {
      calculators: contentRule('calculators'),
      referenceDocuments: contentRule('referenceDocuments'),
      pregnancyProfiles: contentRule('pregnancyProfiles'),
    },
    computedAt: now,
    policyVersion: policy.version,
  };
}
```

The result, `AccessProfile`, is what `GET /v1/me/access` returns to the app. Lists and detail endpoints then decide each item with `decideItem`:

apps/api/src/modules/access/access-policy.ts

```
/** Opens or locks one item for a profile. "Not available" (disabled) is the content module's call.
 */
export function decideItem(profile: AccessProfile, item: AccessibleItem): ItemAccess {
  const rule = profile.content[item.type];
  const opens =
    rule.scope === 'ALL' ||
    (rule.scope === 'STUDENT_TIER' && (item.accessTier === 'STUDENT' || item.demoSample)) ||
    (rule.scope === 'DEMO_SAMPLE' && item.demoSample);
  return opens
    ? { state: 'AVAILABLE', lockReason: null, unlockPath: null }
    : { state: 'LOCKED', lockReason: rule.lockReason, unlockPath: rule.unlockPath };
}
```

Pure and tested as a matrix. `access-policy.ts` does no input/output at all: same inputs and time, same answer. `test/unit/access-policy.matrix.spec.ts` checks every tier against every feature and content rule. To change the policy: edit the table, bump `version`, update the matrix test.

6.6 Re-checking on every endpoint

Two mechanisms enforce the profile on the server:

- `@RequireFeature('…')` on a route (chapter 4.7) refuses a locked feature before the handler runs.
- Content services call `AccessService.decideItems(viewer, items)` and only return a PDF URL, a calculator definition or a profile's content when the item is `AVAILABLE`; a locked item is still listed, with its lock.

6.7 Viewers: user or demo

apps/api/src/infrastructure/security/viewer-auth.guard.ts

```
/** Whoever is looking at content: a signed-in user, or a demo session (ADR-010). */
export type Viewer =
  | ({ accessContext: 'AUTHENTICATED' } & UserPrincipal)
  | ({ accessContext: 'DEMO' } & DemoPrincipal);

/** An Express request after ViewerAuthGuard ran. */
```

apps/api/src/infrastructure/security/viewer-auth.guard.ts

```
/**
 * For read routes open to both user and demo tokens. Write routes use UserAuthGuard, which
```

```
 * refuses demo tokens.
 */
@Injectable()
export class ViewerAuthGuard implements CanActivate {
  constructor(
    private readonly userTokens: UserTokens,
    private readonly demoTokens: DemoTokens,
  ) {}

  /** Accepts either a user token or a demo token and stores the resulting viewer on the request. */
  async canActivate(context: ExecutionContext): Promise<boolean> {
    const request = context.switchToHttp().getRequest<ViewerRequest>();
    const match = /^Bearer ([A-Za-z0-9._~+/=-]+)$/.exec(request.get('authorization') ?? '');
    const token = match?.[1];
    if (!token) throw new DomainError('UNAUTHENTICATED', 'Missing bearer token.');
    request.viewer = isDemoToken(token)
      ? { accessContext: 'DEMO', ...(await this.demoTokens.verify(token)) }
      : { accessContext: 'AUTHENTICATED', ...(await this.userTokens.verify(token)) };
    return true;
  }
}
```

Read routes accept both kinds of token; write routes (favorites, history, verification, payments) use `UserAuthGuard`, which refuses demo tokens. Demo and user tokens are signed with the same key but have different *audiences*, so one can never pass for the other.

6.8 The demo

apps/api/src/modules/demo/demo.service.ts

```
/**
 * Demo sessions (Lock L37, ADR-006 Q5, ADR-010): no account, no contact data, no time limit.
 * A device keeps one active session; nothing a demo does is written to user tables.
 */
@Injectable()
export class DemoService {
  constructor(
    private readonly prisma: PrismaService,
    private readonly tokens: DemoTokens,
    private readonly limiter: RateLimiter,
  ) {}

  /**
   * Starts (or restarts) the demo of a device: one active session per device, a new renewal token
   * each time.
   */
  async start(
```

apps/api/src/modules/demo/demo.service.ts

```
  /**
   * Starts (or restarts) the demo of a device: one active session per device, a new renewal token
   * each time.
   */
  async start(
    demoContext: DemoContext,
    deviceId: string,
    client: ClientInfo,
  ): Promise<DemoSessionGrant> {
    await this.limiter.hit(RATE_LIMITS.startPerIp, clientKey(client));
    const deviceIdHash = sha256Hex(deviceId);
    const renewalToken = randomToken();
    const now = new Date();
    const existing = await this.prisma.demoSession.findFirst({
```

```
      where: { deviceIdHash, status: 'ACTIVE' },
      orderBy: { startedAt: 'desc' },
      select: { id: true },
    });
    const session = existing
      ? await this.prisma.demoSession.update({
          where: { id: existing.id },
          data: { demoContext, renewalTokenHash: sha256Hex(renewalToken), lastSeenAt: now },
        })
      : await this.prisma.demoSession.create({
          data: { deviceIdHash, demoContext, renewalTokenHash: sha256Hex(renewalToken) },
        });
    const access = await this.tokens.issue({ demoSessionId: session.id, demoContext });
    return {
      demoSessionId: session.id,
      demoContext,
      accessToken: access.token,
      accessTokenExpiresAt: access.expiresAt,
      renewalToken,
    };
  }
```

One device keeps one active demo session, identified by the phone's installation id. Nothing a demo does is written to user tables: its favorites live in the phone's memory only. When the same phone confirms a new account, `markConverted` closes the demo as CONVERTED, so the « Démo » analytics funnel can count conversions (another audit fix).

6.9 What the apps do with the profile

- **Mobile:** `AccessController` loads the profile at launch, on resume and after verification or payment. Rows draw a `LockLabel`; tapping a locked row opens `showLockSheet` with the explanation and, if there is one, the single unlock action. Locks are never red.
- **Admin Web:** the user drawer's « Accès » tab shows an account's profile and the inputs it came from (`AccessService.explainForUser`), so support can answer "why can't I open this?".

7. Content: reference PDFs, calculators, pregnancy, search, offline

The four resources of DokTool live in four modules — `reference-library`, `calculators`, `pregnancy` and (for finding them) `search` — helped by `storage`, `clinical-review`, `activity` and `offline`. Each has an **app side** (read-only, access-checked) and an **admin side** (authoring, review, publication, audited).

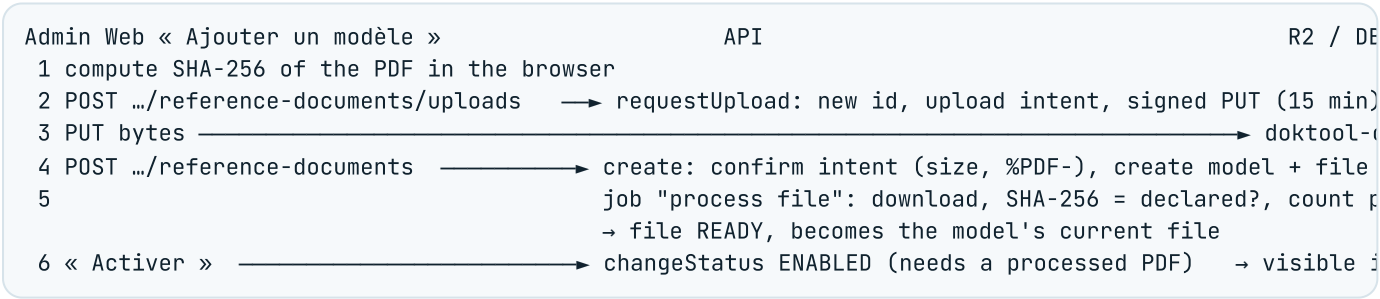
7.1 Reference library: ordonnances and certificates

A reference model is a PDF uploaded by the Super Admin, with minimal metadata: a French title (pre-filled from the file name), an optional English title, the `demoSample` flag and a status. The PDF *is* the content; nobody edits it in DokTool.

apps/api/src/modules/reference-library/reference-documents-admin.service.ts

```ts
/**
 * Super Admin management of Ordonnances and Certificates (section 13). The admin never authors
 * clinical content: the PDF is the content; every mutation needs a reason and is audited.
 */
@Injectable()
export class ReferenceDocumentsAdminService {
  constructor(
    private readonly prisma: PrismaService,
    private readonly intents: UploadIntentsService,
    private readonly storage: StorageService,
    private readonly audit: AuditService,
    private readonly jobs: JobsService,
  ) {}

  /** Step 1 of a new model: a signed upload for version 1 under a freshly allocated id. */
```



apps/api/src/modules/storage/upload-intents.service.ts

```ts
  /**
   * Checks the uploaded object: it exists, has the declared size (within the purpose limit) and
   * starts with the file signature of its type. A failing object is deleted.
   */
  async confirm(
    intentId: string,
    requestedBy: UploadRequester,
  ): Promise<StoredObject & { declaredSha256: string; purpose: UploadPurpose }> {
    const intent = await this.prisma.storageUploadIntent.findUnique({ where: { id: intentId } });
    if (
      !intent ||
      intent.requestedByType !== requestedBy.type ||
      intent.requestedById !== requestedBy.id
    ) {
      throw new DomainError('NOT_FOUND', 'Upload not found.');
```

```
    }
    if (intent.confirmedAt)
      throw new DomainError('UPLOAD_INVALID', 'This upload was already confirmed.');
    if (intent.expiresAt <= new Date())
      throw new DomainError('UPLOAD_EXPIRED', 'The upload window has closed.');

    const rule = UPLOAD_RULES[intent.purpose];
    const object = await this.storage.head(intent.bucket, intent.key);
    if (!object) throw new DomainError('UPLOAD_INVALID', 'The file has not been uploaded yet.');

    const reject = async (reason: string): Promise<never> => {
      await this.storage.delete(intent.bucket, intent.key);
      throw new DomainError('UPLOAD_INVALID', reason);
    };
    if (object.sizeBytes !== intent.declaredSizeBytes || object.sizeBytes > rule.maxBytes) {
      return reject('The uploaded file does not have the declared size.');
    }
    const head = await this.storage.download(intent.bucket, intent.key, {
      start: 0,
      end: Math.min(SIGNATURE_PROBE_BYTES, object.sizeBytes) - 1,
    });
    if (!matchesSignature(rule, intent.contentType, head)) {
      return reject('The uploaded file is not a valid file of the declared type.');
    }

    const { count } = await this.prisma.storageUploadIntent.updateMany({
      where: { id: intent.id, confirmedAt: null },
      data: { confirmedAt: new Date() },
    });
    if (count !== 1) throw new DomainError('UPLOAD_INVALID', 'This upload was already confirmed.');
    return {
      ...object,
      contentType: intent.contentType,
      declaredSha256: intent.declaredSha256,
      purpose: intent.purpose,
    };
  }
```

apps/api/src/modules/reference-library/reference-processing.service.ts

```
  /**
   * Processes one uploaded file: checks it, records the result and makes it current when it is
   * newer.
   */
  async process(fileId: string): Promise<void> {
    const file = await this.prisma.referenceDocumentFile.findUnique({ where: { id: fileId } });
    if (!file || file.processingStatus !== 'PENDING') return;

    const outcome = await this.inspect(file.storageKey, file.declaredSha256);
    const now = new Date();
    const applied = await this.prisma.$transaction(async (tx) => {
      const claimed = await tx.referenceDocumentFile.updateMany({
        where: { id: file.id, processingStatus: 'PENDING' },
        data:
          outcome.status === 'READY'
            ? {
                processingStatus: 'READY',
                sha256: outcome.sha256,
                pageCount: outcome.pageCount,
                processedAt: now,
              }
            : { processingStatus: 'FAILED', processingError: outcome.error, processedAt: now },
      });
      if (claimed.count !== 1) return false;

      // Sequential reads: a relation include here would run parallel queries on the
```

```
      // transaction connection.
      const document = await tx.referenceDocument.findUniqueOrThrow({
        where: { id: file.documentId },
      });
      const currentFile = document.currentFileId
        ? await tx.referenceDocumentFile.findUniqueOrThrow({
            where: { id: document.currentFileId },
            select: { version: true },
          })
        : null;
      if (outcome.status === 'READY') {
        const newer = !currentFile || currentFile.version < file.version;
        if (newer) {
          await tx.referenceDocument.update({
            where: { id: document.id },
            data: {
              currentFileId: file.id,
              ...(document.status === 'PROCESSING'
                ? {
                    status: document.statusAfterProcessing ?? 'DISABLED',
                    statusAfterProcessing: null,
                  }
                : {}),
            },
          });
        }
      } else if (document.status === 'PROCESSING' && !document.currentFileId) {
        await tx.referenceDocument.update({
          where: { id: document.id },
          data: { status: 'DISABLED', statusAfterProcessing: null },
        });
      }
      return true;
    });
    if (!applied) return;

    await this.audit.record({
      actorType: 'SYSTEM',
      actorId: null,
      action: outcome.status === 'READY' ? 'reference.file.processed' : 'reference.file.failed',
      entityType: 'ReferenceDocumentFile',
      entityId: file.id,
      after:
        outcome.status === 'READY'
          ? { documentId: file.documentId, version: file.version, pageCount: outcome.pageCount }
          : { documentId: file.documentId, version: file.version, error: outcome.error },
    });
  }
```

« Remplacer le PDF » creates the next file version under the **same** model id, so favorites, history and links keep working; earlier versions stay stored (immutable keys) and can still be previewed by admins.

7.2 StorageService: one door to files

No module imports the R2 SDK. They all use the abstract `StorageService`; `R2StorageAdapter` implements it in production and `LocalStorageAdapter` (files on disk, signed URLs served by the API itself) in development and tests:

apps/api/src/modules/storage/storage.service.ts

```
/**
 * The only way the backend touches file bytes (ADR-002, ADR-011). PostgreSQL keeps metadata and
 * keys; objects live in R2 (or local files in development). Keys are immutable: an upload to an
 * existing key fails, and a replacement writes a new key.
```

```ts
 */
export abstract class StorageService {
  /** Stores bytes under a new key; fails if the key already exists. */
  abstract upload(input: UploadInput): Promise<StoredObject>;

  /** Deletes an object (no error when it does not exist). */
  abstract delete(bucket: BucketId, key: string): Promise<void>;

  /** Object metadata, or null when it does not exist. */
  abstract head(bucket: BucketId, key: string): Promise<StoredObject | null>;

  /** Object bytes, optionally a range of them (signature checks, hashing jobs). */
  abstract download(bucket: BucketId, key: string, range?: ByteRange): Promise<Uint8Array>;

  /** Short-lived URL to read an object inline. */
  abstract getSignedReadUrl(
    bucket: BucketId,
    key: string,
    options: { ttlSeconds: number },
  ): Promise<string>;

  /**
   * Short-lived URL to PUT exactly `contentLength` bytes of `contentType` at `key`. Both headers
   * are part of the signature, so the storage refuses any other size or type.
   */
  abstract getSignedUploadUrl(
    bucket: BucketId,
    key: string,
    options: { contentType: string; contentLength: number; ttlSeconds: number },
  ): Promise<string>;

  /**
   * Writes a new version under `newKey` and keeps `oldKey`, so every past version stays
   * retrievable (audit §8). The caller moves its pointer to the new key.
   */
  async replace(input: UploadInput & { oldKey: string }): Promise<StoredObject> {
    if (!(await this.head(input.bucket, input.oldKey))) {
      throw new Error(`Cannot replace missing object ${input.bucket}/${input.oldKey}`);
    }
    return this.upload(input);
  }
}
```

7.3 Calculators: a formula as data

A calculator is never code. It is a **definition** — JSON validated by a Zod schema — that both engines (TypeScript on the server, Dart on the phone) interpret the same way. A definition holds:

- **inputs**: `NUMBER` / `INTEGER` (with units from a sourced registry and three ranges: *hard* — impossible values; *validated* — where the formula was validated; *plausible* — beyond it, a « vérifiez la valeur » note), `CHOICE` (answers, optionally with points) and `BOOLEAN`;
- **cross-field rules** (e.g. diastolic must be below systolic);
- the **formula**, a tree over a closed set of operators;
- the **result** (unit, decimals), the interpretation **bands** and the **limitations**.

apps/api/src/modules/calculators/engine/definition.ts

```ts
/** Closed operator set of the formula AST (audit §9). */
export type Expr =
  | { num: number }
  | { bool: boolean }
  | { str: string }
  | { input: string }
```

```
  | { present: string }
  | { points: string }
  | { op: BinaryOp; args: [Expr, Expr] }
  | { op: UnaryOp; args: [Expr] }
  | { op: VariadicOp; args: Expr[] }
```

apps/api/src/modules/calculators/engine/definition.ts

```ts
export const BINARY_OPS = [
  'add',
  'sub',
  'mul',
  'div',
  'pow',
  'lt',
  'le',
  'gt',
  'ge',
  'eq',
  'ne',
] as const;
export const UNARY_OPS = ['sqrt', 'ln', 'log10', 'exp', 'not'] as const;
export const VARIADIC_OPS = ['min', 'max', 'sum', 'and', 'or'] as const;
export type BinaryOp = (typeof BINARY_OPS)[number];
```

There is no `eval`, no free-form expression and no operator outside these lists: a definition cannot do anything the engine does not know. `checkDefinition` then checks what a schema cannot: unique keys, known units, nested ranges, a well-typed formula, ordered bands that do not overlap.

The engine

apps/api/src/modules/calculators/engine/evaluate.ts

```ts
/**
 * Runs a calculator: presence and types, hard ranges, cross-field rules, validated ranges,
 * plausibility notes, then the formula and its band. Never returns a result for invalid or out-of-
 * range inputs.
 */
export function evaluate(definition: CalculatorDefinition, raw: EvaluationInputs): Evaluation {
  const declared = new Map(definition.inputs.map((input) => [input.key, input]));
  const invalid: Issue[] = [];
  const conversions: Conversion[] = [];
  const values = new Map<string, Value>();

  for (const inputKey of Object.keys(raw)) {
    if (!declared.has(inputKey) && raw[inputKey] != null)
      invalid.push({ code: 'UNKNOWN_INPUT', input: inputKey });
  }

  // 1. Presence, type and unit conversion to canonical units.
  for (const input of definition.inputs) {
    const given = raw[input.key];
    if (given == null || given.value === null || given.value === undefined) {
      if (input.required) invalid.push({ code: 'REQUIRED', input: input.key });
      continue;
    }
    const accepted = acceptValue(input, given, conversions);
    if ('issue' in accepted) invalid.push(accepted.issue);
    else values.set(input.key, accepted.value);
  }
  if (invalid.length > 0) return { outcome: 'INVALID', issues: invalid, conversions };

  // 2. Hard ranges: physiologically impossible values block.
  for (const input of numericInputs(definition)) {
    const value = values.get(input.key);
```

```
    if (typeof value === 'number' && !inRange(value, input.hardRange)) {
      invalid.push({
        code: 'HARD_RANGE',
        input: input.key,
        range: { ...input.hardRange, unit: input.canonicalUnit },
      });
    }
  }
  if (invalid.length > 0) return { outcome: 'INVALID', issues: invalid, conversions };

  // 3. Cross-field rules.
  const outOfRange: Issue[] = [];
  for (const rule of definition.crossFieldRules) {
    let holds: Value;
    try {
      holds = run(rule.when, values, definition);
    } catch (error) {
      if (error instanceof MissingInput) {
        invalid.push({ code: 'MISSING_INPUT', input: error.input, ruleId: rule.id });
        continue;
      }
      throw error;
    }
    if (holds !== true) {
      const issue: Issue = { code: 'CROSS_FIELD', ruleId: rule.id, message: rule.message };
      (rule.outcome === 'INVALID' ? invalid : outOfRange).push(issue);
    }
  }
  if (invalid.length > 0) return { outcome: 'INVALID', issues: invalid, conversions };

  // 4. Validated ranges: outside the population the formula was validated on → not computed.
  for (const input of numericInputs(definition)) {
    const value = values.get(input.key);
    if (typeof value === 'number' && !inRange(value, input.validatedRange)) {
      outOfRange.push({
        code: 'VALIDATED_RANGE',
        input: input.key,
        range: { ...input.validatedRange, unit: input.canonicalUnit },
      });
    }
  }
  if (outOfRange.length > 0)
    return { outcome: 'OUT_OF_VALIDATED_RANGE', issues: outOfRange, conversions };

  // 5. Plausibility notes (never block).
  const notes: PlausibilityNote[] = [];
  for (const input of numericInputs(definition)) {
    const value = values.get(input.key);
    if (
      typeof value === 'number' &&
      input.plausibleRange &&
      !inRange(value, input.plausibleRange)
    ) {
      notes.push({
        input: input.key,
        range: { ...input.plausibleRange, unit: input.canonicalUnit },
      });
    }
  }

  // 6. The formula.
  let value: Value;
  try {
    value = run(definition.formula, values, definition);
  } catch (error) {
    if (error instanceof MissingInput) {
      return {
```

```
        outcome: 'INVALID',
        issues: [{ code: 'MISSING_INPUT', input: error.input }],
        conversions,
      };
    }
    throw error;
  }
  if (typeof value !== 'number' || !Number.isFinite(value)) {
    return { outcome: 'INVALID', issues: [{ code: 'NON_FINITE_RESULT' }], conversions };
  }
  const band = definition.interpretation.find(
    (candidate) =>
      (candidate.from === undefined || value >= candidate.from) &&
      (candidate.to === undefined || value < candidate.to),
  );
  return {
    outcome: 'RESULT',
    value,
    unit: definition.result.unit,
    decimals: definition.result.decimals,
    band: band ? { code: band.code, severity: band.severity, label: band.label } : null,
    notes,
    conversions,
  };
}
```

Read the order: presence and types → hard ranges → cross-field rules → validated ranges → plausibility notes → formula → band. The engine **never returns a result** for invalid input or input outside the validated range: it returns the issues instead, and the app explains them. Rounding is "half away from zero" computed on the decimal text, so 2.675 rounds the same way in TypeScript and Dart.

Two engines, one truth. The TypeScript engine is authoritative: admins' test runs and publication use it. The Dart engine lets the phone compute offline; before using a version, the app runs the version's published test cases on its own engine (« self-test ») and refuses the calculator if one fails. `packages/calculator-vectors` holds shared cases both engines must pass in CI.

Governance: draft → review → published

```
DRAFT ──submit──► IN_REVIEW ──approve──► PUBLISHED ──retire──► RETIRED
  ▲  (author)          │ (reviewer ≠ author)     immutable: a change is a new version
  └──── return ────────┘
 gate to submit:  primary source + a passing test run of THIS definition (checksum)
 gate to publish: tests re-run on the server, sources, four-eyes, and 1 or 2 sign-offs (ADR-024)
```

`apps/api/src/modules/calculators/calculators-admin.service.ts`

```
  /**
   * « Approuver » : re-runs every test on the backend engine and checks sources and the four-eyes
   * rule. When two sign-offs are required and none is recorded yet, records the first one (ADR-
024);
   * otherwise publishes, makes it current and retires the previously published version.
   */
  async publish(
    admin: AdminPrincipal,
    versionId: string,
    reviewNote: string,
    client: ClientInfo,
  ): Promise<VersionRecord> {
    const version = await this.requireStatus(versionId, ['IN_REVIEW']);
    await this.assertIndependentReviewer(admin, versionId);
    const run = this.testRunFor(version);
```

```
    this.assertPublishable(version, run);
    const approval = await this.review.approval('CALCULATOR', version, admin);
    if (!approval.publish) return this.signOff(admin, version, reviewNote, run, client);

    const now = new Date();
    const previous = await this.prisma.$transaction(async (tx) => {
      const calculator = await tx.calculator.findUniqueOrThrow({
        where: { id: version.calculatorId },
      });
      await tx.calculatorVersion.update({
        where: { id: versionId },
        data: {
          status: 'PUBLISHED',
          reviewedById: admin.adminId,
          reviewedAt: now,
          reviewNote: reviewNote.trim(),
          requiredSignOffs: approval.requiredSignOffs,
          publishedAt: now,
          engineVersion: ENGINE_VERSION,
          testRun: run as unknown as Prisma.InputJsonValue,
        },
      });
      await tx.calculator.update({
        where: { id: calculator.id },
        data: { currentVersionId: versionId },
      });
      if (calculator.currentVersionId && calculator.currentVersionId !== versionId) {
        await tx.calculatorVersion.update({
          where: { id: calculator.currentVersionId },
          data: { status: 'RETIRED' },
        });
      }
      return calculator.currentVersionId;
    });
    await this.record(
      admin,
      client,
      'calculator.version.published',
      'CalculatorVersion',
      versionId,
      {
        after: {
          version: version.version,
          definitionSha256: version.definitionSha256,
          engineVersion: ENGINE_VERSION,
          testsPassed: run.total,
          retiredVersionId: previous,
          firstSignOffById: version.firstSignOffById,
        },
        reason: reviewNote,
      },
    );
    return this.version(versionId);
  }
```

apps/api/src/modules/calculators/calculators-admin.service.ts

```
  /** Four-eyes: the reviewer is neither the author nor anyone who changed the draft. */
  private async assertIndependentReviewer(admin: AdminPrincipal, versionId: string): Promise<void> {
    const version = await this.version(versionId);
    const contributed =
      version.authoredById === admin.adminId ||
      (await this.prisma.auditLogEntry.count({
        where: {
          entityType: 'CalculatorVersion',
          entityId: versionId,
```

```
          actorType: 'ADMIN',
          actorId: admin.adminId,
          action: { in: [...DRAFTING_ACTIONS] },
        },
      })) > 0;
    if (contributed) {
      throw new DomainError(
        'FORBIDDEN',
        'The reviewer must not have authored or edited this version.',
      );
    }
  }
```

A test run is stored with the SHA-256 of the canonical JSON of the definition it ran on (`canonical-json.ts`). Editing the definition changes the checksum, so an old green run no longer counts — it is impossible to submit a definition that was not tested as it is.

Double sign-off (clinical-review)

apps/api/src/modules/clinical-review/clinical-review.service.ts

```
/**
 * Double sign-off for clinical content (ADR-024). One sign-off — the reviewer who publishes — is
 * the rule of ADR-006 D1; the Super Admin may require two for calculators, pregnancy profiles or
 * both. The second reviewer is never the author nor the first reviewer.
 *
 * Owns: `clinical_review_policies`.
 */
@Injectable()
export class ClinicalReviewService {
  constructor(
    private readonly prisma: PrismaService,
    private readonly audit: AuditService,
  ) {}

  /** The number of sign-offs this kind of content needs now (1 unless set to 2). */
  async required(content: ClinicalContentKind): Promise<SignOffs> {
```

apps/api/src/modules/clinical-review/clinical-review.service.ts

```
  /** Decides what an independent reviewer's approval does: a first sign-off, or publication. */
  async approval(
    content: ClinicalContentKind,
    version: { requiredSignOffs: number; firstSignOffById: string | null },
    admin: AdminPrincipal,
  ): Promise<Approval> {
    if (version.firstSignOffById === admin.adminId) {
      throw new DomainError(
        'FORBIDDEN',
        'The second sign-off must come from another reviewer than the first.',
      );
    }
    const requiredSignOffs = await this.requiredFor(content, version);
    return {
      publish: requiredSignOffs === 1 || version.firstSignOffById !== null,
      requiredSignOffs,
    };
  }
```

7.4 Grossesse · Allaitement

A **substance** (names, ATC code, synonyms, `demoSample`, enabled) has **profile versions**, governed like calculators (draft → review → published, author ≠ reviewer, optional second sign-off). A profile's content

has two blocks: pregnancy (T1, T2, T3, peripartum) and lactation. For each period: a **conduct** (`CONTRAINDICATED`, `AVOID`, `OCCASIONAL_USE` or none) and, separately, an **evidence level** from 0 (« Absence de données ») to 4 (« Données robustes ») with its sources — never merged into a "safe" verdict.

apps/api/src/modules/pregnancy/pregnancy.content.ts

```ts
/** The clinical instructions shown as a badge (icon + word); anything else has no badge. */
export const CONDUCTS = ['CONTRAINDICATED', 'AVOID', 'OCCASIONAL_USE'] as const;
export type Conduct = (typeof CONDUCTS)[number];

/** 4 Données robustes · 3 modérées · 2 limitées · 1 très limitées · 0 Absence de données. */
export const EVIDENCE_LEVELS = [0, 1, 2, 3, 4] as const;
const evidence = z
  .object({
    /** Null only in a draft: an import never invents a level; review requires one. */
    level: z.number().int().min(0).max(4).nullable(),
    /** « cohortes et séries concordantes », « < 100 grossesses exposées »… */
    population: localized.optional(),
  })
  .strict();

/** Two references that disagree, and the reviewers' synthesis (info Alert). */
const conflict = z
  .object({
    sourceIds: z.array(z.uuid()).length(2),
    synthesis: localized,
  })
  .strict();

/** Status exactly as the imported source wrote it (« Non valide »), kept beside the conduct. */
const reportedStatus = z.string().trim().min(1).max(200).optional();
/** Sources (of this version) that support this period specifically. */
const periodSources = z.array(z.uuid()).max(10).optional();
/**
 * Alternative names an import could not link to a substance with certainty. Review is refused
 * until an author links each one to a substance or removes it: nothing is matched by similarity.
 */
```

apps/api/src/modules/pregnancy/pregnancy.content.ts

```ts
/**
 * What still prevents a version from going to review. A draft may be saved incomplete; review needs
 * a complete profile: both blocks, every period, a conduct sentence where a period has a badge, the
 * conduct sentence of lactation, at least one source, conflicts citing the version's own sources,
 * and alternatives that are other substances. « Absence de données » (level 0) is a complete
answer.
 */
export function reviewProblems(
  content: ProfileContent,
  context: {
    substanceId: string;
    sourceIds: readonly string[];
    knownSubstanceIds: ReadonlySet<string>;
  },
): string[] {
  const problems: string[] = [];
  if (context.sourceIds.length === 0) problems.push('sources: at least one source is required');
  /** Reports alternatives that are the substance itself or not an existing substance. */
  const checkAlternatives = (where: string, ids: readonly string[]) => {
```

Three more tools exist on the admin side:

- **Spreadsheet import** (ADR-025): an uploaded XLSX becomes a *preview batch*; the admin maps columns and status words, corrects rows, then commits — which writes **drafts**, never published content.
- **Merging duplicates** (ADR-027): a duplicate substance's names become synonyms of the one kept; favorites and links of the duplicate open the kept substance; the merge can be undone.
- **User reports** (« Signaler une erreur » in the app) arrive in a queue the clinical team resolves.

apps/api/src/modules/pregnancy/pregnancy.service.ts

```
  /**
   * One profile (a merged duplicate opens the substance kept); the content only when the viewer
   * may open it.
   */
  async get(viewer: Viewer, id: string): Promise<SubstanceProfile> {
    const row = await this.prisma.substance.findUnique({
      where: { id },
      include: {
        versions: PUBLISHED,
        _count: { select: { versions: { where: { status: 'IN_REVIEW' } } } },
      },
    });
    if (!row) throw new DomainError('NOT_FOUND', 'Substance not found.');
    // A duplicate merged into another substance opens that one (favorites, links, history keep
    // working). Merges never chain (ADR-027), so this is one step.
    if (row.mergedIntoId) return this.get(viewer, row.mergedIntoId);
    const published = row.versions[0];
    if (!row.enabled || !published) {
      throw new DomainError('CONTENT_UNAVAILABLE', 'This profile is no longer available.', {
        lockReason: 'NO_LONGER_AVAILABLE',
        unlockPath: null,
      });
    }
    const [item] = await this.withAccess(viewer, [{ ...row, published }]);
    const open = item!.access.state === 'AVAILABLE';
    const content = open ? profileContentSchema.parse(published.content) : null;
    const alternatives = content
      ? (await this.nameIndex(alternativeIds(content)))(alternativeIds(content), row.id)
      : [];
    return {
      ...item!,
      content,
      alternatives,
      sources: open ? published.sources.map(({ source }) => sourceView(source)) : [],
      reviewedAt: published.reviewedAt!,
      reviewerName: published.reviewedBy!.fullName,
      underReview: row._count.versions > 0,
    };
  }
```

7.5 Search

Global search (`GET /v1/search?q=…`) runs over enabled reference models, published calculators and listed substances, with the viewer's access on each hit (locked results stay in place). Ranking is in `ranking.ts`: text is normalised (lower case, no accents), each query word may have typos (none up to 2 letters, 1 up to 5, 2 above, Damerau-Levenshtein), exact and prefix matches score higher, and a query naming a category (« certificat… ») puts that group first. Nothing is ever boosted commercially.

apps/api/src/modules/search/ranking.ts

```
/** The entry's score for `query`, or null when it does not match. */
export function score(query: string, entry: SearchEntry): number | null {
```

```
  const q = normalize(query);
  const qWords = q ? q.split(' ') : [];
  if (qWords.length === 0) return null;

  const titles = entry.titles.map(normalize).filter(Boolean);
  const synonyms = entry.synonyms.map(normalize).filter(Boolean);
  const titleWords = titles.flatMap((title) => title.split(' '));
  const synonymWords = synonyms.flatMap((synonym) => synonym.split(' '));

  if (titles.includes(q)) return SCORE.exact;
  if (titles.some((title) => title.startsWith(q))) return SCORE.prefix;
  if (synonyms.includes(q)) return SCORE.synonymExact;
  /** True when every query word passes the test against some word of the pool. */
  const everyWord = (pool: string[], test: (word: string, candidate: string) => boolean) =>
    qWords.every((word) => pool.some((candidate) => test(word, candidate)));
  if (titles.some((title) => everyWord(title.split(' '), (w, c) => c.startsWith(w)))) {
    return SCORE.wordPrefix;
  }
  if (titles.some((title) => qWords.every((word) => title.includes(word)))) return SCORE.contains;
  if (synonyms.some((synonym) => qWords.every((word) => synonym.includes(word)))) {
    return SCORE.synonymPartial;
  }

  const pool = [...titleWords, ...synonymWords];
  let distance = 0;
  for (const word of qWords) {
    const d = closest(word, pool);
    if (d > allowedTypos(word)) {
      distance = -1;
      break;
    }
    distance += d;
  }
  if (distance >= 0) return SCORE.fuzzy - 10 * distance;

  const purposes = entry.purposes.map(normalize).filter(Boolean);
  if (purposes.some((purpose) => qWords.every((word) => purpose.includes(word)))) {
    return SCORE.purpose;
  }
  return null;
}
```

7.6 Favorites and history

apps/api/src/modules/activity/activity.service.ts

```
/**
 * Favorites and the person's own research history (Profile › Activité). The history is private:
 * admins only ever see counts (section 16). Calculator inputs are never recorded (Lock L42).
 */
@Injectable()
export class ActivityService {
  constructor(
    private readonly prisma: PrismaService,
    private readonly references: ReferenceDocumentsService,
    private readonly calculators: CalculatorsService,
    private readonly pregnancy: PregnancyService,
    private readonly access: AccessService,
  ) {}

  /** Account deletion: the person's favorites and research history go with the account. */
```

apps/api/src/modules/activity/activity.service.ts

```
  /** Stores entries recorded on the device (synced in batches, possibly after being offline). */
  async record(
```

```
    viewer: Viewer & { userId: string },
    deviceId: string,
    entries: ActivityInput[],
  ): Promise<void> {
    if (entries.length === 0) return;
    // A restricted account's history is read-only: the device's entries are accepted (so its
    // upload queue empties) but not kept.
    if (!writable(await this.access.profileFor(viewer), 'activity.history')) return;
    const { userId } = viewer;
    const now = Date.now();
    const data: Prisma.RecentActivityCreateManyInput[] = entries.map((entry) => {
      const occurredAt = entry.occurredAt.getTime() > now ? new Date(now) : entry.occurredAt;
      switch (entry.kind) {
        case 'SEARCH':
          if (!entry.queryText?.trim()) {
            throw new DomainError('VALIDATION_FAILED', 'A search entry needs its query.', {
              fields: { queryText: ['Required for SEARCH.'] },
            });
          }
          return {
            userId,
            deviceId,
            kind: 'SEARCH',
            queryText: entry.queryText.trim().slice(0, 200),
            occurredAt,
          };
        case 'REFERENCE_VIEW':
        case 'CALCULATOR_USE':
        case 'PREGNANCY_VIEW': {
          const targetType = TARGETS[entry.kind];
          if (!entry.targetId || (entry.targetType && entry.targetType !== targetType)) {
            throw new DomainError('VALIDATION_FAILED', 'This entry needs its target.', {
              fields: { targetId: [`Required, of type ${targetType}.`] },
            });
          }
          return {
            userId,
            deviceId,
            kind: entry.kind,
            targetType,
            targetId: entry.targetId,
            occurredAt,
          };
        }
      }
    });
    await this.prisma.recentActivity.createMany({ data });
    await this.trim(userId);
  }
```

The phone records history entries locally and sends them in batches. A restricted (read-only) account's entries are dropped silently on purpose — the phone's queue must never get stuck on a refusal.

7.7 The offline manifest

apps/api/src/modules/offline/offline.service.ts

```
  /**
   * Builds the manifest: entitled PDFs, calculators and pregnancy profiles, and a fingerprint that
   * changes whenever any of them changes.
   */
  async manifest(viewer: Viewer): Promise<OfflineManifest> {
    const [referenceItems, calculatorItems, substances] = await Promise.all([
      this.references.openable(viewer),
      this.calculators.openable(viewer),
```

```
      this.pregnancy.openable(viewer),
    ]);
    const referenceDocuments = referenceItems.map((item): ManifestReference => ({
      id: item.id,
      type: item.type,
      titleFr: item.titleFr,
      titleEn: item.titleEn,
      fileVersion: item.fileVersion,
      sha256: item.sha256,
      sizeBytes: item.sizeBytes,
      pageCount: item.pageCount,
      updatedAt: item.fileDate,
    }));
    const calculators = calculatorItems.map((item): ManifestCalculator => ({
      id: item.id,
      slug: item.slug,
      nameFr: item.nameFr,
      nameEn: item.nameEn,
      category: item.category,
      published: item.published,
      testCases: item.testCases,
    }));
    const fingerprint = [
      ...referenceDocuments.map((entry) => `ref:${entry.id}:${entry.fileVersion}:${entry.sha256}`),
      ...calculators.map(
        (entry) =>
          `calc:${entry.id}:${entry.published.version}:${entry.published.definitionSha256}`,
      ),
      ...substances.map((entry) => `sub:${entry.id}:${entry.version}`),
    ]
      .sort()
      .join('\n');
    return {
      generatedAt: new Date(),
      dataVersion: createHash('sha256').update(fingerprint).digest('hex').slice(0, 16),
      referenceDocuments,
      calculators,
      substances,
    };
  }
```

The manifest's `dataVersion` is a fingerprint of everything in it: the phone compares it with the version it stored to know whether anything changed, and shows it in « Données et hors ligne ».

8. Verification, payments, finance, analytics, audit, admin security

These modules turn a sign-up into a paying professional and give the team the tools to run the service: checking diplomas, taking payments, counting money, measuring use without spying, keeping an unchangeable log of every admin action, and protecting the console itself.

8.1 Verification

A person proves two things, in separate **cases**: their **identity** (national ID, passport, or student card for students) and their **status** — enrolment for a student (`STUDENT_STATUS`), residency or the Ordre des médecins for a resident or doctor (`PROFESSIONAL_STATUS`).

```
PREPARING ──submit──► SUBMITTED ──claim──► IN_REVIEW ──approve──► VERIFIED ──(expiry)──► EXPIRED
     ▲                                          │                    renew: a new case from 60 days
     └──────── start again ◄──── reject (reason)┘
                    after 2 rejections: APPEALED → decided by a DIFFERENT reviewer
```

apps/api/src/modules/verification/verification.service.ts

```ts
/**
 * The person's side of verification (section 06): one case per type, prepared step by step,
 * submitted once, then decided by a reviewer. Identity and status are separate cases.
 */
@Injectable()
export class VerificationService {
  constructor(
    private readonly prisma: PrismaService,
    private readonly users: UsersService,
    private readonly intents: UploadIntentsService,
    private readonly files: VerificationFilesService,
    private readonly audit: AuditService,
    private readonly limiter: RateLimiter,
  ) {}
```

apps/api/src/modules/verification/verification.policy.ts

```ts
/** Service level of the review queue: a case waiting longer is flagged (Admin Web « SLA 24 h »). */
export const VERIFICATION_SLA_HOURS = 24;

/** Documents and declared details are deleted this long after the decision (section 06). */
export const DOCUMENT_RETENTION_MONTHS = 24;
/** Reminders before a verification expires (section 06). */
export const EXPIRY_REMINDER_DAYS = [60, 14] as const;
/** A new case may replace a verified one from this many days before it expires. */
export const RENEWAL_WINDOW_DAYS = 60;
/** Cases never submitted are deleted, with their files, after this many days without activity. */
export const ABANDONED_CASE_DAYS = 90;
/** An appeal is possible after this many rejections of the same type (section 06). */
export const REJECTIONS_BEFORE_APPEAL = 2;

/** Reasons a reviewer can choose when rejecting. */
export const REJECTION_REASONS = [
  'ILLEGIBLE',
  'DOCUMENT_EXPIRED',
  'NAME_MISMATCH',
  'WRONG_DOCUMENT',
  'INFORMATION_MISMATCH',
```

On the reviewer's side (`VerificationReviewService`), a reviewer **claims** a case (so two people never decide the same one), views documents in a **restricted viewer** (60-second signed URLs, a watermark with the reviewer's name, every view audited), then approves or rejects with a reason code. Approval copies the proven role and details to the profile and schedules the documents' deletion: files are kept only as long as the retention rule says.

apps/api/src/modules/verification/verification-review.service.ts

```
  /**
   * Approves the case: VERIFIED (with an optional expiry), the proved role and details are copied
   * to the profile, the person is notified, the documents get their deletion date.
   */
  async approve(
    admin: AdminPrincipal,
    caseId: string,
    input: { reason: string; expiresAt?: Date },
    client: ClientInfo,
  ): Promise<ReviewRecord> {
    const record = await this.claimedBy(admin, caseId);
    const now = new Date();
    if (input.expiresAt && input.expiresAt <= now) {
      throw new DomainError('VALIDATION_FAILED', 'The expiry must be in the future.', {
        fields: { expiresAt: ['Must be in the future.'] },
      });
    }
    const target = caseTarget(record);
    const check = checkDetails(record.type, target, detailsOf(record));
    if (!check.ok) throw new DomainError('INVALID_STATE', 'The declared details are incomplete.');

    await this.prisma.$transaction(async (tx) => {
      await this.decide(tx, admin, caseId, now, {
        status: 'VERIFIED',
        expiresAt: input.expiresAt ?? null,
        rejectionReasonCode: null,
        rejectionNote: null,
      });
    });
    const details = check.details;
    if ('professionalType' in details) {
      await this.users.applyVerifiedStatus(record.userId, {
        professionalType: details.professionalType,
        specialtyId: 'specialtyId' in details ? details.specialtyId : undefined,
        institutionId: 'institutionId' in details ? details.institutionId : undefined,
        universityId: 'universityId' in details ? details.universityId : undefined,
        academicYear: 'academicYear' in details ? details.academicYear : undefined,
        residencyYear: 'residencyYear' in details ? details.residencyYear : undefined,
        orderNumberMasked:
          'orderNumber' in details ? maskOrderNumber(details.orderNumber) : undefined,
      });
    }
    await this.notifications.notify({
      userId: record.userId,
      type: 'verification.decided',
      messageKey: 'mobile.notifications.verificationApproved',
      params: { caseId, caseType: record.type },
    });
    await this.record(
      admin,
      client,
      record.appealedAt ? 'verification.appeal_approved' : 'verification.approved',
      caseId,
      { status: 'IN_REVIEW' },
      { status: 'VERIFIED', expiresAt: input.expiresAt?.toISOString() ?? null },
      input.reason,
    );
```

```
    return this.detail(caseId);
  }
```

8.2 Billing: plans, checkout, gateways

Plans are versioned: the Super Admin publishes a plan version with its price, duration and « inclus / non inclus » lines; a price change is a new version, and running subscriptions keep theirs. **Checkout** is open only to an active, professionally verified doctor or resident (students never pay; nobody pays before verification — ADR-006 Q6).

apps/api/src/modules/billing/checkout.service.ts

```ts
  /**
   * Who may pay: an active, verified doctor or resident; students never pay; an active
   * subscription may renew only in its last 14 days.
   */
  async eligibility(userId: string, now = new Date()): Promise<Eligibility> {
    const facts = await this.users.getAccessFacts(userId);
    if (!facts || facts.status !== 'ACTIVE') return { state: 'ACCOUNT_SUSPENDED' };
    if (facts.professionalType === null) return { state: 'ROLE_REQUIRED' };
    if (facts.professionalType === 'STUDENT') return { state: 'STUDENT_FREE' };
    // A renewal under review keeps a still-valid verification counting (section 27).
    const { status, renewalPending } = (await this.verification.accessSnapshot(userId, now))
      .PROFESSIONAL_STATUS;
    if (
      status !== 'VERIFIED' &&
      (renewalPending || status === 'SUBMITTED' || status === 'IN_REVIEW' || status === 'APPEALED')
    ) {
      return { state: 'VERIFICATION_PENDING' };
    }
    if (status !== 'VERIFIED') return { state: 'VERIFICATION_REQUIRED' };
    const live = await this.liveSubscription(userId, now);
    if (!live) return { state: 'OPEN', renewal: false };
    const renewableFrom = addDays(live.currentPeriodEnd!, -RENEWAL_WINDOW_DAYS);
    if (live.status === 'ACTIVE' && now < renewableFrom) return { state: 'ACTIVE', renewableFrom };
    return { state: 'OPEN', renewal: true };
  }
```

```
App « Payer » (plan, CIB or Edahabia, acknowledgement « non remboursable »)
  POST /v1/subscription/checkout ─► CheckoutService.start
                                      eligibility, plan on sale, method available
                                      transaction DT-2026-000123 (PENDING)
                                      gateway.register(order) ──────────────► Chargily / SATIM
                    ◄─ { redirectUrl } ──────────────────────────────────────── payment page
  browser: the person pays ON THE GATEWAY'S PAGE (the API never sees card data)
  return URL / webhook / reconcile job (every 10 min) ─► settle(transaction)
                                      gateway.status(reference)  ← the only truth
                                      PAID → activate: subscription starts or extends
  PaymentResultScreen polls GET /v1/subscription/transactions/:transactionId until final
```

apps/api/src/modules/billing/payments/payment-provider.ts

```ts
/**
 * A card-payment gateway (ADR-015, ADR-021). The API never sees card data: the person pays on the
 * gateway's page and the API asks the gateway, server to server, what happened. Adding a gateway
 * is one class extending this one, its channels in the `PaymentProvider` enum, and its settings in
 * `config/env.ts`; nothing else changes (checkout, finance, the app).
 */
export abstract class PaymentGateway {
  abstract readonly id: GatewayId;

  /** The channel recorded on a transaction for each method this gateway accepts. */
  abstract readonly channels: Readonly<Partial<Record<PaymentMethod, PaymentProvider>>>;
```

```

  /** Creates the order at the gateway and returns its payment page. */
  abstract register(order: RegisterOrder): Promise<RegisteredOrder>;

  /** The gateway's authoritative status of an order. */
  abstract status(providerReference: string, language: 'FR' | 'EN'): Promise<OrderOutcome>;

  /**
   * Checks a notification's signature against the raw body. Returns the order it is about, or
   * null when it is not authentic (or the gateway sends none). The order is then confirmed with
   * {@link status}, never from the notification's content.
   */
  verifyWebhook(_rawBody: Buffer, _headers: IncomingHttpHeaders): WebhookNotice | null {
    return null;
  }
}
```

Each gateway (`ChargilyGateway`, `SatimGateway`, and `FakeGateway` outside production) implements this class. A webhook is accepted only with a valid signature computed over the raw body, and even then the API asks the gateway for the order's status instead of trusting the notification.

apps/api/src/modules/billing/checkout.service.ts

```
  /**
   * Asks the provider for the outcome of a pending transaction and applies it: activation or
   * extension on payment, failure with the provider's message, or cancellation when abandoned.
   */
  async settle(transaction: Transaction, now = new Date()): Promise<Transaction> {
    if (transaction.status !== 'PENDING' || !transaction.providerReference) {
      if (transaction.status === 'PENDING' && this.abandoned(transaction, now)) {
        return this.finish(transaction, 'CANCELED', {});
      }
      return transaction;
    }
    const gateway = this.gateways.forChannel(transaction.paymentProvider);
    if (!gateway) {
      // Its gateway was removed from the configuration: it can no longer be confirmed here.
      this.logger.warn(
        `${transaction.transactionId}: no configured gateway records
${transaction.paymentProvider}`,
      );
      return this.abandoned(transaction, now)
        ? this.finish(transaction, 'CANCELED', {})
        : transaction;
    }
    let outcome;
    try {
      outcome = await gateway.status(
        transaction.providerReference,
        metadataOf(transaction).language === 'EN' ? 'EN' : 'FR',
      );
    } catch (error) {
      this.logger.warn(`Could not confirm ${transaction.transactionId}: ${String(error)}`);
      return transaction;
    }
    switch (outcome.status) {
      case 'PENDING':
        return this.abandoned(transaction, now)
          ? this.finish(transaction, 'CANCELED', {})
          : transaction;
      case 'FAILED':
        return this.finish(transaction, 'FAILED', {
          providerMessage: outcome.providerMessage,
          provider: outcome.details,
        });
      case 'PAID':
```

```
        return this.activate(transaction, outcome);
      }
    }
```

No refund anywhere. There is no refund status, endpoint or button. A real exception (law, accounting, provider error) is an *exceptional adjustment* in Finance, requested by the Super Admin and counted only once a second admin approves it.

apps/api/src/modules/billing/subscription-lifecycle.service.ts

```ts
/**
 * Manual renewal in V1 (section 06): reminders at 14 and 3 days, 7 days of grace with access kept,
 * then « Terminé ». Calm by design: notifications, never banners, countdowns or red (Lock L39).
 */
@Injectable()
export class SubscriptionLifecycleService {
  constructor(
    private readonly prisma: PrismaService,
    private readonly checkout: CheckoutService,
    private readonly notifications: NotificationsService,
  ) {}

  /** Moves ended periods to grace, ended grace to expired, stopped renewals to expired. */
  async advance(now = new Date()): Promise<void> {
    const ended = await this.prisma.subscription.findMany({
      where: { status: 'ACTIVE', currentPeriodEnd: { lte: now } },
      select: { id: true, userId: true, currentPeriodEnd: true },
      take: BATCH,
    });
    for (const subscription of ended) {
      const graceEndsAt = addDays(subscription.currentPeriodEnd!, GRACE_DAYS);
      const { count } = await this.prisma.subscription.updateMany({
        where: { id: subscription.id, status: 'ACTIVE' },
        data: { status: 'GRACE', graceEndsAt },
      });
      if (count === 1) {
        await this.notifications.notify({
          userId: subscription.userId,
          type: 'subscription.grace',
          messageKey: 'mobile.notifications.subscriptionGrace',
          params: { subscriptionId: subscription.id, graceEndsAt: graceEndsAt.toISOString() },
        });
      }
    }

    const expiring = await this.prisma.subscription.findMany({
      where: {
        OR: [
          { status: 'GRACE', graceEndsAt: { lte: now } },
          { status: 'CANCELED', currentPeriodEnd: { lte: now } },
        ],
      },
      select: { id: true, userId: true, status: true },
      take: BATCH,
    });
    for (const subscription of expiring) {
      const { count } = await this.prisma.subscription.updateMany({
        where: { id: subscription.id, status: subscription.status },
        data: { status: 'EXPIRED' },
      });
      if (count === 1) {
        await this.notifications.notify({
          userId: subscription.userId,
          type: 'subscription.ended',
```

```
          messageKey: 'mobile.notifications.subscriptionEnded',
          params: { subscriptionId: subscription.id },
        });
      }
    }
  }

  /** One reminder per threshold before the end of an active period. */
  async remind(now = new Date()): Promise<void> {
    const horizon = new Date(now.getTime() + Math.max(...RENEWAL_REMINDER_DAYS) * DAY_MS);
    const active = await this.prisma.subscription.findMany({
      where: { status: 'ACTIVE', currentPeriodEnd: { gt: now, lte: horizon } },
      select: { id: true, userId: true, currentPeriodEnd: true },
      take: BATCH,
    });
    const thresholds = [...RENEWAL_REMINDER_DAYS].sort((a, b) => a - b);
    for (const subscription of active) {
      const days = Math.ceil((subscription.currentPeriodEnd!.getTime() - now.getTime()) / DAY_MS);
      const threshold = thresholds.find((limit) => days <= limit);
      if (threshold === undefined) continue;
      const params = { subscriptionId: subscription.id, reminder: threshold };
      if (await this.notifications.exists(subscription.userId, 'subscription.renewal', params))
        continue;
      await this.notifications.notify({
        userId: subscription.userId,
        type: 'subscription.renewal',
        messageKey: 'mobile.notifications.subscriptionRenewal',
        params: { ...params, endsAt: subscription.currentPeriodEnd!.toISOString() },
      });
    }
  }

  /** Confirms checkouts nobody came back from, and cancels the abandoned ones. */
  async reconcile(now = new Date()): Promise<void> {
    const pending = await this.prisma.transaction.findMany({
      where: {
        status: 'PENDING',
        createdAt: { lt: new Date(now.getTime() - RECONCILE_AFTER_MINUTES * 60_000) },
      },
      take: BATCH,
    });
    for (const transaction of pending) await this.checkout.settle(transaction, now);
  }
}
```

8.3 Finance

Finance reads transactions (it never changes them) and adds what the business needs around them: payment fees (estimated from a rate per gateway × card, later replaced by the provider's statement), operating expenses (voided, never deleted), exceptional adjustments (two admins) and CSV reports. The ledger always has the same shape (Lock L40):

apps/api/src/modules/finance/finance.policy.ts

```
/** Gross → fees → net revenue; net revenue → expenses → net profit (Lock L40). */
export function ledgerOf(input: {
  revenue: readonly Pick<RevenueRow, 'amountMinor' | 'paymentFeesMinor' | 'paymentProvider'>[];
  rates: ReadonlyMap<PaymentProvider, FeeRate>;
  adjustments: readonly { amountMinor: number }[];
  expenses: readonly { amountMinor: number }[];
}): Ledger {
  let payments = 0;
  let fees = 0;
  let estimated = 0;
  for (const row of input.revenue) {
```

```
    payments += row.amountMinor;
    const fee = feeOf(row, input.rates);
    fees += fee.feesMinor;
    if (fee.estimated) estimated += fee.feesMinor;
  }
  const adjustments = input.adjustments.reduce((total, row) => total + row.amountMinor, 0);
  const expenses = input.expenses.reduce((total, row) => total + row.amountMinor, 0);
  const gross = payments + adjustments;
  const net = gross - fees;
  return {
    grossMinor: gross,
    adjustmentsMinor: adjustments,
    feesMinor: fees,
    estimatedFeesMinor: estimated,
    netRevenueMinor: net,
    expensesMinor: expenses,
    netProfitMinor: net - expenses,
    payments: input.revenue.length,
  };
}
```

CSV cells go through `csvCell()`, which neutralises text a spreadsheet would run as a formula (`= + - @`) — an audit fix.

8.4 Analytics without spying

apps/api/src/modules/analytics/analytics.service.ts

```
/**
 * Pseudonymous event ingestion and aggregates (section 16, Lock L42). The envelope is computed
 * here from the server's own facts: the app sends only the event, its time and its counts/codes.
 * Admins read aggregates only; no per-person timeline exists.
 */
@Injectable()
export class AnalyticsService {
  constructor(
    private readonly prisma: PrismaService,
    private readonly access: AccessService,
    private readonly users: UsersService,
    private readonly verification: VerificationStatusService,
    private readonly demo: DemoService,
    private readonly references: ReferenceDocumentsService,
    private readonly calculators: CalculatorsService,
    private readonly limiter: RateLimiter,
```

The app sends only event names with counts, codes and flags. The server adds the *envelope* from its own facts: a keyed hash of the person (stable, not reversible), tier, profession, specialty, wilaya. Each event type has a strict schema; anything else is dropped. Dashboards hide any cell behind fewer than 10 people (« < 10 »).

apps/api/src/modules/analytics/analytics.policy.ts

```
/**
 * Payload of each event the app may send: counts, codes and flags only — never free text, query
 * text, calculator inputs or PDF content. Unknown keys are rejected.
 */
export const CLIENT_EVENTS = {
  SEARCH_PERFORMED: {
    entity: false,
    properties: z.strictObject({
      resultCount: count,
      groups: z.array(targetType).max(2),
    }),
```

```
  },
  REFERENCE_SEARCHED: {
    entity: false,
    properties: z.strictObject({ referenceType, resultCount: count }),
  },
  REFERENCE_OPENED: {
    entity: true,
    properties: z.strictObject({ referenceType }),
  },
  REFERENCE_VIEWED: {
    entity: true,
    properties: z.strictObject({ fileVersion: version, source: z.enum(['online', 'offline']) }),
  },
  CALCULATOR_STARTED: {
    entity: true,
    properties: z.strictObject({ version }),
  },
  CALCULATOR_COMPLETED: {
    entity: true,
    properties: z.strictObject({
      version,
      resultBand: bandCode.nullable(),
      engine: z.string().regex(/^(ts|dart)-\d+\.\d+\.\d+$/),
```

8.5 The audit log

apps/api/src/modules/audit/audit.service.ts

```
/** Writes the append-only audit log (the table rejects updates and deletes) and reads it back. */
@Injectable()
export class AuditService {
  constructor(private readonly prisma: PrismaService) {}

  /** Appends one entry. Called inside or right after every admin change. */
  async record(entry: AuditEntry): Promise<void> {
    await this.prisma.auditLogEntry.create({
      data: {
        actorType: entry.actorType,
        actorId: entry.actorId,
        action: entry.action,
        entityType: entry.entityType,
        entityId: entry.entityId,
        before: entry.before,
        after: entry.after,
        reason: entry.reason,
        ip: entry.ip ?? null,
        userAgent: entry.userAgent ?? null,
      },
    });
  }

  /** Newest first, `limit` entries (default one page). */
  list(filter: AuditFilter): Promise<AuditLogEntry[]> {
    const action = filter.action?.trim();
    return this.prisma.auditLogEntry.findMany({
      where: {
        ...(filter.actorType ? { actorType: filter.actorType } : {}),
        ...(filter.actorId ? { actorId: filter.actorId } : {}),
        ...(filter.entityType ? { entityType: filter.entityType } : {}),
        ...(filter.entityId ? { entityId: filter.entityId } : {}),
        ...(action ? { action: action.endsWith('.') ? { startsWith: action } : action } : {}),
        ...(filter.from || filter.to
          ? {
              occurredAt: {
                ...(filter.from ? { gte: filter.from } : {}),
                ...(filter.to ? { lt: filter.to } : {}),
```

```
              },
            }
          : {}),
        ...(filter.before !== undefined ? { id: { lt: filter.before } } : {}),
      },
      orderBy: { id: 'desc' },
      take: filter.limit ?? AUDIT_PAGE,
    });
  }

  /** The history of one record (drawer « Historique » tabs), newest first. */
  history(entityType: string, entityId: string, limit = 100): Promise<AuditLogEntry[]> {
    return this.list({ entityType, entityId, limit });
  }
}
```

Every admin change calls `audit.record({...})` with who, what, which record, before/after and the reason the admin typed. The database table itself refuses UPDATE and DELETE, so even a bug cannot rewrite history.

8.6 Admin security: password, TOTP, grants

```
/sign-in  email + password ─► AdminAuthService.signIn   lockout after 5 failures; same timing for unk
          ◄─ 5-minute challenge: ENROL (first time: QR code) or VERIFY
          6-digit TOTP code ──► completeTotp ─► opaque session cookie (idle + absolute timeouts)
every admin request:  AdminBoundaryGuard  (no bearer tokens; writes only from the Admin Web origin)
                      AdminSessionGuard   (live session + the route's @RequireGrants)
```

apps/api/src/modules/admin-auth/grants.ts

```
/** Every permission an admin can hold; routes require them with @RequireGrants. */
export const GRANTS = [
  'users.read',
  'users.write',
  'users.suspend',
  'verification.queue.read',
  'verification.review',
  'verification.documents.view',
  'verification.appeal.decide',
  'library.read',
  'library.write',
  'library.disable',
  'calculators.read',
  'calculators.draft',
  'calculators.review',
  'calculators.publish',
  'calculators.retire',
  'pregnancy.read',
  'pregnancy.draft',
  'pregnancy.review',
  'subscriptions.read',
  'subscriptions.plans.write',
  'transactions.read',
  'transactions.adjustment.request',
  'transactions.adjustment.approve',
  'finance.read',
  'finance.expenses.write',
  'finance.reports.export',
  'analytics.read',
  'analytics.export',
  'audit.read',
  'audit.export',
  'system.roles.write',
  'system.settings.write',
```

```
  'system.notifications.send',
  'system.demo.configure',
] as const;
```

apps/api/src/modules/admin-auth/grants.ts

```
/**
 * Separation of duties for sub-admins (section 03): no account holds both finance grants and
 * clinical content grants, and only the Super Admin may approve exceptional adjustments.
 * Returns the violated rules (empty = allowed).
 */
export function separationOfDutiesViolations(
  grants: readonly Grant[],
  isSuperAdmin: boolean,
): string[] {
  if (isSuperAdmin) return [];
  const violations: string[] = [];
  const has = (prefix: string) => grants.some((grant) => grant.startsWith(prefix));
  if (has('finance.') && (has('calculators.') || has('library.') || has('pregnancy.'))) {
    violations.push('finance grants cannot be combined with clinical content grants');
  }
  if (grants.includes('transactions.adjustment.approve')) {
    violations.push('only the Super Admin approves exceptional adjustments');
  }
  return violations;
}
```

Grants are given per admin, from **presets** (Super Admin, Verification reviewer, Library manager, Calculator editor, Clinical reviewer…) adjustable one by one; a grant can also be temporary (a few hours). Separation of duties is checked both in Admin Web (live, before saving) and by the API.

8.7 The admin module

`modules/admin` holds read-only screens that combine several modules: « Vue d'ensemble » (the ledger strip and work lists), « Utilisateurs » (search and the user drawer: profile, access with its inputs, subscriptions, activity *counts*), and the audit log with its CSV export.

9. How Admin Web is built — Next.js, React and the console kit

Admin Web is a Next.js application written in React and TypeScript. It has no database and no business rule of its own: every screen reads and writes through the API, and every button it hides is also refused by the API. Its job is to make the team's work fast, clear and safe.

9.1 The layers

```
src/app/                    URLs. A folder = a route; page.tsx = the page. Thin: grant check + featu
  admin/layout.tsx          every console page inside ConsoleShell (top bar + navigation)
  admin/users/page.tsx      /admin/users  →  <RequireGrant grant="users.read"><UsersPage/>
  sign-in/                  the two-step sign-in (password, then authenticator code)
src/features/<section>/     the screens: pages, drawers, dialogs, labels, small helpers
src/ui/                     the console kit: Button, Badge, DataTable, Dialog, Drawer, Field, Tabs,
src/api/client.ts           typed calls: api.GET('/admin/v1/users', …) checked against schema.d.ts
src/session/session.ts      who is signed in (useMe) and what they may do (useGrants)
src/i18n/                   FR/EN messages from packages/i18n, formatting of dates, money, bytes
src/shell/                  ConsoleShell, the navigation list, RequireGrant
src/proxy.ts                no session cookie → straight to /sign-in
```

9.2 One origin: the API behind Admin Web

The browser never calls the API's domain. Next.js forwards `/admin/v1/*` to the API, so the session cookie (`__Host-doktool_admin`, HTTP-only) belongs to Admin Web's own origin and every write carries Admin Web's `Origin` header — which the API's boundary guard checks. A strict Content-Security-Policy allows scripts only from the console itself.

apps/admin/next.config.ts

```
38    async rewrites() {
39      return {
40        beforeFiles: [{ source: '/admin/v1/:path*', destination: `${apiOrigin}/admin/v1/:path*` }],
41        afterFiles: [],
42        fallback: [],
43      };
```

apps/admin/src/proxy.ts

```
1   import { type NextRequest, NextResponse } from 'next/server';
2
3   // Kept in sync with session/session.ts (the proxy cannot import client modules).
4   const SESSION_COOKIE = '__Host-doktool_admin';
5
6   /**
7    * Console pages need an admin session cookie; without one the browser goes to sign-in first.
8    * The API still validates the session on every call — this only avoids rendering a shell.
9    */
10  export function proxy(request: NextRequest) {
11    if (request.cookies.has(SESSION_COOKIE)) return NextResponse.next();
12    const url = request.nextUrl.clone();
13    url.pathname = '/sign-in';
14    url.search = `?next=${encodeURIComponent(request.nextUrl.pathname + request.nextUrl.search)}`;
15    return NextResponse.redirect(url);
16  }
17
18  /** Which URLs the proxy runs on: every console page, never the API. */
```

```ts
export const config = {
  // `/admin/v1/*` is the proxied API, never a page.
  matcher: ['/admin', '/admin/((?!v1(?:/|$)).*)'],
};
```

9.3 Calling the API

`openapi-fetch` is typed by `schema.d.ts`, generated from the API contract: a wrong path, a missing parameter or a misspelled field is a compile error. `call()` turns any failure into an `ApiFailure` carrying the API's error code:

apps/admin/src/api/client.ts

```ts
/** Awaits an openapi-fetch call and returns its data, or throws an {@link ApiFailure}. */
export async function call<T>(request: Promise<ApiResult<T>>): Promise<T> {
  let result: ApiResult<T>;
  try {
    result = await request;
  } catch (cause) {
    throw new ApiFailure(0, 'NETWORK', cause instanceof Error ? cause.message : 'Network error');
  }
  if (!result.response.ok) {
    const body = result.error;
    if (isErrorResponse(body)) {
      throw new ApiFailure(
        result.response.status,
        body.error.code,
        body.error.message,
        body.error.details ?? null,
      );
    }
    throw new ApiFailure(result.response.status, 'UNKNOWN', result.response.statusText);
  }
  return result.data as T;
}
```

apps/admin/src/api/client.ts

```ts
/** Fields rejected by the server, keyed by DTO property (VALIDATION_FAILED). */
export function fieldErrors(error: unknown): Record<string, string[]> {
  if (!(error instanceof ApiFailure) || error.code !== 'VALIDATION_FAILED') return {};
  return error.details?.fields ?? {};
}
```

Errors are shown with `useErrorText()` (`src/ui/errors.ts`), which maps codes to catalogue sentences — never the server's developer message. If a call answers "session ended", the provider reloads the sign-in page and remembers where the admin was.

9.4 Server state with TanStack Query

Every list and detail is a `useQuery` (or `useInfiniteQuery` for paged lists) with a *query key*. Every change is a `useMutation`; on success it invalidates or updates the keys it affects, so every screen showing that data refreshes. Read the users page's list as the model for all lists:

apps/admin/src/features/users/users-page.tsx

```ts
  // Pages of users for the current search and filters; the next page starts after the last id.
  const users = useInfiniteQuery({
    queryKey: [...usersKey, deferredSearch, type, status],
    initialPageParam: undefined as string | undefined,
    queryFn: ({ pageParam }) =>
      call(
```

```
        api.GET('/admin/v1/users', {
          params: {
            query: {
              search: deferredSearch || undefined,
              professionalType: type || undefined,
              status: status || undefined,
              cursor: pageParam,
            },
          },
        }),
      ),
    getNextPageParam: (last) => (last.length === PAGE_SIZE ? last.at(-1)?.id : undefined),
  });
  const rows = useMemo(() => users.data?.pages.flat() ?? [], [users.data]);

  // The table's columns (rebuilt when the language changes).
  const columns = useMemo<Column<User>[]>(
```

`useDebounced` waits 300 ms after the last key before asking the server (an audit fix: it used to ask at every key).

apps/admin/src/ui/use-debounced.ts

```ts
'use client';

import { useEffect, useState } from 'react';

/**
 * `value`, once it has stopped changing for `delayMs`: a search box asks the server when the
 * person pauses typing, not at every key.
 */
export function useDebounced<T>(value: T, delayMs = 300): T {
  const [settled, setSettled] = useState(value);
  useEffect(() => {
    const timer = window.setTimeout(() => setSettled(value), delayMs);
    return () => window.clearTimeout(timer);
  }, [value, delayMs]);
  return settled;
}
```

9.5 Who sees what: grants in the UI

apps/admin/src/session/session.ts

```ts
/** Effective grants of the signed-in admin; the navigation and every action read them. */
export function useGrants(): { has: (...grants: Grant[]) => boolean; me: AdminMe | undefined } {
  const me = useMe().data;
  return {
    me,
    has: (...grants) => me !== undefined && grants.every((grant) => me.grants.includes(grant)),
  };
}
```

- The navigation (`shell/nav.ts`) lists each section with the grant it needs; sections the admin lacks are not shown.
- A page wraps its content in `<RequireGrant>`: typing the URL without the grant shows a clear message instead of failing API calls.
- Buttons check `has('…')`. This is only comfort: the API refuses anyway.

apps/admin/src/app/admin/users/page.tsx

```tsx
'use client';

import { UsersPage } from '@/features/users/users-page';
import { RequireGrant } from '@/shell/require-grant';

/** /admin/users, for admins with users.read. */
export default function Page() {
  return (
    <RequireGrant grant="users.read">
      <UsersPage />
    </RequireGrant>
  );
}
```

9.6 The console kit (src/ui)

Component	Role
`Button`	Variants primary · secondary · ghost · danger · teal; `busy` shows a spinner and blocks double clicks.
`DataTable`	Sortable table with keyboard navigation; a row can open a drawer. Clicks on controls inside a row no longer also open the row (audit fix A2).
`Drawer` / `Dialog`	Side panel for a record's detail; centred dialog for a decision. Both trap focus and close with Escape.
`ReasonDialog`	Every admin change asks for a reason (3–500 characters) that goes into the audit log.
`Field`, `TextInput`, `Select`, `Switch`	Labelled, accessible form controls; errors under the field with an icon.
`Badge`, `Alert`, `EmptyState`, `Skeleton`, `PageHeader`	Status words with colour (never colour alone), messages, empty and loading states.
`PeriodPicker`, charts, `MetricTile`	Finance and analytics periods, bar/line charts with a table fallback, ledger figures.
`PdfPages`	Draws a PDF on canvases from a signed URL — preview without download or print.

apps/admin/src/ui/data-table.tsx

```tsx
/**
 * True when an event on a row started on a control inside it (a « Traiter » button, a link): the
 * control acts alone and the row does not open.
 */
function fromControl(event: { target: EventTarget; currentTarget: EventTarget }): boolean {
  return (
    event.target instanceof Element &&
    event.target !== event.currentTarget &&
    event.target.closest('button, a, input, select, textarea, [role="menuitem"]') !== null
  );
}
```

9.7 Forms

Forms use React Hook Form with a Zod schema holding the same limits as the API (lengths, patterns). The API stays the judge: its `VALIDATION_FAILED` field errors are put back under the matching fields with

`fieldErrors()`.

9.8 Signing in

`app/sign-in/sign-in-form.tsx` has two steps. The password step answers with a challenge: **ENROL** the first time (a QR code drawn from the `otpauth://` link, to scan with an authenticator app) or **VERIFY** afterwards. The six-digit code then creates the session cookie. If the 5-minute challenge expires or locks, the form now says why before starting again (audit fix A5).

9.9 Languages and formatting

Messages come from the same catalogue as the app (`packages/i18n`), under `admin.*` keys: `t('admin.users.title')`. A key that does not exist is a compile error (`MessageKey` type). `src/i18n/format.ts` formats dates, numbers, money (DZD), file sizes and ages in the chosen language.

10. Admin Web, screen by screen

Each section of the console has a folder in `apps/admin/src/features/`, a route in `src/app/admin/`, a grant that opens it and an API controller behind it. For each screen: what it is for, every button, and which API call it makes. Appendix A lists every one of these buttons again with its line number.

Section	URL	Grant	Feature folder	API
Vue d'ensemble	/admin/overview	— (blocks follow grants)	overview	/admin/v1/overview
Utilisateurs	/admin/users	users.read	users	/admin/v1/users
Vérifications	/admin/verification	verification.queue.read	verification	/admin/v1/verification
Récupération	/admin/recovery	verification.queue.read	recovery	/admin/v1/account-recoveries
Bibliothèque	/admin/library/…	library.read	library	/admin/v1/reference-documents
Calculateurs	/admin/calculators	calculators.read	calculators	/admin/v1/calculators
Grossesse · Allaitement	/admin/pregnancy	pregnancy.read	pregnancy	/admin/v1/pregnancy
Abonnements	/admin/subscriptions	subscriptions.read	subscriptions	/admin/v1/plans, …/subscriptions
Transactions	/admin/transactions	transactions.read	transactions	/admin/v1/transactions
Finance	/admin/finance	finance.read	finance	/admin/v1/finance
Analytique	/admin/analytics	analytics.read	analytics	/admin/v1/analytics
Journal d'audit	/admin/audit	audit.read	audit	/admin/v1/audit
Permissions	/admin/permissions	system.roles.write	permissions	/admin/v1/admins
Paramètres	/admin/settings	system.settings.write	settings	/admin/v1/reference-data, /admin/v1/clinical-review/policies

10.1 Vue d'ensemble

One ledger strip (net revenue of 30 days, active people…) and three work lists — cases waiting for review, content waiting for review, payment incidents — each shown only to admins whose grants cover it. No wall of numbers: it answers "what needs me today?". A failed file in the library links to its own type's list (audit fix A6).

10.2 Utilisateurs

- Search box (debounced), filters by profession and status, « Charger plus » for the next 50.
- A row opens the **user drawer**: « Profil », « Accès » (the AccessProfile and the facts it came from), « Abonnements », « Activité » (counts only — never what someone searched), « Historique » (audit

entries of this account).

- Actions in the drawer menu (with a reason): « Suspendre » / « Réactiver » the account, and sign it out of every device.

apps/admin/src/features/users/user-drawer.tsx

```tsx
/** User drawer: Profil · Accès · Abonnement · Activité · Historique (section 07); no documents. */
export function UserDrawer({ userId, onClose }: { userId: string | null; onClose: () => void }) {
  const { t } = useI18n();
  const errorText = useErrorText();
  const toast = useToast();
  const queryClient = useQueryClient();
  const { has } = useGrants();
  const [action, setAction] = useState<Action>(null);
  const key = ['admin', 'users', 'detail', userId ?? ''] as const;
  const detail = useQuery({
    queryKey: key,
    queryFn: () =>
      call(api.GET('/admin/v1/users/{id}', { params: { path: { id: userId ?? '' } } })),
    enabled: userId !== null,
  });
  const user = detail.data;

  // After an action: reload the drawer and the list, then confirm with a toast.
  const refresh = async (message: string) => {
    await queryClient.invalidateQueries({ queryKey: key });
    void queryClient.invalidateQueries({ queryKey: usersKey });
    toast(message);
  };

  return (
    <Drawer
      open={userId !== null}
      onClose={onClose}
      mode="inspect"
      title={user ? `${user.firstName} ${user.lastName}` : t('admin.users.user')}
      caption={user ? <span className="dt-mono">{user.id}</span> : null}
```

10.3 Vérifications

- Queue tabs « En attente · Vérifiés · Rejetés · Expirés · Appels », type chips, oldest first; the « Soumis » column turns into a warning past the 24-hour SLA (with a word, never colour alone).
- The case drawer: « Prendre le dossier » (claim), « Libérer » (give back), the documents with « Ouvrir » in the **restricted viewer** (60-second URL, watermark with the reviewer's name, printing and copying blocked, every opening audited; « Rouvrir » asks for a new link), then « Approuver » (optional expiry) or « Rejeter » (reason code).
- An appeal can only be decided by a reviewer with `verification.appeal.decide` who did not reject the case.

apps/admin/src/features/verification/restricted-viewer.tsx

```tsx
/**
 * RestrictedViewer: an identity document behind a 60-second signed URL, watermarked with the
 * viewer's name and time, without download, print, copy or drag. Each opening is audited by the
 * API; once the URL expires the document is hidden until it is opened again.
 */
export function RestrictedViewer({
  document,
  label,
  onClose,
}: {
```

```
  document: Document;
  label: string;
  onClose: () => void;
}) {
  const { t, locale } = useI18n();
  const errorText = useErrorText();
  // The signed URL whose 60 seconds are over (a new opening brings a new URL).
  const [expiredUrl, setExpiredUrl] = useState<string | null>(null);

  // Not cached: every opening asks the API again (a fresh audit entry and signed URL).
  const view = useQuery({
    queryKey: ['admin', 'verification', 'document', document.id],
    queryFn: () =>
      call(
        api.POST('/admin/v1/verification/documents/{id}/view', {
          params: { path: { id: document.id } },
        }),
      ),
    gcTime: 0,
    staleTime: Infinity,
    retry: false,
  });

  useEffect(() => {
    if (!view.data) return;
```

10.4 Récupération

Assisted recovery requests by status. The drawer compares the request (lost contact, names, new contact) with the account found, shows the two photos (« Ouvrir », 60 seconds, audited) beside the verified identity, and offers « Approuver » (only when an account matches; applies after 24 h) or « Rejeter » — also during the 24-hour hold (audit fix A4).

10.5 Bibliothèque (ordonnances, certificats)

- Type tabs (each has its own URL), status chips with counts, a filter box, « Ajouter un modèle ».
- **Upload drawer**: drop or choose a PDF (checked in the browser: type, size, « %PDF » header); the French title is pre-filled from the file name; the button label follows the stage — checksum, sending, saving.
- **Model drawer**: edit titles and « Échantillon démo » (« Enregistrer » asks for a reason), « Voir le PDF » (drawn on canvases, no download), « Remplacer le PDF », and the status menu (activate, deactivate, archive, restore). While a file is being processed the drawer refreshes every 3 seconds without erasing what the admin is typing.
- « Synonymes »: extra words search should match for this model.

apps/admin/src/features/library/upload-drawer.tsx

```
  // Checksum → signed upload → PUT the bytes → create the model; the drawer of the new model opens.
  const submit = form.handleSubmit(async (values) => {
    if (!file) return;
    try {
      setStage('hashing');
      const sha256 = await sha256Hex(file);
      const start = await call(
        api.POST('/admin/v1/reference-documents/uploads', {
          body: { type, fileName: file.name, sizeBytes: file.size, sha256 },
        }),
      );
      setStage('uploading');
```

```
      await putSigned(start.upload, file);
      setStage('saving');
      const created = await call(
        api.POST('/admin/v1/reference-documents', {
          body: {
            documentId: start.documentId,
            intentId: start.upload.intentId,
            type,
            titleFr: values.titleFr.trim(),
            titleEn: values.titleEn.trim() || null,
            demoSample: values.demoSample,
            status: values.enable ? 'ENABLED' : 'DISABLED',
            reason: values.reason.trim(),
          },
        }),
      );
      await queryClient.invalidateQueries({ queryKey: libraryKey(type) });
      toast(t('admin.library.created'));
      onCreated(created.id);
```

10.6 Calculateurs

The list shows each calculator's category, access, published and open version, primary source and status. « Nouveau calculateur » creates the calculator (slug, names, purpose, category). The **detail page** shows the version timeline and, for the selected version, five tabs:

Tab	Buttons
Définition	Read-only view of inputs (ranges, units, answers and points), formula written with the inputs' names, bands and limitations; « Modifier la définition » opens the draft's JSON; « Enregistrer la définition » shows the API's problems by path.
Sources	« Ajouter une source » (role, citation, edition, where the formula is stated, DOI or link, date accessed); bin icon to remove. Links must be http(s) (audit fix A1).
Tests	« Ajouter un cas » (where the example comes from, inputs as JSON, expected outcome / value ± tolerance / band); « Lancer les tests » on the server engine.
Revue	The publication gate as a checklist; « Soumettre à revue », « Supprimer le brouillon » (author); « Approuver et publier » (or first/second sign-off) and « Renvoyer à l'auteur » (reviewer, never the author); « Retirer » a published version.
Aperçu	Type values and « Calculer » on the server engine; nothing is stored.

apps/admin/src/features/calculators/review-panel.tsx

```
/**
 * « Revue »: author, reviewer and note; the publication gate; and the lifecycle actions
 * (submit, return, approve and publish, retire, discard). Author ≠ reviewer (section 28).
 */
export function ReviewPanel({
  calculator,
  version,
  onChanged,
}: {
  calculator: Calculator;
  version: Version;
  onChanged: () => Promise<void>;
}) {
  const { t, locale } = useI18n();
  const errorText = useErrorText();
  const toast = useToast();
```

```
  const { has, me } = useGrants();
  const [action, setAction] = useState<Action>(null);
  const run = currentRun(version.testRun, version.definitionSha256);
  const gates = publicationGates(version, readDefinition(version.definition), run, me?.id);
  const authorGatesOk = gates
    .filter((gate) => gate.label !== 'admin.calculators.gateReviewer')
    .every((gate) => gate.ok);
  const iAmAuthor = me?.id === version.authoredBy.id;
  const step = approvalStep(version, me?.id, 'admin.calculators.publish');

  // « Soumettre à revue »: the API checks the gate again.
  const submit = useMutation({
    mutationFn: () =>
      call(
```

10.7 Grossesse · Allaitement

- Three tabs: **Substances** (search, filters by state and by conduct for a period, « Ajouter une substance », « Approuver les N affichées » for bulk review), **Sources** (the registry: « Ajouter une source », « Modifier »), **Signalements** (user reports; « Traiter » closes one with what was done).
- The **substance page**: « Modifier la substance » (names, ATC, synonyms), « Nouvelle version », « Fusionner avec… » / « Annuler la fusion », the version timeline, and the profile editor (periods, conduct, evidence level and population, sources ticked, alternatives, conflicts between sources) with « Enregistrer le brouillon », « Soumettre à relecture », « Approuver et publier », « Demander des modifications ».
- The **import** pages: « Choisir un fichier .xlsx », then a preview with column mapping (« Appliquer »), the meaning of each status word, rows with issues to correct one by one (« Enregistrer la ligne »), « Abandonner l'import » or « Ajouter à la bibliothèque » (writes drafts), and « Soumettre tout à relecture ».

10.8 Abonnements, Transactions, Finance

- **Abonnements** — « Offres »: the versioned plan catalogue; « Nouveau forfait », « Nouvelle version » (a price change), « Mettre en vente » / « Retirer de la vente ». The plan drawer: code, names, price in dinars, duration, includes / not included (one per line), « Publier ». A second tab lists subscriptions.
- **Transactions**: the immutable ledger with search and filters; the drawer shows the payment, its fee and adjustments; menu « Ajustement exceptionnel… » (Super Admin) and « Approuver » by a different admin.
- **Finance**: the period picker; « Vue d'ensemble » (gross → fees → net, then expenses → net profit, subscriptions, revenue by plan, charts), « Dépenses » (« Ajouter une dépense », « Annuler la dépense », fee rates per channel with « Modifier », « Régler les frais » from a provider statement), « Moyens de paiement » (« Suspendre » / « Reprendre » a card network during an outage), « Rapports » (« Télécharger le CSV », audited).

10.9 Analytique and Journal d'audit

- **Analytique**: period and segment filters (profession, specialty, wilaya); tabs Usage, Bibliothèque, Calculateurs, Utilisateurs (sign-up → verification funnel), Abonnements, Démo. Cells behind fewer than 10 people show « < 10 »; a freshness line says when the last event arrived.
- **Journal d'audit**: filters (admin, action group, record, period), newest first, « Charger plus », « Exporter en CSV » (needs `audit.export`; the export itself is logged).

10.10 Permissions and Paramètres

- **Permissions** (Super Admin): « Nouvel administrateur » (e-mail, name, preset) shows a one-time password once — « Copier », then « J'ai transmis le mot de passe ». The admin drawer: grants by area with live separation-of-duties checks, « Enregistrer les permissions », menu « Accès temporaire », « Réinitialiser l'authentificateur », « Suspendre » / « Réactiver ».
- **Paramètres**: the pickers of the app (specialties, institutions, universities by wilaya: « Ajouter… ») and « Double relecture » — whether calculators and pregnancy profiles need one or two sign-offs (refused if fewer than two admins could approve).

11. How the mobile app is built — Flutter, Riverpod, go_router, drift

The app is written in Dart with Flutter, one code base for iOS and Android. Four libraries shape every file: **Riverpod** (state and dependencies), **go_router** (screens and redirects), **dio** with a generated client (the API), and **drift** (a small SQLite database on the phone). This chapter explains each with the app's own code.

11.1 The folders

```
lib/main.dart                 runApp(ProviderScope(DokToolApp))
lib/src/app.dart              MaterialApp.router: themes, languages, router; wraps every screen with
                              AnalyticsHost (screen views) and OfflineSyncHost (sync on resume)
lib/src/core/                 api/ (dio, interceptors, errors, generated client) · auth/ (session, va
                              database/ (drift) · storage/ (Keychain/Keystore) · network/ · config/
lib/src/features/<feature>/   screens + their providers/controllers, one folder per feature
lib/src/ui/                   the widget kit: DtButton, DtAlert, DtBadge, DtTextField, ContentRow, lo
lib/src/theme/                doktool_tokens.dart (generated) + tokens.dart (context.colors…)
lib/src/router/app_router.dart  every route and the redirect rules
lib/l10n/                     ARB files built from packages/i18n → AppLocalizations
```

11.2 Start-up

apps/mobile/lib/main.dart

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'src/app.dart';
import 'src/core/api/provider_retry.dart';

void main() {
  runApp(const ProviderScope(retry: providerRetry, child: DokToolApp()));
}
```

apps/mobile/lib/src/app.dart

```dart
/// Root widget: light and dark themes from the tokens, FR/EN from packages/i18n, both switched
/// live from Settings.
class DokToolApp extends ConsumerWidget {
  const DokToolApp({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final settings =
        ref.watch(settingsControllerProvider).value ??
        AppSettings(
          appearance: AppearancePreference.system,
          language: AppLanguage.fromDevice(ref.watch(deviceLocaleProvider)),
        );
    return MaterialApp.router(
      routerConfig: ref.watch(routerProvider),
      onGenerateTitle: (context) => AppLocalizations.of(context).appName,
      theme: dtTheme(Brightness.light),
      darkTheme: dtTheme(Brightness.dark),
      themeMode: settings.appearance.themeMode,
      locale: settings.language.locale,
      supportedLocales: AppLocalizations.supportedLocales,
```

```
      localizationsDelegates: AppLocalizations.localizationsDelegates,
      debugShowCheckedModeBanner: false,
      builder: (context, child) => AnalyticsHost(child: OfflineSyncHost(child: child ?? const
SizedBox.shrink())),
    );
  }
}
```

Appearance (light, dark, system) and language (FR, EN) come from `SettingsController` and apply instantly. `providerRetry` makes Riverpod retry a failed load a few times — except when there is no network, where the screen says « Connexion requise » at once.

11.3 Riverpod in five minutes

A **provider** is a named piece of state or service. Widgets `ref.watch(provider)` to rebuild when it changes, or `ref.read(provider)` to use it once (in a button handler). The kinds used here:

Kind	Example	Used for
`Provider`	`apiProvider`, `appDatabaseProvider`, `offlineStoreProvider`	Services (one instance, injected).
`FutureProvider`	`wilayasProvider`, `legalProvider`	Load once, show loading / error / data.
`AsyncNotifier`	`SessionController`, `AccessController`, `FavoritesController`, `OfflineSyncController`	State that loads and then changes through methods (`toggle`, `reload`, `signOut`).
`Notifier`	`OnboardingDraftController`, `PendingCheckoutController`	Synchronous state kept in memory.

Providers depend on each other: `AccessController.build` watches the session, so when the session changes (sign-in, sign-out, demo) the access profile reloads by itself:

apps/mobile/lib/src/features/access/access_controller.dart

```
/// The backend's AccessProfile for this viewer (section 27). The app renders locks from it and
/// never re-derives access from the professional type (Lock L34). Refreshed on launch, on resume
/// and after verification or purchase events.
class AccessController extends AsyncNotifier<AccessProfileDto?> {
  /// Fetches GET /v1/me/access for the current session (null when signed out).
  @override
  Future<AccessProfileDto?> build() async {
    final session = ref.watch(sessionControllerProvider).value;
    if (session == null || session is SignedOut) return null;
    return ref.read(apiProvider).access.accessControllerMe();
  }

  /// Fetches the profile again (after verification, purchase, resume).
  Future<void> reload() async {
    ref.invalidateSelf();
    await future;
  }
}
```

In a screen, the three states are drawn with `.when(...)` or by reading `.value`, `.isLoading`, `.hasError`. Loading lists show skeleton rows, never a spinner (design section 22).

11.4 The session

Who is using the phone is a **sealed class** with three cases — the compiler forces every `switch` to handle all of them:

apps/mobile/lib/src/core/auth/session.dart

```dart
/// Who is using the app on this device (ADR-009, ADR-010).
sealed class Session {
  const Session();
}

/// Nobody signed in: Welcome, sign-in, sign-up and the demo entry only.
final class SignedOut extends Session {
  const SignedOut();
}

/// A real account. Its refresh token lives in the Keychain / Keystore.
final class UserSession extends Session {
  const UserSession({required this.sessionId});

  final String sessionId;
}

/// A demo access context (section 27): no account, read endpoints only.
final class DemoSession extends Session {
  const DemoSession({required this.demoSessionId, required this.demoContext});

  final String demoSessionId;
  final DemoContext demoContext;
}
```

`SessionVault` keeps the secrets: refresh and renewal tokens in the Keychain / Keystore (`SecureStore`), the access token in memory only. When several requests need a new access token at once, they all wait for the **same** refresh ("single-flight") — refresh tokens are single-use, so two parallel refreshes would revoke the session:

apps/mobile/lib/src/core/auth/session_vault.dart

```dart
  /// A valid access token for the current session, or null when signed out.
  ///
  /// Throws [ApiError.network] when a refresh was needed but the server could not be reached. The
  /// session is kept in that case, so the person stays signed in offline.
  Future<String?> accessToken({bool forceRefresh = false}) {
    if (_session is SignedOut) return Future.value();
    final expiresAt = _accessExpiresAt;
    if (!forceRefresh &&
        _accessToken != null &&
        expiresAt != null &&
        DateTime.now().isBefore(expiresAt.subtract(_expiryMargin))) {
      return Future.value(_accessToken);
    }
    return _inFlight ??= _renew().whenComplete(() => _inFlight = null);
  }
```

apps/mobile/lib/src/core/auth/auth_interceptor.dart

```dart
  /// On a 401: renews the token once and replays the request; a second 401 ends the session.
  @override
  Future<void> onError(DioException err, ErrorInterceptorHandler handler) async {
    final options = err.requestOptions;
    final unauthorised = err.response?.statusCode == 401;
    if (!unauthorised || options.extra[_retried] == true ||
!options.headers.containsKey('Authorization')) {
```

```
      return handler.next(err);
    }
    try {
      final token = await _vault.accessToken(forceRefresh: true);
      if (token == null) return handler.next(err);
      options
        ..headers['Authorization'] = 'Bearer $token'
        ..extra[_retried] = true;
      return handler.resolve(await _dio().fetch<Object?>(options));
    } on ApiError {
      return handler.next(err);
    } on DioException catch (retryError) {
      return handler.next(retryError);
    }
  }
```

The installation id that binds a session to the phone is created once; since the October audit concurrent first calls share one future, so they can never create two different ids (fix M1):

apps/mobile/lib/src/core/storage/secure_store.dart

```
/// Keychain / Keystore storage for secrets (audit Appendix D): the installation id and the session
/// tokens. Never the database, never shared preferences.
class SecureStore {
  SecureStore(this._storage);

  final FlutterSecureStorage _storage;

  static const _installationIdKey = 'installation_id';

  /// The id being read or created: concurrent first calls share it, so a fresh install never
  /// creates two ids (a session bound to one would be refused from the other).
  Future<String>? _installationId;

  /// A random id created on first launch. Device sessions and demo sessions are bound to it; it
  /// carries nothing about the person or the hardware.
  Future<String> installationId() => _installationId ??= _loadOrCreateInstallationId();

  /// Reads the installation id, creating a random one the first time.
  Future<String> _loadOrCreateInstallationId() async {
    final existing = await _storage.read(key: _installationIdKey);
    if (existing != null && existing.isNotEmpty) return existing;
    final created = const Uuid().v4();
    await _storage.write(key: _installationIdKey, value: created);
    return created;
  }

  /// Reads one value from the Keychain / Keystore.
  Future<String?> read(String key) => _storage.read(key: key);

  /// Writes one value to the Keychain / Keystore.
  Future<void> write(String key, String value) => _storage.write(key: key, value: value);

  /// Deletes one value from the Keychain / Keystore.
  Future<void> delete(String key) => _storage.delete(key: key);
}
```

`SessionController` exposes the session to the app. Signing out — or the server ending the session — forgets everything personal left on the phone (fixes M2 and M3):

apps/mobile/lib/src/core/auth/session_controller.dart

```
/// The current [Session], restored from secure storage at launch. The router follows it.
class SessionController extends AsyncNotifier<Session> {
  SessionVault get _vault => ref.read(sessionVaultProvider);
```

```dart

  /// Restores the saved session at launch, and listens for the vault ending it (revoked, expired).
  @override
  Future<Session> build() async {
    final vault = ref.watch(sessionVaultProvider);
    vault.onSessionEnded = () {
      unawaited(_forgetPersonalData());
      state = const AsyncData(SignedOut());
    };
    return vault.restore();
  }

  /// Stores the tokens of a confirmed sign-up or a sign-in, without switching the app yet, so the
  /// caller can finish onboarding calls first. Call [enter] afterwards.
  Future<UserSession> store(AuthTokensDto tokens) => _vault.saveUser(tokens);

  /// Switches the app to the stored session.
  void enter() => state = AsyncData(_vault.session);

  /// « Essayer la démo »: asks the API for a short-lived demo token for this device, no sign-up.
  Future<void> startDemo(DemoContext context) async {
    final deviceId = await ref.read(secureStoreProvider).installationId();
    final demo = await ref
        .read(apiProvider)
        .demo
        .demoControllerStart(
          body: StartDemoDto(demoContext: context, deviceId: deviceId),
        );
    state = AsyncData(await _vault.saveDemo(demo));
  }

  /// Signs out on the server when reachable, and always on the device.
  Future<void> signOut() async {
    if (_vault.session is UserSession) {
      try {
        await ref.read(apiProvider).auth.authControllerSignOut();
      } catch (_) {
        // The device forgets the session anyway; the server session expires on its own.
      }
    }
    await _vault.clear();
    await _forgetPersonalData();
    state = const AsyncData(SignedOut());
  }

  /// What the person leaves on the device goes with their session (shared phones).
  Future<void> _forgetPersonalData() async {
    await ref.read(appDatabaseProvider).forgetPersonalSettings();
    ref.invalidate(recentSearchesProvider);
  }

  /// The device descriptor sent at sign-in and sign-up confirmation (sessions are device-bound).
  Future<DeviceDto> device() async => DeviceDto(
    id: await ref.read(secureStoreProvider).installationId(),
    platform: Platform.isIOS ? DevicePlatform.ios : DevicePlatform.android,
    name: Platform.isIOS ? 'iPhone' : 'Android',
  );

  /// The interface language, for server texts (OTP messages).
  Language language() =>
      ref.read(settingsControllerProvider).value?.language == AppLanguage.en ? Language.en :
Language.fr;
}
```

11.5 Routes and redirects

Every screen has a path in `Routes`. The redirect function decides, on every navigation and whenever the session or the account changes, where the person is allowed to be:

apps/mobile/lib/src/router/app_router.dart

```dart
/// Where the router sends a viewer, or null to stay.
@visibleForTesting
String? redirectFor({
  required String location,
  required AsyncValue<Session> session,
  required AsyncValue<AccountSummary?> account,
  required String? afterSignIn,
}) {
  final current = session.value;
  if (current == null) return location == Routes.launch ? null : Routes.launch;
  final isPublic = Routes.public.contains(location);
  switch (current) {
    case SignedOut():
      return isPublic ? null : Routes.welcome;
    case DemoSession():
      final accountOnly = location == Routes.subscription ||
Routes.accountOnly.any(location.startsWith);
      return location == Routes.launch || isPublic || accountOnly ? Routes.home : null;
    case UserSession():
      final summary = account.value;
      if (summary == null) {
        // Still loading (or unreachable with no cached copy): wait on the launch screen only when
        // coming from a signed-out page.
        if (account.isLoading) {
          return location == Routes.launch || (isPublic && location != Routes.role) ? Routes.launch
: null;
        }
        return location == Routes.launch || isPublic ? Routes.home : null;
      }
      if (summary.professionalType == null) return location == Routes.role ? null : Routes.role;
      if (location == Routes.launch || isPublic) return afterSignIn ?? Routes.home;
      return null;
  }
}
```

- Signed out → only public pages (welcome, sign-in, sign-up, codes, demo start, recovery).
- Demo → never public pages again, never account-only pages (subscription, profile edit, verification…).
- Account without a profession → « Qui êtes-vous ? » first.
- Locked content is **never** hidden by a redirect: the screen renders its locked state.

The three tabs (Accueil, Bibliothèque, Profil) are a `StatefulShellRoute`: each tab keeps its own stack, and tapping the active tab again goes back to its first screen. The round button in the middle of the bar opens search over everything; « Activité et favoris » is reached from Profil.

11.6 Talking to the API

The client under `core/api/generated/` is produced from `openapi.json` by `swagger_parser` + `build_runner`: one method per endpoint (`api.calculators.calculatorsControllerList()`) and one immutable class per DTO. Two interceptors are added to dio: the language (`Accept-Language`) and the token (`AuthInterceptor`). Failures become an `ApiError` with the server's code; screens translate their specific codes and fall back to `commonErrorText`.

apps/mobile/lib/src/core/api/api_error.dart

```dart
  /// Turns any failure (network, timeout, HTTP error body) into one ApiError with a stable code.
  static ApiError from(Object error) {
    if (error is ApiError) return error;
    if (error is! DioException) return const ApiError(code: unknownCode);
    switch (error.type) {
      case DioExceptionType.connectionError:
      case DioExceptionType.connectionTimeout:
      case DioExceptionType.receiveTimeout:
      case DioExceptionType.sendTimeout:
      case DioExceptionType.transformTimeout:
        return const ApiError.network();
      case DioExceptionType.badResponse:
      case DioExceptionType.badCertificate:
      case DioExceptionType.cancel:
      case DioExceptionType.unknown:
        break;
    }
    final response = error.response;
    if (response == null) return const ApiError.network();
    final data = response.data;
    final body = data is Map ? data['error'] : null;
    if (body is! Map) return ApiError(code: unknownCode, statusCode: response.statusCode);
    final details = body['details'];
    return ApiError(
      code: body['code'] is String ? body['code'] as String : unknownCode,
      statusCode: response.statusCode,
      fields: details is Map ? _fields(details['fields']) : const {},
      retryAt: details is Map && details['retryAt'] is String
          ? DateTime.tryParse(details['retryAt'] as String)?.toLocal()
          : null,
      attemptsRemaining: details is Map && details['attemptsRemaining'] is num
          ? (details['attemptsRemaining'] as num).toInt()
          : null,
    );
  }
```

11.7 The local database (drift)

A small SQLite database keeps: device settings (key/value), offline documents, calculators and substances, and the analytics queue. Settings that belong to a *person* rather than the phone are listed so they can be forgotten at sign-out:

apps/mobile/lib/src/core/database/app_database.dart

```dart
  /// Settings that belong to the person signed in, not to the device: their cached account (name,
  /// profession), unsent research history and recent searches. Removed when the session ends, so
  /// the next person on a shared phone neither sees them nor sends them under their own account —
  /// nor gets the previous person's account as an offline fallback after signing in.
  static const personalSettings = ['account.summary', 'activity.queue', 'search.recent'];

  /// The last values typed in each calculator (`calculator.inputs.<slug>`): a patient's values,
  /// never left for the next person either.
  static const calculatorInputsPrefix = 'calculator.inputs.';

  /// Forgets [personalSettings] and the remembered calculator inputs (sign-out, session ended,
  /// account deleted).
  Future<void> forgetPersonalSettings() async {
    for (final key in personalSettings) {
      await deleteSetting(key);
    }
    await (delete(localSettings)..where((s) => s.key.like('$calculatorInputsPrefix%'))).go();
  }

```

```
  /// After an account deletion: everything the account left on the device goes — cached account,
  /// recent searches, queued history and analytics, last calculator inputs, the remembered
  /// identifier — except the device preferences in [keep].
  Future<void> forgetAccountData({required Set<String> keep}) => transaction(() async {
    await (delete(localSettings)..where((s) => s.key.isNotIn(keep))).go();
    await delete(analyticsQueue).go();
  });
```

11.8 The widget kit and tokens

Screens are built from `lib/src/ui/`: `DtButton` (one primary per region, a spinner while loading, disabled when `onPressed` is null), `DtTextField` (label above, error on blur), `DtAlert`, `DtBadge` (always a word), `ContentRow` (a list row with its lock), `DtPickerField` (searchable bottom-sheet picker), `DtSegmentedControl`, `DtPillTabs`, `DtChoiceList` (long answers in full), the glass hero (`DtGlass`, `DtHeroAppBar`) and `LockLabel` / `showLockSheet`. Colours, spacing, radii and durations come only from tokens: `context.colors.brand`, `DtSpace.s4`, `DtRadius.lg`. CI rejects a raw colour literal.

apps/mobile/lib/src/ui/locks.dart

```
/// The calm sheet a locked item opens (section 27): what the boundary is and, when there is one,
/// the single unlock path. Students get an explanation only — no purchase action (Lock L38).
Future<void> showLockSheet(BuildContext context, ItemAccessDto access, {required String title}) {
  final l10n = AppLocalizations.of(context);
  final colors = context.colors;
  final (String, String)? action = switch (access.unlockPath) {
    UnlockPath.subscriptionPlans => (l10n.lockSeePlans, Routes.plans),
    UnlockPath.authSignUp => (l10n.lockCreateAccount, Routes.welcome),
    UnlockPath.verificationStart ||
    UnlockPath.verificationStatus => (l10n.lockSeeVerification, Routes.verificationStatus),
    UnlockPath.onboardingRole => (l10n.lockSeeVerification, Routes.role),
    _ => null,
  };
  final demo = access.lockReason == LockReason.demoLimit;
  if (demo) recordDemoLimit(context);
  return showModalBottomSheet<void>(
    // Above the floating tab bar (ADR-020), never under it.
    useRootNavigator: true,
    context: context,
    useSafeArea: true,
    backgroundColor: colors.surface,
    shape: const RoundedRectangleBorder(borderRadius: BorderRadius.vertical(top:
Radius.circular(DtRadius.xl))),
    builder: (sheetContext) => Padding(
      padding: const EdgeInsets.all(DtSpace.s5),
      child: Column(
        mainAxisSize: MainAxisSize.min,
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              Expanded(
                child: Semantics(
                  header: true,
                  child: Text(title, style: DtType.heading.copyWith(color: colors.ink)),
                ),
              ),
              LockLabel(reason: access.lockReason),
            ],
          ),
          const SizedBox(height: DtSpace.s2),
          Text(lockExplanation(l10n, access.lockReason), style: DtType.body.copyWith(color:
colors.inkSecondary)),
          const SizedBox(height: DtSpace.s5),
          if (action != null) ...[
```

```
            // The unlock path the API gives (verify, see plans, create an account): closes the
sheet and goes there.
            DtButton(
              label: action.$1,
              onPressed: () {
                Navigator.of(sheetContext).pop();
                context.push(action.$2);
              },
            ),
            const SizedBox(height: DtSpace.s2),
          ],
          // « Continuer la démo » / « Fermer ».
          DtButton(
            label: demo ? l10n.lockContinueDemo : l10n.commonClose,
            variant: action == null ? DtButtonVariant.primary : DtButtonVariant.secondary,
            onPressed: () => Navigator.of(sheetContext).pop(),
          ),
        ],
      ),
    ),
  );
}
```

11.9 Texts

Every text is `AppLocalizations.of(context).someKey`, generated from the ARB files that `packages/i18n` builds. To add a text: add the key in French and English to the catalogue, rebuild the ARB, run `flutter gen-l10n` (chapter 14).

11.10 Tests

`apps/mobile/test/` has unit tests (engine, vectors, offline store, core) and widget tests that pump real screens with fake providers. `accessibility_test.dart` checks every main screen in light and dark, at 200 % text, for touch size (44 px), labels and contrast.

12. The mobile app, screen by screen

Every screen lives in `apps/mobile/lib/src/features/<feature>/`. For each: what it shows, every button and what it calls. The comments above each button in the code say the same thing in one line; Appendix A lists them with line numbers.

12.1 Welcome and onboarding (features/onboarding)

Screen	Buttons and behaviour
Welcome `welcome_screen.dart`	« Créer un compte », « Se connecter », « Essayer la démo ». On a phone that signed in before, « Se connecter » becomes the primary action and the identifier is pre-filled.
Démo `demo_start_screen.dart`	Choose the context (doctor, resident, student) → « Commencer la démo » → `SessionController.startDemo`. No sign-up, no personal data.
Qui êtes-vous ? `role_select_screen.dart`	One role row per profession; « Continuer ». Before an account exists the choice waits in memory (`OnboardingDraft`); signed in, it is saved at once.
Créer un compte `sign_up_screen.dart`	Names, e-mail or phone, password with strength line, wilaya picker, terms checkbox. Errors appear when a field is left, never per keystroke. « Créer mon compte » stays disabled until everything is valid. « Se connecter » if the account exists.
Code `otp_screen.dart`	Six boxes over one hidden field (paste and SMS autofill work); « Vérifier »; « Renvoyer le code » with a 60-second countdown; after 3 wrong codes a 10-minute lock.
Se connecter `sign_in_screen.dart`	« Se connecter », « Pas encore de compte ? », « Accès perdu ? », « Mot de passe oublié ? ». Messages for wrong password, lock until a time, unconfirmed, suspended.
Mot de passe oublié `password_reset_screens.dart`	« Envoyer le code » (same answer whether or not the account exists), then code + new password → « Réinitialiser » → every device signs out.

apps/mobile/lib/src/features/onboarding/otp_screen.dart

```
47    /// Checks the six-digit code; a confirmed sign-up signs in and continues onboarding.
48    Future<void> _verify(String code) async {
49      final draft = ref.read(onboardingDraftProvider);
50      final challenge = draft.challenge;
51      if (challenge == null || _verifying) return;
52      setState(() {
53        _verifying = true;
54        _error = null;
55        _resent = false;
56      });
57      final sessions = ref.read(sessionControllerProvider.notifier);
58      final api = ref.read(apiProvider);
59      try {
60        final tokens = await api.auth.authControllerConfirmSignUp(
61          body: ConfirmSignUpDto(challengeId: challenge.challengeId, code: code, device: await
62  sessions.device()),
63        );
64        await sessions.store(tokens);
65        final role = draft.role;
```

```
      if (role != null) {
        try {
          await api.account.usersControllerSetProfessionalProfile(
            body: SetProfessionalProfileDto(professionalType: role),
          );
        } catch (error) {
          // The account exists; « Qui êtes-vous ? » is asked again after sign-in if this failed.
          if (ApiError.from(error).code != 'PROFILE_ALREADY_SET') {
            ref.read(onboardingDraftProvider.notifier).reset();
          }
        }
      }
      if (draft.identifier != null) await rememberIdentifier(ref, draft.identifier!);
      ref.read(onboardingDraftProvider.notifier).signedUp(Routes.verificationIntro);
      sessions.enter();
    } catch (error) {
      _code.clear();
      if (mounted) setState(() => _error = ApiError.from(error));
    } finally {
      if (mounted) setState(() => _verifying = false);
    }
  }
```

12.2 The shell and Home

`app_shell.dart` draws the floating glass tab bar: « Accueil », « Bibliothèque », the round search button, « Profil » (with a dot when something needs attention, such as verification). `home_screen.dart`: the identity line (name, role, access; the bell opens notifications), the glass search field, a pregnancy search shortcut, the three resources as tiles, a verification line or « Voir les offres » depending on the access profile, and the recent items. Everything that can be opened comes from the AccessProfile — never from the profession.

12.3 Bibliothèque (features/library)

Pill tabs Ordonnances · Certificats · Calculateurs · Grossesse, a filter field scoped to the section, an A–Z list of `ContentRow`s. A locked row keeps its place with its lock word; tapping it opens the lock sheet. Pull to refresh; « Réessayer » on error; offline, the list comes from the phone's store with the date of the copy. A library search is reported to analytics once typing pauses (the words themselves are never sent).

12.4 Recherche (features/search)

Full screen, keyboard up. Below two characters: recent searches (kept on the phone only; « Effacer », × per item) and example queries. Then: results grouped by type (3 per group, « Voir les N »), type chips with counts, « Vouliez-vous dire … » suggestions, and offline results from the local store when the network fails. An answer arriving late for an older query is ignored.

apps/mobile/lib/src/features/search/search_screen.dart

```
  /// Searches the API (or the offline copy); an older answer arriving late is ignored.
  Future<void> _run() async {
    final query = _query.text.trim();
    if (query.length < 2) return;
    _report?.cancel();
    final request = ++_request;
    setState(() {
      _loading = true;
      _failed = false;
    });
    try {
```

```
      final api = ref.read(apiProvider).search;
      final result = await api.searchControllerQuery(q: query, type: _filter);
      final totals = _filter == null
          ? {for (final group in result.groups) group.type: group.total.toInt()}
          : _totals.isEmpty
          ? {for (final group in (await api.searchControllerQuery(q: query)).groups) group.type:
group.total.toInt()}
          : _totals;
      if (!mounted || request != _request) return;
      setState(() {
        _result = result;
        _totals = totals;
        _loading = false;
      });
      _report = Timer(const Duration(seconds: 1), () {
        ref
            .read(analyticsRecorderProvider)
            .record(
              AnalyticsEvent.searchPerformed(
                resultCount: totals.values.fold(0, (sum, total) => sum + total),
                groups: [
                  for (final entry in totals.entries)
                    if (entry.value > 0) entry.key,
                ],
              ),
            );
      });
    } catch (_) {
      // Without the full search, look through what the device holds (offline store included).
      List<ContentRef> local = const [];
      try {
        final index = await ref.read(contentIndexProvider.future);
        local = [
          for (final content in index.values)
            if (matchesAllWords(query, [content.titleFr, content.titleEn])) content,
        ]..sort((a, b) => a.titleFr.compareTo(b.titleFr));
      } catch (_) {
        local = const [];
      }
      if (!mounted || request != _request) return;
      setState(() {
        _loading = false;
        _failed = true;
        _local = local;
      });
    }
  }
```

12.5 Reading a reference PDF (features/reference)

`reference_viewer_screen.dart` downloads the PDF through a signed URL, checks its SHA-256 against the published one, and draws it with pdfrx. Floating control: page « 2 / 5 », zoom out, fit width, zoom in; the ★ in the top bar adds a favorite. There is no download, print or share button anywhere. Offline, the copy kept on the phone is used.

12.6 A calculator (features/calculators)

```
CalculatorRuntime: detail from API (or offline store) → parse definition → SELF-TEST the published ca
  ready → the form        locked → lock message      unsupported → "update the app"      self-test fa
form: number fields (with unit picker) · choice lists · yes/no switches
  typing → commit after 400 ms idle, "next", or leaving the field → evaluate() on the DEVICE
  result: value + unit + band, formula with the values, notes (« vérifiez la valeur »), conversions
  « Copier le résultat » (value, unit, band — never the inputs) · « Réinitialiser » · « Source » shee
```

apps/mobile/lib/src/features/calculators/calculator_screen.dart

```dart
  /// Reads the field's value (number in its unit, or the choice), stores it and re-evaluates.
  void _commit(_Field field) {
    if (!mounted) return;
    final input = field.input;
    InputValue? value;
    field.unreadable = false;
    if (input.isNumeric) {
      final raw = field.text.text.trim().replaceAll(' ', '').replaceAll(',', '.');
      if (raw.isNotEmpty) {
        final number = double.tryParse(raw);
        if (number == null) {
          field.unreadable = true;
        } else {
          value = InputValue(number, unit: field.unit);
        }
      }
    } else if (field.choice != null) {
      value = InputValue(field.choice);
    }
    setState(() {
      _committed[input.key] = value;
      _touched.add(input.key);
      _evaluate();
    });
    unawaited(ref.read(lastInputsProvider).write(_detail.slug, _committed));
  }
```

apps/mobile/lib/src/features/calculators/calculator_screen.dart

```dart
  /// Runs the device engine on the committed values and saves them locally.
  void _evaluate() {
    final given = {
      for (final entry in _committed.entries)
        if (entry.value != null) entry.key: entry.value,
    };
    _evaluation = evaluate(_definition, given);
    if (_evaluation!.outcome != Outcome.result) return;
    if (!_useRecorded) {
      _useRecorded = true;
      unawaited(ref.read(activityRecorderProvider).calculatorUsed(_detail.id));
    }
    if (!_completionRecorded) {
      _completionRecorded = true;
      ref
          .read(analyticsRecorderProvider)
          .record(
            AnalyticsEvent.calculatorCompleted(
              id: _detail.id,
              version: _version,
              resultBand: _evaluation!.band?.code,
              engine: engineVersion,
            ),
          );
```

```
    }
  }
```

The last values typed are kept on the phone (to find them again after a call) and are forgotten at sign-out. They are never sent anywhere.

12.7 Grossesse · Allaitement (features/pregnancy)

- **Search** (`pregnancy_search.dart`): server search after a 300 ms pause; a period filter (T1, T2, T3, peripartum, lactation) shows each hit's conduct for that period; « Afficher N de plus ».
- **Profile** (`substance_screen.dart`): a switch Grossesse / Allaitement; period tabs; for each period a conduct badge (icon + word) and, separately, the evidence meter (0–4) with population and sources; « Absence de données » as a full block; alternatives (each opens its profile); conflicts between sources; sources with links; « Signaler une erreur » (reason + optional comment; no patient data). A locked profile shows « Débloquer » with its explanation.

12.8 Activité et favoris (features/activity)

Two segments. « Favoris »: grouped by type, « Retirer » on each. « Activité »: the person's own history grouped by day, with filters by kind and « Effacer l'historique » (confirmation sheet). History is recorded on the phone and sent in batches; a demo records nothing.

apps/mobile/lib/src/features/activity/favorites_controller.dart

```dart
  /// Adds or removes a favorite at once on screen, then on the server; undone if the server refuses.
  Future<void> toggle(FavoriteTargetType type, String id) async {
    final current = await future;
    final key = FavoriteKey(type, id);
    final adding = !current.keys.contains(key);
    final keys = {...current.keys};
    adding ? keys.add(key) : keys.remove(key);
    state = AsyncData(Favorites(entries: current.entries, keys: keys));
    final event = AnalyticsEvent.favorite(id: id, targetType: type, added: adding);
    if (ref.read(sessionControllerProvider).value is! UserSession) {
      ref.read(analyticsRecorderProvider).record(event);
      return;
    }
    final api = ref.read(apiProvider).activity;
    try {
      if (adding) {
        await api.activityControllerAddFavorite(targetType: type, targetId: id);
      } else {
        await api.activityControllerRemoveFavorite(targetType: type, targetId: id);
      }
      ref.read(analyticsRecorderProvider).record(event);
      ref.invalidateSelf();
    } catch (_) {
      state = AsyncData(current);
      rethrow;
    }
  }
```

12.9 Profil, Paramètres, Sécurité (features/profile, settings)

- **Profil**: photo or initials, name, role, verification and access badges, « Modifier le profil » (names, photo « Changer » / « Supprimer », bio ≤ 280), then Activité et favoris, Abonnement (students: « Accès étudiant — gratuit »), Vérification, Notifications, Paramètres, « Se déconnecter ».

- **Paramètres**: Apparence (Clair · Sombre · Appareil) and Langue (Français · English), applied at once; « Données et hors ligne »; « Sécurité du compte »; « Supprimer mon compte ».
- **Sécurité du compte**: the e-mail and phone with « Modifier » / « Ajouter » / « Retirer » (password + code to the new contact), « Changer le mot de passe », and, during a recovery's 24-hour hold, « Ce n'est pas moi ».
- **Supprimer mon compte**: what is erased, what the law keeps, the end of a current subscription without refund, then the password.

12.10 Vérification (features/verification)

Step	What happens
Intro	Why, what is checked, where documents go, how long; « Commencer »; « Plus tard » always available.
1 · Identité	Document kind (ID card, passport, student card); « Prendre une photo » opens the camera with corner guides, then « Utiliser la photo » / « Reprendre ». On the phone, ML Kit reads the text: too little text is « illisible », the name must appear — the person can still « Envoyer quand même ». « Importer un fichier » accepts PDF, JPG, PNG.
2 · Statut	Doctor: Ordre number, specialty, attestation or diploma. Resident: year, institution, specialty, attestation. Student: university and proof of enrolment.
3 · Récapitulatif	Checkmarks and file descriptions only; « Envoyer pour vérification ».
Statut	Each case with its timeline (Reçu → En cours de revue → Décision), the refusal reason in plain words and one action: Commencer, Continuer, Corriger, Faire appel, Renouveler; « ⋮ » → « Effacer mes documents ».

apps/mobile/lib/src/features/verification/document_slot.dart

```dart
  /// Checks size and (for photos) readability and name; warns before sending a doubtful photo.
  Future<void> _accept(_Picked picked) async {
    final l10n = AppLocalizations.of(context);
    final size = await File(picked.path).length();
    if (size > maxVerificationFileBytes) {
      await _discard(picked);
      setState(() {
        _phase = _Phase.failed;
        _error = l10n.verificationDocumentTooLarge;
      });
      return;
    }
    _nameRead = null;
    _checkedLegible = false;
    if (picked.contentType == 'application/pdf') return _upload(picked);

    setState(() {
      _phase = _Phase.checking;
      _error = null;
    });
    final account = ref.read(accountControllerProvider).value;
    DocumentCheck? check;
    try {
      check = await ref
          .read(documentCheckerProvider)
          .check(picked.path, firstName: account?.firstName ?? '', lastName: account?.lastName ??
'');
```

```
    } catch (_) {
      // The check is a help, not a gate: if ML Kit is unavailable the reviewer still decides.
    }
    if (!mounted) return;
    if (check != null && !check.legible) {
      setState(() {
        _phase = _Phase.quality;
        _quality = _Quality.illegible;
        _pending = picked;
      });
      return;
    }
    if (check != null && !check.nameFound && _isIdentityStep) {
      setState(() {
        _phase = _Phase.quality;
        _quality = _Quality.nameMismatch;
        _pending = picked;
      });
      return;
    }
    _checkedLegible = check?.legible ?? false;
    if (check != null && check.nameFound && account != null) {
      _nameRead = '${account.lastName.toUpperCase()} ${account.firstName}';
    }
    await _upload(picked);
  }
```

12.11 Abonnement (features/subscription)

- **Offres**: each plan's includes / not included, duration, price, access; the button depends on eligibility: « Choisir », or verification first, or (demo) « Créer un compte ». Students see an explanation, never a purchase.
- **Récapitulatif**: plan, price, period, card (CIB or Edahabia — only those the API accepts now), access, manual renewal, the « non remboursable » line to tick; « Payer » opens the gateway's page in the browser.
- **Résultat du paiement**: asks the API every 4 seconds and on return to the app until the payment is final — the API asks the gateway, never trusting the browser. « Voir mon abonnement », « Réessayer », « Autre moyen de paiement », « Rouvrir la page de paiement ».
- **Mon abonnement**: plan, neutral or teal status badge, period, payments; « Renouveler » (last 14 days), « Arrêter le renouvellement ». No red, no countdown.

apps/mobile/lib/src/features/subscription/payment_result_screen.dart

```
  /// Asks the API for the payment's status; success refreshes the access profile.
  Future<void> _check() async {
    if (_checking || (_transaction != null && _transaction!.status != TransactionStatus.pending))
return;
    setState(() => _checking = true);
    try {
      final transaction = await ref
          .read(apiProvider)
          .subscription
          .billingControllerTransaction(transactionId: widget.transactionId);
      if (!mounted) return;
      setState(() => _transaction = transaction);
      if (transaction.status != TransactionStatus.pending) {
        _timer?.cancel();
        ref.read(pendingCheckoutProvider.notifier).clear();
        ref
          ..invalidate(subscriptionOverviewProvider)
          ..invalidate(plansProvider);
```

```
        await ref.read(accessControllerProvider.notifier).reload();
      }
    } catch (_) {
      // Offline or server busy: the next tick or the next resume tries again.
    } finally {
      if (mounted) setState(() => _checking = false);
    }
  }
```

12.12 Accès perdu ? (features/recovery)

Three ways back in: « Mot de passe oublié » (code), « J'ai changé de contact » (sign in with the other one), « J'ai perdu l'accès aux deux » (assisted recovery: form → code to the new contact → identity photo + selfie → « Envoyer la demande » → answer within 72 h, applied 24 h after approval).

12.13 Données et hors ligne (features/offline)

Last sync, data version, storage used; « Tout » or « Favoris seulement »; mobile data on/off; « Mettre à jour maintenant »; « Supprimer les données hors ligne ». A bar under the status bar says offline, syncing, failed (« Réessayer ») or outdated.

apps/mobile/lib/src/features/offline/offline_sync.dart

```
  /// Downloads the offline manifest, then each missing or changed PDF, calculator and profile;
removes the rest.
  Future<void> _sync() async {
    if (!ref.read(isOnlineProvider)) return;
    state = AsyncData(state.value!.copyWith(phase: SyncPhase.syncing, progress: 0, clearFailure:
true));
    try {
      final access = await ref.read(accessControllerProvider.notifier).future;
      final offline = access?.feature(FeatureKey.offlineSync);
      if (offline == null || offline.state != AccessState.available) {
        final hadCopies =
            (await _store.documents()).isNotEmpty ||
            (await _store.calculators()).isNotEmpty ||
            (await _store.substances()).isNotEmpty;
        await _store.purge();
        if (hadCopies) await _database.writeSetting(_lostKey, 'true');
        final status = await _load(enabled: true);
        if (ref.mounted) state = AsyncData(status);
        return;
      }
      await _database.deleteSetting(_lostKey);
      final manifest = await ref.read(apiProvider).offline.offlineControllerManifest();
      final status = state.value!;
      final onMobile = ref.read(networkProvider).value == NetworkKind.mobile;
      final favorites = (await ref.read(favoritesProvider.future)).keys;
      bool wanted(ManifestReferenceDto item) {
        final favorite = favorites.contains(FavoriteKey(FavoriteTargetType.referenceDocument,
item.id));
        if (favorite) return true;
        if (status.scope == OfflineScope.favorites) return false;
        return !onMobile || status.mobileData;
      }

      final pending = await _syncDocuments(manifest.referenceDocuments, wanted);
      await _syncCalculators(manifest.calculators);
      await _syncSubstances(manifest.substances);
      await _database.writeSetting(_dataVersionKey, manifest.dataVersion);
      await _database.writeSetting(_syncedAtKey, DateTime.now().toIso8601String());
      await _database.writeSetting(_pendingKey, '$pending');
      final loaded = await _load(enabled: true);
```

```
211         if (ref.mounted) state = AsyncData(loaded);
212       } catch (error) {
          // Signed out (or the account changed) while syncing: this controller is gone.
          if (!ref.mounted) return;
```

12.14 Notifications

Newest first; unread ones with a dot; tapping marks one read and opens its link; « Tout marquer comme lu ». Texts come from catalogue keys with parameters, so they follow the app's language.

13. Eight journeys through the whole system

Each journey follows one real story from the first tap to the database and back, naming the file at every step. Read one with the three apps open side by side: it is the fastest way to feel how the pieces fit.

13.1 A doctor joins, gets verified and subscribes

```
MOBILE                                  API                                          ADMIN WEB
welcome_screen « Créer un compte »
role_select_screen  DOCTOR (in memory)
sign_up_screen — POST /v1/auth/sign-up ─► AuthService.signUp → OtpService.issue → SMS/e-mail
otp_screen ──── POST …/sign-up/confirm ► confirmSignUp → session + tokens; DemoService.markConverted
             └─ POST /v1/me/professional-profile (DOCTOR)  UsersService
verification_intro → identity_step (camera, ML Kit check)
   POST /v1/me/verification/cases …/uploads → signed PUT → R2 doktool-private → …/documents (confirm)
status_step (Ordre number, specialty, attestation)  → saveDetails
summary_screen « Envoyer » ──────────────► VerificationService.submit  (SUBMITTED)
                                                                        Vérifications queue (SL
                                                                        « Prendre le dossier »
                                                                        restricted viewer (60 s
                                        VerificationReviewService.approve ◄─ « Approuver »
                                        profile ← role + details; notification; audit entry
notification « Vérification approuvée »  AccessService → tier PROFESSIONAL_UNPAID, purchase OPEN
plans_screen « Choisir » → review_screen « Payer » ─► CheckoutService.start → Chargily page
   browser pays on the gateway's page
payment_result_screen polls ─────────────► settle → gateway.status = PAID → activate subscription
access reloads → tier PROFESSIONAL: every model, calculator and profile opens; offline sync starts
```

13.2 A visitor tries the demo, then creates an account

1. `demo_start_screen` → `POST /v1/demo/sessions` with the installation id → a one-hour demo token + renewal token (`SessionVault.saveDemo`).
2. The router moves to Home; `GET /v1/me/access` answers tier DEMO: only `demoSample` items open; offline, purchase and profile edit are locked with « Créer un compte » as the unlock path.
3. Analytics: `DEMO_STARTED`, then `DEMO_FEATURE_VIEWED` for each module seen or limit met (no account, no personal data).
4. « Créer un compte » → sign-up → on confirmation the same installation id marks the demo CONVERTED; the « Démo » funnel in Admin Web counts it.

13.3 Opening a prescription model

Chapter 3.1 followed this request byte by byte. The things to remember: the list shows locked models in place; `file-url` re-checks access; the PDF comes straight from R2 with a 10-minute URL; its SHA-256 is checked on the phone; `ActivityRecorder.referenceViewed` adds a history entry; `REFERENCE_VIEWED` is counted once the PDF has rendered.

13.4 Using a calculator, offline

1. During the last sync, `OfflineSyncController` stored the published calculator version (definition + test cases) in drift.

2. No network: `calculatorRuntimeProvider` reads the stored version, parses it (`definition.dart`) and runs its published test cases on the Dart engine (`self_test.dart`). A failure refuses the calculator.
3. The doctor types the age, weight, creatinine; each field commits after a pause; `evaluate()` checks ranges and rules and computes; the band and the formula with the values appear.
4. Nothing leaves the phone. `CALCULATOR_COMPLETED` is queued for analytics with the calculator id and version only.

13.5 Authoring and publishing a calculator (two admins)

```
Author (calculators.draft)                          Reviewer (calculators.review + publish), NOT the
« Nouveau calculateur » → createCalculator
« Nouvelle version » → createVersion (DRAFT)
Définition: JSON from the cited source → saveDefinition (checkDefinition)
Sources: PRIMARY source (+ validation sources)
Tests: cases from the source → « Lancer les tests » (TS engine, stored with checksum)
« Soumettre à revue » → submit (gate re-checked)   ──►  IN_REVIEW
                                                          reads definition, sources, tests; « Aperçu »
                                                          « Approuver et publier » + written note
                                                            publish: tests re-run, four-eyes, sign-offs
                                                            (policy 2 → first sign-off; second reviewer pu
                                                          PUBLISHED: immutable; audit entries all along
Phones: next list / sync → new version, self-tested before use
```

13.6 Pregnancy profiles from a spreadsheet

1. « Importer un fichier » → the XLSX is read on the server without running anything in it (`workbook.ts` reads the XML inside the zip).
2. The header row is found, columns mapped, status words listed; each row checked against the current library (duplicates, unknown alternatives).
3. The admin maps columns and status words, corrects rows, then « Ajouter à la bibliothèque » → **draft** profile versions, all or nothing, idempotent on double click.
4. Authors complete evidence levels and explanations; « Soumettre à relecture »; a different admin approves; only then do phones see the profiles.

13.7 Lost both e-mail and phone

Mobile `access_help_screen` → `recovery_screen` (form, code to the new contact, two photos to the private bucket, « Envoyer la demande ») → Admin Web « Récupération » (compare, « Approuver » with a note) → 24-hour hold, owner notified on the old contacts → `auth.account-recovery` job: claim, replace contacts, password unusable, all sessions revoked → the person signs in with the new contact through « Mot de passe oublié ».

13.8 A subscription reaches its end

1. 14 and 3 days before the end: notifications « Votre abonnement se termine bientôt » (`SubscriptionLifecycleService.remind`). « Renouveler » is open in the last 14 days; a renewal extends from the current end.
2. At the end without renewal: GRACE for 7 days, access kept (`advance`).
3. After grace: EXPIRED. `placeInTier` gives PROFESSIONAL_UNPAID; the doctor keeps the sample set and favorites; offline copies of locked items are removed at the next sync.

4. Never: red banners, countdowns, refund buttons.

14. Working on the code: tests, CI and recipes

You now know where everything is. This last chapter is practical: how to check your work, and the exact steps for the changes you will make most often.

14.1 Before every commit

```
DATABASE_URL=postgresql://…/doktool_test npm run ci      # format, lint, types, i18n, tokens, contrac
cd apps/mobile && flutter analyze && flutter test
npm run build -w @doktool/admin                            # when Admin Web changed
```

Use a dedicated test database. Integration tests do not roll back. A test that changes something platform-wide (for example the double sign-off policy) must put it back, as `clinical-review.e2e.spec.ts` does.

14.2 Recipe: add an API endpoint

1. Write the service method in the module that owns the data (with a one-line comment).
2. Add the DTO classes (`@ApiProperty` + class-validator decorators).
3. Add the controller method with the right guard: `UserAuthGuard`, `ViewerAuthGuard`, `@RequireFeature` or `@RequireGrants`. Admin changes call `AuditService.record` with the reason.
4. Write an integration test in `apps/api/test/integration`.
5. Regenerate the contract and both clients:

```
npm run openapi:generate -w @doktool/api
npm run generate:api -w @doktool/admin
cd apps/mobile && dart run swagger_parser && dart run build_runner build --delete-conflicting-outputs
```

14.3 Recipe: add a column or a table

1. Edit `apps/api/prisma/schema.prisma` in the section of the owning module (`// — [module] —`); a unit test checks table ownership.
2. Write the migration by hand in `prisma/migrations/<timestamp>_<name>/migration.sql`. An enum `ADD VALUE` goes in its own migration.
3. Indexes written in SQL (trigram, partial) are also declared in the schema, or `prisma migrate diff` will propose to drop them. Check with `npx prisma migrate diff --from-config-datasource --to-schema prisma/schema.prisma --exit-code`.
4. `npm run prisma:migrate:deploy -w @doktool/api`, then `prisma generate` runs on install.

14.4 Recipe: add or change a text

```
// keys.json
{ "fr": { "mobile.home.newKey": "Bonjour" }, "en": { "mobile.home.newKey": "Hello" }, "remove": [] }

npm run add-keys -w @doktool/i18n -- keys.json
npm run build:arb -w @doktool/i18n
cd apps/mobile && flutter gen-l10n
```

Admin Web reads the catalogue directly; mobile uses `AppLocalizations.of(context).homeNewKey`. CI fails if a key exists in one language only.

14.5 Recipe: add a mobile screen

1. Create the screen in `lib/src/features/<feature>/`, built from `lib/src/ui` widgets and tokens only.
2. Add its path to `Routes` and a `GoRoute` in `app_router.dart`; if it is for real accounts only, add it to `Routes.accountOnly`.
3. Load data through a provider; draw loading (skeleton), error (« Réessayer ») and empty states.
4. Locked things stay visible: use the AccessProfile, `LockLabel` and `showLockSheet`.
5. Add a widget test, and the screen to `accessibility_test.dart`.
6. Format with `dart format --page-width 120` (never on `doktool_tokens.dart`).

14.6 Recipe: add an Admin Web section

1. The API endpoints with `@RequireGrants`; a new grant goes in `grants.ts` and its label in `features/permissions/grants.ts` (the build fails until it is labelled).
2. `src/features/<section>/`: the page, its drawers and dialogs, `labels.ts` for status names and tones.
3. `src/app/admin/<section>/page.tsx` wrapping it in `<RequireGrant>`, and a link in `shell/nav.ts`.
4. Every change asks for a reason (`ReasonDialog`) and shows a toast on success.

14.7 Adding a calculator is not a code change

A new calculator is data, entered in Admin Web by a doctor from a cited source (chapter 13.5), reviewed by someone else, and published. No deployment is needed: phones receive it at their next list or sync. The engines change only when a new *kind* of operator or input is needed — then both engines, `checkDefinition`, the shared vectors and the version number of the engine change together.

14.8 The checklist that keeps DokTool safe

Rule	Where it is enforced
Every endpoint re-checks access	`@RequireFeature`, `AccessService.decideItems`, `@RequireGrants`
No patient data, no free text in analytics	`analytics.policy.ts` schemas; calculator inputs never sent
Files only through StorageService, signed and short-lived	`storage.policy.ts`, ESLint import rule
Tokens: access in memory, refresh in Keychain/Keystore, single-use, device-bound	`session_vault.dart`, `sessions.service.ts`
Admin: cookie + TOTP, writes only from Admin Web, separation of duties	`admin-guards.ts`, `grants.ts`
Clinical content: sourced, tested, author ≠ reviewer, immutable	`calculators-admin.service.ts`, `pregnancy-admin.service.ts`, database constraints
Every admin change audited, log append-only	`AuditService`, table triggers
CSV cells cannot run as formulas	`common/text/csv.ts`

14.9 What the October 2026 audit changed

The full report is `docs/CODE-AUDIT-2026-10.md`. In short: eighteen defects fixed with tests (the most serious: re-verification dropping a paying doctor's access, and a race in account recovery), performance of the offline manifest and favorites improved about tenfold, comments added to every function and button of the three apps, and six recommendations recorded for decisions that need the Product Owner or a real R2 bucket.

14.10 Rebuilding this book

```
python3 docs/learning-book/build.py           # HTML + PDF, English and Arabic (needs Chrome)
python3 docs/learning-book/build.py --html    # HTML only
```

Chapters are in `docs/learning-book/content/en` and `content/ar`; folder introductions in `folders.json`. Code excerpts and Appendix A are read from the sources at build time, so after a code change, rebuilding is enough to bring the book up to date. A placeholder whose anchor no longer exists stops the build with its name, so nothing goes silently stale.

Appendix A — Every file, function and button

This appendix is generated from the code itself. For each folder: what it is for; for each file: every documented declaration (its first line) and every commented button, with the comment written in the code. Read it next to the source: the order is the order of the file.

Admin Web (apps/admin)

apps/admin/src/api/

The typed API client: `client.ts` (openapi-fetch typed by the generated `schema.d.ts`, `call()` and `ApiFailure`) and `download.ts` for CSV exports.

client.ts · 75 lines

5 `export type Schemas = components['schemas'];`
Shapes of every API request and response, generated from the OpenAPI contract.

10 `export class ApiFailure extends Error {`
A failed API call, with the catalogue code the UI translates (never the server's message).

22 `get sessionEnded(): boolean {`
The admin session is gone (expired, signed out elsewhere): back to sign-in.

28 `export const api = createClient<paths>({ baseUrl: '', credentials: 'same-origin' });`
Same-origin client: `/admin/v1/*` is proxied to the API (next.config.ts).

31 `interface ApiResult<T> {`
What an openapi-fetch call resolves to.

38 `function isErrorResponse(value: unknown): value is Schemas['ErrorResponse'] {`
True when a body has the API's `{ error: … }` shape.

48 `export async function call<T>(request: Promise<ApiResult<T>>): Promise<T> {`
Awaits an openapi-fetch call and returns its data, or throws an {@link ApiFailure}.

71 `export function fieldErrors(error: unknown): Record<string, string[]> {`
Fields rejected by the server, keyed by DTO property (VALIDATION_FAILED).

download.ts · 39 lines

7 `export async function downloadFile(`
Downloads a CSV export through the same-origin proxy (the session cookie goes with it) and saves it under `fileName`.

apps/admin/src/app/admin/analytics/

Route of the « Analytique » section: grant check, then the feature component.

page.tsx · 14 lines

7 `export default function Page() {`
/admin/analytics, for admins with analytics.read.

apps/admin/src/app/admin/audit/

Route of the « Journal d'audit » section: grant check, then the feature component.

page.tsx · 14 lines

7 `export default function Page() {`
/admin/audit, for admins with audit.read.

apps/admin/src/app/admin/calculators/[id]/

Route of the « one calculator » section: grant check, then the feature component.

page.tsx · 16 lines

8 `export default function Page({ params }: { params: Promise<{ id: string }> }) {`
/admin/calculators/:id: one calculator.

apps/admin/src/app/admin/calculators/

Route of the « Calculateurs (list) » section: grant check, then the feature component.

page.tsx · 14 lines

```
7    export default function Page() {
     /admin/calculators, for admins with calculators.read.
```

apps/admin/src/app/admin/finance/

Route of the « Finance » section: grant check, then the feature component.

page.tsx · 14 lines

```
7    export default function Page() {
     /admin/finance, for admins with finance.read.
```

apps/admin/src/app/admin/

The console: `layout.tsx` puts every page inside the shell; `page.tsx` opens the first section the admin's grants allow. Every sub-folder is one section; its `page.tsx` only checks the grant and renders the feature component.

layout.tsx · 8 lines

```
5    export default function AdminLayout({ children }: { children: ReactNode }) {
     Every console page sits inside the shell (top bar and navigation).
```

apps/admin/src/app/admin/library/[type]/

Route of the « ordonnances or certificates » section: grant check, then the feature component.

page.tsx · 19 lines

```
10   export default function Page({ params }: { params: Promise<{ type: string }> }) {
     /admin/library/ordonnances or /certificates; any other segment is "not found".
```

apps/admin/src/app/admin/library/

Route of the « Bibliothèque (opens the ordonnances) » section: grant check, then the feature component.

page.tsx · 7 lines

```
4    export default function Page() {
     /admin/library opens the ordonnances.
```

apps/admin/src/app/admin/overview/

Route of the « Vue d'ensemble » section: grant check, then the feature component.

page.tsx · 9 lines

```
6    export default function Page() {
     /admin/overview, for every admin (blocks follow the grants).
```

apps/admin/src/app/admin/

The console: `layout.tsx` puts every page inside the shell; `page.tsx` opens the first section the admin's grants allow. Every sub-folder is one section; its `page.tsx` only checks the grant and renders the feature component.

page.tsx · 27 lines

```
12   export default function AdminHome() {
     `/admin` opens the first section the admin's grants allow.
```

apps/admin/src/app/admin/permissions/

Route of the « Permissions » section: grant check, then the feature component.

page.tsx · 14 lines

```
export default function Page() {
```

/admin/permissions, for admins with system.roles.write.

apps/admin/src/app/admin/pregnancy/[id]/

Route of the « one substance » section: grant check, then the feature component.

page.tsx · 16 lines

```
export default function Page({ params }: { params: Promise<{ id: string }> }) {
```

/admin/pregnancy/:id: one substance.

apps/admin/src/app/admin/pregnancy/import/[id]/

Route of the « one import's preview » section: grant check, then the feature component.

page.tsx · 16 lines

```
export default function Page({ params }: { params: Promise<{ id: string }> }) {
```

/admin/pregnancy/import/:id: one import's preview.

apps/admin/src/app/admin/pregnancy/import/

Route of the « spreadsheet imports » section: grant check, then the feature component.

page.tsx · 14 lines

```
export default function Page() {
```

/admin/pregnancy/import: the import history and the upload.

apps/admin/src/app/admin/pregnancy/

Route of the « Grossesse · Allaitement » section: grant check, then the feature component.

page.tsx · 14 lines

```
export default function Page() {
```

/admin/pregnancy, for admins with pregnancy.read.

apps/admin/src/app/admin/recovery/

Route of the « Récupération » section: grant check, then the feature component.

page.tsx · 14 lines

```
export default function Page() {
```

/admin/recovery, for admins with verification.queue.read.

apps/admin/src/app/admin/settings/

Route of the « Paramètres » section: grant check, then the feature component.

page.tsx · 14 lines

```
export default function Page() {
```

/admin/settings: pickers and the sign-off policy (system.settings.write).

apps/admin/src/app/admin/subscriptions/

Route of the « Abonnements » section: grant check, then the feature component.

page.tsx · 14 lines

```
export default function Page() {
```

/admin/subscriptions, for admins with subscriptions.read.

apps/admin/src/app/admin/transactions/

Route of the « Transactions » section: grant check, then the feature component.

page.tsx · 14 lines

```
export default function Page() {
```

/admin/transactions, for admins with transactions.read.

apps/admin/src/app/admin/users/

Route of the « Utilisateurs » section: grant check, then the feature component.

page.tsx · 14 lines

```
export default function Page() {
```

/admin/users, for admins with users.read.

apps/admin/src/app/admin/verification/

Route of the « Vérifications » section: grant check, then the feature component.

page.tsx · 14 lines

```
export default function Page() {
```

/admin/verification, for admins with verification.queue.read.

apps/admin/src/app/

Next.js App Router: each folder is a URL. The root layout (title, icon, no indexing), the providers (React Query, languages, toasts) and the redirect of `/` to the console.

layout.tsx · 27 lines

```
export const metadata: Metadata = {
```

Browser tab title and icon; search engines are told not to index the console.

```
export default async function RootLayout({ children }: { children: ReactNode }) {
```

The HTML page around everything, in the language remembered in a cookie.

page.tsx · 7 lines

```
export default function Root() {
```

The site root sends to the console.

providers.tsx · 47 lines

```
function onSessionEnded(error: unknown) {
```

Any API call answering "session ended" reloads the sign-in page (remembering where the admin was).

```
window.location.assign(`/sign-in?next=${encodeURIComponent(pathname + search)}`);
```

A full page load drops every cached query (case details, names) of the ended session. eslint-disable-next-line @next/next/no-location-assign-relative-destination

```
function retry(failureCount: number, error: unknown): boolean {
```

Retries network and server errors twice, never a refusal (4xx).

```
export function Providers({ locale, children }: { locale: Locale; children: ReactNode }) {
```

Wraps every page with the language, the API cache and the toasts.

apps/admin/src/app/sign-in/

The two-step sign-in: password, then the authenticator code (QR code enrolment the first time).

page.tsx · 14 lines

```
export default function SignInPage() {
```

/sign-in: the sign-in form, centred.

sign-in-form.tsx · 224 lines

19 `type Step = Schemas['AdminSignInStepDto'];`
What the password step returns: enrol or verify the authenticator code.

22 `const credentialsSchema = z.object({`
Rules of the first step's fields.

27 `const codeSchema = z.object({ code: z.string().regex(/^\d{6}$/) });`
A 6-digit code.

30 `export function SignInForm() {`
Admin sign-in: e-mail + password, then the mandatory TOTP code (ADR-009).

34 `const [restarted, setRestarted] = useState(false);`
The code step ended (expired or too many wrong codes): say why the form starts over.

67 `function CredentialsStep({ onNext }: { onNext: (step: Step) => void }) {`
Step 1: e-mail and password.

71 `const form = useForm({`
The e-mail and password form.

77 `const submit = form.handleSubmit(async (values) => {`
Sends e-mail and password; on success moves to the code step, otherwise shows why.

120 `<Button type="submit" variant="primary" busy={isSubmitting}>`
« Continuer »: sends e-mail and password.

133 `onRestart: (expired: boolean) => void;`
Back to the first step; `expired` when the server ended the code step.

144 `useEffect(() => {`
First sign-in: draws the authenticator QR code from the otpauth link.

159 `const submit = form.handleSubmit(async ({ code }) => {`
Sends the code; on success the session cookie is set and the console opens where the admin wanted to go.

214 `<Button type="submit" variant="primary" busy={isSubmitting}>`
« Vérifier » (or « Activer » the first time): sends the 6-digit code.

218 `<Button variant="ghost" onClick={() => onRestart(false)} disabled={isSubmitting}>`
« Recommencer »: back to e-mail and password.

apps/admin/src/features/analytics/

The analytics dashboards: usage, library, calculators, onboarding funnel, subscriptions, demo; small counts are hidden.

analytics-page.tsx · 488 lines

22 `type ProfessionalType = Schemas['ProfessionalType'];`
A profession, as a segment filter.

24 `interface Filter extends Period {`
Period and segment of the dashboards.

45 `export function AnalyticsPage() {`
« Analytics — DokTool Intelligence » (ScreenAnalytics, section 16): aggregates of activity inside DokTool, never prescriptions or sales; cells under 10 people read « < 10 ».

51 `const freshness = useQuery({`
When the latest event arrived.

137 `function Loading({`
A dashboard's loading, error or ready state.

158 `function Usage({ filter }: { filter: Filter }) {`
« Usage »: active people by profession and per day, against the previous period.

```
162 const query = useQuery({
```
The usage figures.

```
214 function Library({ filter }: { filter: Filter }) {
```
« Bibliothèque »: searches and use of each reference model.

```
218 const query = useQuery({
```
The library figures.

```
283 function Calculators({ filter }: { filter: Filter }) {
```
« Calculateurs »: starts and completions per calculator version.

```
287 const query = useQuery({
```
The calculator figures.

```
359 function Onboarding({ filter }: { filter: Filter }) {
```
« Utilisateurs »: sign-ups and the verification funnel.

```
363 const query = useQuery({
```
The onboarding figures.

```
388 function Subscriptions({ filter }: { filter: Filter }) {
```
« Abonnements »: plan views, starts and renewals.

```
392 const query = useQuery({
```
The subscription figures.

```
439 function Demo({ filter }: { filter: Filter }) {
```
« Démo »: demos started, modules seen, limits met, accounts created.

```
443 const query = useQuery({
```
The demo figures.

apps/admin/src/features/audit/

The audit log page (filters, CSV export) and the « Historique » tab reused by every drawer; human names of audited actions.

audit-history.tsx · 32 lines

```
9  type Entry = Pick<Schemas['AuditEntryDto'], 'id' | 'action' | 'occurredAt' | 'reason'> & {
```
An audit entry as the « Historique » tabs show it.

```
14 export function AuditHistory({ entries }: { entries: Entry[] }) {
```
The « Historique » tab of a drawer: who did what, when, and why.

audit-labels.ts · 94 lines

```
4  const ACTIONS: Record<string, MessageKey> = {
```
Audited actions as people read them; unknown codes are shown as-is (mono) by the caller.

```
70 export function isSensitiveAction(action: string): boolean {
```
Identity-document views and exceptional adjustments are highlighted in the log (section 07).

```
75 export function hasActionLabel(action: string): boolean {
```
True when an audit action has a translated name.

```
85 export const ACTION_GROUPS: { prefix: string; label: MessageKey }[] = [
```
Action prefixes offered in the log's filter.

audit-page.tsx · 283 lines

```
22 type Entry = Schemas['AuditEntryDto'];
```
One audit entry.

```
27 interface Filters {
```
Filters of the audit log.

```
38 export function auditQuery(filters: Filters): Record<string, string> {
```
The query the API takes for the current filters (dates are whole UTC days).

```
53  export function AuditPage() {
```
« Journal d'audit » (ScreenAuditLogs): immutable, filterable, exported with `audit.export`.

```
62  const query = useMemo(() => auditQuery(filters), [filters]);
```
The filters as the API's query.

```
65  const log = useInfiniteQuery({
```
Pages of the log, newest first.

```
138 const set = (patch: Partial<Filters>) => setFilters((current) => ({ ...current, ...patch }));
```
Changes some filters.

```
164 <Button busy={exporting} onClick={() => void exportCsv()}>
```
« Exporter en CSV ».

```
232 <Button size="sm" busy={log.isFetchingNextPage} onClick={() => void log.fetchNextPage()}>
```
« Charger plus ».

apps/admin/src/features/calculators/

Calculator governance: the list, the detail page with its version timeline, and the five tabs Définition · Sources · Tests · Revue · Aperçu; `definition.ts` reads a definition for display.

calculator-detail.tsx · 393 lines

```
40  export function CalculatorDetail({ id }: { id: string }) {
```
Calculator detail (ScreenCalculatorAdmin): version timeline, and the selected version's tabs.

```
51  const detail = useQuery({
```
The calculator with all its versions.

```
58  const refresh = async () => {
```
After any change to a version: refresh this calculator and the list.

```
102 <Button onClick={() => setEditing(true)}>
```
« Modifier »: names, purpose, category (and access with the publish grant).

```
110 <Button
```
« Nouvelle version »: a new draft copied from a version or pasted as JSON (only one draft at a time).

```
139 <button
```
One version of the timeline: shows its tabs.

```
213 function VersionView({
```
The selected version: its checksum and the Définition · Sources · Tests · Revue · Aperçu tabs.

```
279 function NewVersionDialog({
```
« Nouvelle version »: copy of an earlier version (with sources and tests) or a definition pasted as JSON.

```
298 const create = useMutation({
```
Creates the draft; pasted JSON that does not parse is reported before calling the API.

```
338 <Button onClick={() => onOpenChange(false)} disabled={create.isPending}>
```
« Annuler ».

```
342 <Button
```
« Créer le brouillon ».

calculator-form-dialog.tsx · 245 lines

```
27  const schema = z.object({
```
Rules of the calculator form (same limits as the API).

```
41  const TIERS: AccessTier[] = ['STUDENT', 'PROFESSIONAL'];
```
Access tiers a calculator can have.

```
44  const valuesOf = (calculator: Calculator | null): Values => ({
```
The form's starting values: the calculator's, or empty for a new one.

```
57  const orNull = (value: string) => value.trim() || null;
```
An empty text becomes null (cleared).

60 `export function CalculatorFormDialog({`
New calculator, or its catalogue fields (names, purpose, category, access, demo sample).

67 `calculator: Calculator | null;`
Null creates a new calculator.

88 `const submit = form.handleSubmit(async (values) => {`
Sends the form: PATCH for an edit (access only with the publish grant), POST for a new calculator.

157 `<Button onClick={() => onOpenChange(false)} disabled={isSubmitting}>`
« Annuler ».

161 `<Button type="submit" variant="primary" busy={isSubmitting}>`
« Enregistrer ».

calculators-page.tsx · 170 lines

30 `export function CalculatorsPage() {`
« Calculateurs » (ScreenCalculatorAdmin list): row → full-width detail page.

39 `const list = useQuery({`
Every calculator with its versions.

46 `const columns = useMemo<Column<Calculator>[]>(`
The table's columns: name, category, access, published and open version, primary source, review date, status.

136 `<Button variant="primary" onClick={() => setCreating(true)}>`
« Nouveau calculateur »: opens the creation dialog.

definition-panel.tsx · 286 lines

25 `const SEVERITY_TONES: Record<DefinitionView['interpretation'][number]['severity'], BadgeTone> = {`
Badge colour of each band severity.

33 `export function DefinitionPanel({`
« Définition »: inputs, formula, bands and limitations; the author edits a draft's JSON.

52 `const save = useMutation({`
Saves the edited JSON; the API checks it and its problems are listed by path.

110 `<Button variant="primary" busy={save.isPending} onClick={() => save.mutate()}>`
« Enregistrer la définition ».

114 `<Button disabled={save.isPending} onClick={() => setEditing(false)}>`
« Annuler »: back to the read-only view.

133 `<Button`
« Modifier la définition »: opens the draft's definition as editable JSON.

245 `function InputRow({ input }: { input: InputView }) {`
One row of the inputs table: label, kind, units (or answers and points) and the three ranges.

definition.ts · 202 lines

7 `const localized = z.object({ fr: z.string(), en: z.string() });`
Schemas used only to read a definition for display (the API owns the real checks).

11 `export type Expr =`
A formula node, as displayed.

99 `export function readDefinition(raw: unknown): DefinitionView | null {`
The definition as displayed, or null when it does not match schema version 1.

105 `const INFIX: Record<string, string> = {`
Operators written between their two operands.

123 `export function formulaText(node: Expr, labelOf: (key: string) => string): string {`
The formula AST as readable mono text, with input labels instead of keys — the same notation as the mobile app (`formula_text.dart`), e.g. `round(((140 − Âge) × Poids) ÷ (0.815 × Créatinine), 1)`.

```
160 export function rangeText(value: Range | undefined): string {
```
"[30 – 150]", "≥ 18", "≤ 250" or "—".

```
167 export function localizedOf(text: { fr: string; en: string }, locale: Locale): string {
```
A text in the UI language.

```
172 const testCaseResult = z.object({
```
One result of a stored test run.

```
185 export const storedTestRun = z.object({
```
A stored test run and the definition checksum it ran on.

```
198 export function currentRun(raw: unknown, definitionSha256: string): StoredTestRun | null {
```
The stored run of a version, if it is about its current definition (else it must run again).

labels.ts · 121 lines

```
7 export type Calculator = Schemas['AdminCalculatorDto'];
```
API types used by the calculator pages.

```
15 export const SLUG_PATTERN = /^[a-z0-9]+(-[a-z0-9]+)*$/;
```
Same pattern as the API (lowercase words joined by hyphens).

```
21 export const calculatorsKey = ['admin', 'calculators'] as const;
```
Cache key of the list.

```
23 export const calculatorKey = (id: string) => ['admin', 'calculators', id] as const;
```
Cache key of one calculator.

```
26 export const VERSION_STATUS_LABELS: Record<VersionStatus, MessageKey> = {
```
Catalogue key of each version status.

```
34 export const VERSION_STATUS_TONES: Record<VersionStatus, BadgeTone> = {
```
Badge colour of each version status.

```
42 export const CALCULATOR_STATUS_LABELS: Record<Schemas['CalculatorStatus'], MessageKey> = {
```
Catalogue key of each calculator status.

```
48 export const ACCESS_LABELS: Record<AccessTier, MessageKey> = {
```
Catalogue key of each access tier.

```
54 export const ROLE_LABELS: Record<SourceRole, MessageKey> = {
```
Catalogue key of each source role.

```
61 export const OUTCOME_LABELS: Record<TestOutcome, MessageKey> = {
```
Catalogue key of each outcome.

```
68 export function publishedVersion(calculator: Calculator): Version | undefined {
```
The version shown to users (current published one), if any.

```
73 export function openDraft(calculator: Calculator): Version | undefined {
```
The version still being written or reviewed, if any.

```
80 export interface Gate {
```
One condition of the publication gate, met or not.

```
89 export function publicationGates(
```
The publication gate of section 28, as the reviewer sees it before acting (the API enforces it again, with the four-eyes rule on who drafted the version).

preview-panel.tsx · 233 lines

```
18 interface FieldState {
```
What the admin typed in one preview field, and its unit.

```
24 export function toEvaluationInputs(
```
Evaluation inputs of the API (`{ key: { value, unit? } }`); blank fields are left out.

```
44 function initialState(inputs: InputView[]): Record<string, FieldState> {
```
Empty preview fields (booleans false, numbers in their canonical unit).

60 `export function PreviewPanel({ version }: { version: Version }) {`
« Aperçu »: live preview with the backend engine (the authoritative one). Test inputs here are the admin's own, not patient data, and are not stored.

69 `const evaluate = useMutation({`
Runs the version on the API's engine with the typed values (never stored).

84 `const set = (key: string, patch: Partial<FieldState>) =>`
Updates one field of the preview.

112 `<Button type="submit" variant="primary" busy={evaluate.isPending}>`
« Calculer »: evaluates on the server.

159 `function PreviewField({`
One preview field: a switch, a list of answers, or a number with its unit picker.

review-panel.tsx · 348 lines

23 `type Action = 'return' | 'publish' | 'retire' | 'discard' | null;`
Which dialog is open.

29 `export function ReviewPanel({`
« Revue »: author, reviewer and note; the publication gate; and the lifecycle actions (submit, return, approve and publish, retire, discard). Author ≠ reviewer (section 28).

52 `const submit = useMutation({`
« Soumettre à revue »: the API checks the gate again.

117 `<Button`
« Soumettre à revue »: possible once sources and passing tests are there.

126 `<Button variant="danger" onClick={() => setAction('discard')}>`
« Supprimer le brouillon ».

136 `<Button`
« Approuver et publier » (or first / second sign-off): never by the author, never twice by the same reviewer.

145 `<Button disabled={iAmAuthor} onClick={() => setAction('return')}>`
« Renvoyer à l'auteur »: back to draft, with a reason.

152 `<Button variant="danger" onClick={() => setAction('retire')}>`
« Retirer »: the published version leaves the app at the next sync.

235 `function PublishDialog({`
« Approuver et publier », or a first of two sign-offs (ADR-024): the review note is the reviewer's written decision.

255 `const publish = useMutation({`
Sends the reviewer's note with the approval.

278 `<Button onClick={() => onOpenChange(false)} disabled={publish.isPending}>`
« Annuler ».

282 `<Button type="submit" variant="primary" busy={publish.isPending} disabled={!valid}>`
Confirms the approval (needs a written note).

310 `function DiscardDialog({`
Confirms deleting a draft.

334 `<Button onClick={() => onOpenChange(false)} disabled={discard.isPending}>`
« Annuler ».

338 `<Button variant="danger" busy={discard.isPending} onClick={() => discard.mutate()}>`
« Supprimer »: deletes the draft with its sources and tests.

sources-panel.tsx · 278 lines

23 `const ROLES: SourceRole[] = ['PRIMARY', 'VALIDATION', 'UNIT_CONVERSION'];`
Roles a source can have.

26 `const schema = z.object({`
Rules of a source.

39 `type Values = z.input<typeof schema>;`
The source form's values.

42 `export function SourcesPanel({`
« Sources »: citation, edition, where the formula is stated, role (≥ 1 primary, section 28).

57 `const remove = useMutation({`
Removes a source from the draft.

86 `<Button`
Bin icon: removes this source.

128 `<Button size="sm" onClick={() => setAdding(true)}>`
« Ajouter une source ».

151 `function AddSourceDialog({`
The source form: role, citation, edition, where the formula is stated, DOI or link, date accessed.

188 `const submit = form.handleSubmit(async (values) => {`
Adds the source (empty optional fields become null).

242 `<Button onClick={() => onOpenChange(false)} disabled={isSubmitting}>`
« Annuler ».

246 `<Button type="submit" variant="primary" busy={isSubmitting}>`
« Ajouter la source ».

tests-panel.tsx · 387 lines

21 `const OUTCOMES: TestOutcome[] = ['RESULT', 'INVALID', 'OUT_OF_VALIDATED_RANGE'];`
Outcomes a test case can expect.

24 `export function TestsPanel({`
« Tests »: every case with expected vs backend engine result, engine version and run time.

44 `const runTests = useMutation({`
« Lancer les tests » on the server engine; the run is stored with the definition's checksum.

58 `const remove = useMutation({`
Removes a test case from the draft.

104 `<Button size="sm" onClick={() => setAdding(true)}>`
« Ajouter un cas ».

111 `<Button`
« Lancer les tests ».

181 `<Button`
Bin icon: removes this test case.

210 `function AddTestCaseDialog({`
The test-case form: where the example comes from, the inputs as JSON and the expected outcome.

232 `const add = useMutation({`
Adds the case; inputs that are not valid JSON are reported first.

289 `<Button onClick={() => onOpenChange(false)} disabled={add.isPending}>`
« Annuler ».

293 `<Button type="submit" variant="primary" busy={add.isPending} disabled={!ready}>`
« Ajouter le cas ».

apps/admin/src/features/clinical-review/

Shared by calculators and pregnancy: who has signed off a version and what the current admin's approval would do.

sign-offs.tsx · 89 lines

8 `interface Person {`
An admin as shown in sign-off rows.

14 `export interface SignOffState {`
What a version carries about its sign-offs (ADR-024).

23 `export interface ApprovalStep {`
The step an approval by the current admin performs, with its labels.

25 `signedByMe: boolean;`
The current admin gave the first sign-off and may not give the second.

27 `firstOfTwo: boolean;`
This approval records the first of two sign-offs instead of publishing.

30 `title: MessageKey | null;`
Replaces the dialog's own title and text for a first sign-off or a second one.

35 `export function approvalStep(`
What the current admin's approval does: publish, first sign-off, or second sign-off.

63 `export function SignOffRows({ version }: { version: SignOffState }) {`
Rows for a version's description list: « Relectures 1 sur 2 » and the first sign-off.

apps/admin/src/features/finance/

Finance: the ledger and charts, expenses and fee rates, fee settlement, payment methods on/off, CSV reports.

expenses-panel.tsx · 474 lines

26 `type Expense = Schemas['ExpenseDto'];`
API types used by the expenses tab.

37 `export function parseSettlementLines(text: string): {`
One line of a provider statement: `DT-2026-001842;75,00` (fee in DZD).

62 `export function ExpensesPanel({ period }: { period: Period }) {`
« Dépenses »: operating expenses, payment methods, provider fee rates and fee settlement (section 15, ADR-021).

75 `const expenses = useQuery({`
Expenses of the period.

80 `const rates = useQuery({`
Fee rate of each payment channel.

144 `onSelect: () => setVoiding(row),`
« Annuler la dépense » (voided, never deleted).

163 `<Button size="sm" variant="primary" className="ms-auto" onClick={() => setAdding(true)}>`
« Ajouter une dépense ».

192 `<Button size="sm" className="ms-auto" onClick={() => setSettling(true)}>`
« Régler les frais »: replaces estimated fees with the provider's statement.

218 `<Button size="sm" variant="ghost" onClick={() => setRateFor(rate.provider)}>`
« Modifier » the fee rate of this channel.

272 `function ExpenseDialog({ onClose, onDone }: { onClose: () => void; onDone: () => Promise<void> }) {`
The new-expense form: category, label, amount, date and reason.

351 `function RateDialog({`
The fee rate of one channel: percentage and fixed part.

426 `function SettlementDialog({`
Paste the provider's fees (transaction id and fee per line) to settle them.

finance-page.tsx · 333 lines

24 `export function FinancePage() {`
« Finance » (ScreenAdminRevenue, ScreenExpenses, reports): section 15, Lock L40.

62 `function FinanceOverview({ period }: { period: Period }) {`
« Vue d'ensemble » tab: the ledger (gross → fees → net), profit, subscriptions, revenue by plan and the charts.

66 `const overview = useQuery({`
Every figure of the period and the one before.

80 `const money = (minor: number) => formatMoney(locale, minor, data.currency);`
An amount in DZD in the UI language.

289 `function Reports({ period }: { period: Period }) {`
« Rapports »: CSV with the metric definitions on top (section 15).

323 `<Button busy={busy === report.kind} onClick={() => void download(report.kind)}>`
« Télécharger le CSV » of this report.

payment-methods-card.tsx · 121 lines

19 `type Route = Schemas['PaymentMethodRouteDto'];`
A payment method, its gateway and its switch.

30 `export function PaymentMethodsCard() {`
« Moyens de paiement » (ADR-021): the gateway that takes each card, configured on the server, and the switch that pauses a card in the app while its network is down.

40 `const routes = useQuery({`
Every method with the gateway that takes it.

74 `<Button size="sm" variant="ghost" onClick={() => setSwitching(route)}>`
« Suspendre » / « Reprendre » this payment method (an outage), with a reason.

apps/admin/src/features/library/

Reference library: the list per type, the upload drawer (checksum → signed upload → create), the model drawer (metadata, preview, replace, status).

document-drawer.tsx · 572 lines

40 `type Lifecycle = 'enable' | 'disable' | 'archive' | 'restore';`
Status changes offered by the drawer.

43 `const metadataSchema = z.object({`
Rules of the editable fields.

51 `const PROCESSING_POLL_MS = 3_000;`
While a file is processing, the drawer follows its status.

54 `const LIFECYCLE_COPY: Record<`
Dialog texts of each status change.

81 `const LIFECYCLE_PATHS = {`
API path of each status change.

89 `export function DocumentDrawer({`
Reference model drawer (ScreenReferenceLibraryAdmin): file, titles, demo sample, status.

105 `const [previewFile, setPreviewFile] = useState<string | null>(null);`
The file being previewed; another model or a new current file closes the preview.

108 `const detail = useQuery({`
The model; refreshed every 3 seconds while a file is processing.

128 `const saved = useMemo<Metadata>(`
The saved metadata; polling while a file processes never overwrites what the admin typed.

137 `const form = useForm<Metadata>({`
The edit form; a refresh never overwrites what the admin is typing.

197 `<Button`
« Enregistrer »: asks for a reason, then saves titles and demo sample.

241 `<Button`
« Voir le PDF » / « Masquer »: shows the current PDF in the drawer (no download or print).

```
253 <Button size="sm" onClick={() => setReplacing(true)}>
```
« Remplacer le PDF »: uploads a new version under the same model.

```
447 function DocumentPreview({
```
The PDF of one file version, drawn on canvases from a signed URL.

```
485 function ReplaceDialog({
```
« Remplacer le PDF »: a new file version; favourites and links keep working.

```
503 const replace = useMutation({
```
Checksum → signed upload → PUT → register the new version (processed in the background).

```
540 <Button onClick={() => onOpenChange(false)} disabled={replace.isPending}>
```
« Annuler ».

```
544 <Button
```
« Remplacer le PDF »: possible once a PDF and a reason are given.

labels.ts · 72 lines

```
6   export type ReferenceDocument = Schemas['AdminReferenceDocumentDto'];
```
API types used by the library pages.

```
12  export const TYPE_SEGMENTS: Record<string, DocumentType> = {
```
URL segment of each library type (`/admin/library/{ordonnances|certificates}`, section 02).

```
18  export const SEGMENT_OF: Record<DocumentType, string> = {
```
URL segment of each type.

```
24  export const TYPE_LABELS: Record<DocumentType, MessageKey> = {
```
Catalogue key of each type.

```
30  export const STATUS_LABELS: Record<DocumentStatus, MessageKey> = {
```
Catalogue key of each model status.

```
38  export const STATUS_TONES: Record<DocumentStatus, BadgeTone> = {
```
Badge colour of each model status.

```
46  export const FILE_STATUS_LABELS: Record<FileStatus, MessageKey> = {
```
Catalogue key of each file processing status.

```
53  export const FILE_STATUS_TONES: Record<FileStatus, BadgeTone> = {
```
Badge colour of each file processing status.

```
60  export const STATUS_FILTERS: (DocumentStatus | null)[] = [
```
The status chips (null = all).

```
69  export const libraryKey = (type: DocumentType) => ['admin', 'library', type] as const;
```
Cache key of one type's list.

```
71  export const documentKey = (id: string) => ['admin', 'library', 'document', id] as const;
```
Cache key of one model.

library-page.tsx · 206 lines

```
32  const TYPES: DocumentType[] = ['ORDONNANCE', 'CERTIFICATE'];
```
The two library types, in tab order.

```
35  export function LibraryPage({ type }: { type: DocumentType }) {
```
« Bibliothèque de référence » (ScreenReferenceLibraryAdmin): one surface, type switch on top.

```
45  const list = useQuery({
```
Every model of this type (archived ones included; the status chips filter them).

```
51  const rows = useMemo(() => {
```
The rows shown: filtered by status chip and by the search text, in the browser.

```
63  const counts = useMemo(() => {
```
How many models each status chip has.

```
71  const columns = useMemo<Column<ReferenceDocument>[]>(
```
The table's columns.

129 `<Button variant="primary" onClick={() => setAdding(true)}>`
« Ajouter un modèle »: opens the upload drawer.

166 `<button`
Status chip with its count: filters the table.

pdf-drop.tsx · 93 lines

11 `const PROBLEMS: Record<PdfProblem, MessageKey> = {`
Catalogue key of each refusal reason.

18 `export function PdfDrop({`
PDF drop zone (component FileUpload): drag and drop or click to choose; one PDF, ≤ 20 MB.

48 `<button`
The drop zone: click to choose a PDF, or drop one on it.

upload-drawer.tsx · 219 lines

22 `const schema = z.object({`
Rules of the « Ajouter un modèle » form.

32 `type Stage = 'idle' | 'hashing' | 'uploading' | 'saving';`
Where the upload is: computing the checksum, sending the file, creating the model.

38 `export function UploadDrawer({`
« Ajouter un modèle » (section 13): PDF → display title pre-filled from the file name (editable) → optional English title → demo sample → status. Everything else is derived from the file.

69 `const submit = form.handleSubmit(async (values) => {`
Checksum → signed upload → PUT the bytes → create the model; the drawer of the new model opens.

123 `<Button onClick={onClose} disabled={busy}>`
« Annuler ».

128 `<Button variant="primary" busy={busy} disabled={!file} onClick={() => void submit()}>`
« Ajouter »: its label follows the stage (checksum, sending, saving).

upload.ts · 57 lines

4 `export const PDF_MAX_BYTES = 20 * 1024 * 1024;`
Same limits as the API's REFERENCE_DOCUMENT storage policy (checked again server-side).

8 `export function suggestTitle(fileName: string): string {`
Display title proposed from the file name, as the API does: "angine_adulte-v2.pdf" → "angine adulte v2".

19 `export type PdfProblem = 'type' | 'size' | 'empty';`
Why a file is refused before uploading.

22 `export function pdfProblem(file: { name: string; type: string; size: number }): PdfProblem | null {`
Why a picked file cannot be uploaded as a reference model, or null.

31 `export async function sha256Hex(file: Blob): Promise<string> {`
Lowercase hex SHA-256 of the file, declared before upload and verified by the API.

37 `export class UploadFailed extends Error {`
The PUT to the signed URL failed (shown as « Envoi impossible »).

48 `export async function putSigned(ticket: Schemas['UploadTicketDto'], file: Blob): Promise<void> {`
Sends the bytes to the signed URL (R2, or the API's local driver in development). The browser sets Content-Length itself; Content-Type must match the signature.

apps/admin/src/features/overview/

The overview page: one ledger strip and the work lists of the admin's grants.

overview-page.tsx · 202 lines

18 `export function OverviewPage() {`
« Vue d'ensemble » (ScreenAdminOverview): one ledger strip, then three work lists — no KPI wall.

apps/admin/src/features/permissions/

Admin accounts: creation with a one-time password, grants by area with separation-of-duties checks, temporary grants, authenticator reset, suspension.

admin-drawer.tsx · 248 lines

```
20  type Admin = Schemas['AdminAccountDto'];
```
One admin account.

```
22  type Action = 'save' | 'temporary' | 'suspend' | 'reinstate' | 'resetTotp' | null;
```
Which dialog is open.

```
27  export function AdminDrawer({ admin, onClose }: { admin: Admin | null; onClose: () => void }) {
```
One admin: effective grants with their origin, temporary access, status and TOTP (section 07).

```
32  function AdminDrawerBody({ admin, onClose }: { admin: Admin; onClose: () => void }) {
```
The drawer's content: the grants by area, with the separation-of-duties rules checked live.

```
39  const permanent = useMemo(
```
The admin's permanent grants (temporary ones expire on their own).

```
83  { label: t('admin.permissions.temporary'), onSelect: () => setAction('temporary') },
```
« Accès temporaire »: a grant that ends after a few hours.

```
85  { label: t('admin.permissions.resetTotp'), onSelect: () => setAction('resetTotp') },
```
« Réinitialiser l'authentificateur »: lost phone; enrols again at next sign-in.

```
89  onSelect: () => setAction('suspend'),
```
« Suspendre »: blocks sign-in and ends sessions.

```
96  onSelect: () => setAction('reinstate'),
```
« Réactiver ».

```
103 <Button
```
« Enregistrer les permissions »: needs a change that breaks no separation rule.

grants.ts · 95 lines

```
11  export const GRANT_LABELS: Record<Grant, MessageKey> = {
```
Every grant with its label, in section 03 order. Typed as a record of the API's grant enum, so a grant added to the contract fails the build until it is labelled here.

```
51  export const GRANT_GROUPS: { domain: string; label: MessageKey; grants: Grant[] }[] = [
```
Grants grouped by their domain (the part before the first dot).

```
82  export function separationProblems(grants: readonly Grant[]): MessageKey[] {
```
Section 03 separation of duties, checked before saving (the API checks again): finance never goes with calculators or the library, and only the Super Admin approves adjustments.

permissions-page.tsx · 285 lines

```
22  type Admin = Schemas['AdminAccountDto'];
```
One admin account.

```
25  export function PermissionsPage() {
```
« Permissions » (ScreenPermissions): admins, their presets and grants (section 03).

```
32  const admins = useQuery({
```
Every admin with their grants.

```
121 <Button variant="primary" onClick={() => setCreating(true)}>
```
« Nouvel administrateur ».

```
166 function CreateAdminDialog({
```
New admin: e-mail, name and preset; the one-time password is then shown once.

```
242 function PasswordDialog({
```
The one-time password, shown once: the new admin enrols TOTP at first sign-in.

261 `<Button variant="primary" onClick={onClose}>`
« J'ai transmis le mot de passe »: closes; the password is never shown again.

269 `<Button`
Copies the one-time password to the clipboard.

apps/admin/src/features/pregnancy/

Grossesse · Allaitement: substances, sources and reports tables, the substance page with the profile editor and viewer, merging duplicates, spreadsheet import pages.

dialogs.tsx · 323 lines

23 `const ATC = /^[A-Z]([0-9]{2}([A-Z]([A-Z]([0-9]{2})?)?)?)?$/;`
Shape of an ATC code (A, A01, A01B, A01BC, A01BC01).

26 `const substanceSchema = z.object({`
Rules of the substance form.

38 `type SubstanceForm = z.infer<typeof substanceSchema>;`
The substance form's values.

41 `export function SubstanceDialog({`
« Nouvelle substance » or its names: DCI, English name, ATC code, synonyms.

65 `const save = useMutation({`
Creates the substance, or saves its names, ATC code and synonyms.

102 `<Button onClick={() => onOpenChange(false)} disabled={save.isPending}>`
« Annuler ».

106 `<Button type="submit" variant="primary" busy={save.isPending}>`
« Enregistrer ».

174 `const sourceSchema = z.object({`
Rules of the source form.

192 `type SourceForm = z.infer<typeof sourceSchema>;`
The source form's values.

195 `export function SourceDialog({`
A source of the registry: CRAT, LactMed, RCP ANSM, Briggs... (section 25 Appendix B).

220 `const save = useMutation({`
Adds the source to the registry, or saves its changes.

255 `<Button onClick={() => onOpenChange(false)} disabled={save.isPending}>`
« Annuler ».

259 `<Button type="submit" variant="primary" busy={save.isPending}>`
« Enregistrer la source ».

import-labels.ts · 151 lines

6 `export type ImportPreview = Schemas['ImportPreviewDto'];`
API types used by the import pages.

14 `export const importsKey = ['admin', 'pregnancy', 'imports'] as const;`
Cache keys of the import pages.

18 `export const FIELDS = [`
The fields a column can be mapped to.

30 `export const FIELD_LABELS: Record<Field | 'ignore' | 'row', MessageKey> = {`
Catalogue key of each field.

43 `export const STATUS_TARGETS: StatusTarget[] = [`
What a status word of the sheet can mean.

50 `export const STATUS_TARGET_LABELS: Record<StatusTarget, MessageKey> = {`
Catalogue key of each meaning.

58 `export const FILTERS: RowFilter[] = [`
Row filters of the preview.

68 `export const FILTER_LABELS: Record<RowFilter, MessageKey> = {`
Catalogue key of each filter.

79 `export const STATUS_LABELS: Record<Schemas['ImportStatus'], MessageKey> = {`
Catalogue key of each import status.

85 `export const STATUS_TONES: Record<Schemas['ImportStatus'], BadgeTone> = {`
Badge colour of each import status.

120 `export function issueKey(code: string): MessageKey {`
Message key of an issue code; an unknown code (newer server) falls back to a generic text.

130 `export function rowState(row: ImportRow): { tone: BadgeTone; key: MessageKey } {`
The row's technical state in one word: never colour alone.

140 `export const SEVERITY_TONES: Record<string, BadgeTone> = {`
Badge colour of each issue severity.

146 `export const SEVERITY_LABELS: Record<string, MessageKey> = {`
Catalogue key of each issue severity.

import-pages.tsx · 862 lines

52 `export function ImportsPage() {`
« Importer un fichier » (ADR-025): the history of imports and the upload.

60 `const list = useQuery({`
The last 100 imports.

66 `const upload = useMutation({`
Sends the chosen .xlsx; the preview page opens with what the server read.

142 `<Button`
« Choisir un fichier .xlsx ».

191 `const preview = useQuery({`
The import with the current page of rows and filter.

261 `<Button`
« Précédent »: the previous page of rows.

269 `<Button`
« Suivant »: the next page of rows.

285 `function useRefresh(id: string) {`
Reloads the import after a change.

291 `function Summary({ data }: { data: ImportPreview }) {`
Counts at the top: rows, valid, errors, new substances.

318 `function ColumnMapping({ data }: { data: ImportPreview }) {`
Which field each column of the sheet holds.

325 `const save = useMutation({`
Saves the column mapping; every row is checked again.

377 `<Button`
« Appliquer »: saves the mapping.

390 `function StatusMapping({ data, editable }: { data: ImportPreview; editable: boolean }) {`
What each status word of the sheet means (contre-indiqué, à éviter, possible, rien).

396 `const save = useMutation({`
Saves what the status words mean; every row is checked again.

458 `function IssueList({ issues, all = false }: { issues: ImportIssue[]; all?: boolean }) {`
In the table, only what needs action; the row dialog (`all`) also lists the notes.

```
480 function RowsTable({ data, editable }: { data: ImportPreview; editable: boolean }) {
```
The rows with their issues; a row opens its correction dialog.

```
566 function FieldMark({ row, field }: { row: ImportRow; field: ImportField }) {
```
Marks a cell by the worst issue of its field: an icon-free word badge, never colour alone.

```
575 function Value({ row, field }: { row: ImportRow; field: ImportField }) {
```
One cell: the corrected value, with the original shown when it was changed.

```
586 function RowDialog({
```
Corrects one row: values, exclusion, and what to do when it conflicts with existing content.

```
617 const save = useMutation({
```
Saves the row's correction; the row is checked again.

```
652 <Button variant="ghost" onClick={onClose}>
```
« Annuler ».

```
657 <Button variant="primary" type="submit" disabled={save.isPending}>
```
« Enregistrer la ligne ».

```
720 function Confirm({ data }: { data: ImportPreview }) {
```
The bottom bar: discard, or add the valid rows as drafts (refused while errors remain).

```
745 <Button variant="ghost" onClick={() => setDiscarding(true)}>
```
« Abandonner l'import ».

```
749 <Button variant="primary" disabled={blocked} onClick={() => setConfirming(true)}>
```
« Ajouter à la bibliothèque »: writes the valid rows into draft profiles.

```
796 function Committed({ data }: { data: ImportPreview }) {
```
After a commit: what was created, and sending every new draft to review.

```
825 <Button variant="primary" onClick={() => setSubmitting(true)}>
```
« Soumettre tout à relecture ».

labels.ts · 152 lines

```
6   export type Substance = Schemas['AdminSubstanceDto'];
```
API types used by the pregnancy pages.

```
21  export const substancesKey = ['admin', 'pregnancy', 'substances'] as const;
```
Cache keys of the pregnancy pages.

```
28  export const PERIODS = ['T1', 'T2', 'T3', 'PERIPARTUM'] as const;
```
The four pregnancy periods, in order.

```
32  export const PERIOD_LABELS: Record<PeriodKey, MessageKey> = {
```
Catalogue key of each period.

```
40  export const CONDUCTS: Conduct[] = ['CONTRAINDICATED', 'AVOID', 'OCCASIONAL_USE'];
```
Conducts a period can state (or none).

```
43  export const CONDUCT_LABELS: Record<Conduct, MessageKey> = {
```
A clinical instruction (icon + word); never a safe/unsafe verdict.

```
50  export const EVIDENCE_LABELS: Record<number, MessageKey> = {
```
Evidence strength is neutral: the word carries the meaning, never a colour (Lock L6).

```
59  export const VERSION_STATUS_LABELS: Record<VersionStatus, MessageKey> = {
```
Catalogue key of each version status.

```
68  export const VERSION_STATUS_TONES: Record<VersionStatus, BadgeTone> = {
```
Badge colour of each version status.

```
77  export const SOURCE_TYPES: SourceType[] = [
```
Kinds of source.

```
86  export const SOURCE_TYPE_LABELS: Record<SourceType, MessageKey> = {
```
Catalogue key of each kind of source.

95 `export const SECTION_LABELS: Record<Schemas['ProfileSection'], MessageKey> = {`
Catalogue key of each section a report points at.

101 `export const REPORT_REASON_LABELS: Record<Schemas['ContentReportReason'], MessageKey> = {`
Catalogue key of each report reason.

109 `export function sourceCitation(source: ClinicalSource): string {`
« CRAT · 09/2026 », the way the evidence footer cites a source.

115 `export function isOpenStatus(status: VersionStatus): boolean {`
True for a version not decided yet.

120 `export function isEditable(status: VersionStatus): boolean {`
True for a version its author may still edit.

125 `export function localized(text: Localized | undefined, locale: string): string | null {`
The French text, or English in the English console when it was written.

131 `export function emptyPeriod(): Period {`
A period with nothing stated.

136 `export function emptyPregnancy(): PregnancyBlock {`
A pregnancy block with four empty periods.

149 `export function emptyLactation(): LactationBlock {`
A lactation block with nothing stated.

merge-dialog.tsx · 212 lines

16 `const BLOCKER_LABELS: Record<string, MessageKey> = {`
Catalogue key of each reason a merge is refused.

30 `export function MergeDialog({`
« Fusionner avec… » (ADR-027): the duplicate's names join the substance kept; its profile is never copied. The reviewer sees exactly what changes before confirming, with a reason for the audit log.

51 `const candidates = useMemo(() => {`
Substances the duplicate can be merged into (others, visible, not merged), filtered by the words typed.

65 `const preview = useQuery({`
What the merge would change, and what blocks it.

76 `const merge = useMutation({`
Performs the merge.

109 `<Button variant="ghost" onClick={() => onOpenChange(false)}>`
« Annuler ».

113 `<Button variant="danger" type="submit" disabled={!ready || merge.isPending}>`
« Fusionner ».

pregnancy-page.tsx · 512 lines

39 `export function PregnancyPage() {`
« Grossesse · Allaitement » (ADR-019): substances, the sources registry and user reports.

54 `<Link href="/admin/pregnancy/import" className="dt-btn secondary">`
« Importer un fichier » (or the import history without the draft grant).

62 `<Button variant="primary" onClick={() => setCreating(true)}>`
« Ajouter une substance ».

103 `type SubstanceQuery = NonNullable<`
Filters of the substances table, as the API names them.

108 `const STATES = ['PUBLISHED', 'DRAFT', 'IN_REVIEW', 'CHANGES_REQUESTED', 'NONE'] as const;`
Profile states offered by the state filter.

110 `const TARGETS = ['T1', 'T2', 'T3', 'PERIPARTUM', 'LACTATION'] as const;`
Periods offered by the period filter.

```
120 function SubstancesTable() {
```
« Substances » tab: search and filters, and approving every version shown in review at once.

```
130 const requested = useDebounced(query);
```
Typing in the search box asks the server once the text settles, not at every key.

```
138 const set = (patch: Partial<SubstanceQuery>) => {
```
Changes one filter; empty values are dropped, and the conduct filter needs a period.

```
283 <Button variant="primary" onClick={() => setApproving(true)}>
```
« Approuver les N affichées »: approves every version in review shown, with one reason.

```
331 function SourcesTable() {
```
« Registre des sources » tab: the sources profiles cite.

```
367 <Button size="sm" variant="ghost" onClick={() => setEditing(row)}>
```
« Modifier » this source.

```
394 <Button size="sm" onClick={() => setEditing('new')}>
```
« Ajouter une source ».

```
417 function ReportsTable() {
```
« Signalements » tab: errors reported by users, open ones first.

```
466 <Button size="sm" onClick={() => setResolving(row)}>
```
« Traiter »: closes the report with what was done (the row itself opens the substance).

problems.ts · 63 lines

```
5  const FIELD_LABELS: Record<string, MessageKey> = {
```
Fields of a profile named by the review gate (`pregnancy.content.ts`, `reviewProblems`).

```
16 function place(path: string, t: Translate): string {
```
Where a problem is: « 1er trimestre · Niveau de preuve », « Allaitement », « Sources ».

```
38 function what(message: string, t: Translate): string {
```
What is missing, in words; an unknown message (newer server) is shown as the server wrote it.

```
58 export function describeProblem(problem: string, t: Translate): string {
```
« 1er trimestre · Niveau de preuve : à renseigner ».

profile-editor.tsx · 802 lines

```
38 type Conflict = NonNullable<PregnancyBlock['conflict']>;
```
A disagreement between two sources.

```
41 function cleanText(text: Localized | undefined): Localized | undefined {
```
Drops empty texts so the server receives only what the author wrote.

```
49 function keepImported<T extends Period | LactationBlock>(value: T) {
```
Import traceability (ADR-025) is kept as is: status word, period sources and alternatives.

```
60 function cleanPeriod(period: Period): Period {
```
A period without its empty optional parts (sent to the API).

```
82 function cleanConflict(conflict: Conflict | undefined): Conflict | undefined {
```
A conflict without empty parts, or nothing when unset.

```
144 function LocalizedField({
```
French (required when the field is used) and English side by side.

```
189 function ConductSelect({
```
The conduct picker of a period.

```
222 function EvidenceFields({
```
Evidence level and its note.

```
271 function AlternativesField({
```
Other substances, as chips; a substance is never its own alternative.

```
301 <button
```
Removes this alternative.

336 `function ConflictFields({`
Two references that disagree, and the reviewers' synthesis (info Alert for doctors).

403 `function PeriodFields({`
The fields of one pregnancy period.

473 `function PregnancyFields({`
The pregnancy block: four periods and its alternatives.

530 `function LactationFields({`
The lactation block.

648 `export function ProfileEditor({`
Profile editor (ScreenDrugEditorAdmin pattern): tabs mirror what doctors read, structured fields, « Prévisualiser comme médecin ». Saving keeps a draft; publishing never happens here.

670 `const selected = useMemo(`
The registry sources ticked for this version.

682 `const save = useMutation({`
Saves the draft: content without empty parts, and the sources ticked.

711 `<Button`
« Enregistrer le brouillon ».

781 `<Button size="sm" onClick={onAddSource}>`
« Ajouter une source » to the registry, without leaving the editor.

profile-view.tsx · 435 lines

31 `export function ConductBadge({ conduct }: { conduct: Conduct }) {`
The conduct badge: an instruction, as icon + word (ScreenPregnancyDesktop).

43 `export function EvidenceMeter({`
Four neutral segments, the level word and the population note — never a colour verdict.

72 `function Labelled({ label, text }: { label: string; text: Localized | undefined }) {`
A labelled text in the UI language (nothing when empty).

84 `function NoData({ alternatives }: { alternatives: string[] }) {`
« Absence de données — ce n'est pas une preuve d'innocuité »: a full block, never empty.

99 `function PeriodTrace({`
What an import recorded for one period (ADR-025): the source's own word, its links, pending names.

145 `function Alternatives({ names }: { names: string[] }) {`
The alternatives of a period or block.

163 `function Conflict({`
Two sources that disagree, and the synthesis written.

186 `export function EvidenceFooter({`
Sources (title · edition), reviewer and date, version — the mandatory evidence footer.

212 `function PregnancyView({`
The pregnancy block: one card per period.

311 `function LactationView({`
The lactation block.

368 `export function ProfileView({`
A profile as doctors read it (ScreenPregnancyDesktop, ScreenLactationMobile, EvidenceBlock): « Prévisualiser comme médecin » and the review screen.

378 `nameOf: (id: string) => string;`
Substance names for alternative ids.

424 `export function ReviewComment({ comment }: { comment: string }) {`
Shown to the author when the reviewer sent the version back.

substance-detail.tsx · 547 lines

```
38  type Action = 'newVersion' | 'submit' | 'discard' | 'approve' | 'changes' | null;
```
Which dialog is open.

```
41  export function SubstanceDetailPage({ id }: { id: string }) {
```
One substance: versions on the left, the selected version (editor, review) on the right.

```
58  const detail = useQuery({
```
The substance with its versions.

```
64  const substances = useQuery({
```
Every substance (alternatives picker and merge target list).

```
69  const registry = useQuery({
```
The sources registry (the editor's source picker).

```
79  const version = useQuery({
```
The selected version with its content and review problems.

```
139 <Button onClick={() => setEditing(true)}>
```
« Modifier la substance »: names, ATC code, synonyms.

```
147 <Button
```
« Nouvelle version »: a new draft profile (one open version at a time, never on a merged substance).

```
168 <Button onClick={() => setUnmerging(true)}>{t('admin.pregnancyMerge.undo')}</Button>
```
« Annuler la fusion ».

```
200 <Button className="ms-auto" onClick={() => setMerging(true)}>
```
« Fusionner avec… »: this substance is a duplicate of another one.

```
247 <button
```
One version of the timeline: shows it below.

```
311 <Button
```
« Soumettre à relecture »: possible once nothing blocks the review.

```
320 <Button variant="ghost" onClick={() => setAction('discard')}>
```
« Supprimer le brouillon ».

```
329 <Button variant="primary" onClick={() => setAction('approve')}>
```
« Approuver et publier » (or first / second sign-off).

```
333 <Button onClick={() => setAction('changes')}>
```
« Demander des modifications »: back to the author with a comment.

```
508 function VersionHeader({ version }: { version: ProfileVersion }) {
```
Lifecycle of the selected version: who wrote it, who reviewed it, when it went live.

apps/admin/src/features/recovery/

Assisted account recovery: the queue by status, the comparison drawer, watermarked photos, approve and reject.

labels.ts · 67 lines

```
6   export type RecoveryStatus = Schemas['RecoveryStatus'];
```
API enums used by the recovery pages.

```
11  export const recoveryKey = ['admin', 'account-recoveries'] as const;
```
Cache key of the recovery pages.

```
14  export const RECOVERY_TABS: RecoveryStatus[] = [
```
The queue tabs, in the order a reviewer works them.

```
23  export const RECOVERY_STATUS_LABELS: Record<RecoveryStatus, MessageKey> = {
```
Catalogue key of each status.

```
34  export const RECOVERY_STATUS_TONES: Record<RecoveryStatus, BadgeTone> = {
```
Badge colour of each status.

45 `export const RECOVERY_REJECTION_REASONS: RecoveryRejectionReason[] = [`
Rejection reasons, in the picker's order.

54 `export const RECOVERY_REJECTION_LABELS: Record<RecoveryRejectionReason, MessageKey> = {`
Catalogue key of each rejection reason.

63 `export const RECOVERY_SLOT_LABELS: Record<RecoveryPhotoSlot, MessageKey> = {`
Catalogue key of each photo.

recovery-page.tsx · 440 lines

35 `type Row = Schemas['AdminRecoveryDto'];`
One request in the list.

37 `type Detail = Schemas['AdminRecoveryDetailDto'];`
A request with the matching account, for the drawer.

44 `export function RecoveryPage() {`
« Récupérations de compte » (ADR-023): people who lost both their e-mail and phone. The reviewer compares the identity photo and the selfie with the account; an approval applies after 24 hours, during which the owner can still stop it by signing in.

51 `const list = useQuery({`
The requests of the selected status.

57 `const columns = useMemo<Column<Row>[]>(`
The list's columns.

108 `<Button`
Status tab (submitted, approved, applied, rejected…).

145 `function RecoveryDrawer({ requestId, onClose }: { requestId: string | null; onClose: () => void }) {`
The drawer of one request: what the person said beside the account, the two photos, and the decision.

183 `<Button variant="danger" onClick={() => setDecision('reject')}>`
« Rejeter »: refuses the request (also during the 24-hour hold of an approval).

188 `<Button`
« Approuver »: possible only when an account matches; it applies after 24 hours.

264 `<Button`
Opens one photo (identity document or selfie) for 60 seconds; audited.

360 `function Compare({ record }: { record: Detail }) {`
What the person said beside what the account holds: the reviewer compares the two.

recovery-photo.tsx · 121 lines

15 `const WATERMARK_TILES = 12;`
How many times the watermark text is repeated over the photo.

22 `export function RecoveryPhoto({`
A recovery photo behind a 60-second signed URL, watermarked with the viewer's name and time, without download, copy or drag (same rules as the verification viewer). Each opening is audited by the API.

95 `<Button size="sm" onClick={() => void view.refetch()}>`
« Rouvrir »: a new 60-second link (a new audit entry).

apps/admin/src/features/search/

The synonyms editor: extra words search should match for a model or a calculator.

synonyms-editor.tsx · 163 lines

17 `type EntityType = Schemas['SearchEntityType'];`
What the synonyms belong to.

23 `export function addTerm(terms: readonly string[], raw: string): string[] {`
Adds a term unless it is blank, too long, a case-insensitive duplicate, or over the limit.

```
35  export function SynonymsEditor({
```
Search synonyms of one model or calculator (section 12): lay terms, abbreviations, frequent misspellings, FR and EN. Saving replaces the whole list and asks for a reason.

```
50  const saved = useQuery({
```
The synonyms saved now.

```
60  const [edited, setEdited] = useState<string[] | null>(null);
```
Unsaved edits; null shows the saved list.

```
66  const save = useMutation({
```
Replaces the synonyms (with a reason).

```
101 <button
```
Removes this synonym from the list (not saved yet).

```
130 <Button onClick={add} disabled={!draft.trim() || terms.length >= MAX_SYNONYMS}>
```
« Ajouter » the typed synonym.

```
136 <Button
```
« Enregistrer »: asks for a reason, then saves the list.

```
146 <Button size="sm" variant="ghost" onClick={() => setEdited(null)}>
```
« Annuler »: back to the saved list.

apps/admin/src/features/settings/

Settings: the pickers of the app (specialties, institutions, universities) and the double sign-off policies.

reference-data-page.tsx · 301 lines

```
19  type Specialty = Schemas['SpecialtyDto'];
```
Picker entries as the API returns them.

```
22  type PlaceKind = 'institutions' | 'universities';
```
The two kinds of place list.

```
25  const referenceKey = (kind: string) => ['admin', 'reference-data', kind] as const;
```
Cache key of one list.

```
28  const WILAYA_PATTERN = /^\d{1,3}$/;
```
Wilaya codes as the API accepts them.

```
36  export function ReferenceDataPage() {
```
« Paramètres système » — reference data of the onboarding and verification pickers: specialties, institutions (by wilaya) and universities, added with a reason, never typed by users — and the clinical review rule (ADR-024).

```
66  function Specialties() {
```
The specialties list and its form.

```
76  const list = useQuery({
```
Every specialty.

```
122 <Button
```
« Ajouter une spécialité »: opens an empty form.

```
190 function Places({ kind }: { kind: PlaceKind }) {
```
Institutions or universities, and their form.

```
203 const list = useQuery({ queryKey: referenceKey(kind), queryFn: () => call(api.GET(path)) });
```
Every institution or university.

```
250 <Button
```
« Ajouter »: opens an empty institution or university form.

review-policies.tsx · 114 lines

```
17  type Policy = Schemas['ReviewPolicyDto'];
```
A sign-off policy as the API returns it.

32 `export function ReviewPolicies() {`
« Relecture clinique » (ADR-024): one review by default — the person who publishes, never the author — or two independent reviews, per kind of clinical content. Every change asks for a reason and is audited.

36 `const policies = useQuery({`
Both policies (calculators, pregnancy profiles).

59 `function PolicyCard({ policy }: { policy: Policy }) {`
One policy: one or two sign-offs, how many admins may approve, who last changed it.

apps/admin/src/features/subscriptions/

Plans and subscriptions: the versioned catalogue, the plan drawer, the subscriptions list.

labels.ts · 47 lines

18 `export const SUBSCRIPTION_STATUS_TONES: Record<SubscriptionStatus, BadgeTone> = {`
Calm tones only (Lock L39): no red for a subscription.

27 `export const PROVIDER_LABELS: Record<PaymentProvider, MessageKey> = {`
A channel is gateway × card (ADR-021): « Chargily · CIB ».

plan-drawer.tsx · 225 lines

27 `export function PlanDrawer({`
Publishes a plan version (ScreenSubscriptionsAdmin « Offres »). Editing never changes a sold version: it publishes the next one; running subscriptions keep theirs.

34 `base: Plan | null;`
The version to start from; null for a new plan code.

83 `const list = (name: 'includesFr' | 'includesEn' | 'excludesFr' | 'excludesEn', label: string) => (`
One "includes / not included" list (one item per line).

117 `<Button onClick={onClose} disabled={isSubmitting}>`
« Annuler ».

122 `<Button variant="primary" busy={isSubmitting} onClick={() => void submit()}>`
« Publier »: a new plan or version; earlier versions of the code stop being sold.

plan-form.ts · 108 lines

8 `export const PLAN_LIMITS = {`
API bounds of a published plan (PublishPlanDto).

17 `export function linesOf(text: string): string[] {`
One line per included / excluded item, blank lines ignored.

33 `export function parsePrice(text: string): number | null {`
"1 500,50" or "1500.5" DZD → 150050 minor units; null when not a price.

70 `export function planFormValues(plan: Plan | null): PlanFormInput {`
Form values for a new version of `plan`, or for a new plan code.

89 `export function toPublishBody(values: PlanFormOutput): Schemas['PublishPlanDto'] {`
The form's values as the API expects them (prices in centimes, lists without empty lines).

subscriptions-list.tsx · 161 lines

24 `type Subscription = Schemas['AdminSubscriptionDto'];`
One subscription row.

30 `export function SubscriptionsList() {`
« Abonnements » tab: who holds which plan version, until when, last payment (section 07).

35 `const list = useInfiniteQuery({`
Pages of subscriptions for the filters.

149 `<Button`
« Charger plus ».

subscriptions-page.tsx · 212 lines

```
26  export function SubscriptionsPage() {
```
« Abonnements » — Offres (ScreenSubscriptionsAdmin): the versioned plan catalogue.

```
37  const plans = useQuery({ queryKey: plansKey, queryFn: () => call(api.GET('/admin/v1/plans')) });
```
Every version of every plan.

```
39  const latestByCode = useMemo(() => {
```
The latest version number of each plan code (only it can be the base of a new version).

```
109 onSelect: () => setDrawer({ base: row }),
```
« Nouvelle version » of this plan (a price change is a new version).

```
115 onSelect: () => setSale(row),
```
« Mettre en vente » / « Retirer de la vente ».

```
141 <Button variant="primary" onClick={() => setDrawer({ base: null })}>
```
« Nouveau forfait ».

apps/admin/src/features/transactions/

The payments ledger and its drawer with fees and exceptional adjustments (two admins).

transaction-drawer.tsx · 363 lines

```
30  type Detail = Schemas['AdminTransactionDetailDto'];
```
API types used by the transaction drawer.

```
47  export function TransactionDrawer({
```
Transaction drawer: every field, the linked plan and person, adjustments and history. The `•••` holds « Ajustement exceptionnel… », Super Admin only, approved by another admin; it is not a refund and has no tab, list or status of its own (section 15).

```
63  const detail = useQuery({
```
The payment with its fee and adjustments.

```
80  const money = (minor: number) => formatMoney(locale, minor, row?.currency ?? 'DZD');
```
An amount in the payment's currency.

```
111 onSelect: () => setAdjusting(true),
```
« Ajustement exceptionnel… »: a reversal or correction that counts only once a second admin approves it.

```
200 <Button
```
« Approuver » this adjustment (never by the admin who requested it).

```
261 function AdjustmentDialog({
```
The exceptional adjustment form: kind, amount, reason code and note.

transactions-page.tsx · 226 lines

```
27  type Transaction = Schemas['AdminTransactionDto'];
```
One payment row.

```
35  export function TransactionsPage() {
```
« Transactions » (ScreenTransactions): the immutable payment ledger; no refund anywhere.

```
45  const ledger = useInfiniteQuery({
```
Pages of payments for the filters and search.

```
213 <Button
```
« Charger plus ».

apps/admin/src/features/users/

Users: the searchable list and the user drawer (profile, access, subscriptions, activity counts, history).

labels.ts · 69 lines

```
5   export type EntitlementTier = Schemas['EntitlementTier'];
```
Access tiers, as the API names them.

10 `export const TIER_LABELS: Record<EntitlementTier, MessageKey> = {`
Catalogue key of each tier's name.

19 `export const FEATURE_LABELS: Record<FeatureKey, MessageKey> = {`
Catalogue key of each feature's name.

32 `export const LOCK_REASON_LABELS: Record<LockReason, MessageKey> = {`
Catalogue key of each lock reason.

44 `export const ACTIVITY_KIND_LABELS: Record<Schemas['ActivityKind'], MessageKey> = {`
Catalogue key of each kind of history entry.

52 `export const FAVORITE_TYPE_LABELS: Record<Schemas['FavoriteTargetType'], MessageKey> = {`
Catalogue key of each kind of favorite.

59 `export const usersKey = ['admin', 'users'] as const;`
Cache key of the user list (invalidated after an action).

65 `export function personName(person: { firstName: string; lastName: string }, t: Translate): string {`
A person's name in the ledgers. A deleted account is an anonymous tombstone whose payments are kept (ADR-005): it reads « Compte supprimé ».

user-drawer.tsx · 402 lines

31 `type Detail = Schemas['AdminUserDetailDto'];`
The user drawer data.

33 `type Action = 'suspend' | 'reinstate' | 'signOut' | null;`
Which confirmation dialog is open.

36 `export function UserDrawer({ userId, onClose }: { userId: string | null; onClose: () => void }) {`
User drawer: Profil · Accès · Abonnement · Activité · Historique (section 07); no documents.

185 `function Profile({ user }: { user: Detail }) {`
« Profil » tab: contacts, wilaya, profession and its details.

246 `function Access({ user }: { user: Detail }) {`
The derived AccessProfile and its five inputs, read-only (section 27).

335 `function Subscriptions({ user }: { user: Detail }) {`
« Abonnement » tab: every subscription with its period and last payment (needs subscriptions.read).

377 `function Activity({ user }: { user: Detail }) {`
Counts by module only — never which searches, models or calculators (section 07).

users-page.tsx · 210 lines

22 `type User = Schemas['AdminUserDto'];`
One row of the user list.

26 `const PAGE_SIZE = 50;`
Rows per page of the API (a full page means there may be more).

29 `export function UsersPage() {`
« Utilisateurs » (ScreenUsersAdmin): who uses DokTool, with their verification and plan.

39 `const users = useInfiniteQuery({`
Pages of users for the current search and filters; the next page starts after the last id.

60 `const columns = useMemo<Column<User>[]>(`
The table's columns (rebuilt when the language changes).

197 `<Button`
« Charger plus »: the next 50 users.

apps/admin/src/features/verification/

Verification review: the queue, the case drawer, claim and release, the restricted viewer, approve / reject / release dialogs, SLA badge.

case-drawer.tsx · 384 lines

```
32  type Case = Schemas['AdminVerificationCaseDto'];
```
A case as the review drawer shows it.

```
35  type Decision = 'approve' | 'reject' | 'release' | null;
```
Which decision dialog is open.

```
38  export function CaseDrawer({ caseId, onClose }: { caseId: string | null; onClose: () => void }) {
```
Review drawer: Résumé · Documents (restricted) · Historique, and the decisions (section 07).

```
64  const claim = useMutation({
```
« Prendre le dossier »: the case becomes IN_REVIEW for this reviewer.

```
84  <Button variant="primary" busy={claim.isPending} onClick={() => claim.mutate(record.id)}>
```
« Prendre le dossier ».

```
91  <Button onClick={() => setDecision('release')}>{t('admin.verification.release')}</Button>
```
« Libérer »: gives the case back to the queue.

```
94  <Button variant="danger" onClick={() => setDecision('reject')}>
```
« Rejeter »: opens the rejection dialog.

```
98  <Button variant="primary" onClick={() => setDecision('approve')}>
```
« Approuver »: opens the approval dialog.

```
186 function Summary({ record }: { record: Case }) {
```
« Résumé » tab: the person, the declared details, the dates and which steps were provided.

```
298 function Documents({ record, mine }: { record: Case; mine: boolean }) {
```
« Documents » tab: listed only with the time-limited documents grant; opened only by the reviewer holding the case.

```
332 <Button size="sm" disabled={!mine} onClick={() => setOpen(document)}>
```
« Ouvrir »: shows the document in the restricted viewer (audited).

```
352 function History({ record }: { record: Case }) {
```
« Historique » tab: the person's earlier cases of this type.

decisions.tsx · 166 lines

```
11  type Case = Schemas['AdminVerificationCaseDto'];
```
The case a decision is about.

```
14  interface DecisionProps {
```
What every decision dialog receives.

```
22  const path = (record: Case) => ({ params: { path: { id: record.id } } });
```
URL parameters of the case.

```
25  export function ApproveDialog({ record, open, onOpenChange, onDone }: DecisionProps) {
```
« Approuver »: optional end of validity (the document's own expiry), and the reason.

```
79  export function RejectDialog({ record, open, onOpenChange, onDone }: DecisionProps) {
```
« Rejeter »: reason from the closed list, and the note shown to the person saying which step to redo — this is how a reviewer asks for a complement.

```
144 export function ReleaseDialog({ record, open, onOpenChange, onDone }: DecisionProps) {
```
« Libérer »: back to the queue for another reviewer.

keys.ts · 6 lines

```
2   export const queueKey = ['admin', 'verification', 'queue'] as const;
```
Cache key of the queue.

```
5   export const caseKey = (id: string) => ['admin', 'verification', 'case', id] as const;
```
Cache key of one case.

labels.ts · 113 lines

```
6   export type VerificationStatus = Schemas['VerificationStatus'];
```
API enums used by the verification pages.

13 `export const VERIFICATION_SLA_HOURS = 24;`
Verification service level: a case waiting longer is flagged (Admin Web preview « SLA 24 h »).

16 `export const TYPE_LABELS: Record<VerificationType, MessageKey> = {`
Catalogue key of each case type.

23 `export const STATUS_LABELS: Record<VerificationStatus, MessageKey> = {`
Catalogue key of each case status.

34 `export const STATUS_TONES: Record<VerificationStatus, BadgeTone> = {`
Badge colour of each case status.

45 `export const REJECTION_LABELS: Record<RejectionReason, MessageKey> = {`
Catalogue key of each rejection reason.

55 `export const REJECTION_REASONS = Object.keys(REJECTION_LABELS) as RejectionReason[];`
The rejection reasons, in the order the picker lists them.

58 `export const STEP_LABELS: Record<StepType, MessageKey> = {`
Catalogue key of each document step.

67 `export const PROFESSIONAL_TYPE_LABELS: Record<ProfessionalType, MessageKey> = {`
Catalogue key of each profession.

74 `export const DOCUMENT_KIND_LABELS: Record<`
Catalogue key of each identity document kind.

84 `export const QUEUE_TABS = ['PENDING', 'VERIFIED', 'REJECTED', 'EXPIRED', 'APPEALED'] as const;`
Queue tabs (section 07): « En attente » is the API default (submitted, in review, appealed).

88 `export const QUEUE_TAB_LABELS: Record<QueueTab, MessageKey> = {`
Catalogue key of each tab.

97 `export function queueStatus(tab: QueueTab): VerificationStatus | undefined {`
The status a tab asks the API for (none: the default waiting list).

102 `export function personName(person: { firstName: string; lastName: string }): string {`
A person's full name.

107 `export function waitingSince(item: {`
The moment a case started waiting: its appeal, or its submission.

restricted-viewer.tsx · 143 lines

16 `type Document = Schemas['AdminVerificationDocumentDto'];`
A document of the case.

19 `const WATERMARK_TILES = 12;`
How many times the watermark text is repeated over the document.

26 `export function RestrictedViewer({`
RestrictedViewer: an identity document behind a 60-second signed URL, watermarked with the viewer's name and time, without download, print, copy or drag. Each opening is audited by the API; once the URL expires the document is hidden until it is opened again.

38 `const [expiredUrl, setExpiredUrl] = useState<string | null>(null);`
The signed URL whose 60 seconds are over (a new opening brings a new URL).

41 `const view = useQuery({`
Not cached: every opening asks the API again (a fresh audit entry and signed URL).

64 `useEffect(() => {`
Printing and copying are refused while a document is on screen.

106 `<Button size="sm" onClick={() => void view.refetch()}>`
« Rouvrir »: asks for a new 60-second link (a new audit entry).

sla-badge.tsx · 28 lines

10 `export function SlaBadge({ since, waiting }: { since: string | null; waiting: boolean }) {`
« Soumis »: age as a word plus colour; past the SLA it also says so (never colour alone).

verification-page.tsx · 204 lines

32 `type QueueItem = Schemas['AdminVerificationQueueItemDto'];`
One row of the review queue.

35 `const PAGE_SIZE = 50;`
The API returns 50 cases per page, oldest first.

39 `export function VerificationPage() {`
« Vérification » (ScreenVerificationCenter): queue by status, oldest first, row → drawer.

47 `const queue = useInfiniteQuery({`
Pages of cases for the selected tab and type, oldest first.

69 `const columns = useMemo<Column<QueueItem>[]>(`
The queue's columns.

137 `<button`
Queue tab (En attente, Vérifiés, Rejetés, Expirés, Appels).

164 `<button`
Type filter chip « Tous ».

174 `<button`
Type filter chip: identity, professional status or student status.

191 `<Button`
« Charger plus »: the next 50 cases.

apps/admin/src/i18n/

Languages: the FR/EN catalogue (`admin.*` keys), the provider and `useI18n`, formatting of dates, numbers, money and sizes.

format.ts · 62 lines

9 `export function formatDate(locale: Locale, value: string | Date): string {`
A date in the UI language ("12 oct. 2026").

14 `export function formatDateTime(locale: Locale, value: string | Date): string {`
A date and time in the UI language.

21 `export function formatNumber(locale: Locale, value: number, maximumFractionDigits = 6): string {`
A number in the UI language (spaces or commas for thousands).

26 `export function formatMoney(locale: Locale, minor: number, currency: string): string {`
Minor units (centimes) → "1 500,00 DZD" / "DZD 1,500.00".

31 `export function formatBytes(locale: Locale, bytes: number): string {`
A file size in B, kB or MB.

47 `export function formatAge(locale: Locale, from: string | Date, now: Date = new Date()): string {`
Time elapsed since `from`, in the largest whole unit: "45 min", "6 h", "3 j".

59 `export function hoursSince(from: string | Date, now: Date = new Date()): number {`
Hours elapsed since a moment (SLA badges).

i18n-provider.tsx · 44 lines

7 `interface I18nValue {`
What usel18n gives a component: the language, the translator and how to switch language.

18 `export function I18nProvider({`
Holds the UI language; switching it saves a cookie and retranslates every page at once.

39 `export function useI18n(): I18nValue {`
The language and translator of the page.

messages.ts · 38 lines

```
6   export const LOCALES = ['fr', 'en'] as const;
```
The two UI languages.

```
10  export const LOCALE_COOKIE = 'doktool_admin_locale';
```
Admin Web's UI language, remembered per browser (read on the server for `<html lang>`).

```
13  export type MessageKey = Extract<keyof typeof fr, `admin.${string}` | `common.${string}`>;
```
Keys Admin Web may use: its own namespace and the shared one (packages/i18n README).

```
20  export function isLocale(value: unknown): value is Locale {
```
True for "fr" or "en".

```
25  export function createTranslator(locale: Locale): Translate {
```
ICU formatter over the shared catalogue; compiled messages are cached per translator.

apps/admin/src/

Root of Admin Web: `proxy.ts` sends the browser to sign-in when there is no session cookie.

proxy.ts · 23 lines

```
10  export function proxy(request: NextRequest) {
```
Console pages need an admin session cookie; without one the browser goes to sign-in first. The API still validates the session on every call — this only avoids rendering a shell.

```
19  export const config = {
```
Which URLs the proxy runs on: every console page, never the API.

apps/admin/src/session/

Who is signed in (`useMe`) and what they may do (`useGrants`); safe redirect after sign-in.

session.ts · 40 lines

```
7   export type AdminMe = Schemas['AdminMeDto'];
```
The signed-in admin, as GET /admin/v1/me returns it.

```
11  export const SESSION_COOKIE = '__Host-doktool_admin';
```
Name of the httpOnly admin session cookie set by the API (ADR-009).

```
14  export const meQueryKey = ['admin', 'me'] as const;
```
Cache key of the signed-in admin.

```
17  export function useMe() {
```
Loads the signed-in admin (cached for a minute); a 401 sends the browser back to sign-in.

```
26  export function useGrants(): { has: (...grants: Grant[]) => boolean; me: AdminMe | undefined } {
```
Effective grants of the signed-in admin; the navigation and every action read them.

```
35  export function safeNext(value: string | null): string {
```
A `next` target is only ever a console page of this origin (no open redirect).

apps/admin/src/shell/

The console frame: top bar, navigation list filtered by grants, and `RequireGrant` for pages opened without their grant.

console-shell.tsx · 130 lines

```
18  const ENVIRONMENTS: Record<string, MessageKey> = {
```
Label of each environment shown in the top bar.

```
25  const environment = ENVIRONMENTS[process.env.NEXT_PUBLIC_DOKTOOL_ENVIRONMENT ?? 'development'];
```
Environment label of the console bar, set at build time (NEXT_PUBLIC_DOKTOOL_ENVIRONMENT).

```
31  export function ConsoleShell({ children }: { children: ReactNode }) {
```
The console frame: top bar (environment, language, account menu) and the navigation the admin's grants allow.

62 `<button`
Language button: switches the console between French and English.

nav.ts · 151 lines

21 `export interface NavItem {`
One link of the navigation.

26 `grant: Grant | null;`
Shown when the admin holds this grant (a missing grant hides the item, section 02); null for every admin.

30 `export interface NavGroup {`
A titled group of links.

36 `export const NAVIGATION: NavGroup[] = [`
Admin navigation (design system section 02).

134 `export function visibleNavigation(grants: readonly Grant[]): NavGroup[] {`
Groups and items the admin may see.

142 `export function homeFor(grants: readonly Grant[]): string | null {`
The first section an admin can open (landing page of `/admin`).

147 `export function isActive(item: NavItem, pathname: string): boolean {`
`/admin/library/certificates` belongs to the library item, `/admin/calculators/x` to calculators.

require-grant.tsx · 23 lines

10 `export function RequireGrant({ grant, children }: { grant: Grant; children: ReactNode }) {`
A section opened without its grant (typed URL) says so instead of failing API calls.

apps/admin/src/test/

Admin Web tests (Vitest): the data table, helpers, pregnancy review problems.

data-table.test.tsx · 104 lines

helpers.test.ts · 265 lines

76 `expect(homeFor([])).toBe('/admin/overview');`
The overview needs no grant: every admin lands there first.

pregnancy-problems.test.ts · 36 lines

apps/admin/src/ui/

The console kit: buttons, badges, tables, dialogs, drawers, fields, menus, tabs, toasts, charts, period picker, PDF pages, debounce.

badge.tsx · 11 lines

4 `export type BadgeTone =`
Colours a badge can take.

8 `export function Badge({ tone = 'neutral', children }: { tone?: BadgeTone; children: ReactNode }) {`
Status and age cells: always a word (and icon), never colour alone.

button.tsx · 45 lines

5 `export type ButtonVariant = 'primary' | 'secondary' | 'ghost' | 'danger' | 'teal';`
Visual styles of a button (primary, secondary, ghost, danger, teal).

8 `interface ButtonProps extends ButtonHTMLAttributes<HTMLButtonElement> {`
A button: style, size, icon-only, and a busy spinner that also disables it.

12 `icon?: boolean;`
Square icon button; `aria-label` is then required.

18 `export function Button({`
The console's button; disabled while busy so a click is never sent twice.

charts.tsx · 210 lines

18 `export const SERIES_TOKENS = ['--brand', '--teal', '--viz-3', '--viz-5'] as const;`
Series colours, in order (tokens: brand, teal, viz-3, viz-5; previous period is viz-4).

23 `export interface Series {`
One line or bar series of a chart.

27 `previous?: boolean;`
Previous-period series: dashed `viz-4` (section 16).

31 `function readTokens(): Record<string, string> {`
Reads the colour tokens from the page's CSS.

44 `function useTokens(): Record<string, string> | null {`
Token colours resolved once in the browser (SVG attributes cannot read CSS variables everywhere).

55 `function colorOf(tokens: Record<string, string>, series: Series[], index: number): string {`
The colour of a series: dashed grey for the previous period, otherwise the next palette colour.

63 `interface ChartProps<T> {`
Data and labels of a chart.

65 `label: string;`
Accessible name; also the caption of the hidden data table.

72 `formatAxis?: (value: number) => string;`
Short labels for the value axis (e.g. « 23 k »); the tooltip and the table use `formatY`.

77 `function DataTableFallback<T>({ label, data, xKey, series, formatX, formatY }: ChartProps<T>) {`
The chart's data as a table for screen readers (every chart has one, section 16).

114 `function axes<T>(tokens: Record<string, string>, props: ChartProps<T>) {`
Grid, axes, tooltip and legend shared by both chart kinds.

151 `export function BarSeriesChart<T>(props: ChartProps<T> & { stacked?: boolean }) {`
Bars only or lines only (section 07): no pies, no areas.

180 `export function LineSeriesChart<T>(props: ChartProps<T>) {`
A line chart (previous period dashed), with its data table for screen readers.

data-table.tsx · 251 lines

20 `export interface Column<T> {`
One column of a DataTable: header, how to draw a cell, optional sort value.

25 `sortValue?: (row: T) => string | number | null;`
Makes the column sortable by this value.

28 `hideable?: boolean;`
Can be hidden from the column chooser (non-essential columns, 1024–1439 px).

32 `interface DataTableProps<T> {`
What a table shows and what a click on a row does.

42 `toolbar?: ReactNode;`
Toolbar content shown at the start of the table header bar (filters, search).

46 `const features = tableFeatures({`
TanStack Table features used: sorting and column visibility.

58 `function fromControl(event: { target: EventTarget; currentTarget: EventTarget }): boolean {`
True when an event on a row started on a control inside it (a « Traiter » button, a link): the control acts alone and the row does not open.

70 `export function DataTable<T extends RowData>({`
The console's primary element (component DataTable): dense 36 px rows, sticky header, sort, column chooser; a row opens the drawer, ↑/↓ move between records.

108 `const onRowKey = (event: KeyboardEvent<HTMLTableRowElement>, index: number) => {`
Keyboard on a row: Enter or Space opens it, ↑/↓ move to the next row (and follow with the open drawer).

135 `<Button size="sm" variant="secondary">`
« Colonnes »: chooses which optional columns are shown.

185 `<button type="button" onClick={header.column.getToggleSortingHandler()}>`
Header button: sorts by this column (ascending, descending, none).

dialog.tsx · 72 lines

7 `interface DialogProps {`
A centred dialog: title, optional description, body and footer buttons.

16 `onSubmit?: () => void;`
Wraps body and footer (a form submits from its footer button).

23 `export function Dialog({`
A modal dialog (Radix): focus stays inside, Escape closes; with onSubmit, Enter submits the form.

drawer.tsx · 81 lines

10 `interface DrawerProps {`
What a drawer shows and how it behaves.

17 `mode: 'inspect' | 'edit' | 'review';`
`inspect` keeps the list usable (non-modal); `edit` and `review` are modal.

19 `dirty?: boolean;`
Unsaved changes: closing asks for confirmation first.

25 `export function Drawer({`
Right-hand drawer (480 px, 560 px from 1440 px) opened from a table row (component Drawer).

errors.ts · 37 lines

8 `const KNOWN: Partial<Record<ApiFailure['code'], MessageKey>> = {`
API error codes and the sentence the console shows for each.

30 `export function useErrorText(): (error: unknown) => string {`
Error → sentence in the UI language (the server's own message is never shown).

feedback.tsx · 95 lines

5 `type AlertTone = 'critical' | 'warning' | 'info' | 'success' | 'neutral';`
Kinds of alert.

16 `export function Alert({`
A coloured message box with an icon; critical ones are announced to screen readers.

41 `export function EmptyState({`
What a page shows when it has nothing (or failed to load).

65 `export function Skeleton({ lines = 3 }: { lines?: number }) {`
Grey placeholder lines while data loads.

76 `export function PageHeader({`
Page title, caption and the page's action buttons.

field.tsx · 137 lines

16 `interface FieldProps {`
A form field: label, required mark, help or error text, and the control (given the ids to wire).

29 `export function Field({ label, required, help, error, children }: FieldProps) {`
Label + control + help/error, wired for assistive technology.

61 `type Invalid = { invalid?: boolean; unit?: ReactNode; mono?: boolean };`
Extra options of the inputs below.

64 `export const TextInput = forwardRef<`
A text input with an optional unit at its end.

83 `export const TextArea = forwardRef<`
A multi-line input (monospace for JSON).

95 `export const Select = forwardRef<`
A drop-down list.

109 `export function Switch({`
An on/off switch (role="switch", readable by screen readers).

menu.tsx · 45 lines

9 `export interface MenuAction {`
One entry of an overflow menu.

18 `export function OverflowMenu({ label, actions }: { label: string; actions: MenuAction[] }) {`
The `•••` overflow menu of a drawer or a row. Hidden actions (missing grant) are not listed.

metric.tsx · 69 lines

11 `export function Metric({`
One figure of a ledger strip (component MetricTile): label, value, change against the previous period as a word and an arrow (never colour alone), optional note (« estimé »).

62 `export function useCellText() {`
« < 10 » for suppressed analytics cells.

pdf-pages.tsx · 88 lines

6 `type Outcome = 'ready' | 'error';`
Whether the PDF could be drawn.

9 `const PAGE_WIDTH = 720;`
CSS pixels of a rendered page width; the canvas is drawn at device resolution.

15 `export function PdfPages({`
Renders a PDF (signed URL) onto canvases with pdf.js: no browser PDF toolbar, hence no download or print control (reference models section 13, identity documents in the RestrictedViewer).

period.tsx · 95 lines

9 `export const PRESETS = ['today', '7d', '30d', '3m', '12m', 'custom'] as const;`
Periods of section 15: today, 7 d, 30 d, 3 months, 12 months, custom.

11 `export type PeriodPreset = (typeof PRESETS)[number];`
One of the period presets.

23 `export interface Period {`
A period sent to the API: from (inclusive) and to (exclusive), ISO dates.

31 `export function presetPeriod(preset: Exclude<PeriodPreset, 'custom'>, now = new Date()): Period {`
The period ending now (tomorrow 00:00 UTC, so today is complete) for a preset.

38 `export function customPeriod(fromDay: string, toDay: string): Period | null {`
A custom period from two calendar days (inclusive), or null while it is incomplete or reversed.

46 `export function usePeriod(initial: Exclude<PeriodPreset, 'custom'> = '30d') {`
State of a period picker: preset or custom days, and the resulting period.

58 `export function PeriodPicker({ state }: { state: ReturnType<typeof usePeriod> }) {`
The period buttons (today, 7 d, 30 d…) and, for « Personnalisée », two date inputs.

65 `<button`
One preset button; aria-pressed marks the one selected.

reason-dialog.tsx · 110 lines

15 `export const reasonSchema = z.string().trim().min(3).max(500);`
Same bounds as the API's reason fields (3–500 characters, trimmed).

18 `interface ReasonDialogProps {`
What a reason dialog shows and does when confirmed.

26 `children?: ReactNode;`
Extra controls above the reason (e.g. the rejection reason picker).

28 `ready?: boolean;`
Disables confirmation while the extra controls are incomplete.

36 `export function ReasonDialog({`
Every decision that affects a user, money or content asks for a reason; the API writes it to the audit log with the decision (section 07).

58 `const submit = form.handleSubmit(async ({ reason }) => {`
Validates the reason, calls onConfirm, closes on success, shows the error otherwise.

78 `<Button onClick={() => onOpenChange(false)} disabled={isSubmitting}>`
« Annuler »: closes without doing anything.

82 `<Button type="submit" variant={confirmVariant} busy={isSubmitting} disabled={!ready}>`
Confirm button: submits the reason with the decision.

tabs.tsx · 35 lines

7 `export const Tabs = RadixTabs.Root;`
Tabs (Radix): keyboard arrows move between tabs.

11 `export function TabList({ label, children }: { label: string; children: ReactNode }) {`
The row of tabs, with an accessible name.

20 `export function Tab({`
One tab.

toast.tsx · 51 lines

6 `interface Toast {`
One short message at the bottom of the screen.

17 `export function ToastProvider({ children }: { children: ReactNode }) {`
Short confirmations after a decision ("Dossier approuvé"), announced politely.

46 `export function useToast() {`
Shows a toast: `toast('Dossier approuvé')`.

use-debounced.ts · 17 lines

9 `export function useDebounced<T>(value: T, delayMs = 300): T {`
`value`, once it has stopped changing for `delayMs`: a search box asks the server when the person pauses typing, not at every key.

The API (apps/api)

apps/api/src/

The server entry point (`main.ts`), the HTTP settings shared with tests (`app.setup.ts`) and the root module (`app.module.ts`).

app.module.ts · 37 lines

18 `export class AppModule {`
Root module: the infrastructure modules (config, database, jobs, crypto, rate limits, messaging, security, health) plus every domain module listed in modules/index.ts.

20 `static forRoot(env: Env): DynamicModule {`
Builds the root module for one validated environment (the tests pass their own).

app.setup.ts · 55 lines

13 `export function configureApp(app: INestApplication, env: Env): void {`
HTTP settings shared by the server entry point and the end-to-end tests.

apps/api/src/common/errors/

The error contract: `DomainError` with its stable codes, the response shape, and the global filter that renders every failure the same way.

domain-error.ts · 64 lines

7 `export const ErrorCode = {`
Stable error codes of the API contract. Clients translate them with their own catalogue; the `message` field is for developers and logs only.

40 `export interface ErrorDetails {`
Extra information the client may use to explain an error.

42 `retryAt?: string;`
When the client may retry (lockouts, rate limits, resend timers).

46 `lockReason?: string;`
FEATURE_LOCKED: the entitlement lock to render (ADR-010).

51 `export class DomainError extends Error {`
An expected, client-facing failure. Anything else is a 500.

error-response.ts · 42 lines

5 `export class ErrorDetailsBody {`
Optional details attached to an error (OpenAPI shape of ErrorDetails).

26 `export class ErrorBody {`
The `error` object: a stable code clients translate, a developer message and details.

38 `export class ErrorResponse {`
Shape of every error the API returns.

error.filter.ts · 81 lines

23 `export class ErrorFilter implements ExceptionFilter {`
Renders every failure as `{ error: { code, message, details } }`.

27 `catch(exception: unknown, host: ArgumentsHost): void {`
Called by Nest for any exception thrown while handling a request; writes the JSON error.

37 `private toBody(exception: unknown): [number, ErrorResponse] {`
Maps an exception to [HTTP status, body]: DomainError as is, validation errors as VALIDATION_FAILED, other HTTP errors by status, anything else as a logged 500.

apps/api/src/common/http/

Who is calling (IP, user agent), for rate limits and audit entries.

client-info.ts · 27 lines

```
5   export interface ClientInfo {
```
Who is calling, for audit entries and rate limits.

```
7   ip: string | null;
```
Client IP as resolved by Express (`trust proxy` = TRUST_PROXY_HOPS).

```
15  export function clientInfoOf(request: Request): ClientInfo {
```
Reads the caller's IP and a truncated user agent from an Express request (for audit and rate limits).

```
24  export const Client = createParamDecorator((_: unknown, context: ExecutionContext) =>
```
Injects the caller's IP and user agent.

apps/api/src/common/text/

Text helpers: safe CSV cells, accent-insensitive search text.

csv.ts · 14 lines

```
8   export function csvCell(value: string | number | null): string {
```
One CSV cell: quoted when it holds a separator, a quote or a line break. Text a spreadsheet would run as a formula (=, +, -, @, tab, carriage return first) gets a leading apostrophe; numbers, negative ones included, are kept as numbers.

search-text.ts · 29 lines

```
4   export function normaliseForSearch(text: string): string {
```
Accent- and case-insensitive form of a text ("Céphalée" ~ "cephalee").

```
14  export function matchesAllWords(query: string, texts: (string | null | undefined)[]): boolean {
```
True when every word of `query` appears in one of `texts`, in any order.

```
26  export function compareFrench(a: string, b: string): number {
```
French A–Z order.

apps/api/src/config/

Environment variables, validated at start-up and injectable as `ENV`.

config.module.ts · 17 lines

```
7   export class ConfigModule {
```
Makes the parsed environment injectable everywhere as {@link ENV}.

```
9   static forRoot(env: Env): DynamicModule {
```
Registers one frozen copy of the parsed environment, so nothing can change it at runtime.

env.ts · 325 lines

```
4   export const GATEWAY_IDS = ['chargily', 'satim', 'fake'] as const;
```
The payment gateways the API can talk to (ADR-021). `fake` exists only outside production.

```
7   export const SMS_GATEWAY_IDS = ['twilio', 'infobip'] as const;
```
The SMS providers the API can send through (ADR-022).

```
10  const idList = <const T extends readonly [string, ...string[]]>(
```
A comma-separated list of ids, trimmed and lower-cased, without repeats.

```
28  const booleanFlag = z
```
A "true"/"false" environment value (default true) turned into a boolean.

```
34  const secret = (name: string) =>
```
A secret of at least 32 characters; shorter values are refused at boot.

```
38  const envSchema = z
```
Runtime configuration. Parsed once at boot; the process refuses to start on invalid values.

50 `TRUST_PROXY_HOPS: z.coerce.number().int().min(0).max(5).default(0),`
Number of reverse proxies (e.g. Cloudflare) in front of the API, for the client IP.

53 `USER_JWT_SECRET: secret('USER_JWT_SECRET'),`
HS256 key of the user access tokens (ADR-009).

55 `OTP_HMAC_SECRET: secret('OTP_HMAC_SECRET'),`
Key of the OTP code HMAC.

57 `ANALYTICS_SUBJECT_SECRET: secret('ANALYTICS_SUBJECT_SECRET'),`
Key of the pseudonymous analytics subject hash (section 16); changing it splits the history.

59 `ADMIN_TOTP_ENCRYPTION_KEY: z`
Base64 AES-256 key encrypting admin TOTP secrets.

66 `ADMIN_WEB_ORIGIN: z`
Exact origin of Admin Web; every non-GET admin request must come from it (CSRF).

72 `TERMS_VERSION: z.string().trim().min(1),`
Version of the terms of use a new account accepts.

74 `TERMS_URL: z.string().url().optional(),`
Public pages of the terms of use and the privacy policy, linked from sign-up.

78 `SMTP_URL: z`
SMTP transport for OTP e-mails, e.g. smtps://user:pass@smtp.example.com:465

90 `SMS_GATEWAYS: idList(SMS_GATEWAY_IDS, 'SMS provider'),`
SMS providers by priority (ADR-022, docs/operations/sms-setup.md), e.g. `infobip,twilio`: a provider that fails for a temporary reason hands over to the next. Empty: no SMS (sign-up and codes by phone are refused in production; in development codes go to the log).

92 `SMS_ALLOWED_COUNTRY_CODES: z`
Calling codes texts may go to (SMS pumping protection), e.g. `213` or `213,216`.

103 `SMS_DAILY_LIMIT: z.coerce.number().int().min(1).max(1_000_000).default(2000),`
Texts the whole server may send in 24 hours: a cost ceiling if something goes wrong.

111 `TWILIO_MESSAGING_SERVICE_SID: z`
Recommended: a Messaging Service picks the right sender for each country.

117 `TWILIO_FROM: z.string().trim().min(3).max(16).optional(),`
Otherwise the sender: a Twilio number (E.164) or a registered alphanumeric sender id.

119 `INFOBIP_BASE_URL: z`
Your personal Infobip API base, e.g. https://xxxxx.api.infobip.com

127 `INFOBIP_SENDER: z.string().trim().min(3).max(16).optional(),`
Registered sender: alphanumeric (3–11 characters, e.g. DokTool) or a number.

130 `STORAGE_DRIVER: z.enum(['local', 'r2']).default('local'),`
Object storage (ADR-011): Cloudflare R2, or local files for development and tests.

133 `PUBLIC_API_URL: z`
Public base URL of this API; local-driver signed URLs point here.

143 `R2_PRIVATE_BUCKET: z.string().trim().min(3).default('doktool-private'),`
Separate API token for the private (verification) bucket (ADR-002).

152 `PAYMENT_GATEWAYS: idList(GATEWAY_IDS, 'gateway', 'fake').refine(`
Card payment gateways by priority (ADR-021), e.g. `chargily` or `chargily,satim`: each method (CIB, Edahabia) goes to the first gateway that accepts it. `fake` simulates the payment page for development and tests and is refused in production.

157 `CHARGILY_MODE: z.enum(['test', 'live']).default('test'),`
Chargily Pay mode; its secret key must be of the same mode (test_sk_... / live_sk_...).

159 `CHARGILY_SECRET_KEY: z.string().trim().min(1).optional(),`
Chargily Pay API secret key (dashboard › Developers corner); also signs its webhooks.

161 `SATIM_BASE_URL: z`
SATIM e-payment REST base, e.g. https://test2.satim.dz/payment/rest (test) — from SATIM.

169 `SATIM_TERMINAL_ID: z.string().trim().min(1).optional(),`
Terminal id SATIM assigns to the merchant (sent as force_terminal_id).

172 `JOBS_ENABLED: booleanFlag,`
Run pg-boss background jobs in this process.

299 `export type Env = z.infer<typeof envSchema>;`
The typed, validated configuration every service receives through the ENV token.

302 `export const ENV = Symbol('ENV');`
Injection token for the parsed {@link Env}.

305 `export class InvalidEnvironmentError extends Error {`
Thrown at boot with one line per invalid or missing variable.

316 `export function loadEnv(source: Record<string, string | undefined>): Env {`
Parses process.env (or a test object) against the schema; throws InvalidEnvironmentError listing every problem at once.

apps/api/src/health/

The public health endpoint (`GET /v1/health`).

health.controller.ts · 31 lines

11 `export class HealthController {`
Public health endpoint: no token needed.

21 `async check(@Res({ passthrough: true }) response: Response): Promise<HealthResponse> {`
Liveness and database readiness, for load balancers and uptime checks.

health.module.ts · 9 lines

8 `export class HealthModule {}`
Exposes GET /v1/health.

health.response.ts · 14 lines

4 `export class HealthResponse {`
Body of GET /v1/health.

apps/api/src/http/

URL prefixes: `/v1` for the app, `/admin/v1` for Admin Web.

api-prefix.ts · 6 lines

2 `export const USER_API_PREFIX = 'v1';`
User-facing API (Flutter app, future web), authenticated by the user token issuer.

5 `export const ADMIN_API_PREFIX = 'admin/v1';`
Admin Web API, authenticated by the separate admin issuer (ADR-001, audit Appendix D).

apps/api/src/infrastructure/crypto/

Cryptography: Argon2id passwords, AES-GCM secret box (admin TOTP secrets), random tokens, hashes and HMACs.

crypto.module.ts · 11 lines

10 `export class CryptoModule {}`
Global module that gives every module the password hasher.

password-hasher.ts · 42 lines

5 `const ARGON2ID = 2;`
`Algorithm.Argon2id` — an ambient const enum, which isolated modules cannot reference.

8 `const OPTIONS = { algorithm: ARGON2ID, memoryCost: 19_456, timeCost: 2, parallelism: 1 };`
OWASP minimum for Argon2id: 19 MiB memory, 2 iterations, 1 lane (ADR-009).

12 `export class PasswordHasher {`
Hashes and checks passwords (users and admins) with Argon2id.

14 `private readonly decoy: Promise<string> = hash('decoy-password-for-timing', OPTIONS);`
Hash of a random string, used to spend the same time when the account does not exist.

20 `hash(password: string): Promise<string> {`
Returns a self-describing Argon2id hash (algorithm, parameters, salt) to store instead of the password.

28 `async verify(passwordHash: string, password: string): Promise<boolean> {`
True when `password` matches the stored hash; a malformed hash counts as a mismatch instead of an error.

37 `async verifyDecoy(password: string): Promise<false> {`
Runs one verification against a decoy hash and always returns false.

secret-box.ts · 47 lines

11 `export class SecretBox {`
AES-256-GCM envelope for small secrets at rest (admin TOTP secrets). Layout: version (1 byte) · IV (12) · auth tag (16) · ciphertext.

23 `seal(plaintext: string): Uint8Array<ArrayBuffer> {`
Encrypts a text with a fresh random IV; the result carries everything needed to decrypt it except the key.

32 `open(sealed: Uint8Array): string {`
Decrypts a value made by seal(); throws if it was altered or sealed with another key.

tokens.ts · 34 lines

4 `export function randomToken(bytes = 32): string {`
256-bit random token, base64url (refresh tokens, admin session and challenge tokens).

9 `export function sha256Hex(value: string): string {`
Hex SHA-256; tokens are stored only in this form.

17 `export function hmacSha256Hex(key: string, value: string): string {`
Hex HMAC-SHA-256 of `value` with `key` (OTP codes, analytics subjects): unlike a plain hash it cannot be recomputed without the secret.

22 `export function hexDigestsEqual(a: string, b: string): boolean {`
Constant-time comparison of two hex digests.

29 `export function randomNumericCode(digits: number): string {`
Uniform numeric code, zero-padded (OTP).

apps/api/src/infrastructure/database/

The single Prisma client, connected lazily, sessions in UTC.

database.module.ts · 11 lines

10 `export class DatabaseModule {}`
Global module exposing the single PrismaService to every module.

prisma.service.ts · 38 lines

14 `export class PrismaService extends PrismaClient implements OnModuleDestroy {`
The single database client. Connects lazily on the first query, so booting the app (and generating the OpenAPI document) never needs a reachable database. Table ownership (ADR-001): a domain module only writes the tables its section of prisma/schema.prisma assigns to it.

24 `async isReachable(): Promise<boolean> {`
True when the database answers a trivial query.

34 `async onModuleDestroy(): Promise<void> {`
Closes the database connections when the application shuts down.

apps/api/src/infrastructure/jobs/

Background jobs on pg-boss (in PostgreSQL): registration of cron schedules, server analytics events.

analytics-job.ts · 14 lines

7 `export interface ServerAnalyticsEvent extends Record<string, unknown> {`
An analytics event produced by the server itself (not by the app), queued as a job.

jobs.module.ts · 11 lines

10 `export class JobsModule {}`
Global module giving every module the JobsService (background and scheduled jobs).

jobs.service.ts · 105 lines

12 `interface ScheduledJob {`
A job pg-boss runs on a cron schedule.

19 `interface Worker {`
A queue consumer: runs once per job queued with enqueue().

30 `export class JobsService implements OnApplicationBootstrap, OnApplicationShutdown {`
Background jobs on pg-boss (ADR-007), in its own `pgboss` schema. Modules register recurring jobs and on-demand workers during `onModuleInit`; they start once the application has bootstrapped, only when JOBS_ENABLED is true.

39 `register(name: string, cron: string, handler: () => Promise<void>): void {`
Declares a recurring job (cron in UTC). Names are unique.

45 `registerWorker(name: string, handler: (data: Record<string, unknown>) => Promise<void>): void {`
Declares an on-demand queue fed by {@link enqueue}. Names are unique.

54 `async enqueue(name: string, data: Record<string, unknown>): Promise<void> {`
Queues one job. When jobs are disabled in this process the call is a no-op, and the owning module's recurring sweep picks the work up in the process that runs jobs.

62 `registeredJobs(): string[] {`
Names of the registered recurring jobs and workers.

70 `async onApplicationBootstrap(): Promise<void> {`
Starts pg-boss once every module has registered its jobs: creates the queues, the schedules and the workers. Does nothing when JOBS_ENABLED is false.

95 `async onApplicationShutdown(): Promise<void> {`
Stops pg-boss, letting running jobs finish.

101 `private assertUnique(name: string): void {`
Refuses two jobs with the same name (a programming error, caught at boot).

apps/api/src/infrastructure/messaging/

E-mails and SMS: delivery with daily limits, server texts in the account's language.

message-delivery.ts · 156 lines

13 `export interface OutgoingMessage {`
One e-mail or SMS to send.

16 `to: string;`
E-mail address or E.164 phone number.

18 `subject: string;`
E-mail subject; ignored for SMS.

23 `const SMS_PER_NUMBER_PER_DAY = 10;`
SMS cost guards (ADR-022): a number gets at most this many texts a day, whatever the purpose.

35 `export class MessageDelivery {`
Sends one-time codes and security notices (ADR-009, ADR-022). - E-mail: SMTP when SMTP_URL is set (mandatory in production). - SMS: the providers of SMS_GATEWAYS, in order; a provider that fails for a temporary reason hands over to the next. Only numbers of SMS_ALLOWED_COUNTRY_CODES are texted (SMS pumping), each at most 10 times a day, and the whole server at most SMS_DAILY_LIMIT times a day. - Development without a provider: the message is written to the log. Test: kept in `sent` only.

40 `readonly sent: OutgoingMessage[] = [];`
Messages delivered without a real provider (development and test), newest last, capped.

58 `isAvailable(channel: DeliveryChannel): boolean {`
Whether this channel is configured. Outside production it is always "available" (messages go to the log or to `sent`).

64 `canReach(channel: DeliveryChannel, to: string): boolean {`
Whether a code can be sent to this destination at all (channel configured, country allowed).

73 `async send(message: OutgoingMessage): Promise<void> {`
Sends an e-mail through SMTP or an SMS through the providers; without a provider (development, tests) the message is kept in `sent` and logged.

108 `private smsCountryAllowed(phone: string): boolean {`
True when the number starts with one of the allowed calling codes.

116 `private async sendSms(to: string, text: string): Promise<void> {`
Applies the per-number and per-server daily limits, then tries each SMS provider in order until one accepts.

messaging.module.ts · 14 lines

13 `export class MessagingModule {}`
Global module for sending e-mails and SMS and formatting server-side texts.

server-messages.ts · 33 lines

7 `export type MessageLocale = 'FR' | 'EN';`
Languages of server texts (the account's language).

14 `export class ServerMessages {`
Server-rendered texts (e-mails, SMS) from the `server.*` keys of packages/i18n.

21 `format(locale: MessageLocale, key: string, values: Record<string, string | number> = {}): string {`
Formats one ICU message of the shared catalogue (e.g. a code e-mail) with its values; formatters are cached per locale and key.

apps/api/src/infrastructure/messaging/sms/

SMS providers (Infobip, Twilio) behind one interface, with fallback and masked numbers in logs.

infobip.sms.ts · 75 lines

12 `interface InfobipMessage {`
The part of Infobip's answer we read.

23 `export class InfobipSms extends SmsGateway {`
Infobip SMS (docs/operations/sms-setup.md): `POST {base}/sms/3/messages` with `Authorization: App <key>` and `{ messages: [{ sender, destinations: [{ to }], content: { text } }] }`. Numbers go without the `+`. A message in the REJECTED group is refused for this number.

34 `async send(to: string, text: string): Promise<SmsSent> {`
Posts one SMS to Infobip; network errors and 5xx are retryable on the next provider, a rejected number is not.

sms-gateway.ts · 49 lines

6 `export interface SmsSent {`
What a provider returns once it has accepted a message.

8 `providerMessageId: string | null;`
The provider's id of the message, for its delivery reports.

15 `export class SmsDeliveryError extends Error {`
The provider refused or could not be reached. `retryable` failures (network, 5xx, throttling) let the next configured provider try; a refusal of the number itself does not.

31 `export abstract class SmsGateway {`
An SMS provider (ADR-022). Adding one is one class extending this, its id in `SMS_GATEWAY_IDS` and its settings in `config/env.ts`. Messages are short and carry a one-time code: an implementation never logs the text.

35 `abstract send(to: string, text: string): Promise<SmsSent>;`
Sends `text` to an E.164 number (`+213…`).

41 `export function maskPhone(phone: string): string {`
`+213661234567` → `+21366•••••67`, for logs.

46 `export function retryableStatus(status: number): boolean {`
4xx other than throttling means the request itself is wrong: another provider won't help.

twilio.sms.ts · 62 lines

17 `export class TwilioSms extends SmsGateway {`
Twilio Programmable Messaging (docs/operations/sms-setup.md): `POST /2010-04-01/Accounts/{sid}/Messages.json`, HTTP Basic with the account SID and auth token, form-encoded `To`, `Body` and either `MessagingServiceSid` (recommended: it picks the sender per country) or `From`.

25 `async send(to: string, text: string): Promise<SmsSent> {`
Posts one SMS to Twilio with Basic authentication; returns Twilio's message id.

apps/api/src/infrastructure/rate-limit/

Rate limits stored in PostgreSQL (fixed windows), valid across several API instances.

rate-limit.module.ts · 24 lines

11 `export class RateLimitModule implements OnModuleInit {`
Global module exposing the RateLimiter, and scheduling the hourly clean-up of old counters.

18 `onModuleInit(): void {`
Registers the purge job at minute 17 of every hour.

rate-limiter.ts · 50 lines

7 `export interface RateLimitRule {`
A limit: at most `limit` hits per `windowSeconds` for one subject.

9 `name: string;`
Stable rule name, part of the counter key.

20 `export class RateLimiter {`
Fixed-window rate limiter on PostgreSQL (ADR-007, ADR-009): one row per key, updated with one atomic upsert, so the limit holds across API instances. Subjects (IPs, identifiers) are hashed before they are stored.

24 `async hit(rule: RateLimitRule, subject: string): Promise<void> {`
Records one hit for `subject`; throws RATE_LIMITED once the window's limit is exceeded.

43 `async purgeExpired(): Promise<number> {`
Deletes expired counters (hourly maintenance job).

apps/api/src/infrastructure/security/

Tokens and guards: user tokens (15 min, live session check), demo tokens (1 h), `UserAuthGuard`, `ViewerAuthGuard`.

demo-tokens.ts · 73 lines

11 `export const DEMO_TOKEN_TTL_SECONDS = 60 * 60;`
Demo access-token lifetime (ADR-010: 1 hour, renewed with the renewal token).

14 `export interface DemoPrincipal {`
Who a demo token stands for: a demo session (no user account) and the profession it previews.

26 `export class DemoTokens {`
Demo access tokens: same issuer and key as user tokens but audience `doktool-demo`, so a demo token can never pass {@link UserTokens.verify} (and the reverse).

34 `async issue(principal: DemoPrincipal): Promise<{ token: string; expiresAt: Date }> {`
Signs a one-hour demo access token for a demo session.

53 `async verify(token: string): Promise<DemoPrincipal> {`
Checks the signature, issuer, audience and expiry of a demo token and returns its session; throws UNAUTHENTICATED otherwise.

security.module.ts · 17 lines

16 `export class SecurityModule {}`
Global module with the token services and the two guards (user-only routes, user-or-demo routes).

user-auth.guard.ts · 39 lines

12 `type AuthenticatedRequest = Request & { user?: UserPrincipal };`
An Express request after UserAuthGuard ran.

16 `export class UserAuthGuard implements CanActivate {`
Requires a valid user access token (`Authorization: Bearer …`). Admin cookies are ignored.

23 `async canActivate(context: ExecutionContext): Promise<boolean> {`
Runs before the route: extracts the bearer token, verifies it (including that its session is still live) and stores the principal on the request.

34 `export const CurrentUser = createParamDecorator((_: unknown, context: ExecutionContext) => {`
The principal set by {@link UserAuthGuard}.

user-tokens.ts · 94 lines

11 `export const ACCESS_TOKEN_TTL_SECONDS = 15 * 60;`
Access-token lifetime (ADR-009: 15 min).

14 `export interface UserPrincipal {`
Who is calling, as proven by a valid user access token.

29 `export class UserTokens {`
Signs and verifies user access tokens (HS256, issuer `doktool-api`, audience `doktool-user`). Lives outside the auth module so any module can guard its routes without importing auth. A token is accepted only while its session is live (ADR-023): signing a device out, changing the password, a recovery, a suspension or a deletion take effect on the next request rather than when the 15-minute token expires. This is a read-only, primary-key lookup of the auth module's `user_sessions` — the one place outside auth that reads it (ADR-001 exception).

40 `async issue(principal: UserPrincipal): Promise<{ token: string; expiresAt: Date }> {`
Signs a 15-minute access token for a user session (subject = user id, `sid` = session id).

56 `async verify(token: string): Promise<UserPrincipal> {`
Verifies a user access token and that its session has not ended; returns who is calling.

85 `private async sessionIsLive(sessionId: string, userId: string): Promise<boolean> {`
True when the session exists for this user, is not revoked and has not expired (one indexed lookup per request).

viewer-auth.guard.ts · 62 lines

14 `export type Viewer =`
Whoever is looking at content: a signed-in user, or a demo session (ADR-010).

```
19  type ViewerRequest = Request & { viewer?: Viewer };
```
An Express request after ViewerAuthGuard ran.

```
26  export class ViewerAuthGuard implements CanActivate {
```
For read routes open to both user and demo tokens. Write routes use UserAuthGuard, which refuses demo tokens.

```
33  async canActivate(context: ExecutionContext): Promise<boolean> {
```
Accepts either a user token or a demo token and stores the resulting viewer on the request.

```
46  function isDemoToken(token: string): boolean {
```
Routes by the unverified audience claim only; the chosen verifier then checks everything.

```
57  export const CurrentViewer = createParamDecorator((_: unknown, context: ExecutionContext) => {
```
Route parameter decorator returning the viewer set by ViewerAuthGuard.

apps/api/src/

The server entry point (`main.ts`), the HTTP settings shared with tests (`app.setup.ts`) and the root module (`app.module.ts`).

main.ts · 15 lines

```
10  const env = loadEnv(process.env);
```
Server entry point: read and check the environment, build the Nest application, apply the shared HTTP settings, then listen on PORT.

apps/api/src/modules/access/

The entitlement policy as data, its pure evaluation, the access service, `GET /v1/me/access` and `@RequireFeature`.

access-policy.config.ts · 161 lines

```
13  export type FeatureSetting =
```
A feature state in the config: available, available read-only, locked, or decided at runtime.

```
17  | { state: 'RESTRICTION' }
```
Locked with the reason that restricted the account (RESTRICTED tier only).

```
19  | { state: 'PURCHASE_AFTER_VERIFICATION' };
```
Purchase for PROFESSIONAL_UNPAID: open once professional verification is VERIFIED (Q6).

```
22  export interface TierSetting {
```
What one tier opens: which items of each content type, and the state of each feature.

```
28  const available: FeatureSetting = { state: 'AVAILABLE' };
```
Small builders that keep the policy table below readable.

```
30  const locked = (lockReason: LockReason, unlockPath: UnlockPath | null): FeatureSetting => ({
```
A locked feature with its reason and unlock path.

```
36  const demoSamplesOnly = (lockReason: LockReason, unlockPath: UnlockPath | null): ContentRule => ({
```
Only demo-sample items open; the others show this lock.

```
51  export const ACCESS_POLICY = {
```
The whole V1 policy, one entry per tier (DEMO, STUDENT, PROFESSIONAL_UNPAID, PROFESSIONAL, RESTRICTED).

```
54  studentPendingWindowDays: 14,
```
A student whose status case is submitted keeps student access this long (section 27).

```
56  subscriptionGraceFallbackDays: 7,
```
ACTIVE past its period end still counts as live this long, if billing has not moved it to GRACE yet.

```
83  referenceDocuments: demoSamplesOnly('PROFESSIONAL_ONLY', null),
```
ADR-006 Q2 + D2: titles only, except the demo samples.

```
85  pregnancyProfiles: demoSamplesOnly('PROFESSIONAL_ONLY', null),
```
ADR-019: like the reference models, names only except the demo samples.

103 `calculators: demoSamplesOnly('SUBSCRIPTION_REQUIRED', 'subscription.plans'),`
ADR-006 Q1: the demo sample set, with a real account.

160 `export type AccessPolicy = typeof ACCESS_POLICY;`
Type of the policy object; tests pass modified copies.

access-policy.ts · 208 lines

24 `interface Restriction {`
Why an account is RESTRICTED: shown on every locked item and feature.

30 `interface Placement {`
The tier a person lands in, why they are restricted (if so) and whether they may buy.

34 `purchase: Restriction | 'OPEN' | null;`
PROFESSIONAL_UNPAID only: 'OPEN' once professionally verified (Q6), else why it is locked.

38 `const restricted = (lockReason: LockReason, unlockPath: UnlockPath | null): Placement => ({`
Shortcut for a RESTRICTED placement with its reason and unlock path.

48 `export function isSubscriptionLive(`
True while a subscription gives Pro access: ACTIVE (with a grace fallback), CANCELED until the end of the paid period, GRACE until its end date.

69 `function isPendingWithin(snapshot: CaseSnapshot, days: number, now: Date): boolean {`
True when a verification case is waiting for review and was submitted less than `days` ago.

78 `export function placeInTier(inputs: AccessInputs, now: Date, policy: AccessPolicy): Placement {`
Places a person in exactly one tier (section 27 table, ADR-010 §3–4).

121 `function decideFeature(`
Turns one feature setting of the tier into the decision sent to the app (available, locked, read-only).

157 `export function evaluateAccess(`
Computes the full AccessProfile: tier, every feature decision and every content rule. Pure: same inputs and time give the same profile.

191 `export function decideItem(profile: AccessProfile, item: AccessibleItem): ItemAccess {`
Opens or locks one item for a profile. "Not available" (disabled) is the content module's call.

203 `export function featureOf(profile: AccessProfile, key: FeatureDecision['key']): FeatureDecision {`
Returns one feature decision of a profile; a missing key is a programming error.

access.controller.ts · 33 lines

16 `export class AccessController {`
GET /v1/me/access.

29 `async me(@CurrentViewer() viewer: Viewer): Promise<AccessProfileDto> {`
The caller's AccessProfile (user or demo token). The app renders every lock from it and refreshes it on launch, on resume and after purchase or verification events.

access.dto.ts · 102 lines

17 `export class FeatureDecisionDto {`
One feature as the app receives it: available or locked, with the reason and where to go to unlock it.

35 `export class ContentRuleDto {`
Which items of a content type open for this person, and the lock shown on the others.

47 `export class ContentRulesDto {`
The content rule of each content type.

59 `export class AccessProfileDto {`
JSON form of AccessProfile (dates as ISO strings).

79 `static from(profile: AccessProfile): AccessProfileDto {`
Converts the internal profile into the response body.

92 `export class ItemAccessDto {`
Server-computed decision attached to every item in lists, search and viewers.

access.module.ts · 22 lines

21 `export class AccessModule {}`
The entitlement policy engine — the only place access is decided (ADR-004, ADR-006, ADR-010). Owns no table: AccessProfile is derived on every request. Other modules reach it only through ./index.ts.

access.service.ts · 61 lines

15 `export class AccessService {`
The only place access is decided (ADR-004). Gathers the inputs from the owning modules and evaluates the policy for each request; nothing is cached or stored.

23 `async profileFor(viewer: Viewer): Promise<AccessProfile> {`
The access profile of the caller (user or demo), computed fresh on every call.

28 `async explainForUser(userId: string): Promise<{ profile: AccessProfile; inputs: AccessInputs }> {`
Admin Web « Accès »: the profile of one account and its inputs (read-only).

34 `async decideItems<T extends AccessibleItem>(`
Decides many items at once for list and search endpoints.

46 `private async inputsFor(viewer: Viewer): Promise<AccessInputs> {`
Collects the account facts, verification cases and subscription in parallel; a deleted account is treated as signed out.

access.types.ts · 103 lines

7 `export const FEATURE_KEYS = [`
V1 feature keys (section 27).

21 `export const LOCK_REASONS = [`
Section 27 lock reasons, plus ACCOUNT_SUSPENDED ("or suspension notice", ADR-010).

34 `export const UNLOCK_PATHS = [`
Where the app sends the person to lift a lock; null = explanation only.

44 `export type Tier = EntitlementTier;`
The five tiers, as stored in the database enum.

47 `export interface FeatureDecision {`
The decision about one feature.

53 `readOnly: boolean;`
Available for reading only (RESTRICTED favorites and history).

57 `export type ContentScope = 'ALL' | 'STUDENT_TIER' | 'DEMO_SAMPLE' | 'NONE';`
Which items of a content type open: all, student-tier (+ demo samples), demo samples, none.

60 `export interface ContentRule {`
Content rule of one content type.

63 `lockReason: LockReason;`
Reason shown on items outside the scope.

70 `export interface AccessProfile {`
Everything the app needs to show or lock: computed by evaluateAccess, never stored.

80 `export type AccessInputs =`
Everything the policy needs, gathered from users, verification and billing.

90 `export interface AccessibleItem {`
The access facts of one item, as stored by its content module.

93 `accessTier?: 'STUDENT' | 'PROFESSIONAL';`
Calculators only; reference documents have no per-item tier (ADR-006 D3).

98 `export interface ItemAccess {`
The decision about one item (a calculator, a reference model, a pregnancy profile).

index.ts · 15 lines

require-feature.ts · 60 lines

22 `export class FeatureGuard implements CanActivate {`
Re-evaluates the policy on every request and refuses a LOCKED feature (L34: every endpoint re-checks).

32 `async canActivate(context: ExecutionContext): Promise<boolean> {`
Reads the feature named by @RequireFeature on the route, computes the caller's profile and refuses with FEATURE_LOCKED (and the unlock path) when that feature is locked.

52 `export function RequireFeature(key: FeatureKey) {`
Guards a route with a user or demo token and the given feature being AVAILABLE.

apps/api/src/modules/activity/

Favorites and the person's own research history (private; admins see counts only).

activity.controller.ts · 119 lines

39 `const asViewer = (user: UserPrincipal) => ({ accessContext: 'AUTHENTICATED' as const, ...user });`
These routes accept only real accounts; the services expect a "viewer", so the user is wrapped as one.

47 `export class ActivityController {`
Favorites and the person's own research history (ScreenFavoritesRecents). Real accounts only.

56 `async favorites(@CurrentUser() user: UserPrincipal): Promise<FavoriteDto[]> {`
GET /v1/me/favorites: the person's favorites, newest first, each with its current title and lock (or null when no longer published).

64 `async addFavorite(`
PUT /v1/me/favorites/:targetType/:targetId: adds a favorite (idempotent).

75 `async removeFavorite(`
DELETE /v1/me/favorites/:targetType/:targetId: removes a favorite (idempotent).

86 `async record(@CurrentUser() user: UserPrincipal, @Body() body: RecordActivityDto): Promise<void> {`
Entries recorded on the device, synced in batches (up to 100).

97 `async history(`
Newest first, 50 per page.

112 `async clear(`
DELETE /v1/me/activity: clears the history, or only one kind of entry.

activity.dto.ts · 167 lines

20 `export class FavoriteTargetParams {`
URL parameters naming a favorite: its type and id.

31 `export class FavoriteItemDto {`
The item a favorite points at, with the caller's access to it.

49 `export class FavoriteDto {`
One favorite in GET /v1/me/favorites.

72 `export class ActivityEntryDto {`
One history entry sent by the app.

99 `export class RecordActivityDto {`
Body of POST /v1/me/activity: a batch of entries from one device.

116 `export class ActivityQuery {`
Filters of GET /v1/me/activity.

132 `export class ClearActivityQuery {`
Filter of DELETE /v1/me/activity.

144 `export class ActivityDto {`
One history entry returned to the app.

activity.module.ts · 22 lines

21 `export class ActivityModule {}`
Favorites and recent activity (« Activité »). Owns: `favorites`, `recent_activities`. Other modules reach it only through ./index.ts.

activity.service.ts · 316 lines

17 `const TARGETS = {`
What each kind of history entry points at.

24 `export const MAX_FAVORITES = 500;`
Favorites a person can keep (section 13: favorites first everywhere).

26 `export const MAX_ACTIVITY = 500;`
Research history kept per person; older entries are dropped.

30 `export interface FavoriteView {`
A favorite as the app shows it.

35 `item: {`
Null when the model or calculator is no longer published (« N'est plus disponible »).

45 `export interface ActivityInput {`
One history entry as the device sends it.

54 `export interface ActivityRecord {`
One stored history entry.

68 `export class ActivityService {`
Favorites and the person's own research history (Profile › Activité). The history is private: admins only ever see counts (section 16). Calculator inputs are never recorded (Lock L42).

78 `async eraseUser(userId: string): Promise<void> {`
Account deletion: the person's favorites and research history go with the account.

88 `async countsFor(userId: string) {`
Admin Web « Activité »: how many favorites and history entries, by kind — never which.

111 `async favorites(viewer: Viewer & { userId: string }): Promise<FavoriteView[]> {`
The person's favorites with the title and access of each item; items no longer published come back with `item: null`.

126 `async addFavorite(`
Idempotent. Only a currently published model or calculator can become a favorite.

134 `throw new DomainError('FEATURE_LOCKED', 'Favorites are read-only for this account.', {`
Read-only (RESTRICTED): the lock is the one that restricts the account.

155 `async removeFavorite(`
Removes one favorite; nothing happens if it was not there.

166 `async record(`
Stores entries recorded on the device (synced in batches, possibly after being offline).

174 `if (!writable(await this.access.profileFor(viewer), 'activity.history')) return;`
A restricted account's history is read-only: the device's entries are accepted (so its upload queue empties) but not kept.

220 `async history(`
One page of the person's history, newest first; `before` continues from the last entry shown.

244 `async clearHistory(userId: string, kind?: ActivityKind): Promise<void> {`
Deletes the whole history, or only one kind of entry.

249 `private async trim(userId: string): Promise<void> {`
Keeps only the newest MAX_ACTIVITY entries of a person.

268 `private async itemsFor(`
The items behind some favorites, still published, with the viewer's access, keyed `TYPE:id`. Only those ids are read; a substance merged into another answers with the one kept.

312 `function writable(profile: AccessProfile, key: FeatureKey): boolean {`
True when the feature is available and not read-only for this account.

index.ts · 4 lines

apps/api/src/modules/admin-auth/

Admin authentication and permissions: password + TOTP, cookie sessions, guards, grants and presets, separation of duties, admin accounts.

admin-accounts.controller.ts · 317 lines

48 `const trimmed = Transform(({ value }: { value: unknown }) =>`
Trims a text field before validation.

55 `class ReasonBody {`
A reason, required for every change to an admin account.

65 `export class NewAdminBody extends ReasonBody {`
Body of « Nouvel administrateur ».

84 `export class GrantsBody extends ReasonBody {`
Body of « Modifier les permissions ».

92 `export class TemporaryGrantBody extends ReasonBody {`
Body of a temporary grant.

110 `export class StatusBody extends ReasonBody {}`
Body of suspend, reinstate and reset TOTP.

118 `export class AdminGrantDto {`
One grant of an admin, with its expiry and who granted it.

131 `export class AdminAccountDto {`
One admin in « Permissions ».

151 `export class CreatedAdminDto {`
A new admin and the one-time password to hand over.

160 `export class TemporaryGrantDto {`
When a temporary grant ends.

173 `export class AdminAccountsController {`
« Permissions » (ScreenPermissions): admins, presets, grants and temporary access (section 03).

184 `async list(): Promise<AdminAccountDto[]> {`
GET /admin/v1/admins: every admin with their grants, and the preset their permanent grants match (if any).

222 `async create(`
POST /admin/v1/admins: creates an admin; the one-time password is shown once.

237 `async setGrants(`
PUT /admin/v1/admins/:id/grants: replaces the permanent grants.

252 `async grantTemporary(`
POST /admin/v1/admins/:id/temporary-grants: a grant that expires after a few hours.

275 `async suspend(`
POST /admin/v1/admins/:id/suspend: blocks sign-in and ends sessions.

290 `async reinstate(`
POST /admin/v1/admins/:id/reinstate: lifts the suspension.

308 `async resetTotp(`
POST /admin/v1/admins/:id/reset-totp: the admin enrols a new authenticator at the next sign- in.

admin-accounts.service.ts · 292 lines

20 `export const TEMPORARY_GRANTS = ['verification.documents.view'] as const;`
Grants an admin may receive for a few hours only (section 03).

25 `export interface NewAdminAccount {`
What is needed to create an admin account.

31 `grantedById: string | null;`
Admin granting the preset; null only when bootstrapping the first Super Admin.

36 `export class AdminAccountsService {`
Owner of `admin_accounts` and `admin_grants`.

47 `async create(input: NewAdminAccount): Promise<{ id: string }> {`
Creates an admin with the grants of a preset; refuses presets that break the separation of duties, and duplicate e-mails.

74 `async effectiveGrants(adminId: string): Promise<Grant[]> {`
Non-expired grants of an admin, sorted.

86 `async names(ids: string[]): Promise<Map<string, string>> {`
Full names shown in review trails, by admin id.

99 `list(now = new Date()) {`
Every admin with the grants in force (permanent, and temporary until they expire).

125 `async createByAdmin(`
A new admin from a preset; the one-time password is returned once and never stored.

144 `async setGrants(`
Replaces the permanent grants of another admin (temporary grants run out on their own). The separation of duties of section 03 applies; nobody edits their own grants.

184 `async grantTemporary(`
Time-limited access, e.g. `verification.documents.view` for two hours (section 03).

214 `async setStatus(`
Suspension ends every session of the admin at once.

240 `async resetTotp(`
Lost phone: the next sign-in enrols a new authenticator; current sessions end.

256 `private async assertOther(actor: AdminPrincipal, adminId: string): Promise<void> {`
Refuses changes to one's own account, and unknown admins.

265 `private async endSessions(adminId: string): Promise<void> {`
Revokes every open session of an admin.

273 `private record(`
Writes the audit entry of a change to an admin account.

admin-auth.controller.ts · 90 lines

29 `async signIn(`
Step 1: e-mail + password. Never creates a session; answers with the TOTP step.

42 `async totp(`
Step 2: TOTP code (verification, or first enrolment). Sets the session cookie.

67 `async signOut(`
POST /admin/v1/auth/sign-out: ends the session and deletes the cookie.

86 `me(@CurrentAdmin() admin: AdminPrincipal): AdminMeDto {`
The signed-in admin and effective grants; Admin Web renders its navigation from these.

admin-auth.dto.ts · 90 lines

8 `export class AdminSignInDto {`
Body of the password step.

22 `export class AdminTotpDto {`
Body of the TOTP step.

35 `export class TotpEnrollmentDto {`
First sign-in only: the secret to add to an authenticator app.

44 `export class AdminSignInStepDto {`
Answer of the password step.

68 `export class AdminMeDto {`
The signed-in admin, as Admin Web receives it.

admin-auth.module.ts · 36 lines

23 `export class AdminAuthModule implements OnModuleInit {`
Admin authentication: separate issuer, cookie sessions, mandatory TOTP, admin accounts and grants (ADR-009). Owns: `admin_accounts`, `admin_grants`, `admin_sessions`, `admin_auth_challenges`. Other modules reach it only through ./index.ts.

30 `onModuleInit(): void {`
Schedules the hourly clean-up of expired challenges and old sessions.

admin-auth.policy.ts · 20 lines

8 `export const ADMIN_SESSION_TOUCH_SECONDS = 60;`
lastSeenAt is written at most this often, to avoid a write per request.

admin-auth.service.ts · 322 lines

30 `export interface AdminSignInStep {`
What the password step answers: the next step (enrol or verify TOTP) and a short-lived challenge token.

35 `enrollment: TotpEnrollment | null;`
Only for TOTP_ENROLL: the secret to add to an authenticator app.

39 `export interface AdminPrincipal {`
The signed-in admin, with the grants in force now.

48 `const clientKey = (client: ClientInfo) => client.ip ?? 'unknown';`
Rate-limit key: the caller's IP.

55 `export class AdminAuthService {`
Admin login: password, then TOTP at every login; opaque cookie sessions with idle and absolute timeouts (ADR-009). No session exists before the TOTP step succeeds.

73 `async signIn(rawEmail: string, password: string, client: ClientInfo): Promise<AdminSignInStep> {`
Step 1: checks the password (with lockout after repeated failures) and opens a 5-minute TOTP challenge. An unknown e-mail costs the same time as a wrong password.

133 `async completeTotp(`
Second step: a valid TOTP code turns the challenge into a session. Returns the session token.

248 `async resolveSession(sessionToken: string): Promise<AdminPrincipal> {`
Resolves a session cookie; enforces the idle and absolute timeouts.

280 `async signOut(principal: AdminPrincipal, client: ClientInfo): Promise<void> {`
Revokes the current admin session.

297 `async purge(): Promise<void> {`
Removes expired challenges and sessions ended more than a day ago (hourly job).

309 `private async registerFailedSignIn(adminId: string): Promise<Date | null> {`
Counts a wrong password in one atomic update; the fifth in a row locks the account for 15 minutes. Returns the lock end when this failure locked it.

admin-guards.ts · 100 lines

21 `type AdminRequest = Request & { admin?: AdminPrincipal };`
An Express request after AdminSessionGuard ran.

24 `const cookieOf = (request: Request, name: string): string | undefined =>`
cookie-parser types `request.cookies` as `any`; read it as the string map it is.

35 `export class AdminBoundaryGuard implements CanActivate {`
Trust boundary of every admin route (ADR-009): user bearer tokens are refused outright, and a state-changing request must come from the Admin Web origin (CSRF defence on top of SameSite).

41 `canActivate(context: ExecutionContext): boolean {`
Refuses bearer tokens and, for anything but GET/HEAD/OPTIONS, an Origin other than Admin Web.

55 `export class AdminSessionGuard implements CanActivate {`
Requires a live admin session cookie and every grant listed by {@link RequireGrants}.

65 `async canActivate(context: ExecutionContext): Promise<boolean> {`
Resolves the session cookie, checks the grants the route requires and stores the admin on the request.

85 `export function RequireGrants(...grants: Grant[]) {`
Guards a route with an admin session holding all of `grants`.

94 `export const CurrentAdmin = createParamDecorator((_: unknown, context: ExecutionContext) => {`
Route parameter decorator returning the admin set by AdminSessionGuard.

grants.ts · 96 lines

4 `export const GRANTS = [`
Every permission an admin can hold; routes require them with @RequireGrants.

44 `export const PRESETS = {`
Ready-made sets of grants offered when creating an admin.

47 `VERIFICATION_REVIEWER: ['verification.queue.read', 'verification.review'],`
`verification.documents.view` is granted separately, time-limited (default 2 h).

50 `CALCULATOR_EDITOR: ['calculators.read', 'calculators.draft', 'pregnancy.read', 'pregnancy.draft'],`
Authors clinical content: calculator drafts and pregnancy · lactation profiles (ADR-019).

52 `CLINICAL_REVIEWER: [`
Reviews and publishes clinical content, never their own (author ≠ reviewer).

72 `export function isGrant(value: string): value is Grant {`
True when a stored string is a known grant (old grants are ignored).

81 `export function separationOfDutiesViolations(`
Separation of duties for sub-admins (section 03): no account holds both finance grants and clinical content grants, and only the Super Admin may approve exceptional adjustments. Returns the violated rules (empty = allowed).

index.ts · 12 lines

totp.ts · 46 lines

4 `const PERIOD_SECONDS = 30;`
RFC 6238 parameters understood by every authenticator app (ADR-009).

8 `const WINDOW = 1;`
Accept the previous and next 30-second step to absorb clock drift.

11 `export interface TotpEnrollment {`
A new authenticator secret and its otpauth:// link.

17 `export function newTotpEnrollment(accountEmail: string): TotpEnrollment {`
A new random TOTP secret and the otpauth:// link an authenticator app scans as a QR code.

34 `export function matchTotpStep(secretBase32: string, code: string, now = Date.now()): number | null {`
Returns the time step the code belongs to, or null when the code is wrong. The caller must refuse a step that is not newer than the last one accepted (replay).

apps/api/src/modules/admin/

Admin read screens across modules: overview, users and the user drawer, audit log and export.

admin-read.service.ts · 347 lines

35 `type AdminUserRow = NonNullable<Awaited<ReturnType<UsersService['adminDetail']>>>;`
An account row as UsersService returns it to admins.

42 `export class AdminReadService {`
Admin-facing read models behind /admin/v1 that combine several modules (ADR-001: this module owns no table; every read and write goes through the owning module's service).

64 `async userList(query: UserQuery): Promise<AdminUserDto[]> {`
The « Utilisateurs » list: one row per account with its verification and subscription state.

73 `async userDetail(admin: AdminPrincipal, userId: string): Promise<AdminUserDetailDto> {`
The user drawer: the row, the access profile and the facts it was computed from, subscriptions (with `subscriptions.read`), activity counts and audit history.

125 `async setUserStatus(`
« Suspendre »: sign-in and refresh are refused and every session ends (ADR-010).

154 `async signOutEverywhere(`
« Réinitialiser la connexion »: every device signs in again.

180 `private async userRow(row: AdminUserRow): Promise<AdminUserDto> {`
Builds one list row: identity, profession, verification state of the case that matters for the profession, subscription status.

218 `async presentAudit(entries: AuditLogEntry[]): Promise<AuditEntryDto[]> {`
Turns audit rows into the response shape, adding the admin's name to each admin action.

244 `async overview(admin: AdminPrincipal, now = new Date()): Promise<OverviewDto> {`
« Vue d'ensemble »: each block only for the admins whose grants cover it (section 07).

admin.controllers.ts · 229 lines

49 `export class OverviewController {`
« Vue d'ensemble »: the ledger strip and the work lists the admin's grants cover.

57 `overview(@CurrentAdmin() admin: AdminPrincipal): Promise<OverviewDto> {`
GET /admin/v1/overview: the blocks of the home page the admin's grants allow.

66 `export class AdminUsersController {`
« Utilisateurs » (ScreenUsersAdmin): profile, derived access, subscription, activity counts.

73 `list(@Query() query: UserQuery): Promise<AdminUserDto[]> {`
GET /admin/v1/users: search and list user accounts.

84 `detail(`
GET /admin/v1/users/:id: one account with its derived access, subscriptions, activity counts and history.

96 `async suspend(`
POST /admin/v1/users/:id/suspend: suspends the account and signs it out everywhere.

110 `async reinstate(`
POST /admin/v1/users/:id/reinstate: lifts a suspension.

124 `async signOut(`
POST /admin/v1/users/:id/sign-out: ends every session of the account.

138 `export class AuditLogController {`
« Journal d'audit » (ScreenAuditLogs): immutable, filterable, exported with `audit.export`.

148 `async list(@Query() query: AuditQuery): Promise<AuditEntryDto[]> {`
GET /admin/v1/audit: one page of the audit log, newest first.

160 `async export(`
The filtered log as CSV (at most 10 000 entries); the export is itself audited.

187 `const cell = (value: unknown) =>`
One CSV cell: objects as JSON, then quoted and guarded against formulas by csvCell.

admin.dto.ts · 277 lines

```
32  export class AdminReasonBody {
```
A reason, required for every admin change (kept in the audit log).

```
44  export class UserQuery {
```
Filters of the user list.

```
79  export class VerificationStateDto {
```
Verification state shown in the user list.

```
93  export class AdminUserDto {
```
One row of the user list.

```
113 export class AccessInputsDto {
```
The facts the access profile was computed from (shown to explain it).

```
142 export class AuditEntryDto {
```
One audit log entry.

```
163 export class AdminUserDetailDto extends AdminUserDto {
```
The user drawer.

```
180 export class AuditQuery {
```
Filters of the audit log and of its export.

```
257 export class OverviewDto {
```
The « Vue d'ensemble » page; each block is present only with the matching grant.

admin.module.ts · 47 lines

```
46  export class AdminModule {}
```
Admin-facing read models behind /admin/v1 that span several modules: overview, users, audit log (section 07). Owns no table. Other modules reach it only through ./index.ts.

index.ts · 3 lines

apps/api/src/modules/analytics/

Pseudonymous events: ingestion with strict schemas, server envelope, dashboards with small-count suppression.

analytics.controller.ts · 127 lines

```
43  export class AnalyticsController {
```
Event upload from the app (accounts and demo sessions); the server adds the envelope.

```
53  async ingest(
```
POST /v1/analytics/events: a batch of up to 100 events from the app or a demo session.

```
68  export class AnalyticsAdminController {
```
« Analytics — DokTool Intelligence » (section 16): aggregates only, cells under 10 people suppressed, demo events only in the demo funnel.

```
75  async freshness(): Promise<FreshnessDto> {
```
GET /admin/v1/analytics/freshness: when the latest event arrived.

```
83  usage(@Query() query: DashboardQuery): Promise<UsageDashboardDto> {
```
GET /admin/v1/analytics/usage: active people.

```
91  library(@Query() query: DashboardQuery): Promise<LibraryDashboardDto> {
```
GET /admin/v1/analytics/library: searches and reference model use.

```
99  calculators(@Query() query: DashboardQuery): Promise<CalculatorsDashboardDto> {
```
GET /admin/v1/analytics/calculators: starts and completions per calculator version.

```
107 users(@Query() query: DashboardQuery): Promise<OnboardingDashboardDto> {
```
GET /admin/v1/analytics/users: sign-ups and verification funnel.

115 `subscriptions(@Query() query: DashboardQuery): Promise<SubscriptionsDashboardDto> {`
GET /admin/v1/analytics/subscriptions: plan views, starts and renewals.

123 `demo(@Query() query: DashboardQuery): Promise<DemoDashboardDto> {`
GET /admin/v1/analytics/demo: the demo funnel.

analytics.dto.ts · 205 lines

22 `export class AnalyticsEventDto {`
One event in an upload.

50 `export class AnalyticsBatchDto {`
Body of POST /v1/analytics/events.

82 `export class IngestResultDto {`
Answer of an upload.

94 `export class DashboardQuery {`
Period (from inclusive, to exclusive) and optional segment of a dashboard.

analytics.module.ts · 50 lines

37 `export class AnalyticsModule implements OnModuleInit {`
Pseudonymous event ingestion and aggregates (Lock L42, section 16). Owns: `analytics_events`. Other modules reach it only through ./index.ts.

44 `onModuleInit(): void {`
Registers the worker that stores subscription events published by billing.

analytics.policy.ts · 173 lines

6 `export const SUPPRESSION_THRESHOLD = 10;`
Any aggregate cell behind fewer distinct subjects than this is shown as « < 10 ».

8 `export const MAX_BATCH = 100;`
Events per upload (the app buffers up to 5 000 offline and sends them in batches).

10 `export const MAX_EVENT_AGE_DAYS = 30;`
Offline events older than this are dropped; so are events dated in the future.

13 `export const MAX_DASHBOARD_DAYS = 400;`
Longest period of an analytics dashboard.

22 `const favoriteTarget = z.enum(['REFERENCE_DOCUMENT', 'CALCULATOR', 'SUBSTANCE']);`
What a favorite points at, pregnancy · lactation profiles included (ADR-019).

25 `export const DEMO_MODULES = [`
Screens the demo can open or be stopped at (DEMO_FEATURE_VIEWED `module`).

41 `export const CLIENT_EVENTS = {`
Payload of each event the app may send: counts, codes and flags only — never free text, query text, calculator inputs or PDF content. Unknown keys are rejected.

74 `PREGNANCY_VIEWED: {`
A pregnancy · lactation profile opened on one of its two blocks (ADR-019).

102 `export interface ClientEventInput {`
One event as the app sends it.

113 `export type RejectionReason = 'PROPERTIES' | 'ENTITY' | 'TIME' | 'CONTEXT';`
Why an event was dropped: bad properties, bad entity id, out-of-range time, or a demo-only event from an account.

116 `export interface EventColumns {`
Columns an accepted event fills besides the envelope.

125 `export function checkEvent(`
Validates one client event; returns its columns or why it is dropped.

170 `export function suppress(value: number, subjects: number): number | null {`
A count behind fewer than {@link SUPPRESSION_THRESHOLD} people is not shown.

analytics.service.ts · 414 lines

30 `const INGEST_LIMIT: RateLimitRule = {`
At most 60 uploads a minute per person or demo session.

37 `export interface IngestResult {`
How many events of a batch were kept, and how many were dropped for each reason.

43 `export interface DashboardFilter {`
Period and segment of a dashboard.

51 `export type Cell = number | null;`
A count and the number of distinct people behind it; null when fewer than 10 (« < 10 »).

54 `interface Envelope {`
Columns the server adds to every event from its own facts (never sent by the app).

69 `export class AnalyticsService {`
Pseudonymous event ingestion and aggregates (section 16, Lock L42). The envelope is computed here from the server's own facts: the app sends only the event, its time and its counts/codes. Admins read aggregates only; no per-person timeline exists.

88 `async ingest(`
Stores a batch of app events: computes the envelope, rate-limits, checks every event against its schema and keeps only the valid ones.

124 `async recordServerEvent(event: ServerAnalyticsEvent): Promise<void> {`
SUBSCRIPTION_STARTED / RENEWED, published by billing through the jobs queue.

147 `private async envelopeOf(viewer: Viewer): Promise<Envelope> {`
The pseudonymous envelope of the caller: keyed hash, tier, profession, specialty and wilaya (demo: none of those).

174 `private subject(kind: 'user' | 'demo', id: string): string {`
Keyed hash: stable for one person, not reversible without the secret.

181 `assertPeriod(filter: DashboardFilter): void {`
Refuses empty, reversed or too long dashboard periods.

195 `private where(filter: DashboardFilter, from = filter.from, to = filter.to): Prisma.Sql {`
Real-account events of the period and segment (demo events stay in the demo funnel).

203 `async usage(filter: DashboardFilter) {`
« Usage »: active people by type, and daily actives against the previous period.

234 `private async dailyActives(filter: DashboardFilter, from: Date, to: Date) {`
Distinct people active per day (UTC) between two dates.

243 `async library(filter: DashboardFilter) {`
« Bibliothèque »: searches, and opens / views / favorites per model.

293 `async calculatorUse(filter: DashboardFilter) {`
« Calculateurs »: starts vs completions by calculator and version.

329 `async onboarding(filter: DashboardFilter) {`
« Utilisateurs »: account → submitted case → verified (people, not events).

345 `async subscriptionActivity(filter: DashboardFilter) {`
« Abonnements »: subscriptions and renewals by plan (money lives in Finance).

371 `async demoFunnel(filter: DashboardFilter) {`
« Démo »: starts → modules viewed → limits met → accounts (demo events only).

398 `async freshness(): Promise<Date | null> {`
When the latest event arrived (« mis à jour il y a … »).

407 `async activeSubjectsSince(from: Date, to: Date): Promise<Cell> {`
People active since `from` (overview ledger strip).

index.ts · 4 lines

apps/api/src/modules/audit/

The append-only audit log: record, list, history of one record.

audit.module.ts · 15 lines

14 `export class AuditModule {}`
The append-only audit log written on every admin mutation and on admin sign-in events. Owns: `audit_log_entries`. Other modules reach it only through ./index.ts.

audit.service.ts · 93 lines

8 `export interface AuditEntry {`
One line of the audit log: who did what to which record, with before/after values and a reason.

12 `action: string;`
Dotted verb, e.g. "admin.sign_in.succeeded".

23 `export interface AuditFilter {`
Filters of the audit log page and its CSV export.

29 `action?: string;`
Exact action, or a prefix ending with "." (e.g. "verification.").

33 `before?: bigint;`
Entries older than this id (newest first).

42 `export class AuditService {`
Writes the append-only audit log (the table rejects updates and deletes) and reads it back.

46 `async record(entry: AuditEntry): Promise<void> {`
Appends one entry. Called inside or right after every admin change.

64 `list(filter: AuditFilter): Promise<AuditLogEntry[]> {`
Newest first, `limit` entries (default one page).

89 `history(entityType: string, entityId: string, limit = 100): Promise<AuditLogEntry[]> {`
The history of one record (drawer « Historique » tabs), newest first.

index.ts · 10 lines

apps/api/src/modules/auth/

Proving who someone is: sign-up, codes, sign-in, sessions and refresh tokens, password reset, contacts and password changes, assisted recovery, account deletion.

account-deletion.service.ts · 78 lines

26 `export class AccountDeletionService {`
« Supprimer le compte » (section 08, Settings): after the password is confirmed, each module erases what it holds about the person — favorites and history, notifications, verification documents, codes, profile — then every session ends. A live subscription stops without refund (ADR-005); payments and past verification decisions are kept against an anonymous tombstone. Every step can run again, so a failure part-way is retried by the person asking again.

46 `async delete(userId: string, password: string, client: ClientInfo): Promise<void> {`
Deletes the account after checking the password: ends subscriptions, erases each module's data, anonymises the account, ends every session and writes an audit entry.

account-recovery.service.ts · 684 lines

47 `const OPEN: RecoveryStatus[] = ['SUBMITTED', 'APPROVED'];`
Statuses of a recovery that can still change the account.

51 `const hoursFromNow = (hours: number, now = new Date()) =>`
The moment `hours` after `now`.

57 `export interface RecoveryStarted {`
Answer of the first recovery step: the request and the code challenge sent to the new contact.

63 `export interface RecoveryAdminFilter {`
Filter of the recovery queue.

68 `function requireIdentifier(raw: string, field: string): Identifier {`
Parses an e-mail or phone, with the error on the given form field.

79 `const destinationOf = (request: Pick<AccountRecoveryRequest, 'newChannel' | 'newDestination'>) =>`
The new contact of a request, as a message destination.

95 `export class AccountRecoveryService {`
Assisted account recovery (ADR-023): « J'ai perdu l'accès à mon e-mail et à mon téléphone ». 1. The person names the lost contact, their name and a NEW contact, and proves the new contact with a code. A recovery token (kept by the app only) then allows two photo uploads: an identity document and a selfie holding it. 2. The account's current contacts are warned; the owner signing in cancels the request. 3. A reviewer (`verification.review`) compares the photos with the account and decides. 4. An approval applies after RECOVERY_HOLD_HOURS: the new contact becomes the only contact, the password becomes unusable and every device signs out; the person then sets a password with « Mot de passe oublié » on the new contact. Photos are deleted 30 days after the decision.

119 `async start(`
Step 1: records the lost contact, the name and the new contact, and sends a code to the new contact.

151 `tokenHash: sha256Hex(randomToken()),`
Unusable until the new contact is proven (the real token replaces it).

159 `async confirmContact(`
The new contact is proven: returns the token that allows the photos and the submission.

190 `async requestUpload(`
A signed upload for one of the two photos (private bucket, JPEG or PNG, 10 MB).

218 `async attach(`
Confirms an uploaded photo and puts it in its slot (replacing a previous one).

242 `async submit(recoveryToken: string, client: ClientInfo): Promise<{ answerWithinHours: number }> {`
« Envoyer »: both photos are there; the request goes to the review queue.

285 `async openForUser(`
A recovery of this account waiting for review or for its waiting period, if any.

300 `async cancelForOwner(userId: string): Promise<void> {`
Signing in with the password, or « Ce n'est pas moi » in Settings, proves the owner still has the account: open requests stop.

328 `async list(filter: RecoveryAdminFilter) {`
Admin queue: submitted and decided requests, oldest first.

338 `async detail(id: string) {`
One request with the matching account and its verification state, for the reviewer.

346 `async viewPhoto(admin: AdminPrincipal, id: string, slot: RecoverySlot, client: ClientInfo) {`
Restricted viewer: a 60-second signed URL to one photo; every view is audited.

375 `async approve(admin: AdminPrincipal, id: string, note: string, client: ClientInfo) {`
Approves a submitted request: it applies after the 24-hour hold unless the owner signs in meanwhile. Both the new contact and the account are told.

420 `async reject(`
Rejects a submitted or approved request with a reason; the new contact is told why.

465 `async applyDue(now = new Date()): Promise<number> {`
Applies approvals whose waiting period is over (every 10 minutes).

483 `async maintain(now = new Date()): Promise<void> {`
Expires requests whose photos never came, and deletes photos past their retention.

513 `await this.prisma.accountRecoveryRequest.deleteMany({`
Requests that never got past the code keep nothing worth keeping after a week.

523 `async eraseUser(userId: string): Promise<void> {`
Account deletion: open requests stop and their photos are deleted now.

541 `private async apply(request: AccountRecoveryRequest, now: Date): Promise<void> {`
Applies an approved request whose hold is over: the new contact replaces the old ones, the password becomes unusable and every device signs out.

544 `const { count } = await this.prisma.accountRecoveryRequest.updateMany({`
Claim the request before touching the account: the owner signing in meanwhile cancels it (cancelForOwner), and a cancelled request must never change the account.

607 `private async awaiting(id: string): Promise<AccountRecoveryRequest> {`
A request still waiting for its code and photos, within the time allowed.

620 `private async byToken(recoveryToken: string): Promise<AccountRecoveryRequest> {`
The request a recovery token opens, while photos may still be sent.

635 `private async find(id: string): Promise<AccountRecoveryRequest> {`
A request visible to admins (past the code step).

644 `private documentsDeadline(request: Pick<AccountRecoveryRequest, 'createdAt'>): Date {`
When the code and photos stop being accepted.

649 `private purgeDate(now = new Date()): Date {`
When a request's photos are deleted.

654 `private async deleteObject(key: string): Promise<void> {`
Deletes a photo; a failure is logged and retried by the next maintenance run.

663 `private async record(`
Audit entry of an admin decision on a request.

account-security.service.ts · 163 lines

17 `const channelName = (channel: Identifier['channel']) => (channel === 'EMAIL' ? 'EMAIL' : 'PHONE');`
Name of a contact kind in audit entries and notices.

26 `export class AccountSecurityService {`
Settings › Compte (ADR-023): add, change or remove the e-mail or phone, and change the password. Every change asks for the current password (a borrowed unlocked phone is not enough), a new contact is proven with a code sent to it, and the person is told on their other contact — the earliest warning if someone else is at work.

39 `async requestContact(`
Sends a code to the e-mail or phone to add (or to replace the current one of its kind).

67 `async confirmContact(`
The code was received: the contact is added (or replaces the previous one of its kind).

99 `async removeContact(`
Removes the e-mail or the phone, keeping the other (confirmed) one.

123 `async changePassword(`
Changes the password; every other device signs out, this one stays signed in.

152 `private async checkPassword(userId: string, password: string, field = 'password'): Promise<void> {`
Rate-limited check of the current password; a wrong one is a field error, not a sign-out.

account.controller.ts · 322 lines

67 `export class AccountSecurityController {`
Settings › Compte: contacts and password (ADR-023).

77 `async contacts(@CurrentUser() user: UserPrincipal): Promise<AccountContactsDto> {`
The e-mail and phone, and whether both are confirmed (« récupérable »).

88 `async cancelRecovery(@CurrentUser() user: UserPrincipal): Promise<AccountContactsDto> {`
« Ce n'est pas moi »: stops a recovery of this account someone else asked for.

98 `async requestContact(`
Sends a code to the contact to add; confirm it with `contacts/confirm`.

118 `async confirmContact(`
Confirms the code sent to the new contact; the contact is then added or replaces the old one.

132 `async removeContact(`
Removes the e-mail or the phone; the other one must be confirmed.

146 `async changePassword(`
Changes the password; other devices are signed out.

161 `export class AccountRecoveryController {`
« J'ai perdu l'accès à mon e-mail et à mon téléphone » (ADR-023). No session: the new contact is proven by a code, then a short-lived recovery token carries the two photos.

171 `async start(`
POST /v1/account-recovery: starts a recovery (no account token: the person has lost access).

184 `async confirm(`
Confirms the code sent to the new contact; returns the recovery token used for the photos.

197 `async upload(@Body() body: RecoveryUploadRequestDto): Promise<RecoveryUploadTicketDto> {`
A signed upload URL for one photo.

208 `async attach(@Body() body: RecoveryAttachDto): Promise<void> {`
Attaches an uploaded photo to the request.

217 `async submit(`
Sends the request for review once both photos are there.

232 `export class AccountRecoveryAdminController {`
Admin Web « Récupérations de compte »: the review queue of ADR-023.

239 `async list(@Query() query: RecoveryListQuery): Promise<AdminRecoveryDto[]> {`
Admin: the recovery requests.

247 `async detail(@Param('id', id) requestId: string): Promise<AdminRecoveryDetailDto> {`
Admin: one recovery request.

257 `async photo(`
A 60-second view of one photo, audited; needs the time-limited documents grant.

273 `async approve(`
Admin: approves a request (applies after the hold).

287 `async reject(`
Admin: rejects a request with a reason.

297 `private toDetail(`
Builds the recovery drawer response.

account.dto.ts · 361 lines

auth.controller.ts · 196 lines

59 `async signUp(@Body() body: SignUpDto, @Client() client: ClientInfo): Promise<OtpChallengeDto> {`
Creates (or restarts) an unconfirmed account and sends a 6-digit code.

69 `async confirmSignUp(`
Confirms the identifier with the code and signs the new account in on this device.

83 `async resend(@Body() body: ResendCodeDto): Promise<OtpChallengeDto> {`
Sends a new code for a pending challenge (after 60 s).

94 `async signIn(@Body() body: SignInDto, @Client() client: ClientInfo): Promise<AuthTokensDto> {`
POST /v1/auth/sign-in: password sign-in on a device.

105 `async refresh(@Body() body: RefreshDto, @Client() client: ClientInfo): Promise<AuthTokensDto> {`
Rotates the refresh token. Call it once at a time: a reused token revokes the session.

116 `async signOut(@CurrentUser() user: UserPrincipal): Promise<void> {`
POST /v1/auth/sign-out: ends this device's session.

125 `async requestPasswordReset(`
Always 202 with a challenge, whether or not an account exists.

139 `async confirmPasswordReset(`
Sets the new password and signs every device out.

157 `async list(@CurrentUser() user: UserPrincipal): Promise<SessionDto[]> {`
GET /v1/me/sessions: the devices signed in.

167 `async revoke(`
DELETE /v1/me/sessions/:id: signs one device out.

180 `export class AccountDeletionController {`
Settings « Supprimer le compte » (section 08): confirmed with the password, not reversible.

188 `async delete(`
POST /v1/me/account/deletion: deletes the account (password required).

auth.dto.ts · 258 lines

251 `export class DeleteAccountDto {`
« Supprimer le compte »: the password confirms it is the person.

auth.module.ts · 95 lines

72 `export class AuthModule implements OnModuleInit {`
User authentication: sign-up, OTP confirmation, sign-in, access/refresh token rotation, device sessions and password reset (ADR-009); contacts, password change, security notices and assisted account recovery (ADR-023); and account deletion, which asks each owning module to erase the person's data. Owns: `user_sessions`, `refresh_tokens`, `otp_challenges`, `account_recovery_requests`. Other modules reach it only through ./index.ts.

84 `onModuleInit(): void {`
Schedules the hourly purge of codes and sessions, and the 10-minute recovery job (apply due approvals, expire and clean up).

auth.policy.ts · 57 lines

7 `export const OTP_MAX_FAILED_ATTEMPTS = 3;`
Wrong codes allowed before the challenge locks (spec: 3 tries then 10 minutes).

10 `export const OTP_RESEND_COOLDOWN_SECONDS = 60;`
Minimum delay between two sends of the same challenge (spec: 60 s).

16 `export const SESSION_RETENTION_DAYS = 30;`
Revoked or ended sessions stay listable this long before the purge job removes them.

31 `accountChangePerUser: { name: 'account-change:user', limit: 10, windowSeconds: 3600 },`
Password checks of signed-in changes (contacts, password): a stolen phone can't brute-force.

42 `export const RECOVERY_DOCUMENTS_TTL_MINUTES = 60;`
The code and the photos must be sent within this delay after the request.

44 `export const RECOVERY_HOLD_HOURS = 24;`
An approved recovery applies after this delay; the owner signing in meanwhile cancels it.

46 `export const RECOVERY_DOCUMENT_RETENTION_DAYS = 30;`
Recovery photos are deleted this long after the decision (or the cancellation).

48 `export const RECOVERY_REVIEW_HOURS = 72;`
Requests answered by the team within this delay (shown to the person).

auth.service.ts · 217 lines

18 `export interface SignUpInput {`
What the sign-up form sends.

31 `function requireIdentifier(raw: string): Identifier {`
Parses an e-mail or Algerian mobile number; refuses anything else with a field error.

42 `function assertPasswordAllowed(password: string, identifier: Identifier): void {`
Refuses a password equal to the e-mail or phone it protects.

52 `export class AuthService {`
User sign-up, confirmation, sign-in, refresh, sign-out and password reset (ADR-009).

69 `async signUp(input: SignUpInput, client: ClientInfo): Promise<OtpChallengeView> {`
Creates (or restarts) an unconfirmed account and sends a 6-digit code to the e-mail or phone.

104 `async confirmSignUp(`
Checks the sign-up code, marks the contact confirmed and signs the new account in on this device.

115 `await this.demo.markConverted(device.id, userId);`
A demo tried on this device became an account (analytics « Démo » funnel).

120 `resendCode(challengeId: string, language: Language): Promise<OtpChallengeView> {`
Sends a new code for the same challenge (cooldown and limits apply).

128 `async signIn(`
Password sign-in: rate limit, lockout after repeated failures, suspended and unconfirmed accounts refused. A successful sign-in cancels any recovery someone else requested.

166 `await this.recovery.cancelForOwner(candidate.id);`
The owner has the account: a recovery someone else asked for stops here (ADR-023).

171 `refresh(refreshToken: string, deviceId: string, client: ClientInfo): Promise<AuthTokens> {`
Rotates the refresh token and returns new tokens.

178 `signOut(userId: string, sessionId: string): Promise<void> {`
Ends the current device session.

183 `async requestPasswordReset(`
Always answers with a challenge, whether or not the identifier has an account.

204 `async confirmPasswordReset(`
Checks the reset code, stores the new password, signs every device out and tells the person.

index.ts · 4 lines

otp.service.ts · 310 lines

31 `export interface OtpChallengeView {`
What the app needs to show the code screen.

35 `destinationHint: string;`
Masked destination for the OTP screen, e.g. "a•••@example.dz".

41 `export interface ConsumedChallenge {`
A code that was accepted: whose account and which contact it proves.

45 `destination: string;`
The e-mail or E.164 phone the code went to (the contact being added, for CONTACT_CHANGE).

49 `export interface ProvenContact {`
A proven contact: the person received the code sent there (account recovery).

55 `const MESSAGE_KEYS: Record<OtpPurpose, { subject: string; text: string }> = {`
Catalogue keys of the code message, per purpose.

76 `const delivers = (purpose: OtpPurpose, userId: string | null) =>`
Whether a challenge's code is really sent. A password reset for an unknown identifier sends nothing (no enumeration); an account recovery proves a contact that belongs to no account yet, so its code is always sent.

80 `export function maskDestination(channel: Identifier['channel'], destination: string): string {`
Hides most of an e-mail or phone for display ("a•••@x.dz", "+2136•••••67").

89 `const minutesFromNow = (minutes: number) => new Date(Date.now() + minutes * 60_000);`
The moment `minutes` from now.

93 `export class OtpService {`
One-time codes: sign-up, password reset (ADR-009), contact change and account recovery (ADR-023).

```
107 async issue(
```
Creates a challenge and sends its code. With `userId = null` (password reset for an unknown identifier) a challenge is still created and answered identically, but nothing is sent and no code can ever match it.

```
131 async resend(challengeId: string, locale: MessageLocale): Promise<OtpChallengeView> {
```
Sends a new code for the same challenge (60 s cooldown, 5 sends max, blocked while locked).

```
182 async consume(
```
Checks a code and consumes the challenge. Wrong codes count; the third locks the challenge for 10 minutes. Only a challenge of the expected purpose can be consumed.

```
197 async consumeContactProof(challengeId: string, code: string): Promise<ProvenContact> {
```
Account recovery: the code proves the person receives messages at this new contact.

```
206 private async verify(challengeId: string, purpose: OtpPurpose, code: string) {
```
Checks a code: right purpose, not used, not expired, not locked. A right code consumes the challenge once; a wrong one counts towards the lock.

```
258 async eraseUser(userId: string): Promise<void> {
```
Account deletion: no code of the person is left to confirm.

```
263 async purgeExpired(): Promise<number> {
```
Deletes challenges expired for more than a day (hourly maintenance job).

```
271 private hash(challengeId: string, code: string): string {
```
HMAC of the code bound to its challenge: the code itself is never stored.

```
276 private async send(
```
Formats the code message in the person's language and sends it.

```
292 private view(challenge: {
```
Builds the public view of a challenge (masked destination, timers).

security-notices.service.ts · 152 lines

```
9   export type SecurityEvent =
```
What happened to the account; each has `server.security.<kind>.subject|text` texts.

```
19  export type RecoveryContactEvent =
```
Messages to the new contact of a recovery, which is not (yet) on the account.

```
26  | 'recoveryApplied';
```
To the contacts an applied recovery replaced.

```
37  export type NoticeValues = Record<string, string | Date>;
```
Values of a notice: identifiers and codes as text, moments as dates (formatted per language).

```
46  export interface Destination {
```
Where a message goes: a channel and an address.

```
58  export class SecurityNoticesService {
```
Tells the person when something changes how their account is reached (ADR-023): password, contacts, recovery. Sent to the account's confirmed contacts (e-mail first; SMS only when it is the only contact, since texts cost money) and, while signed in, as an in-app notification. A notice that cannot be sent is logged and never blocks the change itself.

```
72  async notify(
```
Notifies the account. `extraDestinations` also receive it (e.g. the contact that was just replaced or removed, which is no longer on the account).

```
114 async toRecoveryContact(
```
A message to the new contact of an account recovery.

```
127 private async send(
```
Formats a notice in the person's language (dates in Algiers time) and sends it; failures are only logged.

sessions.service.ts · 233 lines

17 `export interface DeviceInput {`
The device a session is opened on.

24 `export interface AuthTokens {`
Tokens returned at sign-in and refresh.

33 `export interface SessionView {`
A session as listed in Settings › Appareils.

42 `type RefreshOutcome =`
What the refresh transaction decided.

47 `interface RefreshRow {`
The refresh token and its session, read together and locked.

60 `export class SessionsService {`
Device sessions and rotating refresh tokens (ADR-009).

71 `async open(userId: string, device: DeviceInput, client: ClientInfo): Promise<AuthTokens> {`
Opens a device session: a 180-day session, a 30-day refresh token and a 15-minute access token.

102 `async refresh(refreshToken: string, deviceId: string, client: ClientInfo): Promise<AuthTokens> {`
Rotates a refresh token. A reused token, a token from another device, an expired or revoked session, or a suspended account all revoke the session and answer SESSION_INVALID.

169 `async revoke(userId: string, sessionId: string, reason: SessionRevocationReason): Promise<void> {`
Revokes one of the user's sessions; revoking an unknown or foreign session is a 404.

178 `async revokeOthers(`
Every other device signs out; this one stays signed in (password change).

190 `async revokeAll(userId: string, reason: SessionRevocationReason): Promise<void> {`
Ends every session of the account (password reset, suspension, deletion, recovery).

198 `listActive(userId: string): Promise<SessionView[]> {`
The account's live sessions, most recently used first.

207 `async purge(): Promise<void> {`
Removes expired refresh tokens and long-ended sessions (hourly maintenance job).

217 `private async tokensFor(`
Issues the access token and returns it with the refresh token.

apps/api/src/modules/billing/

Plans, checkout, payment confirmation, webhooks, subscription lifecycle (reminders, grace, expiry), ledgers for Admin Web.

billing-admin.controller.ts · 124 lines

44 `export class BillingAdminController {`
The plan catalogue in Admin Web: prices are set by the Super Admin (ADR-006 D4).

51 `async list(): Promise<PlanDto[]> {`
Every version of every plan, newest version first.

61 `async publish(`
A new plan, or a new version of a plan code (a price change is a new version).

75 `async sale(`
POST /admin/v1/plans/:id/sale: starts or stops selling one version.

92 `export class PaymentMethodsAdminController {`
Finance › « Moyens de paiement » (ADR-021): which gateway takes each card, and the switch that pauses a card in the app (e.g. while its network is down). Gateways themselves are configured on the server, never from the browser.

99 `async list(): Promise<PaymentMethodRouteDto[]> {`
GET /admin/v1/payment-methods: methods with their gateway and switch.

115 `async setEnabled(`
PUT /admin/v1/payment-methods/:method: switches a method on or off.

billing-ledger.controller.ts · 127 lines

21 `export class LedgerPlanDto {`
A plan version as shown in the ledgers.

31 `export class LedgerPersonDto {`
The person a subscription or payment belongs to.

38 `export class LedgerTransactionRefDto {`
The latest payment of a subscription.

49 `export class AdminSubscriptionDto {`
One subscription in Admin Web.

65 `static from(row: LedgerSubscription): AdminSubscriptionDto {`
Converts a ledger row into the response.

82 `export function planRef(plan: LedgerSubscription['plan']): LedgerPlanDto {`
The plan fields shown in the ledgers.

94 `export class SubscriptionQuery {`
Filters of the subscriptions page.

116 `export class BillingLedgerController {`
« Abonnements » tab of Admin Web: who holds which plan, until when (section 07).

123 `async list(@Query() query: SubscriptionQuery): Promise<AdminSubscriptionDto[]> {`
50 per page, newest first; payments still pending are not subscriptions yet.

billing-ledger.service.ts · 353 lines

17 `export interface TransactionFilter {`
Filters of the transactions page.

22 `from?: Date;`
Created (or paid, for SUCCEEDED) on or after.

24 `to?: Date;`
...and before.

26 `search?: string;`
Exact public id (DT-...) or provider reference.

32 `export interface SubscriptionFilter {`
Filters of the subscriptions page.

40 `type PlanRef = Pick<`
The plan fields the ledgers need.

56 `export type LedgerTransaction = Transaction & {`
A transaction with its plan and the person's name.

62 `export type LedgerSubscription = Subscription & {`
A subscription with its plan, the person's name and its latest transaction.

72 `export interface RevenueRow {`
A SUCCEEDED payment as revenue sees it.

83 `firstOfSubscription: boolean;`
The first payment of its subscription (new), or a later one (renewal).

87 `export interface SubscriptionMetrics {`
Subscription figures for finance and the overview.

89 `mrrMinor: number;`
Normalised monthly value of live subscriptions at `at` (an annual plan counts ÷ 12).

91 `payingProfessionals: number;`
Distinct doctors and residents with a live subscription at `at`.

94 `ended: number;`
Subscriptions whose access ended (after grace, or at the end of a stopped renewal) in the period.

98 `export type FeeSettlementOutcome = 'SETTLED' | 'NOT_FOUND' | 'NOT_SUCCEEDED' | 'FEES_ABOVE_AMOUNT';`
What happened to one line of a fee settlement.

101 `export function planMonths(durationDays: number): number {`
Months a plan covers, for MRR: 30 days → 1, 90 → 3, 365 → 12.

106 `export function accessEnd(`
When a subscription's access ends: its grace end if it went through grace, else its period end.

117 `export class BillingLedgerService {`
Read models of billing for finance and Admin Web (transactions, subscriptions, revenue rows, subscription metrics) and the one write finance may ask for: fee settlement (ADR-005).

124 `async transactions(filter: TransactionFilter): Promise<LedgerTransaction[]> {`
One page of transactions, newest first.

153 `async transaction(id: string): Promise<LedgerTransaction | null> {`
One transaction with its plan and person.

162 `async subscriptions(filter: SubscriptionFilter): Promise<LedgerSubscription[]> {`
One page of subscriptions, newest first.

195 `async revenue(from: Date, to: Date): Promise<RevenueRow[]> {`
SUCCEEDED payments dated in [from, to).

225 `async subscriptionMetrics(at: Date, from: Date, to: Date): Promise<SubscriptionMetrics> {`
MRR and paying professionals at `at`; subscriptions ended in [from, to).

258 `async endedBetween(from: Date, to: Date): Promise<Date[]> {`
When each subscription that ended in [from, to) lost its access (for charts by bucket).

276 `async paymentIncidents(from: Date, to: Date, pendingSince: Date) {`
Transactions that failed or were abandoned in [from, to), and those still pending now.

292 `async settleFees(`
Replaces estimated fees with the provider's actual fee (the only change a final transaction accepts, enforced by the database). Each item is reported with its outcome.

334 `async forUser(userId: string) {`
Counts for the admin user drawer: the live subscription and the payments of one person.

343 `private async withUsers<T extends { userId: string }>(`
Adds the person's first and last name to each row.

billing.controller.ts · 237 lines

61 `export class BillingController {`
Subscription screens of the app (ScreenPlans, ScreenSubscriptionMobile). Real accounts only.

75 `async overview(@CurrentUser() user: UserPrincipal): Promise<SubscriptionOverviewDto> {`
GET /v1/subscription: the subscription screen.

97 `async listPlans(@CurrentViewer() viewer: Viewer): Promise<PlansDto> {`
The offers, with whether checkout is open (only once professional verification is VERIFIED). The demo sees the same offers with ACCOUNT_REQUIRED (ScreenDemo « Voir les offres »).

124 `async start(`
« Payer »: registers the order with the provider and returns its payment page.

146 `async transaction(`
A payment's status, confirmed with the provider while pending (the app polls it).

166 `async stopRenewal(@CurrentUser() user: UserPrincipal): Promise<void> {`
« Ne pas renouveler »: access continues to the period end.

178 `async paymentReturn(@Query() query: ReturnQuery): Promise<string> {`
Where the payment page sends the person back. The outcome is taken from the provider, never from this request; the page only tells the person to return to the app.

193 `fakePage(@Query('orderId') orderId: string): string {`
Development only: the simulated payment page of the fake provider.

210 `fakeSettle(`
Development only: the fake page's buttons choose the outcome and send the person back.

221 `private fake(): FakeGateway {`
The fake gateway, only when it is configured.

229 `function escapeHtml(value: string): string {`
Escapes text placed in the small HTML pages.

234 `function page(title: string, body: string): string {`
A minimal HTML page (payment return and fake page).

billing.dto.ts · 342 lines

254 `const TextList = (description?: string) =>`
A list of short plan lines (what is included / not included), up to 20 of 200 characters.

314 `export class PaymentMethodRouteDto {`
A payment method in Admin Web: the gateway that takes it and its switch.

billing.module.ts · 74 lines

54 `export class BillingModule implements OnModuleInit {`
Plans, subscriptions, checkout, payment gateways and confirmation, transactions (ADR-005, ADR-015, ADR-021). Owns: `subscription_plans`, `subscriptions`, `transactions`, `payment_method_settings`. Other modules reach it only through ./index.ts.

64 `onModuleInit(): void {`
Schedules the daily subscription advance and reminders, and the 10-minute reconciliation of pending checkouts.

billing.policy.ts · 26 lines

4 `export const RENEWAL_WINDOW_DAYS = 14;`
Renewal opens this many days before the period ends (« Renouveler »).

6 `export const GRACE_DAYS = 7;`
Access is kept this many days after the end, with a neutral notice.

8 `export const RENEWAL_REMINDER_DAYS = [14, 3] as const;`
Reminders before the end of a period (notifications, never banners).

10 `export const PENDING_CHECKOUT_MINUTES = 60;`
A checkout left unpaid is canceled after this long.

12 `export const RECONCILE_AFTER_MINUTES = 5;`
The provider is asked about pending checkouts older than this.

18 `export const ACKNOWLEDGEMENT_VERSION = 'pro-no-refund-v1';`
The acknowledgement the person ticks before paying (section 06). Its version is kept with the transaction, so what was acknowledged is known for every payment.

23 `export function addDays(date: Date, days: number): Date {`
The moment `days` after `date`.

checkout.service.ts · 510 lines

32 `export type Eligibility =`
Whether the person may pay now, and if not, why (shown on the plans screen).

43 `export interface CheckoutStarted {`
Answer of « Payer »: our transaction and the payment page to open.

49 `const LIVE = ['ACTIVE', 'GRACE', 'CANCELED'] as const;`
Subscription statuses that can still give access.

```
57  export class CheckoutService {
```
Checkout and payment confirmation (ADR-015). Nobody pays before professional verification is VERIFIED (ADR-006 Q6), every payment is confirmed server to server with the provider, and a transaction, once final, is never changed or deleted (ADR-005: no refund workflow exists).

```
75  async eligibility(userId: string, now = new Date()): Promise<Eligibility> {
```
Who may pay: an active, verified doctor or resident; students never pay; an active subscription may renew only in its last 14 days.

```
81  const { status, renewalPending } = (await this.verification.accessSnapshot(userId, now))
```
A renewal under review keeps a still-valid verification counting (section 27).

```
102 async start(
```
« Payer »: checks eligibility, plan and method, reuses a payment already in progress, creates the transaction and registers the order with the gateway. A gateway refusal marks the transaction FAILED.

```
192 async transaction(userId: string, transactionId: string): Promise<Transaction> {
```
The person's transaction, confirmed with the provider when still pending.

```
200 async settleByPublicId(transactionId: string): Promise<Transaction | null> {
```
Called by the payment page's return URL: confirms the order, whoever is calling.

```
209 async settleByProviderReference(
```
Called by a gateway's authenticated webhook: confirms that gateway's order with the gateway itself. Unknown orders (another application on the same merchant account) are ignored.

```
225 async settle(transaction: Transaction, now = new Date()): Promise<Transaction> {
```
Asks the provider for the outcome of a pending transaction and applies it: activation or extension on payment, failure with the provider's message, or cancellation when abandoned.

```
268 async stopRenewal(userId: string): Promise<void> {
```
« Ne pas renouveler »: records the choice; access continues to the period end.

```
278 async overview(userId: string): Promise<{
```
The subscription screen: the live subscription (or the latest one) and every payment.

```
300 async liveSubscription(userId: string, now = new Date()): Promise<Subscription | null> {
```
The subscription giving access now (active, in grace, or stopped but not yet ended), if any.

```
317 private abandoned(transaction: Transaction, now: Date): boolean {
```
True when a pending checkout is older than PENDING_CHECKOUT_MINUTES.

```
325 private async subscriptionFor(
```
The subscription a payment is for: the live one when renewing, otherwise a waiting one (reused) or a new one.

```
348 private async createTransaction(
```
DT-YYYY-NNNNNN, numbered per year under an advisory lock.

```
386 private async finish(
```
Closes a pending transaction as FAILED or CANCELED (only if still pending).

```
404 private async activate(
```
Records a confirmed payment and starts or extends the subscription (from the current end when renewing early). A short payment or a deleted account records the money but grants no access.

```
419 const deleted = (await this.users.getAccessFacts(transaction.userId))?.status === 'DELETED';
```
The money is recorded whatever happens; access is granted only for the full price and to an account that still exists (ADR-018). Anything else is left to the Super Admin.

```
494 private publicUrl(): string {
```
Base URL the gateway sends the person back to.

```
500 export function metadataOf(transaction: Pick<Transaction, 'metadata'>): Prisma.JsonObject {
```
The transaction's JSON metadata as an object (empty when missing).

```
506 export function orderNumberOf(transactionId: string): string {
```
Short order number (SATIM takes at most 10 characters): DT-2026-000123 → 26000123.

index.ts · 28 lines

payment-webhooks.controller.ts · 60 lines

19 `export const PAYMENT_WEBHOOKS_PATH = `/${USER_API_PREFIX}/payments/webhooks`;`
Path whose bodies are kept raw (app.setup.ts): signatures are computed over the exact bytes.

28 `export class PaymentWebhooksController {`
Gateway notifications (ADR-021), e.g. Chargily's `checkout.paid`. A notification is accepted only with a valid signature, and even then only names an order: its outcome is read back from the gateway, so a replayed or forged body can never activate anything by itself.

42 `async receive(@Param('gateway') gatewayId: string, @Req() request: Request): Promise<void> {`
POST /v1/payments/webhooks/:gateway: checks the signature, then confirms the named order with the gateway.

apps/api/src/modules/billing/payments/

Payment gateways behind one abstract class: Chargily, SATIM, and a fake gateway outside production.

chargily.gateway.ts · 182 lines

17 `export const CHARGILY_API = {`
Chargily Pay v2 API roots (dev.chargily.com/pay-v2/api-reference/introduction).

23 `const METHOD_NAMES: Record<PaymentMethod, string> = { CIB: 'cib', EDAHABIA: 'edahabia' };`
Chargily's names for the methods it accepts.

26 `const MINIMUM_DINARS = 50;`
Chargily's smallest accepted checkout amount, in dinars.

30 `type Json = Record<string, unknown>;`
A parsed JSON object.

33 `const integer = (value: unknown): number | null =>`
A whole number from a JSON field, or null.

45 `export class ChargilyGateway extends PaymentGateway {`
Chargily Pay (CIB of SATIM and EDAHABIA of Algérie Poste) over its v2 REST API (docs/operations/chargily-setup.md). A checkout is created for each payment; its status is read back server to server, and Chargily's signed webhook only triggers that read. Amounts: Chargily counts whole dinars, the API counts centimes. Fees are always the merchant's (`chargily_pay_fees_allocation: merchant`): the person pays exactly the price shown in the app.

61 `async register(order: RegisterOrder): Promise<RegisteredOrder> {`
Creates a Chargily checkout (whole dinars, at least 50 DZD, fees on the merchant) and returns its page.

92 `async status(providerReference: string): Promise<OrderOutcome> {`
Reads the checkout back from Chargily: paid (amount, fee, time), pending, or failed/canceled/expired.

123 `override verifyWebhook(rawBody: Buffer, headers: IncomingHttpHeaders): WebhookNotice | null {`
`signature` header = hex HMAC-SHA256 of the raw body with the API secret key.

143 `private async call(method: 'GET' | 'POST', path: string, body?: Json): Promise<Json> {`
One authenticated call to the Chargily API; network errors and refusals become PaymentProviderError.

176 `private secret(): string {`
The Chargily secret key (API and webhook signatures).

fake.gateway.ts · 83 lines

13 `const FAKE_FEE_RATE = 0.025;`
The fee the fake gateway reports, like Chargily's 2.5 % (illustrative only).

16 `interface FakeOrder {`
An order kept in memory with the outcome chosen on the fake page.

27 `export class FakeGateway extends PaymentGateway {`
Development and test stand-in for Chargily, the default gateway (refused in production by the environment check): it records Chargily's channels and reports a fee like Chargily does. Its payment page is served by the API itself and lets the developer choose the outcome.

40 `register(order: RegisterOrder): Promise<RegisteredOrder> {`
Keeps the order in memory and points to the API's own fake payment page.

51 `status(providerReference: string): Promise<OrderOutcome> {`
The outcome chosen on the fake page (pending until a button is pressed).

71 `settle(providerReference: string, outcome: 'PAID' | 'FAILED'): string | null {`
The fake page's buttons: returns where the person goes back, or null for an unknown order.

79 `describe(providerReference: string): RegisterOrder | null {`
The order shown on the fake page.

payment-gateways.ts · 124 lines

18 `export const PAYMENT_METHODS: readonly PaymentMethod[] = Object.values(PaymentMethods);`
Card networks in the order the app lists them.

21 `export interface MethodRoute {`
One payment method, the gateway that takes it and whether it is switched on.

24 `gateway: GatewayId | null;`
The configured gateway that takes this method, or null when none accepts it.

26 `enabled: boolean;`
Switched on or off by an admin (on unless a row says otherwise).

35 `export class PaymentGateways {`
The configured gateways (ADR-021). PAYMENT_GATEWAYS lists them by priority: each method is routed to the first gateway that accepts it, so a second gateway can take over a method, or be added for a new one, by configuration alone. Admins can switch a method off (an outage).

51 `async forMethod(method: PaymentMethod): Promise<PaymentGateway | null> {`
The gateway that takes a method now, or null (none configured, or switched off).

58 `forChannel(channel: PaymentProvider): PaymentGateway | null {`
The gateway that recorded a channel, to confirm its transactions (even if since demoted).

63 `byId(id: string): PaymentGateway | null {`
A configured gateway by id (null when not configured).

68 `async routes(): Promise<MethodRoute[]> {`
Every method with its gateway and switch, for the app's choice and Admin Web.

78 `async setEnabled(`
Admin Web « Moyens de paiement »: switches a method on or off, with a reason (audited).

105 `async availableMethods(): Promise<PaymentMethod[]> {`
The methods a person can pay with right now.

113 `const CHANNEL_METHODS: Record<PaymentProvider, PaymentMethod> = {`
The card network of each channel; a new channel does not compile until it is listed here.

121 `export function methodOfChannel(channel: PaymentProvider): PaymentMethod {`
The card network of a recorded channel.

payment-provider.ts · 88 lines

8 `export interface RegisterOrder {`
What is sent to a gateway to open its payment page.

10 `transactionId: string;`
Our public transaction id (DT-YYYY-NNNNNN), kept by the gateway as order metadata.

12 `orderNumber: string;`
Short order number derived from the transaction id (≤ 10 characters, for SATIM).

15 `currency: string;`
ISO 4217 alphabetic code; DZD in V1.

20 `returnUrl: string;`
Where the payment page sends the person back (success or failure).

22 `webhookUrl: string;`
Where the gateway posts its signed notifications, for gateways that send them.

26 `export interface RegisteredOrder {`
What a gateway answers when the order is created.

28 `providerReference: string;`
The gateway's order id, kept as the transaction's provider reference.

30 `redirectUrl: string;`
The gateway's payment page.

34 `export type OrderOutcome =`
The gateway's verdict on an order: paid (with amount and fee), still pending, or failed.

40 `feesMinor?: number;`
The fee the gateway kept, when it reports it; otherwise finance estimates it.

47 `export interface WebhookNotice {`
A verified notification names the order it is about; its content is never trusted as is.

52 `export class PaymentProviderError extends Error {`
The gateway refused or could not be reached; its message is shown to the person.

67 `export abstract class PaymentGateway {`
A card-payment gateway (ADR-015, ADR-021). The API never sees card data: the person pays on the gateway's page and the API asks the gateway, server to server, what happened. Adding a gateway is one class extending this one, its channels in the `PaymentProvider` enum, and its settings in `config/env.ts`; nothing else changes (checkout, finance, the app).

71 `abstract readonly channels: Readonly<Partial<Record<PaymentMethod, PaymentProvider>>>;`
The channel recorded on a transaction for each method this gateway accepts.

74 `abstract register(order: RegisterOrder): Promise<RegisteredOrder>;`
Creates the order at the gateway and returns its payment page.

77 `abstract status(providerReference: string, language: 'FR' | 'EN'): Promise<OrderOutcome>;`
The gateway's authoritative status of an order.

84 `verifyWebhook(_rawBody: Buffer, _headers: IncomingHttpHeaders): WebhookNotice | null {`
Checks a notification's signature against the raw body. Returns the order it is about, or null when it is not authentic (or the gateway sends none). The order is then confirmed with {@link status}, never from the notification's content.

satim.gateway.ts · 146 lines

13 `const NUMERIC_CURRENCY: Record<string, string> = { DZD: '012' };`
ISO 4217 numeric code SATIM expects.

15 `const ORDER_DEPOSITED = 2;`
SATIM's OrderStatus: 2 = amount deposited (paid).

20 `type SatimResponse = Record<string, unknown>;`
A parsed SATIM response.

23 `function text(value: unknown): string | null {`
A scalar field of a SATIM response as text (numbers and strings vary between versions).

36 `export class SatimGateway extends PaymentGateway {`
SATIM e-payment (CIB and Edahabia), contracted directly, over its REST API (docs/operations/satim-setup.md): `register.do` creates the order and returns the payment page; `confirmOrder.do` answers, server to server, whether it was paid. Credentials come from the merchant contract with SATIM.

49 `async register(order: RegisterOrder): Promise<RegisteredOrder> {`
register.do: creates the SATIM order and returns its payment page.

75 `async status(providerReference: string, language: 'FR' | 'EN'): Promise<OrderOutcome> {`
confirmOrder.do: whether the order was paid (OrderStatus 2), is still pending, or failed.

83 `if (!Number.isInteger(amountMinor) || amountMinor < 0) {`
Without a readable amount the payment cannot be checked against the price: the order stays pending and the reconciliation job asks again.

98 `private async call(method: string, params: Record<string, string>): Promise<SatimResponse> {`
One form-encoded call to SATIM with the merchant credentials.

123 `private message(response: SatimResponse): string | null {`
SATIM's human-readable message, wherever the response version puts it.

137 `private details(response: SatimResponse): Record<string, string> {`
What is kept with the transaction: never a full card number (SATIM returns it masked).

apps/api/src/modules/billing/

Plans, checkout, payment confirmation, webhooks, subscription lifecycle (reminders, grace, expiry), ledgers for Admin Web.

plans.service.ts · 155 lines

10 `export interface PlanInput {`
A plan version as the Super Admin writes it.

31 `export class PlansService {`
The plan catalogue (ADR-006 D4: Pro Monthly and Pro Annual, prices set by the Super Admin). A plan is never edited: a new price or inclusion list is a new version of the same code, and the older versions stop being sold. Subscriptions and transactions keep the version they bought.

38 `async purchasable(professionalType: ProfessionalType): Promise<SubscriptionPlan[]> {`
The purchasable plans for a professional type, one (latest) version per code.

46 `all(): Promise<SubscriptionPlan[]> {`
Every version of every plan (Admin Web).

53 `async byId(planId: string): Promise<SubscriptionPlan> {`
One plan version, or NOT_FOUND.

60 `async publish(`
Publishes a plan code or a new version of it; earlier versions of the code stop being sold.

121 `async setPurchasable(`
Starts or stops selling one version (only one version of a code is sold at a time).

subscription-lifecycle.service.ts · 113 lines

20 `export class SubscriptionLifecycleService {`
Manual renewal in V1 (section 06): reminders at 14 and 3 days, 7 days of grace with access kept, then « Terminé ». Calm by design: notifications, never banners, countdowns or red (Lock L39).

28 `async advance(now = new Date()): Promise<void> {`
Moves ended periods to grace, ended grace to expired, stopped renewals to expired.

77 `async remind(now = new Date()): Promise<void> {`
One reminder per threshold before the end of an active period.

102 `async reconcile(now = new Date()): Promise<void> {`
Confirms checkouts nobody came back from, and cancels the abandoned ones.

subscription-status.service.ts · 47 lines

6 `export interface SubscriptionSnapshot {`
The subscription facts the access policy looks at.

14 `export class SubscriptionStatusService {`
Read side of billing used by the access policy.

21 `async endForDeletedAccount(userId: string): Promise<number> {`
Account deletion: a live subscription ends now, without refund (ADR-005). Its payments stay in the ledger; a checkout still waiting for the provider is left to the payment flow.

```
30  async current(userId: string): Promise<SubscriptionSnapshot | null> {
```
The user's most relevant subscription: a live one first, otherwise the most recent.

apps/api/src/modules/calculators/

Calculators for the app (list, detail, published version) and their governance for Admin Web (drafts, sources, tests, review, publication).

calculators-admin.controller.ts · 314 lines

```
46  export class CalculatorsAdminController {
```
Calculator governance for Admin Web (section 28, grants `calculators.*`).

```
54  async list(): Promise<AdminCalculatorDto[]> {
```
GET /admin/v1/calculators.

```
63  async detail(@Param('id', uuid) id: string): Promise<AdminCalculatorDto> {
```
GET /admin/v1/calculators/:id.

```
73  async create(
```
POST /admin/v1/calculators.

```
86  async update(
```
Names and category; access tier and demo sample additionally need `calculators.publish`.

```
103 async createVersion(
```
« Nouvelle version » (draft).

```
117 async version(@Param('versionId', uuid) versionId: string): Promise<AdminVersionDto> {
```
GET /admin/v1/calculators/versions/:versionId.

```
126 async saveDefinition(
```
PUT .../versions/:versionId/definition: saves the draft's definition.

```
143 async addSource(
```
POST .../versions/:versionId/sources.

```
157 async removeSource(
```
DELETE .../versions/:versionId/sources/:sourceId.

```
174 async addTestCase(
```
POST .../versions/:versionId/test-cases.

```
195 async removeTestCase(
```
DELETE .../versions/:versionId/test-cases/:testCaseId.

```
212 async runTests(
```
« Lancer les tests » (authors and reviewers).

```
228 async submit(
```
« Soumettre à revue ».

```
242 async returnToDraft(
```
« Renvoyer à l'auteur ».

```
259 async publish(
```
« Approuver et publier » — reviewer ≠ author, primary source, all tests passing.

```
276 async retire(
```
« Retirer ».

```
293 async discard(
```
DELETE .../versions/:versionId: deletes a draft.

```
307 async preview(
```
« Aperçu » with the backend engine; inputs are not stored.

calculators-admin.service.ts · 822 lines

```
28  export interface NewCalculator {
```
A new calculator as Admin Web creates it.

```
export interface CalculatorPatch {
```
Changes to a calculator's names, category or access.

```
export interface NewSource {
```
A cited source of a version.

```
export interface NewTestCase {
```
A test case written by the author from a cited example.

```
type StoredTestRun = TestRun & { definitionSha256: string };
```
A stored test run also records which definition it ran against.

```
const VERSION_INCLUDE = {
```
Relations loaded with a version in Admin Web.

```
export class CalculatorsAdminService {
```
Calculator governance for Admin Web (section 28, ADR-012 §6). No formula is written here: the definition is data authored from a cited source; this service enforces the review workflow.

```
async list(): Promise<CalculatorRecord[]> {
```
Every calculator with all its versions.

```
async detail(calculatorId: string): Promise<CalculatorRecord> {
```
One calculator with all its versions.

```
async createCalculator(
```
Creates a calculator (no version yet); a used slug is refused.

```
async updateCalculator(
```
Names and category need `calculators.draft`; access tier and demo sample change who can use it.

```
async createVersion(
```
« Nouvelle version »: from a definition, or copying an earlier version with its sources and tests.

```
async saveDefinition(
```
Replaces the definition of a draft (validated first); the previous test run no longer counts.

```
async addSource(
```
Adds a source to a draft.

```
async removeSource(
```
Removes a source from a draft.

```
async addTestCase(
```
Adds a test case to a draft; a RESULT case needs a value and a tolerance.

```
async removeTestCase(
```
Removes a test case from a draft.

```
async runTests(
```
« Lancer les tests » on the authoritative engine; the run is stored with the definition checksum.

```
async submit(
```
« Soumettre à revue »: needs a passing run of the current definition and a primary source.

```
async returnToDraft(
```
« Renvoyer à l'auteur ».

```
async publish(
```
« Approuver »: re-runs every test on the backend engine and checks sources and the four-eyes rule. When two sign-offs are required and none is recorded yet, records the first one (ADR-024); otherwise publishes, makes it current and retires the previously published version.

```
private async signOff(
```
The first of two sign-offs: the version stays in review for a second, different reviewer.

```
async retire(
```
« Retirer »: devices drop it at the next sync; a retired current version leaves the calculator unavailable.

```
async discard(admin: AdminPrincipal, versionId: string, client: ClientInfo): Promise<void> {
```
Deletes an unpublished draft together with its sources and tests.

677 `async preview(versionId: string, inputs: EvaluationInputs): Promise<Evaluation> {`
« Aperçu »: any version, on the backend engine. Inputs are not stored.

682 `async version(versionId: string): Promise<VersionRecord> {`
One version with its sources, tests and people, and the sign-offs it currently needs.

695 `private versionsOf(calculatorId: string): Promise<VersionRecord[]> {`
All versions of a calculator, newest first.

704 `private async requireStatus(`
Loads a version and refuses the action if it is not in one of the allowed statuses.

719 `private requireValid(raw: unknown): CalculatorDefinition {`
Checks a definition and turns its problems into field errors.

730 `private definitionOf(version: VersionRecord): CalculatorDefinition {`
The parsed definition of a stored version.

735 `private testRunFor(version: VersionRecord): StoredTestRun {`
Runs a version's tests now and tags the run with the definition it ran on.

751 `private storedRun(version: VersionRecord): StoredTestRun | null {`
The stored test run, only if it ran on the current definition.

757 `private assertPublishable(version: VersionRecord, run: StoredTestRun | null): void {`
The publication gate of section 28 (the database enforces it again).

778 `private async assertIndependentReviewer(admin: AdminPrincipal, versionId: string): Promise<void> {`
Four-eyes: the reviewer is neither the author nor anyone who changed the draft.

800 `private record(`
Audit entry of a calculator change.

calculators.controller.ts · 103 lines

45 `export class CalculatorsController {`
Calculators for the app (section 28). Works with user and demo tokens.

52 `async list(`
GET /v1/calculators: the catalogue (optional search words).

65 `async get(`
The published version (definition, sources, review) when the calculator is available to the viewer.

82 `async evaluate(`
Authoritative evaluation of the current published version. Inputs are never stored.

calculators.dto.ts · 649 lines

76 `const PURPOSE_MAX_LENGTH = 160;`
One-line purpose shown under the name in lists and search.

607 `export class PublishedTestCaseDto {`
A reference test case of a published version; the device runs them before first use.

calculators.module.ts · 25 lines

24 `export class CalculatorsModule {}`
Calculator definitions, versions, sources, test cases and the authoritative TypeScript engine (ADR-003, ADR-012). No formula lives in this module's code: definitions are reviewed data. Owns: `calculators`, `calculator_versions`, `calculator_sources`, `calculator_test_cases`. Other modules reach it only through ./index.ts.

calculators.policy.ts · 22 lines

4 `export const SLUG_PATTERN = /^[a-z0-9]+(-[a-z0-9]+)*$/;`
URL name of a calculator: lower-case words joined by dashes.

13 `export const DRAFTING_ACTIONS = [`
Audit actions that change a draft. An admin who performed any of them on a version may not review or publish it (four-eyes, ADR-012 §6).

calculators.service.ts · 350 lines

21 `export interface CalculatorSummary {`
A calculator in the app's lists, with the viewer's access.

34 `export interface PublishedSource {`
A source shown under a calculator.

47 `export interface PublishedVersion {`
The published version the app runs: definition, checksum, review and sources.

59 `export interface PublishedTestCase {`
A test case sent to the app so its offline engine can check itself.

68 `const PUBLISHED_INCLUDE = {`
Relations loaded with a calculator's published version.

79 `const SUMMARY_SELECT = {`
What a list needs of a calculator: its names, access facts and current version number.

93 `type SummaryRow = Omit<Row, 'currentVersion'> & { currentVersion: { version: number } | null };`
A calculator row as lists read it.

96 `type Row = {`
A calculator row with its published version, as read here.

127 `export class CalculatorsService {`
What users see of the calculators (section 28): active calculators with a published version, each with the server's access decision. The definition, sources and review are returned only when the viewer may use the calculator. Inputs are evaluated, never stored.

134 `async labels(`
Names of calculators by id (admin read models).

147 `async awaitingReview(limit: number) {`
Versions submitted for review, oldest first.

165 `async list(viewer: Viewer, query?: string): Promise<CalculatorSummary[]> {`
Active calculators with a published version, optionally filtered by words, A–Z, each with its lock.

176 `async get(`
One calculator by slug; its definition and tests are sent only when the viewer may use it.

206 `async evaluate(`
Online evaluation with the authoritative engine, on the current published version only.

233 `async openable(`
Every calculator the viewer may use, with its test cases for the device self-test (offline manifest).

250 `async summariesByIds(viewer: Viewer, ids: readonly string[]): Promise<CalculatorSummary[]> {`
The published calculators among `ids` (favorites), with access; others are left out.

257 `private async summaryRows(ids?: string[]): Promise<SummaryRow[]> {`
Active calculators with a published version (only what a list shows), optionally by id.

269 `private async rows(): Promise<Row[]> {`
Every active calculator with a published version.

278 `private async withAccess<R extends SummaryRow>(`
Adds the viewer's access decision to each calculator.

308 `function testCasesOf(row: Row): PublishedTestCase[] {`
The published version's test cases in the app's shape.

319 `function published(row: Row): PublishedVersion {`
The published version in the app's shape (its definition is checked again).

apps/api/src/modules/calculators/engine/

The authoritative calculator engine: the definition schema, deeper checks, evaluation, test runs, canonical JSON checksum.

canonical-json.ts · 25 lines

4 `export function canonicalJson(value: unknown): string {`
JSON with object keys sorted at every level, so equal definitions hash equally.

9 `function sortKeys(value: unknown): unknown {`
A copy of a JSON value with object keys sorted, recursively.

22 `export function definitionSha256(definition: unknown): string {`
The checksum recorded on a version (`definitionSha256`).

check-definition.ts · 278 lines

11 `export interface DefinitionProblem {`
One problem found in a definition, with where it is.

17 `export type CheckResult =`
Either the parsed definition, or the problems.

21 `type ValueType = 'number' | 'boolean' | 'string';`
The types a formula node can have.

30 `export function checkDefinition(raw: unknown): CheckResult {`
Validates a definition beyond its schema: unique keys, units, nested ranges, scored choices, formula types, rules and ordered, non-overlapping bands.

121 `function within(inner: Range | undefined, outer: Range | undefined): boolean {`
True when `inner` lies inside `outer` (missing bounds are unbounded).

129 `class Typer {`
Infers the type of every formula node and reports type errors.

138 `type(node: Expr, path: string, depth: number): ValueType | null {`
Returns the type of `node`, or null after reporting a problem.

252 `private inputType(inputKey: string, path: string): ValueType | null {`
The type an input produces in a formula (CHOICE gives its value as text).

262 `private checkChoiceLiteral(args: [Expr, Expr], path: string): void {`
`eq(input(choice), str("X"))`: "X" must be one of the declared choice values.

definition.ts · 191 lines

7 `export const localizedText = z`
A text in French and English (both required).

16 `const key = z.string().regex(/^[a-z][a-zA-Z0-9]{0,39}$/, 'keys are camelCase, 1-40 characters');`
Identifier of an input or rule: camelCase, 1–40 characters.

21 `export const range = z`
A numeric interval; either bound may be open.

28 `export type Expr =`
Closed operator set of the formula AST (audit §9).

62 `export const expr: z.ZodType<Expr> = z.lazy(() =>`
Validation schema of the formula tree (recursive).

95 `const unitConversion = z`
A non-canonical unit and how it converts: canonical = value × factorToCanonical.

100 `source: z.string().trim().min(1).max(200),`
Where the factor is stated in the UNIT_CONVERSION source (page, table…).

105 `const choice = z`
One answer of a CHOICE input, with its optional points.

114 `const baseInput = {`
Fields every input has.

122 `const numericInput = {`
Fields of NUMBER and INTEGER inputs: what is measured, units and the three ranges.

```
125 quantity: z.string().regex(/^[A-Z][A-Z0-9_]{1,39}$/),
```
What is measured, e.g. BODY_WEIGHT (free code, used for display and analytics).

```
130 decimals: z.number().int().min(0).max(6).default(0),
```
Display decimals of the field.

```
137 export const calculatorInput = z.discriminatedUnion('kind', [
```
One input of a calculator: NUMBER, INTEGER, CHOICE or BOOLEAN.

```
149 export const crossFieldRule = z
```
A rule between several inputs (e.g. "diastolic below systolic").

```
153 when: expr,
```
Must be true for the inputs to be acceptable.

```
160 export const interpretationBand = z
```
A result band: from ≤ value < to, with a severity and a label.

```
172 export const calculatorDefinition = z
```
A complete calculator definition (schema version 1).

evaluate.ts · 352 lines

```
17 export type InputValue = { value: number | string | boolean | null; unit?: string };
```
One value typed or chosen by the person, with its unit for numbers.

```
19 export type EvaluationInputs = Record<string, InputValue | null | undefined>;
```
All values, by input key.

```
22 export type IssueCode =
```
Why a calculation gives no result.

```
36 export interface Issue {
```
One reason, pointing at the input, range or rule involved.

```
40 range?: Range & { unit: string };
```
The range that was not met, in the input's canonical unit.

```
46 export interface Conversion {
```
A value converted to the input's canonical unit (shown to the person).

```
53 export interface PlausibilityNote {
```
A value outside the usual range: shown, never blocking.

```
59 export type Evaluation =
```
What the engine answers: a result with its band, or why there is none.

```
72 type Value = number | string | boolean;
```
A value during evaluation.

```
75 class MissingInput extends Error {
```
Thrown inside the formula when an optional input it needs was left empty.

```
82 const inRange = (value: number, range: Range | undefined) =>
```
True when a value lies in a range (missing bounds are open).

```
92 export function evaluate(definition: CalculatorDefinition, raw: EvaluationInputs): Evaluation {
```
Runs a calculator: presence and types, hard ranges, cross-field rules, validated ranges, plausibility notes, then the formula and its band. Never returns a result for invalid or out-of- range inputs.

```
117 for (const input of numericInputs(definition)) {
```
2. Hard ranges: physiologically impossible values block.

```
150 for (const input of numericInputs(definition)) {
```
4. Validated ranges: outside the population the formula was validated on → not computed.

```
213 function numericInputs(definition: CalculatorDefinition): NumericInput[] {
```
The NUMBER and INTEGER inputs.

```
220 function acceptValue(
```
Checks one value against its input and converts numbers to the canonical unit.

257 `function run(node: Expr, values: Map<string, Value>, definition: CalculatorDefinition): Value {`
Evaluates one node of the formula tree recursively.

340 `export function roundHalfAwayFromZero(value: number, digits: number): number {`
Rounds to `digits` decimals, halves away from zero (2.5 → 3, −2.5 → −3). Shifting the decimal exponent of the shortest representation avoids the binary error of multiplying by 10^digits (1.005 → 1.01), and handles numbers written with an exponent (1.5e-7, 1e21).

348 `function shiftDecimal(value: number, places: number): number {`
value × 10^places, computed on the decimal representation.

index.ts · 24 lines

run-test-cases.ts · 101 lines

7 `export interface TestCaseSpec {`
A test case: inputs and the expected outcome, value (± tolerance) and band.

17 `export interface TestCaseResult {`
The outcome of one test case, with the reason when it fails.

27 `failure: string | null;`
Why the case failed, in plain words.

31 `export interface TestRun {`
A full run of a version's test cases.

41 `export function runTestCases(`
Runs a version's test cases on the authoritative engine (publication gate, admin "Lancer les tests").

apps/api/src/modules/calculators/

Calculators for the app (list, detail, published version) and their governance for Admin Web (drafts, sources, tests, review, publication).

index.ts · 22 lines

apps/api/src/modules/clinical-review/

The double sign-off policy for clinical content (one or two reviewers).

clinical-review.controller.ts · 42 lines

15 `export class ClinicalReviewController {`
« Double relecture »: the Super Admin's sign-off policy for clinical content (ADR-024).

22 `async list(): Promise<ReviewPolicyDto[]> {`
GET /admin/v1/clinical-review/policies.

31 `async update(`
PUT /admin/v1/clinical-review/policies/:content: one or two sign-offs.

clinical-review.dto.ts · 48 lines

13 `export class ReviewPolicyDto {`
How many independent sign-offs a kind of clinical content needs (ADR-024).

clinical-review.module.ts · 21 lines

20 `export class ClinicalReviewModule {}`
Double sign-off for clinical content (ADR-024): how many independent reviewers calculators and pregnancy profiles need before publication. Owns: `clinical_review_policies`. Other modules reach it only through ./index.ts.

clinical-review.service.ts · 160 lines

14 `const REVIEW_GRANTS: Record<ClinicalContentKind, readonly Grant[]> = {`
The grants that let an admin approve each kind of content.

20 `export interface ReviewPolicy {`
The sign-off rule of one kind of clinical content, as Admin Web shows it.

24 `eligibleReviewers: number;`
Active admins who may approve this content (the author of a version never counts for it).

30 `export interface Approval {`
What an approval does: record the first of two sign-offs, or publish.

43 `export class ClinicalReviewService {`
Double sign-off for clinical content (ADR-024). One sign-off — the reviewer who publishes — is the rule of ADR-006 D1; the Super Admin may require two for calculators, pregnancy profiles or both. The second reviewer is never the author nor the first reviewer. Owns: `clinical_review_policies`.

50 `async required(content: ClinicalContentKind): Promise<SignOffs> {`
The number of sign-offs this kind of content needs now (1 unless set to 2).

56 `async policies(): Promise<ReviewPolicy[]> {`
Both policies with how many admins could approve.

78 `async setPolicy(`
Two sign-offs need two people who may approve; with fewer, nothing could ever be published, so the change is refused.

120 `async requiredFor(`
Sign-offs an open version needs: the stricter of the rule when it was submitted and the rule now, and always two once a first sign-off is recorded. Lowering the policy never relaxes a version already in review.

129 `async approval(`
Decides what an independent reviewer's approval does: a first sign-off, or publication.

148 `private eligibleReviewers(content: ClinicalContentKind): Promise<number> {`
Active admins holding every grant needed to approve this content.

index.ts · 4 lines

apps/api/src/modules/demo/

Demo sessions: start, renew, mark converted, statistics.

demo.controller.ts · 45 lines

18 `export class DemoController {`
POST /v1/demo/sessions and /v1/demo/tokens (no account).

25 `async start(@Body() body: StartDemoDto, @Client() client: ClientInfo): Promise<DemoSessionDto> {`
« Explorer la démo »: no sign-up, no contact data. Returns a demo token and its renewal token.

34 `async renew(`
POST /v1/demo/tokens: renews the demo access token.

demo.dto.ts · 66 lines

7 `export class StartDemoDto {`
Body of « Essayer la démo »: the profession to preview and the device id.

20 `export class RenewDemoTokenDto {`
Body of a demo token renewal.

35 `export class DemoTokenDto {`
A demo access token and its expiry.

44 `export class DemoSessionDto extends DemoTokenDto {`
A started demo session.

demo.module.ts · 18 lines

17 `export class DemoModule {}`
Demo sessions and their tokens (Lock L37, ADR-006 Q5, ADR-010). The demo content subset is the `demoSample` flag on calculators and reference documents, decided by the access module. Owns: `demo_sessions`. Other modules reach it only through ./index.ts.

demo.service.ts · 129 lines

12 `const RATE_LIMITS = {`
Limits on starting and renewing demo sessions, per IP.

18 `export interface DemoSessionGrant {`
What starting a demo returns: the session, a one-hour token and a renewal token.

27 `const clientKey = (client: ClientInfo) => client.ip ?? 'unknown';`
Rate-limit key: the caller's IP.

34 `export class DemoService {`
Demo sessions (Lock L37, ADR-006 Q5, ADR-010): no account, no contact data, no time limit. A device keeps one active session; nothing a demo does is written to user tables.

45 `async start(`
Starts (or restarts) the demo of a device: one active session per device, a new renewal token each time.

78 `async renew(`
A new one-hour demo token, from the renewal token and the same device.

111 `async markConverted(deviceId: string, userId: string): Promise<void> {`
A new account was confirmed on this device: its active demo session ends as CONVERTED (the « Démo » funnel counts it). The app uses one installation id for the demo and the account.

119 `async stats(from: Date, to: Date): Promise<{ started: number; converted: number }> {`
Demo sessions started in [from, to), and how many of them became accounts.

index.ts · 4 lines

apps/api/src/modules/finance/

Revenue, fees, expenses, adjustments, reports, and the transactions ledger for Admin Web.

finance.controller.ts · 172 lines

50 `export class FinanceController {`
« Finance » of Admin Web (ScreenAdminRevenue · ScreenExpenses · reports), section 15.

57 `async overview(@Query() query: PeriodQuery): Promise<FinanceOverviewDto> {`
GET /admin/v1/finance/overview.

65 `async feeRates(): Promise<FeeRateDto[]> {`
GET /admin/v1/finance/fee-rates.

81 `async setFeeRate(`
PUT /admin/v1/finance/fee-rates/:provider.

100 `async expenses(@Query() query: PeriodQuery): Promise<ExpenseDto[]> {`
GET /admin/v1/finance/expenses.

110 `async addExpense(`
POST /admin/v1/finance/expenses.

123 `async voidExpense(`
POST /admin/v1/finance/expenses/:id/void.

137 `async settleFees(`
Actual fees from the provider's statement replace the estimates.

154 `async report(`
CSV with the metric definitions on its first lines (section 15).

finance.dto.ts · 461 lines

finance.module.ts · 23 lines

22 `export class FinanceModule {}`
Revenue aggregates, operating expenses, fee estimates and settlement, reports and exceptional adjustments (section 15, ADR-005, ADR-017). Owns: `transaction_adjustments`, `operating_expenses`, `payment_fee_rates`. Other modules reach it only through ./index.ts.

finance.policy.ts · 177 lines

9 `export const ADJUSTMENT_REASONS = [`
Reason codes of an exceptional adjustment (section 15: law, accounting, provider error).

19 `export const MAX_PERIOD_DAYS = 400;`
Longest period one overview or report covers.

24 `export interface FeeRate {`
A channel's fee: rate in basis points (1 % = 100) plus a fixed amount in centimes.

30 `export interface Period {`
A period: from inclusive, to exclusive.

38 `export function previousPeriod({ from, to }: Period): Period {`
The equal period just before (comparison of section 15).

44 `export function bucketFor({ from, to }: Period): Bucket {`
Days for ≤ 45 days, ISO weeks for ≤ 6 months, months beyond.

50 `export function bucketStart(date: Date, bucket: Bucket): Date {`
Start (UTC) of the bucket holding `date`.

59 `export function nextBucket(start: Date, bucket: Bucket): Date {`
Start of the bucket after `start`.

66 `export function bucketStarts(period: Period, bucket: Bucket): Date[] {`
Every bucket start overlapping the period.

79 `export function estimateFee(amountMinor: number, rate: FeeRate): number {`
Estimated provider fee of one payment: rate + fixed part, never above the amount.

84 `export interface FeeView {`
The fee of one payment: actual or estimated.

87 `estimated: boolean;`
True while the provider has not settled the actual fee.

89 `rateMissing: boolean;`
No rate is configured for the provider: the estimate is 0 and must be shown as unknown.

96 `export function feeOf(`
The fee of one payment: the settled one, else an estimate from the channel's rate (0 and flagged when no rate is set).

110 `export interface Ledger {`
The figures of the Finance ledger.

112 `grossMinor: number;`
Payments plus approved exceptional adjustments.

114 `adjustmentsMinor: number;`
Sum of the approved exceptional adjustments (negative for reversals).

117 `estimatedFeesMinor: number;`
Part of the fees that is still an estimate.

125 `export function ledgerOf(input: {`
Gross → fees → net revenue; net revenue → expenses → net profit (Lock L40).

157 `export function csvDocument(`
CSV with `;` separators (Excel in French locales) and a UTF-8 BOM; definitions first.

```
172 export function decimal(minor: number): string {
```
Minor units as a plain decimal for spreadsheets: 150050 → "1500.50".

finance.service.ts · 647 lines

```
38 const PROVIDERS = Object.values(PaymentProviders);
```
Every payment channel (ADR-021): a fee rate is set per gateway × card.

```
40 const STALE_PENDING_MINUTES = 60;
```
A checkout pending longer than this is an incident on the overview.

```
43 export type ExpenseWithActors = OperatingExpense & {
```
An expense with who recorded it and who voided it.

```
49 export type AdjustmentWithActors = TransactionAdjustment & {
```
An exceptional adjustment with who requested and who approved it.

```
55 export interface SeriesPoint {
```
One bar of the revenue chart (a day, week or month).

```
58 partial: boolean;
```
The bucket is not over yet (« période en cours »).

```
67 export interface FinanceOverview {
```
Everything the Finance page shows for a period and the one before.

```
75 feeRatesMissing: PaymentProvider[];
```
Providers without a fee rate: their estimated fees are unknown.

```
93 export type ReportKind = 'revenue-by-plan' | 'subscriptions' | 'provider-fees';
```
The CSV reports Finance can export.

```
101 export class FinanceService {
```
Revenue aggregates, operating expenses, fee estimates and settlement, exceptional adjustments and reports (section 15, ADR-005). Transactions stay billing's; finance only reads them, except for the fee settlement billing performs on its behalf.

```
112 assertPeriod(period: Period): void {
```
Refuses empty, reversed or too long periods.

```
128 async feeRates(): Promise<Map<PaymentProvider, FeeRate>> {
```
The configured fee rate of each payment channel.

```
136 async feeRateRows() {
```
Every channel with its rate (or none), for the fee settings table.

```
147 async setFeeRate(
```
Sets the estimated fee of one channel (percentage in basis points + fixed part); audited.

```
170 expenses(period: Period): Promise<ExpenseWithActors[]> {
```
Operating expenses of a period, voided ones included (shown struck through).

```
182 async addExpense(
```
Records an operating expense; audited.

```
220 async voidExpense(
```
Voids an expense (never deleted), with a reason; audited.

```
249 async settleFees(
```
Replaces estimated fees with the actual fees of the provider's statement; audited.

```
273 adjustmentsOf(transactionId: string): Promise<AdjustmentWithActors[]> {
```
The adjustments of one transaction, oldest first.

```
288 async requestAdjustment(
```
« Ajustement exceptionnel… » (section 15): a Super Admin records it; it counts only once a second admin approves. A reversal is negative and never exceeds what is left of the payment.

```
356 async approveAdjustment(
```
Second admin's approval of an adjustment (never the one who requested it); only then does it count.

402 `async overview(period: Period, now = new Date()): Promise<FinanceOverview> {`
Builds the Finance page: ledger of the period and the previous one, subscription figures, revenue per plan, the chart series and payment incidents.

485 `async netRevenueSince(from: Date, to: Date): Promise<number> {`
Net revenue of the last 30 days (overview ledger strip).

493 `async report(`
CSV reports with the metric definitions on top (section 15).

611 `private async periodData(period: Period, rates: Map<PaymentProvider, FeeRate>) {`
Payments, approved adjustments and live expenses of a period, and their ledger.

627 `private record(`
Audit entry of a finance action.

index.ts · 4 lines

transactions.controller.ts · 132 lines

40 `export class TransactionsController {`
« Transactions » (ScreenTransactions): the immutable ledger and its drawer. The drawer's `•••` holds the exceptional adjustment, Super Admin only, approved by a second admin.

51 `async list(@Query() query: TransactionQuery): Promise<AdminTransactionDto[]> {`
50 per page, newest first.

62 `async detail(@Param('id', id) transactionId: string): Promise<AdminTransactionDetailDto> {`
GET /admin/v1/transactions/:id: one payment with its adjustments.

74 `async requestAdjustment(`
POST /admin/v1/transactions/:id/adjustments: records an exceptional adjustment (waits for a second admin).

89 `async approveAdjustment(`
POST /admin/v1/transactions/adjustments/:adjustmentId/approve.

100 `private async find(transactionId: string): Promise<LedgerTransaction> {`
Loads a transaction or answers NOT_FOUND.

107 `private async present(row: LedgerTransaction): Promise<AdminTransactionDetailDto> {`
Builds the transaction drawer: the payment, its fee and its adjustments.

apps/api/src/modules/

The list of the 20 domain modules (`index.ts`).

index.ts · 45 lines

apps/api/src/modules/notifications/

In-app notifications stored as catalogue keys with parameters.

index.ts · 8 lines

notifications.controller.ts · 70 lines

41 `async list(`
Newest first, 50 per page; the next page uses `before` = the last `createdAt`.

55 `async markAllRead(@CurrentUser() user: UserPrincipal): Promise<void> {`
POST /v1/me/notifications/read-all.

63 `async markRead(`
POST /v1/me/notifications/:id/read.

notifications.dto.ts · 62 lines

notifications.module.ts · 30 lines

17 `export class NotificationsModule implements OnModuleInit {`
In-app notifications rendered from catalogue keys. Owns: `notifications`. Other modules reach it only through ./index.ts.

24 `onModuleInit(): void {`
Schedules the daily deletion of notifications older than the retention period.

notifications.service.ts · 128 lines

8 `export type NotificationParams = Record<string, string | number>;`
Parameters are identifiers, dates and numbers only — never free text typed by a person.

11 `export interface NewNotification {`
A notification to create for one person.

14 `type: string;`
Stable machine type, e.g. `verification.decided`.

16 `messageKey: string;`
A `mobile.notifications.*` catalogue key the app renders with `params`.

21 `export interface NotificationRecord {`
A stored notification.

33 `export const NOTIFICATION_RETENTION_DAYS = 180;`
Notifications are kept this long, read or not.

46 `export class NotificationsService {`
In-app notifications, rendered by the clients from catalogue keys (section 19).

50 `async eraseUser(userId: string): Promise<void> {`
Account deletion: the person's notifications go with the account.

58 `async notify(input: NewNotification): Promise<void> {`
Creates a notification; its message key must exist in the catalogue (checked so the app can always render it).

73 `async exists(userId: string, type: string, match: NotificationParams): Promise<boolean> {`
True when a notification of this type has all of `match` among its parameters (idempotent reminders).

87 `async list(userId: string, before?: Date): Promise<NotificationRecord[]> {`
One page of notifications, newest first.

98 `unreadCount(userId: string): Promise<number> {`
Number of unread notifications (badge).

103 `async markRead(userId: string, id: string): Promise<void> {`
Marks one of the person's notifications read.

112 `async markAllRead(userId: string): Promise<void> {`
Marks every unread notification read.

120 `async purgeOld(): Promise<number> {`
Deletes notifications past retention (daily job).

apps/api/src/modules/offline/

The offline manifest: what an account may keep on the phone, with a data version.

index.ts · 3 lines

offline.controller.ts · 112 lines

13 `export class ManifestReferenceDto {`
A reference PDF of the manifest.

43 `export class ManifestCalculatorDto {`
A calculator of the manifest.

67 `export class OfflineManifestDto {`
GET /v1/offline/manifest response.

86 `export class OfflineController {`
The offline manifest endpoint.

94 `async manifest(@CurrentViewer() viewer: Viewer): Promise<OfflineManifestDto> {`
Entitled content for the offline store. Locked for Demo and unpaid professionals.

offline.module.ts · 21 lines

20 `export class OfflineModule {}`
Offline manifests of the content each account is entitled to (Lock L44). Owns no table. Other modules reach it only through ./index.ts.

offline.service.ts · 110 lines

13 `export interface ManifestReference {`
A reference PDF the device may keep.

26 `export interface ManifestCalculator {`
A calculator version the device may keep, with its self-test cases.

34 `testCases: PublishedTestCase[];`
Run by the device before first use of this version (ADR-003).

38 `export interface OfflineManifest {`
The list of everything the device may keep offline, with a data version fingerprint.

41 `dataVersion: string;`
Changes whenever any entitled file or calculator version changes ("update available").

45 `substances: SubstanceProfile[];`
Pregnancy · lactation profiles the account opens, in full (ADR-019).

54 `export class OfflineService {`
What an account may keep on its device (audit Appendix A, Lock L44): the reference PDFs, the published calculator versions and the pregnancy · lactation profiles its profile opens. The app diff-syncs against this list and deletes anything no longer listed.

65 `async manifest(viewer: Viewer): Promise<OfflineManifest> {`
Builds the manifest: entitled PDFs, calculators and pregnancy profiles, and a fingerprint that changes whenever any of them changes.

apps/api/src/modules/pregnancy/import/

Reading XLSX workbooks safely, finding and mapping columns, checking rows, applying them as drafts.

apply.ts · 99 lines

14 `export interface Resolved {`
How names found in the sheet become ids: the registry source of a URL, and the substance of an alternative.

16 `sourceId: (url: string) => string;`
Source registry id for a URL.

18 `substanceId: (key: string) => string | null;`
Substance id for an alternative name, when it is known for certain.

22 `const union = <T>(current: T[] | undefined, added: T[]): T[] | undefined => {`
Union of two lists without repeats; undefined when empty (so empty lists are not stored).

33 `export function applyRows(`
Writes the accepted rows of one substance into its draft content and returns the new content (the input is not modified).

58 `function apply(`
Writes one row into one period (or lactation): conduct and status word replaced, sources and alternatives added.

rows.ts · 512 lines

24 `export const STATUS_TARGETS = [...CONDUCTS, 'NONE'] as const;`
Conduct chosen by the admin for a status word; NONE = no badge.

32 `export type RowValues = Record<Field, string | null>;`
Editable values of a row, as text (what the admin sees and corrects).

36 `export interface Issue {`
One problem or note about a row, shown in the preview.

41 `params?: Record<string, string | number>;`
Data for the message: a row number, a name… never free clinical text.

72 `export function fieldForHeader(header: string | null): Field | null {`
Field a header names, or null. Longer keywords win (« titre de la source » before « source »).

86 `export function findHeaderRow(rows: Cell[][]): number {`
Header row: the first of the first ten rows with at least two filled cells.

94 `export function detectColumns(headers: (string | null)[]): ColumnMapping {`
Guesses which column holds which field from the header names.

106 `export function missingColumns(mapping: ColumnMapping): Field[] {`
The required fields (substance, period, status) no column is mapped to.

112 `export function valuesOf(cells: Cell[], mapping: ColumnMapping): RowValues {`
Editable values of a data row under a column mapping.

149 `export function parsePeriod(raw: string | null): PeriodResult {`
« T1 », « 1er trimestre », « T1, T2 », « Toute la grossesse » (= the four periods), « Allaitement ».

165 `export function parseContext(raw: string | null): ContextValue | null | 'UNKNOWN' {`
Reads a "pregnancy / lactation" column (null when empty, UNKNOWN when unreadable).

174 `export const statusKey = (raw: string | null) => normaliseForSearch(raw ?? '');`
The form a status word is mapped under: « Non valide », « non valide » → « non valide ».

177 `export function parseUrl(`
Reads a source URL: only http(s) with a host; anything else is an error for the row.

195 `export const splitAlternatives = (raw: string | null) =>`
Several alternatives in one cell: « paracétamol ; ibuprofène ». Commas stay inside names.

204 `export interface LibrarySubstance {`
A substance as the import sees it: its names and its open and published content.

209 `open: { id: string; status: string; authoredById: string; content: ProfileContent } | null;`
The open (DRAFT / CHANGES_REQUESTED / IN_REVIEW) version, if any.

214 `export interface Library {`
Substances indexed by every normalised name and synonym.

216 `byName: Map<string, LibrarySubstance[]>;`
Normalised name or synonym → substances bearing it.

220 `export function libraryOf(substances: LibrarySubstance[]): Library {`
Builds the name index of the existing substances.

233 `export interface RowInput {`
One stored row of the batch: its values, original cells, and the admin's choices.

243 `export interface PlanSource {`
The source a row cites.

250 `export interface CheckedRow {`
The result of checking a row: its issues and, without errors, what it would write.

254 `plan: {`
What the row would write, when it has no error.

268 `const blocking = (issues: Issue[]) => issues.some((issue) => issue.severity === 'ERROR');`
True when one of the issues is an error.

271 `function existingPeriod(content: ProfileContent | null, target: Target) {`
The period (or lactation block) already in a profile, if any.

281 `export function checkRows(`
Checks every row against the others and the library, for the admin who will commit (`authorId`). Rows marked excluded are reported but take no part in duplicates.

444 `const seen = new Map<string, number>();`
Duplicates inside the file: the same drug and period twice.

478 `export interface Summary {`
Counts shown at the top of the preview.

490 `export function summarise(rows: RowInput[], checked: Pick<CheckedRow, 'issues'>[]): Summary {`
Counts rows by state for the preview header.

workbook.ts · 221 lines

18 `export interface Cell {`
One cell as read: its text, its hyperlink and whether it was a formula.

21 `hyperlink: string | null;`
External hyperlink target on the cell, or the URL of a HYPERLINK() formula.

23 `formula: boolean;`
The cell held a formula: its stored value was read, nothing was evaluated.

27 `export interface Workbook {`
The first sheet of a workbook, as rows of cells.

30 `otherSheets: string[];`
Other sheets of the file, not read.

32 `rows: Cell[][];`
Rows from row 1, cells from column A, as positioned in the sheet.

36 `const invalid = (message: string) =>`
An UPLOAD_INVALID error shown on the file field.

40 `function inflate(buffer: Buffer, wanted: (name: string) => boolean): Map<string, Uint8Array> {`
Inflates the parts whose names pass `wanted`, refusing anything over the limits.

80 `const xml = (part: Uint8Array | undefined) => (part ? decoder.decode(part) : '');`
A zip part decoded as text.

83 `const unescapeXml = (text: string) =>`
Turns XML entities back into characters.

92 `function attributes(tag: string): Map<string, string> {`
The attributes of one XML tag.

100 `export function cellAddress(ref: string): { row: number; column: number } | null {`
"AB12" → { row: 11, column: 27 } (0-based).

109 `function cellText(value: unknown): string | null {`
A cell value as trimmed, bounded text (dates as YYYY-MM-DD).

124 `export async function readWorkbook(buffer: Buffer): Promise<Workbook> {`
Reads an uploaded .xlsx safely: size and signature checks, bounded inflation, first sheet only, formulas never evaluated, hyperlinks kept.

128 `if (buffer.readUInt32LE(0) !== 0x04034b50) throw invalid('The file is not an .xlsx workbook.');`
An .xlsx is a zip archive: check its real signature, not the extension or the MIME type.

apps/api/src/modules/pregnancy/

Grossesse · Allaitement: the app side (list, search, profile, reports) and the admin side (substances, versions, sources, merges, imports).

index.ts · 10 lines

pregnancy-admin.controller.ts · 370 lines

54 `export class PregnancyAdminController {`
« Grossesse · Allaitement » governance for Admin Web (ADR-019, grants `pregnancy.*`).

63 `async list(@Query() query: AdminSubstancesQuery): Promise<AdminSubstanceDto[]> {`
GET /admin/v1/pregnancy/substances.

71 `async detail(@Param('id', uuid) id: string): Promise<AdminSubstanceDetailDto> {`
GET /admin/v1/pregnancy/substances/:id.

80 `async create(`
POST /admin/v1/pregnancy/substances.

93 `async update(`
Names, ATC and synonyms.

107 `async visibility(`
Who sees the profile: demo sample, enabled. A reviewer's decision.

124 `async createVersion(`
POST .../substances/:id/versions: a new draft profile.

139 `async mergePreview(`
What merging this duplicate into another substance would change, and what prevents it.

151 `async merge(`
Merges a duplicate into the substance to keep (ADR-027).

166 `async unmerge(`
Undoes a merge; the duplicate stays archived until a reviewer shows it again.

181 `async submitMany(`
Submits several drafts for review (e.g. after an import); each is checked on its own.

198 `async approveMany(`
Approves several versions in review; author ≠ reviewer and sign-offs apply to each.

214 `async version(@Param('versionId', uuid) versionId: string): Promise<AdminProfileVersionDto> {`
GET /admin/v1/pregnancy/versions/:versionId.

222 `async save(`
PUT .../versions/:versionId: saves the draft's content and sources.

238 `async submit(`
POST .../versions/:versionId/submit.

254 `async approve(`
Approve and publish (never the author); the previous version is superseded.

270 `async requestChanges(`
POST .../versions/:versionId/request-changes.

286 `async discard(`
DELETE .../versions/:versionId: deletes a draft.

301 `async sources(): Promise<ClinicalSourceDto[]> {`
GET /admin/v1/pregnancy/sources.

310 `async createSource(`
POST /admin/v1/pregnancy/sources.

322 `async updateSource(`
PATCH /admin/v1/pregnancy/sources/:id.

337 `async reports(@Query() query: ReportsQuery): Promise<AdminContentReportDto[]> {`
GET /admin/v1/pregnancy/reports.

350 `async resolve(`
POST /admin/v1/pregnancy/reports/:id/resolve.

```
360  private async detailDto(id: string): Promise<AdminSubstanceDetailDto> {
```
The substance drawer response.

pregnancy-admin.service.ts · 896 lines

```
19   export interface SubstanceInput {
```
A new substance as Admin Web sends it.

```
28   export interface SubstancePatch extends Partial<Omit<SubstanceInput, 'reason'>> {
```
Changes to a substance.

```
30   demoSample?: boolean;
```
Visibility: set only through the reviewer's endpoint.

```
36   export interface SourceInput {
```
A source of the clinical sources registry.

```
56   export type AdminVersion = Prisma.SubstanceProfileVersionGetPayload<{
```
A profile version with its people, sources, substance and review problems.

```
63   export interface SubstanceFilter {
```
Filters of the substances page.

```
78   export class PregnancyAdminService {
```
Governance of « Grossesse · Allaitement » for Admin Web (ADR-019, section 25 Appendix B): substances, versioned profiles DRAFT → IN_REVIEW → PUBLISHED (author ≠ reviewer), the sources registry and user reports. Every mutation is audited with its reason.

```
88   async listSubstances(filter: SubstanceFilter = {}) {
```
The substances page: search, archive filter, profile state and conduct filters.

```
153  async substance(id: string) {
```
One substance with its versions and open reports.

```
170  async createSubstance(admin: AdminPrincipal, input: SubstanceInput, client: ClientInfo) {
```
Creates a substance (no profile yet); a name already used is refused.

```
197  async updateSubstance(
```
Names, ATC and synonyms need `pregnancy.draft`; demo sample and enabling change who sees it.

```
253  async mergePreview(sourceId: string, targetId: string) {
```
What merging `sourceId` into `targetId` would do, and what prevents it. The source's names become synonyms of the target; its profile versions stay on it as history; nothing clinical is copied. Blockers are codes Admin Web translates.

```
285  async merge(
```
Merges a duplicate: one transaction, audited on both substances.

```
314  const { count } = await tx.substance.updateMany({
```
Guarded: only a substance still unmerged is merged (two reviewers at once).

```
364  async unmerge(admin: AdminPrincipal, sourceId: string, reason: string, client: ClientInfo) {
```
Undoes a merge: the names it added leave the target, the duplicate stands alone again. It stays archived: showing it again is a separate visibility decision.

```
412  private async mergeRow(id: string) {
```
What a merge needs to know about a substance.

```
427  async createVersion(
```
« Nouvelle version »: starts from the published profile, or empty for a first profile.

```
473  async version(versionId: string): Promise<AdminVersion> {
```
One profile version with the problems that would block its review.

```
490  async saveVersion(
```
Saves the draft as written: incomplete is fine until submission.

```
538  async submit(admin: AdminPrincipal, versionId: string, reason: string, client: ClientInfo) {
```
The author sends a draft for review; it must have no blocking problem.

574 `async approve(admin: AdminPrincipal, versionId: string, comment: string, client: ClientInfo) {`
Approves. When two sign-offs are required and none is recorded yet, records the first one (ADR-024); otherwise publishes, and the previous published version is superseded in the same step.

646 `async requestChanges(`
The reviewer sends a version back to its author with a comment.

680 `async discard(admin: AdminPrincipal, versionId: string, reason: string, client: ClientInfo) {`
Drops a draft that will not be published; versions in review or published are kept.

701 `listSources() {`
The sources registry.

706 `async createSource(admin: AdminPrincipal, input: SourceInput, client: ClientInfo) {`
Adds a source to the registry.

717 `async updateSource(admin: AdminPrincipal, id: string, input: SourceInput, client: ClientInfo) {`
Corrects a source; every version citing it shows the correction (title, edition, link).

734 `listReports(filter: { open: boolean; substanceId?: string }) {`
User reports (« Signaler une erreur »), open ones or all.

751 `async resolveReport(admin: AdminPrincipal, id: string, resolution: string, client: ClientInfo) {`
Closes a user report with what was done.

763 `/**`
Open reports and versions waiting for a reviewer (Admin Web overview).

767 `async many(`
Bulk submit / approve: each version goes through the single-version rules (author ≠ reviewer, review gate, sign-offs); one failure does not stop the others and is reported with its code.

785 `async pending() {`
Counts for the overview: open reports and versions in review.

796 `private async problemsOf(row: {`
Review problems of a version (missing sources, alternatives that do not exist…).

813 `private assertAuthor(admin: AdminPrincipal, version: { authoredById: string }) {`
Only the author edits, submits or discards a draft.

820 `private assertReviewer(admin: AdminPrincipal, version: { authoredById: string }) {`
A version is never reviewed by its author.

827 `private assertStatus(status: string, allowed: readonly string[], message: string) {`
Refuses an action in the wrong status.

832 `private record(`
Audit entry of a pregnancy change.

856 `function isOpen(status: string): boolean {`
True for a version not yet decided.

861 `function cleanSynonyms(values: string[]): string[] {`
Trims synonyms and drops empty ones and repeats.

876 `function normaliseSource(input: SourceInput) {`
Trims the fields of a source.

888 `function nameTaken(error: unknown): unknown {`
Turns a duplicate-name database error into a field error.

pregnancy-import.controller.ts · 201 lines

59 `export class PregnancyImportController {`
Spreadsheet import into « Grossesse · Allaitement » (ADR-025). Every write needs `pregnancy.draft`; nothing here publishes: the result is drafts a reviewer checks.

80 `async upload(`
POST /admin/v1/pregnancy/imports: uploads a spreadsheet and returns its preview.

```
97  async list(@Query() query: PageQuery): Promise<ImportListDto> {
```
GET /admin/v1/pregnancy/imports: past imports.

```
126 async view(
```
GET /admin/v1/pregnancy/imports/:id: the preview with its rows.

```
140 async mapping(
```
PUT .../imports/:id/mapping: columns and status words.

```
163 async row(
```
PATCH .../imports/:id/rows/:rowId: corrects one row.

```
178 async commit(
```
POST .../imports/:id/commit: writes the valid rows into drafts.

```
192 async discard(
```
POST .../imports/:id/discard: abandons the import.

pregnancy-import.dto.ts · 356 lines

pregnancy-import.service.ts · 849 lines

```
53  interface HeaderInfo {
```
The header row found in the sheet.

```
60  export interface StatusWord {
```
A status word of the sheet, how often it appears and what it is mapped to.

```
63  sample: string;
```
The word as written the first time it appears.

```
69  export interface CommitResult {
```
What a commit created or updated.

```
80  type BatchRow = Prisma.PregnancyImportBatchGetPayload<{
```
A batch with its uploader, as read for the preview.

```
88  export interface BatchView {
```
One import as the preview page shows it.

```
103 interface AuditItem {
```
An audit entry prepared during a commit, written after it succeeds.

```
111 const hash = (buffer: Buffer) => createHash('sha256').update(buffer).digest('hex');
```
SHA-256 of the uploaded file (to spot the same file uploaded twice).

```
113 const json = (value: unknown) => value as Prisma.InputJsonValue;
```
Typed cast for JSON columns.

```
120 export class PregnancyImportService {
```
Spreadsheet import into « Grossesse · Allaitement » (ADR-025). A batch is a preview until its commit, which writes drafts (never published content) in one transaction, at most once.

```
133 async upload(
```
Reads an uploaded workbook, finds its header row, maps its columns and stores it as a preview batch (nothing is written to the library yet).

```
141 const open = await this.prisma.pregnancyImportBatch.findFirst({
```
The same file uploaded again by the same admin reopens its preview instead of a copy.

```
209 async list(limit: number, offset: number) {
```
Past imports, newest first.

```
227 async view(id: string, filter: RowFilter, limit: number, offset: number): Promise<BatchView> {
```
One import with its rows (filtered and paged); rows are re-checked against today's library.

```
307 async setMapping(
```
Column and status mapping. Rows the admin has not edited are re-read from their cells.

```
365 async updateRow(
```
Correction of one row by the admin (values, exclusion, conflict choice).

417 `async discard(`
Abandons an import; nothing is written to the library.

441 `async commit(`
Writes the batch as drafts, all or nothing. The batch row is locked; a second call (double click, retry) returns the stored result. The admin names the revision they reviewed, so a change made meanwhile in another tab is never committed unseen.

495 `const groups = new Map<string, typeof written>();`
Substances: the existing one, or created here.

708 `private async editable(id: string, revision: number) {`
An import still in preview, at the revision the admin saw (refuses stale edits).

721 `private async bump(`
Saves a change to the batch and moves its revision forward.

736 `private async recheck(id: string, authorId: string): Promise<void> {`
Re-checks every row (duplicates depend on the others) and stores the issues that changed.

764 `private async record(`
Audit entry of an import action.

787 `function toInput(row: {`
A stored row in the shape the checker expects.

809 `async function loadLibrary(`
Every substance with its names and contents, for matching rows (a merged duplicate stands for the substance kept).

813 `const substances = await db.substance.findMany({`
A merged duplicate is represented by the substance it was merged into (ADR-027).

pregnancy.content.ts · 211 lines

10 `const localized = z`
Authored text: French required, English optional (the app falls back to French).

19 `export const CONDUCTS = ['CONTRAINDICATED', 'AVOID', 'OCCASIONAL_USE'] as const;`
The clinical instructions shown as a badge (icon + word); anything else has no badge.

23 `export const EVIDENCE_LEVELS = [0, 1, 2, 3, 4] as const;`
4 Données robustes · 3 modérées · 2 limitées · 1 très limitées · 0 Absence de données.

27 `level: z.number().int().min(0).max(4).nullable(),`
Null only in a draft: an import never invents a level; review requires one.

29 `population: localized.optional(),`
« cohortes et séries concordantes », « < 100 grossesses exposées »…

34 `const conflict = z`
Two references that disagree, and the reviewers' synthesis (info Alert).

42 `const reportedStatus = z.string().trim().min(1).max(200).optional();`
Status exactly as the imported source wrote it (« Non valide »), kept beside the conduct.

44 `const periodSources = z.array(z.uuid()).max(10).optional();`
Sources (of this version) that support this period specifically.

49 `const unresolvedAlternatives = z.array(z.string().trim().min(1).max(200)).max(10).optional();`
Alternative names an import could not link to a substance with certainty. Review is refused until an author links each one to a substance or removes it: nothing is matched by similarity.

56 `conductDetail: localized.optional(),`
One sentence: « Contre-indiqué à partir de 24 SA (5e mois révolu) ».

62 `alternatives: z.array(z.uuid()).max(10).optional(),`
Alternatives specific to this period (the block's alternatives apply to every period).

93 `conductText: localized.optional(),`
Conduct first, in one sentence: « Compatible avec l'allaitement. Surveiller les selles. »

95 `relativeInfantDose: z.string().trim().min(1).max(40).optional(),`
Relative infant dose as the source states it: « 0,5 – 1 % ».

97 `maternalHalfLife: z.string().trim().min(1).max(40).optional(),`
« 1 – 1,5 h ».

100 `specialPopulations: localized.optional(),`
Premature infants, infants under one month.

116 `export const EMPTY_CONTENT: ProfileContent = { pregnancy: null, lactation: null };`
A new profile starts empty: both blocks are filled by the author.

124 `export function reviewProblems(`
What still prevents a version from going to review. A draft may be saved incomplete; review needs a complete profile: both blocks, every period, a conduct sentence where a period has a badge, the conduct sentence of lactation, at least one source, conflicts citing the version's own sources, and alternatives that are other substances. « Absence de données » (level 0) is a complete answer.

135 `const checkAlternatives = (where: string, ids: readonly string[]) => {`
Reports alternatives that are the substance itself or not an existing substance.

147 `const checkPeriod = (`
Reports a period without its evidence level, citing a source not attached to this version, or with alternatives not yet linked to a substance.

164 `const checkConflict = (where: string, value: { sourceIds: string[] } | undefined) => {`
Reports a conflict between sources that cites sources not attached to this version.

202 `export function alternativeIds(content: ProfileContent): string[] {`
The substances a profile links to as alternatives.

pregnancy.controller.ts · 99 lines

42 `export class PregnancyController {`
« Grossesse · Allaitement » for the app (ADR-019). Locked substances stay listed with their reason; only an AVAILABLE one returns its content. Works with user and demo tokens.

49 `async list(`
GET /v1/pregnancy/substances: the A–Z list (optional filter).

63 `async search(`
Server-side search (ADR-025): partial, accent-insensitive words of the name or a synonym, bounded pages, with each period's conduct and evidence level when the profile is open.

79 `async get(`
The published profile; CONTENT_UNAVAILABLE (410) once disabled.

91 `async report(`
« Signaler une erreur » on the pregnancy or the lactation block.

pregnancy.dto.ts · 761 lines

233 `export class PeriodSummariesDto {`
One entry per period; null when the published profile says nothing for it.

428 `export class SubstanceVisibilityDto extends ReasonField {`
Who sees a profile: the demo sample flag and enabling (grant `pregnancy.review`).

pregnancy.module.ts · 29 lines

28 `export class PregnancyModule {}`
« Grossesse · Allaitement » (ADR-019): substances, versioned pregnancy and lactation profiles reviewed by someone other than their author, the clinical sources registry and user reports. Not a drug database: no posology, products or interactions. Owns: `substances`, `substance_profile_versions`, `clinical_sources`, `profile_version_sources`, `content_reports`, `pregnancy_import_batches`, `pregnancy_import_rows` (ADR-025). Other modules reach it only through ./index.ts.

pregnancy.search.ts · 22 lines

6 `export function searchTextOf(`
The normalised text searched by the app: names and synonyms, accents removed, lower case.

15 `export function likeWords(query: string): string[] {`
Words of a query, each safe inside a LIKE pattern.

pregnancy.service.ts · 430 lines

25 `export interface PeriodSummary {`
What a search result shows for one period of a published profile.

31 `export interface SubstanceHit extends SubstanceItem {`
A search result with a per-period summary.

33 `periods: Record<SummaryTarget, PeriodSummary | null> | null;`
Per period of the published profile; null while locked (no content for a locked item).

37 `function summaries(content: ProfileContent): Record<SummaryTarget, PeriodSummary | null> {`
The conduct and evidence level of each period, for search results.

51 `export interface SubstanceItem {`
A substance as the app lists it: published profiles only, each with the server's decision.

62 `export interface SourceView {`
A source as the app shows it.

73 `export interface SubstanceProfile extends SubstanceItem {`
An opened profile: the content only when AVAILABLE, never for a locked item.

80 `underReview: boolean;`
A newer version is in review: the published one remains the reference (info Alert).

101 `export class PregnancyService {`
« Grossesse · Allaitement » for the app (ADR-019): enabled substances with a published profile. Locked substances stay listed with their reason; only an AVAILABLE one yields its content.

108 `async list(viewer: Viewer, query?: string): Promise<SubstanceItem[]> {`
A–Z list, optionally filtered by name or synonym (accent- and case-insensitive).

121 `async get(viewer: Viewer, id: string): Promise<SubstanceProfile> {`
One profile (a merged duplicate opens the substance kept); the content only when the viewer may open it.

162 `async openable(viewer: Viewer): Promise<SubstanceProfile[]> {`
The profiles this viewer may open, in full, for the offline store (Lock L44). Built in bulk — one read of the published profiles, one access decision, one lookup of the alternatives' names — so the cost does not grow with a query per substance.

197 `async itemsByIds(viewer: Viewer, ids: readonly string[]): Promise<Map<string, SubstanceItem>> {`
The listed substances among `ids` (favorites), keyed by the id asked for. A duplicate merged into another substance is answered with the substance kept (ADR-027), so its favorite keeps working; disabled or unpublished ones are left out.

233 `async report(`
« Signaler une erreur »: kept for the clinical team; a demo report has no account.

266 `async search(`
Server-side search for the app (ADR-025): every word of the query must appear in the name or a synonym (accent- and case-insensitive, partial words), names starting with the first word first, then A–Z. Bounded pages; only published profiles of enabled substances.

313 `async searchable(viewer: Viewer): Promise<SubstanceItem[]> {`
Every substance the viewer can see listed, for global search. Names, synonyms and access only: profile content and sources are not loaded (ADR-025, measured on 5 000 substances).

332 `async eraseUser(userId: string): Promise<void> {`
Account deletion: the person's reports stay for the clinical team, without their account.

337 `private async published() {`
Enabled substances with their published version.

346 `private async withAccess(`
Adds the viewer's access decision to each substance.

377 `private async nameIndex(`
Looks up, in two queries, the names of every alternative among `ids`, and returns how to list the alternatives of one profile: still-enabled substances only, each once; an alternative merged into another substance is shown as that one (ADR-027), never as the profile itself.

411 `export function sourceView(source: {`
The public fields of a source.

apps/api/src/modules/reference-library/

Ordonnances and certificates: app list and signed PDF URLs; admin upload, replacement, metadata, status; PDF processing job.

index.ts · 4 lines

reference-documents-admin.service.ts · 401 lines

24 `export interface UploadRequest {`
What Admin Web declares before uploading a PDF.

32 `export interface NewDocument {`
Step 2 of « Ajouter un modèle ».

44 `export interface DocumentPatch {`
Display metadata changes.

52 `export const LIFECYCLE = {`
Lifecycle actions and the statuses they move a document between (section 13).

63 `export type LifecycleAction = keyof typeof LIFECYCLE;`
enable, disable, archive or restore.

69 `const DETAIL_INCLUDE = {`
Relations shown to Admin Web. Only ever used in reads outside transactions: with a write (an implicit transaction) Prisma would query these relations in parallel on one connection.

77 `export type AdminDocumentRecord = Prisma.ReferenceDocumentGetPayload<{`
A model with its files and people, as Admin Web shows it.

82 `const auditSnapshot = (document: {`
The fields recorded in audit entries of a model.

101 `export class ReferenceDocumentsAdminService {`
Super Admin management of Ordonnances and Certificates (section 13). The admin never authors clinical content: the PDF is the content; every mutation needs a reason and is audited.

111 `async requestUpload(`
Step 1 of a new model: a signed upload for version 1 under a freshly allocated id.

126 `async create(`
Step 2: validates the upload and creates the model; processing sets its final status.

182 `async requestReplacement(`
A signed upload for the next version of an existing model.

200 `async replace(`
New file version under the same id; favorites keep working; the current file moves once processed.

253 `async update(`
Display metadata only; the PDF is never touched (section 13).

279 `async changeStatus(`
Enables, disables, archives or restores a model; enabling needs a processed PDF.

311 `list(filter: {`
Admin list: archived models only when asked for (section 13).

326 `async detail(documentId: string): Promise<AdminDocumentRecord> {`
One model with all its files.

331 `async previewUrl(documentId: string, fileId: string): Promise<{ url: string; expiresAt: Date }> {`
Preview of any stored version for `library.read` holders (views are not logged, section 13).

346 `private async require(documentId: string): Promise<AdminDocumentRecord> {`
Loads a model or answers NOT_FOUND.

356 `private ticket(`
Signs the upload of one PDF version under its immutable key.

380 `private record(`
Audit entry of a library change.

reference-documents.service.ts · 228 lines

11 `const byTitle = (a: { titleFr: string }, b: { titleFr: string }) =>`
French A–Z order by title.

15 `export interface ReferenceItem {`
A reference model as the app shows it.

25 `fileDate: Date;`
When the current file was processed ("file date" in the viewer).

30 `const PUBLISHED_SELECT = {`
Columns read for a published model.

50 `type PublishedRow = {`
A model row with its current file, as read here.

73 `export class ReferenceDocumentsService {`
What users see of the Reference Library (section 13): enabled models only, each with the server's access decision. Disabled and archived models are gone from lists; opening one answers CONTENT_UNAVAILABLE ("N'est plus disponible").

81 `async labels(`
Titles and type of reference models by id (admin read models; status does not matter).

96 `async processingFailures(limit: number) {`
Models whose latest file failed processing (overview « Incidents »).

111 `async list(`
A–Z list of one type, optionally filtered by title (accent- and case-insensitive).

125 `async get(viewer: Viewer, id: string): Promise<ReferenceItem> {`
One model; CONTENT_UNAVAILABLE once disabled or archived.

142 `async fileUrl(`
A 10-minute signed URL to the current PDF, only when the viewer's profile opens the model.

169 `async openable(viewer: Viewer): Promise<ReferenceItem[]> {`
Every model the viewer may open, for the offline manifest.

175 `async byIds(viewer: Viewer, ids: readonly string[]): Promise<ReferenceItem[]> {`
The published models among `ids` (favorites), with access; unknown or hidden ids are left out.

181 `private async published(filter: {`
Enabled models whose current file is processed.

198 `private async withAccess(viewer: Viewer, rows: PublishedRow[]): Promise<ReferenceItem[]> {`
Adds the viewer's access decision to each model.

222 `function isPublished(row: PublishedRow): boolean {`
True when the model's current file has been processed (checksum, page count, date).

reference-library-admin.controller.ts · 231 lines

46 `export class ReferenceLibraryAdminController {`
Reference Library management for Admin Web (section 13, grants `library.*`).

54 `async list(@Query() query: AdminListQuery): Promise<AdminReferenceDocumentDto[]> {`
GET /admin/v1/reference-documents.

65 `async detail(@Param('id', id) documentId: string): Promise<AdminReferenceDocumentDto> {`
GET /admin/v1/reference-documents/:id.

74 `async previewUrl(`
« Voir le PDF » for any stored version.

88 `async requestUpload(`
Step 1 of « Ajouter un modèle »: a signed upload for the PDF (≤ 20 MB).

102 `async create(`
Step 2: creates the model once the PDF is uploaded; it shows « Traitement… » until processed.

115 `async update(`
Display metadata only (titles, demo sample); never the PDF.

132 `async requestReplacement(`
« Remplacer le PDF », step 1: a signed upload for the next version.

147 `async replace(`
« Remplacer le PDF », step 2: same id, new version, previous files kept.

164 `enable(`
Enables a disabled model.

179 `disable(`
Hides a model from users.

194 `archive(`
Replaces hard delete: gone for users and from default admin views; files kept.

209 `restore(`
Brings an archived model back as disabled.

219 `private async lifecycle(`
Shared handler of enable, disable, archive and restore.

reference-library.controller.ts · 72 lines

28 `export class ReferenceLibraryController {`
Ordonnances and Certificates for the app (section 13). Locked models stay listed with their reason; only an AVAILABLE model yields a PDF URL. Works with user and demo tokens.

35 `async list(`
A–Z list of one type (favorites-first ordering is applied by the app with its favorites).

48 `async get(`
One model; CONTENT_UNAVAILABLE (410) once disabled or archived.

59 `async fileUrl(`
Signed 10-minute URL to the PDF; FEATURE_LOCKED with the reason when the model is locked.

reference-library.dto.ts · 399 lines

reference-library.module.ts · 44 lines

30 `export class ReferenceLibraryModule implements OnModuleInit {`
Reference documents (ordonnances, certificates) and their immutable file versions (section 13). Owns: `reference_documents`, `reference_document_files`. Other modules reach it only through ./index.ts.

37 `onModuleInit(): void {`
Registers the processing worker and the minute sweep of pending files.

reference-library.policy.ts · 26 lines

4 `export const TITLE_MAX_LENGTH = 200;`
Limits of titles and reasons.

10 `export const PROCESS_FILE_JOB = 'reference-library.process-file';`
Queue fed after each upload, and the minute sweep that catches anything the queue missed.

17 `export function suggestTitle(fileName: string): string {`
Display title proposed from the uploaded file name (section 13: "pre-filled from the file name"). "angine_adulte-v2.pdf" → "angine adulte v2".

reference-processing.service.ts · 146 lines

9 `type Outcome =`
Result of inspecting an uploaded file.

18 `export class ReferenceProcessingService {`
Turns an uploaded PDF into a usable file version: verifies the declared checksum, counts pages, then makes it the document's current file. A new document takes the status the admin chose; a failed first file leaves the document DISABLED (admin state "PDF unavailable").

31 `async process(fileId: string): Promise<void> {`
Processes one uploaded file: checks it, records the result and makes it current when it is newer.

54 `const document = await tx.referenceDocument.findUniqueOrThrow({`
Sequential reads: a relation include here would run parallel queries on the transaction connection.

103 `async processPending(limit = 20): Promise<void> {`
Processes files still pending (the minute sweep, in case a queued job was lost).

122 `private async inspect(storageKey: string, declaredSha256: string): Promise<Outcome> {`
Downloads the file, compares its SHA-256 with the declared one and counts its PDF pages.

apps/api/src/modules/search/

Global search: ranking with typo tolerance, intent detection, suggestions, admin synonyms.

index.ts · 4 lines

ranking.ts · 227 lines

14 `export interface SearchEntry {`
What a search candidate can be matched on.

17 `titles: string[];`
Display titles in both languages (either may match, whatever the UI language).

20 `purposes: string[];`
Calculator one-line purpose; matched last.

24 `export const SCORE = {`
Scores, highest first. Never boosted by commerce (Lock L12).

36 `export function normalize(text: string): string {`
Lower case, no accents, apostrophes and hyphens as spaces, single spaces.

55 `export function words(text: string): string[] {`
The normalised words of a text.

61 `export function allowedTypos(word: string): number {`
Typos tolerated in a query word: none up to 2 letters, 1 up to 5, 2 above (section 12).

70 `export function editDistance(a: string, b: string, max: number): number {`
Optimal-string-alignment (Damerau-Levenshtein) distance, stopping early once it exceeds `max` (returns max + 1 then).

108 `function closest(queryWord: string, candidates: string[]): number {`
Distance of a query word to the closest word of `candidates` (also against their prefixes).

123 `export function score(query: string, entry: SearchEntry): number | null {`
The entry's score for `query`, or null when it does not match.

137 `const everyWord = (pool: string[], test: (word: string, candidate: string) => boolean) =>`
True when every query word passes the test against some word of the pool.

183 `export function intentOf(query: string): SearchType | null {`
A query naming a category puts that group first (section 12 « intent boosts »).

196 `export function withoutIntent(query: string, intent: SearchType | null = intentOf(query)): string {`
The query without the words naming the detected category, so « certificat grossesse » matches the model « Grossesse … » in the certificates group and « ibuprofène grossesse » matches the substance. A query made only of such words stays whole.

206 `export function suggestions(`
« Vouliez-vous dire » when nothing matched: up to three titles whose words are closest to the query's longest word (distance ≤ 3).

search.controllers.ts · 87 lines

30 `export class SearchController {`
Global search of the app (section 12), for user and demo tokens.

40 `async query(`
GET /v1/search: the global search of the app.

54 `export class SearchAdminController {`
Synonyms of a model (grant `library.write`) or a calculator (grant `calculators.draft`), checked in the service since the grant depends on the entity type.

62 `async list(`
GET /admin/v1/search/synonyms: the synonyms of one model or calculator.

75 `async replace(`
PUT /admin/v1/search/synonyms: replaces them.

search.dto.ts · 140 lines

23 `export class SearchQuery {`
Query of GET /v1/search: the text and an optional type.

42 `export class SearchHitDto {`
One search result.

73 `export class SearchGroupDto {`
One group of results.

85 `export class SearchResultDto {`
The search response.

108 `export class SynonymsQuery {`
Which item's synonyms to read.

115 `export class ReplaceSynonymsDto {`
The new list of synonyms, with a reason.

136 `export class SynonymsDto {`
An item's synonyms.

search.module.ts · 29 lines

28 `export class SearchModule {}`
The V1 search index over reference documents and calculators (section 12, ADR-014). Owns: `search_synonyms`. Other modules reach it only through ./index.ts.

search.service.ts · 291 lines

26 `export interface SearchHit {`
One result of the global search.

30 `slug: string | null;`
Calculators open by slug.

37 `terms?: string[];`
Other names the item is found by (substance synonyms); not returned to the app.

```
41  export interface SearchGroup {
```
Results of one type (ordonnances, certificates, calculators, pregnancy).

```
48  export interface SearchResult {
```
The grouped results and, when nothing matched, suggestions.

```
51  suggestions: string[];
```
« Vouliez-vous dire » titles, only when nothing matched.

```
55  const GROUP_PREVIEW = 3;
```
Rows shown per group in the grouped view; a type filter shows up to the full limit.

```
67  export class SearchService {
```
The V1 search index (section 12) over enabled reference models and published calculators. Results keep locked items in place with the server's access decision (never hidden, Lock L38). The query text is never stored or logged (Lock L42).

```
83  async search(viewer: Viewer, query: string, type?: SearchType): Promise<SearchResult> {
```
Global search: rate limit, detect the category asked for, score every candidate, group by type (best group first), suggest titles when nothing matched.

```
132 async synonymsFor(
```
Synonyms as Admin Web reads them (grant `library.read` or `calculators.read`).

```
145 private async synonymsOf(entityType: SearchEntityType, entityId: string): Promise<string[]> {
```
The synonyms of one model or calculator, A–Z.

```
155 async replaceSynonyms(
```
Replaces the synonyms of one model or calculator (duplicates after normalisation are merged).

```
213 private async candidates(viewer: Viewer, type?: SearchType): Promise<SearchHit[]> {
```
Everything the viewer could find: reference models, calculators and substances, with access.

```
263 private entryOf(hit: SearchHit, synonyms: Map<string, string[]>): SearchEntry {
```
The texts a candidate is matched on.

```
275 private async synonyms(): Promise<Map<string, string[]>> {
```
Every admin-entered synonym, cached for a minute.

apps/api/src/modules/storage/

The only door to files: abstract `StorageService`, R2 and local adapters, immutable keys, upload intents, limits and signed URL lifetimes.

index.ts · 19 lines

local-storage.adapter.ts · 178 lines

```
18  export interface LocalGrant {
```
What a local signed URL allows, and until when.

```
23  expiresAt: number;
```
Unix seconds.

```
29  interface Metadata {
```
What is kept next to each local object.

```
42  export class LocalStorageAdapter extends StorageService {
```
Development and test storage on the local disk (ADR-011). Objects are never served statically: only through {@link LOCAL_OBJECTS_PATH} with an HMAC-signed, expiring grant, exactly like presigned R2 URLs. Production refuses this driver.

```
57  async upload(input: UploadInput): Promise<StoredObject> {
```
Writes the file (never over an existing one) and its metadata.

```
63  await writeFile(file, input.body, { flag: 'wx' });
```
`wx`: fail if the key exists — keys are immutable.

80 `async delete(bucket: BucketId, key: string): Promise<void> {`
Deletes the file and its metadata.

87 `async head(bucket: BucketId, key: string): Promise<StoredObject | null> {`
Size, type and checksum of a local object, or null.

106 `async download(bucket: BucketId, key: string, range?: ByteRange): Promise<Uint8Array> {`
The bytes of a local object (or a range of them).

112 `getSignedReadUrl(`
A signed, expiring GET URL served by LocalStorageController.

123 `getSignedUploadUrl(`
A signed, expiring PUT URL fixing type and size.

141 `verifyGrant(token: string, method: LocalGrant['method'], now = Date.now()): LocalGrant {`
Validates a signed URL token for `method`; throws FORBIDDEN when invalid or expired.

155 `private sign(grant: LocalGrant): string {`
Encodes and signs a grant into a URL.

162 `private expiry(ttlSeconds: number): number {`
Unix time `ttlSeconds` from now.

167 `private paths(bucket: BucketId, key: string): { file: string; meta: string } {`
The file and metadata paths of a key, refusing anything outside the bucket folder.

local-storage.controller.ts · 79 lines

25 `export class LocalStorageController {`
Serves the local driver's signed URLs (development and tests only; not part of the API contract). With STORAGE_DRIVER=r2 every request here is a 404.

33 `async read(@Query('token') token: string | undefined, @Res() response: Response): Promise<void> {`
GET with a signed token: streams the object.

52 `async write(@Query('token') token: string | undefined, @Req() request: Request): Promise<void> {`
PUT with a signed token: stores the body if type and size match the grant.

72 `private local(): LocalStorageAdapter {`
The local adapter, or 404 when R2 is configured.

r2-storage.adapter.ts · 167 lines

20 `interface BucketTarget {`
One R2 bucket and the client holding its own credentials.

26 `function required(value: string | undefined, name: string): string {`
Reads a required setting (the environment check already guarantees it).

32 `const isNotFound = (error: unknown) =>`
True when R2 answered "no such object".

42 `export class R2StorageAdapter extends StorageService {`
Cloudflare R2 through its S3-compatible API (ADR-002). Each bucket has its own client and API token, so the private (verification) bucket never shares credentials with content.

69 `async upload(input: UploadInput): Promise<StoredObject> {`
PutObject, refusing to overwrite an existing key.

92 `async delete(bucket: BucketId, key: string): Promise<void> {`
DeleteObject.

98 `async head(bucket: BucketId, key: string): Promise<StoredObject | null> {`
HeadObject, or null when absent.

118 `async download(bucket: BucketId, key: string, range?: ByteRange): Promise<Uint8Array> {`
GetObject (optionally a byte range).

132 `getSignedReadUrl(`
A presigned GET URL (shown inline).

146 `getSignedUploadUrl(`
A presigned PUT URL that also signs Content-Type and Content-Length.

storage-keys.ts · 62 lines

5 `export const STORAGE_KEY_PATTERN = /^doktool\/[A-Za-z0-9/_.-]+$/;`
Same rule as the `storage_upload_intents_key_chk` constraint.

8 `export function isValidStorageKey(key: string): boolean {`
True for a well-formed key (no "..", no empty segment).

20 `export const StorageKeys = {`
Builders of every object key, so all modules use the same layout.

22 `referenceDocument(`
doktool/reference-documents/{ordonnances|certificates}/{documentId}/v{n}-{sha8}.pdf

33 `verificationDocument(`
doktool/verification/{userId}/{caseId}/{documentId}.{ext} — private bucket only.

43 `recoveryDocument(`
doktool/recovery/{requestId}/{slot}-{uuid}.{jpg|png} — private bucket only (ADR-023).

53 `profileImageOriginal(userId: string, imageId: string, extension: 'jpg' | 'png') {`
doktool/profile-images/{userId}/originals/{uuid}.{jpg|png} — deleted once re-encoded.

58 `profileImage(userId: string, imageId: string) {`
doktool/profile-images/{userId}/{uuid}.webp

storage.module.ts · 43 lines

30 `export class StorageModule implements OnModuleInit {`
StorageService and its Cloudflare R2 adapter — the only code allowed to use the S3 SDK (ADR-002, ADR-011). A local-disk adapter serves development and tests. Owns: `storage_upload_intents`. Other modules reach it only through ./index.ts.

37 `onModuleInit(): void {`
Schedules the hourly clean-up of uploads never confirmed.

storage.policy.ts · 86 lines

6 `export const SIGNED_READ_TTL = {`
Signed read URL lifetimes, in seconds.

14 `export const UPLOAD_URL_TTL_SECONDS = 15 * 60;`
A direct upload must happen within this delay.

19 `interface FileSignature {`
The first bytes a file type always starts with ("magic number").

22 `magic: number[];`
Leading bytes the file must start with.

36 `export interface UploadRule {`
Where an upload purpose goes, how big it may be and which file types it accepts.

44 `export const UPLOAD_RULES: Record<UploadPurpose, UploadRule> = {`
Rules of each kind of upload.

52 `PROFILE_IMAGE: {`
The original photo; the API re-encodes it (WebP, 512 px, no metadata) and deletes it.

65 `RECOVERY_DOCUMENT: {`
Identity photo and selfie of an account recovery (ADR-023): photos only.

74 `export const SIGNATURE_PROBE_BYTES = 8;`
Longest magic number, i.e. how many leading bytes confirmation reads.

77 `export function matchesSignature(rule: UploadRule, contentType: string, head: Uint8Array): boolean {`
True when the first bytes match the declared file type.

83 `export function allowsContentType(rule: UploadRule, contentType: string): boolean {`
True when the purpose accepts this file type.

storage.service.ts · 74 lines

6 `export interface StoredObject {`
What storage knows about an object.

15 `export interface UploadInput {`
Bytes to store under a new key.

23 `export interface ByteRange {`
Inclusive byte range, as in an HTTP `Range: bytes=start-end` header.

33 `export abstract class StorageService {`
The only way the backend touches file bytes (ADR-002, ADR-011). PostgreSQL keeps metadata and keys; objects live in R2 (or local files in development). Keys are immutable: an upload to an existing key fails, and a replacement writes a new key.

35 `abstract upload(input: UploadInput): Promise<StoredObject>;`
Stores bytes under a new key; fails if the key already exists.

38 `abstract delete(bucket: BucketId, key: string): Promise<void>;`
Deletes an object (no error when it does not exist).

41 `abstract head(bucket: BucketId, key: string): Promise<StoredObject | null>;`
Object metadata, or null when it does not exist.

44 `abstract download(bucket: BucketId, key: string, range?: ByteRange): Promise<Uint8Array>;`
Object bytes, optionally a range of them (signature checks, hashing jobs).

47 `abstract getSignedReadUrl(`
Short-lived URL to read an object inline.

57 `abstract getSignedUploadUrl(`
Short-lived URL to PUT exactly `contentLength` bytes of `contentType` at `key`. Both headers are part of the signature, so the storage refuses any other size or type.

67 `async replace(input: UploadInput & { oldKey: string }): Promise<StoredObject> {`
Writes a new version under `newKey` and keeps `oldKey`, so every past version stays retrievable (audit §8). The caller moves its pointer to the new key.

upload-intents.service.ts · 175 lines

16 `export interface UploadRequester {`
Who asked for an upload (only they can confirm it).

22 `export interface NewUploadIntent {`
An upload a module wants to allow.

25 `key: string;`
Allocated by the owning module with StorageKeys.

33 `export interface UploadTicket {`
What the client needs to upload: the signed URL and the headers to send.

39 `headers: { 'Content-Type': string; 'Content-Length': string };`
Headers the client must send unchanged (they are part of the signature).

48 `export class UploadIntentsService {`
Direct uploads to storage (ADR-011): the server allocates the key and signs a PUT fixing type and exact size; confirmation proves the object is what was declared before anyone uses it.

58 `async create(input: NewUploadIntent): Promise<UploadTicket> {`
Checks type, size and checksum format, records the intent and signs a PUT URL valid 15 minutes.

112 `async confirm(`
Checks the uploaded object: it exists, has the declared size (within the purpose limit) and starts with the file signature of its type. A failing object is deleted.

162 `async purgeExpired(): Promise<number> {`
Deletes expired, unconfirmed intents and anything uploaded under them (hourly job).

apps/api/src/modules/users/

Accounts: identifiers, profile, photo, preferences, access facts, reference data (wilayas, specialties, institutions, universities).

identifier.ts · 26 lines

2 `export type Identifier = { channel: 'EMAIL'; value: string } | { channel: 'SMS'; value: string };`
A sign-in identifier after normalisation (ADR-009).

5 `const EMAIL = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;`
Loose e-mail shape (the code sent to it is the real check).

8 `const ALGERIAN_LOCAL_MOBILE = /^0[5-7]\d{8}$/;`
Algerian mobile numbers written locally: 05..., 06..., 07... (10 digits).

15 `export function parseIdentifier(raw: string): Identifier | null {`
Parses what a person typed as e-mail or phone. E-mails are lower-cased; phones become E.164, with Algerian local mobiles mapped to +213. Returns null when the input is neither.

index.ts · 11 lines

profile-photo.service.ts · 128 lines

16 `const PHOTO_SIZE = 512;`
Stored photo size (square, cover) — enough for the profile header on any screen density.

26 `export class ProfilePhotoService {`
Profile photos (ScreenProfileMobile): the original goes to storage through a signed upload, then the API re-encodes it to a 512 px WebP — which drops EXIF, GPS and every other metadata — stores that under a new key and deletes the original.

37 `async requestUpload(`
Signs a direct upload of the original photo (JPEG or PNG).

61 `async attach(userId: string, input: { intentId: string; imageId: string }): Promise<void> {`
Confirms the upload, re-encodes it and makes it the profile photo.

96 `async remove(userId: string): Promise<void> {`
Removes the profile photo.

102 `async urlOf(key: string | null): Promise<string | null> {`
A signed URL for the photo (1 hour), or null without one.

110 `private async replaceKey(userId: string, key: string | null): Promise<string | null> {`
Points the account at a new photo key and returns the previous one (to delete it).

120 `async deleteObject(key: string): Promise<void> {`
Deletes a photo no row points to any more; a failure is logged with the key, not raised.

reference-data.controllers.ts · 255 lines

28 `const trimmed = Transform(({ value }: { value: unknown }) =>`
Trims a text field before validation.

38 `export class SpecialtyDto {`
A specialty in the pickers.

46 `export class PlaceDto {`
An institution or university in the pickers.

53 `export class InstitutionQuery {`
Optional wilaya filter of the institution picker.

64 `class ReasonField {`
A reason, required for admin changes.

74 `export class CreateSpecialtyDto extends ReasonField {`
Admin: a new specialty.

81 `export class CreatePlaceDto extends ReasonField {`
Admin: a new institution or university.

89 `export class ReferenceDataController {`
Pickers of the onboarding and verification steps (never typed free text, section 06).

95 `specialties(): Promise<SpecialtyDto[]> {`
GET /v1/reference-data/specialties.

102 `institutions(@Query() query: InstitutionQuery): Promise<PlaceDto[]> {`
GET /v1/reference-data/institutions.

109 `universities(): Promise<PlaceDto[]> {`
GET /v1/reference-data/universities.

117 `export class ReferenceDataAdminController {`
Admin management of the pickers (grant `system.settings.write`). The lists are operator data.

128 `listSpecialties(): Promise<SpecialtyDto[]> {`
GET /admin/v1/reference-data/specialties.

137 `listInstitutions(@Query() query: InstitutionQuery): Promise<PlaceDto[]> {`
GET /admin/v1/reference-data/institutions.

146 `listUniversities(): Promise<PlaceDto[]> {`
GET /admin/v1/reference-data/universities.

156 `async createSpecialty(`
POST /admin/v1/reference-data/specialties.

184 `async createInstitution(`
POST /admin/v1/reference-data/institutions.

211 `async createUniversity(`
POST /admin/v1/reference-data/universities.

233 `private record(`
Audit entry of a picker change.

users.controller.ts · 168 lines

58 `legal(): LegalDto {`
Current terms version and the legal pages linked at sign-up (public).

69 `listWilayas(): Promise<WilayaDto[]> {`
Wilayas for the sign-up form (public).

79 `async me(@CurrentUser() user: UserPrincipal): Promise<MeDto> {`
GET /v1/me: the signed-in account, with a signed URL of its photo.

89 `async updateProfile(`
« Modifier »: name and bio (≤ 280 characters).

101 `async updatePreferences(`
PATCH /v1/me/preferences.

115 `async setProfessionalProfile(`
« Qui êtes-vous ? » — Doctor, Resident or Medical student (ADR-006 Q7).

129 `async requestPhotoUpload(`
Signs a direct upload of a new profile photo (JPEG or PNG, 8 MB at most).

144 `async attachPhoto(`
Re-encodes the uploaded photo (512 px WebP, no metadata) and makes it the profile photo.

158 `async removePhoto(@CurrentUser() user: UserPrincipal): Promise<MeDto> {`
DELETE /v1/me/photo.

164 `private async present(record: MeRecord): Promise<MeDto> {`
Turns an account record into the response (photo key → signed URL).

users.dto.ts · 285 lines

33 `export class LegalDto {`
What the sign-up screen shows and sends back (the server refuses a stale terms version).

users.module.ts · 39 lines

26 `export class UsersModule implements OnModuleInit {`
Accounts, professional profiles, preferences and the reference pickers (wilayas, specialties, institutions, universities). Owns: `users`, `professional_profiles`, `user_preferences`, `wilayas`, `specialties`, `institutions`, `universities`. Other modules reach it only through ./index.ts.

33 `onModuleInit(): void {`
Schedules the hourly deletion of sign-ups never confirmed.

users.service.ts · 924 lines

15 `export const SIGN_IN_MAX_FAILURES = 5;`
Sign-in lockout (ADR-009): 5 wrong passwords lock the account for 15 minutes.

18 `export const UNCONFIRMED_ACCOUNT_TTL_DAYS = 7;`
Accounts never confirmed are deleted after this many days.

21 `export interface PendingAccountInput {`
What sign-up stores before the contact is confirmed.

32 `export interface AccountContacts {`
The contacts of an account (ADR-023): only confirmed ones are used to reach the person.

43 `export interface SignInCandidate {`
What sign-in needs to know about the account an identifier opens.

52 `const ADMIN_USER_SELECT = {`
What Admin Web shows of a person: identity and profile, never the password or tokens.

79 `export interface AccessFacts {`
What the access policy needs to know about an account.

86 `export interface ProfessionalProfileInput {`
« Qui êtes-vous ? »: the profession and its details (pickers only).

96 `export interface PreferencesInput {`
App preferences synced to the account.

103 `const identifierWhere = (identifier: Identifier): Prisma.UserWhereUniqueInput =>`
Database lookup of an account by e-mail or by phone.

107 `const ME_SELECT = {`
Columns returned by GET /v1/me.

134 `export type MeRecord = Prisma.UserGetPayload<{ select: typeof ME_SELECT }>;`
Shape of an account as read for GET /v1/me.

138 `export class UsersService {`
Owner of `users`, `professional_profiles`, `user_preferences` and the reference pickers.

148 `async createOrRestartPendingAccount(input: PendingAccountInput): Promise<{ userId: string }> {`
Creates the account, or restarts an unconfirmed one with the new details. A confirmed identifier is refused with ACCOUNT_EXISTS (design: « Ce compte existe déjà »).

201 `async markIdentifierConfirmed(userId: string, channel: Identifier['channel']): Promise<void> {`
Marks the e-mail or phone as confirmed (after the sign-up code).

210 `async findSignInCandidate(identifier: Identifier): Promise<SignInCandidate | null> {`
The account an identifier opens, with what sign-in checks (null when none).

```
234 async getAccessFacts(userId: string): Promise<AccessFacts | null> {
```
What the access policy needs to know about an account (ADR-010).

```
251 async isActive(userId: string): Promise<boolean> {
```
True when the account exists and is ACTIVE (checked on every token refresh).

```
260 async findConfirmedActiveUserId(identifier: Identifier): Promise<string | null> {
```
User id of a confirmed, active account for this identifier (password reset).

```
268 async registerFailedSignIn(userId: string): Promise<Date | null> {
```
Counts one wrong password atomically; returns the lock end when this failure locks the account.

```
282 async clearSignInFailures(userId: string): Promise<void> {
```
Resets the wrong-password counter and records the sign-in as activity.

```
290 async contactsOf(userId: string): Promise<AccountContacts | null> {
```
The account's contacts and language (for notices and contact changes).

```
316 async findAccountIdByContact(identifier: Identifier): Promise<string | null> {
```
The live account whose confirmed contact this is (account recovery, ADR-023).

```
328 async setContact(userId: string, identifier: Identifier): Promise<string | null> {
```
Sets an account's e-mail or phone to a contact just proven by a code, confirmed now, and returns the value it replaces. A sign-up never confirmed that holds the same contact is dropped (it could never be used); an account that confirmed it keeps it (ACCOUNT_EXISTS).

```
354 async removeContact(userId: string, channel: Identifier['channel']): Promise<string> {
```
Removes the e-mail or the phone; the other one must be confirmed, so the person stays reachable.

```
378 async replaceContactsForRecovery(
```
Applies an approved account recovery (ADR-023): the proven contact becomes the only one, and the password is replaced by an unusable one, so only « Mot de passe oublié » on the new contact opens the account. Returns the contacts it replaced.

```
423 private async releaseFromUnconfirmed(
```
Drops a never-confirmed sign-up holding this contact, so a confirmed owner can take it.

```
441 async setPassword(userId: string, passwordHash: string): Promise<void> {
```
Stores a new password hash and lifts any lockout.

```
449 async getMe(userId: string): Promise<MeRecord> {
```
The signed-in account (GET /v1/me); a deleted account counts as signed out.

```
457 async updateProfile(
```
« Modifier »: name and bio (null clears the bio). Verification keeps the names it checked.

```
473 async updatePreferences(userId: string, input: PreferencesInput): Promise<MeRecord> {
```
Saves language, theme or default weight unit.

```
486 async setProfessionalProfile(userId: string, input: ProfessionalProfileInput): Promise<MeRecord> {
```
Records who the person is (onboarding « Qui êtes-vous ? »). Set once; changing type later is a re-verification (Phase 9).

```
510 async professionalTypeOf(userId: string): Promise<ProfessionalType | null> {
```
Current professional type, or null before « Qui êtes-vous ? ».

```
519 async assertProfessionalReferences(input: ProfessionalProfileInput): Promise<void> {
```
Checks that the pickers referenced by declared details exist (VALIDATION_FAILED otherwise).

```
528 async describeReferences(ids: {
```
Display names of picker references, for a reviewer reading declared details.

```
563 async applyVerifiedStatus(
```
Applies a verified status case to the profile: the proved type and its details replace the previous ones (a role transition keeps favorites, activity and preferences). The profile keeps only a masked Ordre number.

```
584 async names(ids: string[]): Promise<Map<string, { firstName: string; lastName: string }>> {
```
First and last names by user id, for admin lists.

597 `async affiliationNames(ids: { specialtyIds: string[]; universityIds: string[] }): Promise<{`
Specialty (FR/EN) and university names by id, for the verification queue columns.

624 `async analyticsFacts(userId: string): Promise<{`
Segments of the analytics envelope (section 16): never names or contact data.

647 `countSignUps(from: Date, to: Date): Promise<number> {`
Confirmed accounts created in [from, to) (onboarding funnel).

657 `async touchLastSeen(userId: string, now = new Date()): Promise<void> {`
Records activity for « Dernière activité »; written at most once an hour per person.

668 `adminList(filter: {`
Admin Web « Utilisateurs »: 50 per page, newest first; search on name, e-mail or phone.

704 `adminDetail(userId: string) {`
One account as Admin Web shows it.

709 `async setStatus(userId: string, status: 'ACTIVE' | 'SUSPENDED'): Promise<boolean> {`
Suspension blocks sign-in and refresh (ADR-010); the account and its data are kept.

718 `async passwordHashOf(userId: string): Promise<string | null> {`
The password hash of a live account, for a re-authentication (account deletion).

732 `async eraseAccount(userId: string): Promise<boolean> {`
Account deletion (section 08, Settings « Supprimer le compte »): the person's details, professional profile and preferences are erased and the identifiers freed for a new account. The row stays as a tombstone because payments and past verification decisions keep pointing at it (legal retention, ADR-005). Returns false when the account was already deleted.

764 `async reviewSummary(userId: string) {`
What a verification reviewer sees about the person (never the password or tokens).

794 `listSpecialties() {`
Medical specialties, A–Z (picker).

802 `listInstitutions(wilayaCode?: number) {`
Hospitals and institutions, optionally of one wilaya (picker).

811 `listUniversities() {`
Universities (picker).

819 `async createSpecialty(input: { code: string; nameFr: string; nameEn: string }) {`
Admin: adds a specialty; a duplicate code is refused.

831 `async createInstitution(input: { name: string; wilayaCode: number }) {`
Admin: adds an institution in a wilaya.

844 `async createUniversity(input: { name: string; wilayaCode: number }) {`
Admin: adds a university in a wilaya.

857 `private conflict(error: unknown, message: string): unknown {`
Turns a unique-constraint violation into a readable conflict error.

864 `listWilayas(): Promise<{ code: number; name: string }[]> {`
The 58 wilayas, by code.

872 `async purgeUnconfirmedAccounts(): Promise<number> {`
Deletes accounts whose identifier was never confirmed (hourly maintenance job).

886 `private async assertWilaya(code: number): Promise<void> {`
Refuses an unknown wilaya code.

896 `private async assertReferencesExist(`
Adds a field error for each picker id that does not exist.

apps/api/src/modules/verification/

Identity and status verification: the person's cases, the reviewer side, files, maintenance (expiry, reminders, deletion), the access snapshot.

index.ts · 10 lines

verification-admin.controller.ts · 138 lines

34 `export class VerificationAdminController {`
Admin Web « Vérifications »: queue, review drawer, restricted viewer, decisions (section 07).

41 `async queue(@Query() query: VerificationQueueQuery): Promise<AdminVerificationQueueItemDto[]> {`
GET /admin/v1/verification/cases: the review queue.

49 `async detail(@Param('id', id) caseId: string): Promise<AdminVerificationCaseDto> {`
One case in the review drawer.

58 `async claim(`
Takes the case for review.

71 `async release(`
Gives the case back to the queue.

87 `async approve(`
Approves the case.

108 `async reject(`
Rejects the case with a reason.

129 `async viewDocument(`
Restricted viewer: 60-second URL plus watermark text; every view is audited.

verification-cases.ts · 87 lines

12 `export const CASE_SELECT = {`
Columns of a case and its documents read by both sides.

42 `export type CaseRecord = Prisma.VerificationCaseGetPayload<{ select: typeof CASE_SELECT }>;`
A case as read with CASE_SELECT.

45 `export const LATEST_FIRST = [`
Newest case first.

51 `export function detailsOf(record: { declaredDetails: Prisma.JsonValue }): Record<string, unknown> {`
The declared details as an object (empty when cleared).

57 `export function caseTarget(record: CaseRecord): ProfessionalType | null {`
The role a case proves.

62 `export interface StepState extends StepRule {`
A required or optional step, and whether its file is there.

67 `export function stepStates(record: CaseRecord): StepState[] {`
The steps of a case with what is already provided.

79 `export function ownerDetails(record: CaseRecord): Partial<DeclaredDetails> {`
Details as the owner sees them: complete while the case is being prepared, then with the Ordre number masked.

verification-files.service.ts · 51 lines

7 `export class VerificationFilesService {`
Deletes verification files: the object in the private bucket first, then its row.

16 `async eraseCase(caseId: string): Promise<number> {`
Deletes every document of a case and clears its declared details. Returns the count deleted.

33 `async deleteDocument(document: { id: string; storageKey: string }): Promise<void> {`
Deletes one document: the stored object first, then its row (so a failure leaves the row to retry).

43 `async deleteObject(storageKey: string): Promise<void> {`
Deletes the object of a replaced or removed document whose row is already gone. No row points to it any more, so a failure is logged with the key for manual cleanup instead of failing the person's request.

verification-maintenance.service.ts · 152 lines

16 `const UNDECIDED: ReadonlySet<VerificationStatus> = new Set([`
Cases no reviewer has decided: nothing about them needs keeping once the account is gone.

25 `export class VerificationMaintenanceService {`
Daily verification housekeeping: expiry, reminders and the retention purge (section 06).

33 `async expireCases(now = new Date()): Promise<number> {`
VERIFIED cases past their expiry become EXPIRED, and the person is told.

60 `async sendExpiryReminders(now = new Date()): Promise<number> {`
One reminder per threshold (60 and 14 days before expiry). A threshold already passed when the case was approved is skipped in favour of the nearer one.

99 `async eraseUser(userId: string): Promise<number> {`
Account deletion: every document of the person is deleted now, cases still waiting for a decision leave the queue, and decided cases keep only the decision (who, when, outcome) with their declared details cleared. Returns the number of documents deleted.

118 `async purge(now = new Date()): Promise<{ documents: number; abandoned: number }> {`
Deletes documents past retention, clears declared details of cases decided more than the retention period ago, and deletes cases abandoned before submission.

verification-review.service.ts · 453 lines

28 `export interface QueueFilter {`
Filters of the review queue.

35 `const QUEUE_SELECT = {`
Columns read for each queue row.

47 `export type QueueRecord = Prisma.VerificationCaseGetPayload<{ select: typeof QUEUE_SELECT }> & {`
One queue row with the names the table shows.

51 `specialty: { fr: string; en: string } | null;`
Declared specialty (professional status) or university (student status), by name.

59 `export interface ReviewRecord {`
Everything the review drawer shows: the case, who handles it, the person, their picker names and earlier cases.

75 `export interface DocumentView {`
A short-lived link to one document and the watermark the viewer draws over it.

88 `export class VerificationReviewService {`
Reviewer side of verification (Admin Web « Vérifications »): queue, claim, restricted viewer and decisions. Every step is audited; appeals go to a different reviewer holding `verification.appeal.decide`.

99 `async queue(filter: QueueFilter): Promise<QueueRecord[]> {`
Oldest submissions first.

144 `async detail(caseId: string): Promise<ReviewRecord> {`
The review drawer of one submitted case.

176 `async claim(admin: AdminPrincipal, caseId: string, client: ClientInfo): Promise<ReviewRecord> {`
Takes a waiting case: it becomes IN_REVIEW for this reviewer only.

199 `async release(`
Gives the case back to the queue.

228 `async approve(`
Approves the case: VERIFIED (with an optional expiry), the proved role and details are copied to the profile, the person is notified, the documents get their deletion date.

288 `async reject(`
Rejects the case with a reason code; the person is notified and may start again (or appeal after enough rejections).

326 `async viewDocument(`
Restricted viewer (section 06): a 60-second signed URL for one document of a case the admin is reviewing, with the watermark text the viewer overlays. Every view is audited.

363 `private async find(caseId: string): Promise<CaseRecord> {`
Loads a case or answers NOT_FOUND.

373 `private async claimedBy(admin: AdminPrincipal, caseId: string): Promise<CaseRecord> {`
A case this admin has claimed (and may decide).

383 `private assertMayDecide(admin: AdminPrincipal, record: CaseRecord): void {`
An appeal needs `verification.appeal.decide` and a reviewer other than the one who rejected.

400 `private async decide(`
Writes the decision if the case is still claimed by this admin, and schedules its documents' deletion.

424 `private async adminName(id: string): Promise<{ id: string; fullName: string } | null> {`
Name of an admin, for the drawer.

430 `private record(`
Audit entry of a reviewer action.

verification-status.service.ts · 160 lines

6 `export interface CaseSnapshot {`
The facts of one case the access policy looks at.

14 `renewalPending?: boolean;`
A newer case of the same type is being prepared or reviewed while this earlier decision still counts (renewal, role change): see {@link VerificationStatusService.accessSnapshot}.

18 `export type VerificationSnapshot = Record<VerificationType, CaseSnapshot>;`
Latest case of each type; a type with no case is NOT_STARTED.

22 `const UNDECIDED: ReadonlySet<VerificationStatus> = new Set([`
Statuses of a case not decided yet.

32 `export class VerificationStatusService {`
Read side of verification used by the access policy.

36 `async snapshot(userId: string): Promise<VerificationSnapshot> {`
The latest case of each type (NOT_STARTED when none).

65 `async accessSnapshot(userId: string, now = new Date()): Promise<VerificationSnapshot> {`
What access and checkout rely on (section 27: re-verification never interrupts access). While the latest case of a type is not decided yet, the decision before it keeps counting: a VERIFIED case until its expiry and, for professional status, a lapsed (EXPIRED) one too, since with a live subscription an expiry does not end access before the period end. Without such an earlier decision, the open case itself is reported, as {@link snapshot} does.

106 `async waiting(now: Date, slaHours: number, limit: number) {`
Cases waiting for a decision, the oldest ones, and how many wait longer than the SLA.

140 `async funnel(`
Onboarding funnel (analytics « Utilisateurs »): people who submitted, and who got verified.

verification.controller.ts · 150 lines

39 `export class VerificationController {`
The person's verification (section 06). Real accounts only — never a demo token.

45 `async overview(@CurrentUser() user: UserPrincipal): Promise<VerificationOverviewDto> {`
Latest case of each type and whether a new one can start.

53 `async start(`
POST /v1/me/verification/cases: starts a case.

65 `async saveDetails(`
PUT /v1/me/verification/cases/:id/details: saves the declared details.

77 `async requestUpload(`
Signs a direct upload of one step's file to private storage.

91 `async attach(`
Confirms an uploaded file and attaches it to its step.

102 `async removeDocument(`
Removes a file from a case being prepared.

116 `async submit(`
« Envoyer ».

128 `async appeal(`
Appeal after a second rejection; decided by a different reviewer.

140 `async erase(`
Deletes the files and declared details of a decided case now (deletion request).

verification.dto.ts · 700 lines

57 `export class VerificationDetailsDto {`
Complete details of the case (only the fields of its type are accepted).

verification.module.ts · 52 lines

35 `export class VerificationModule implements OnModuleInit {`
Identity, professional-status and student-status cases, their documents and decisions (section 06, ADR-013). Owns: `verification_cases`, `verification_documents`. Other modules reach it only through ./index.ts.

42 `onModuleInit(): void {`
Schedules the daily expiry and reminders, and the hourly purge of documents past retention.

verification.policy.ts · 162 lines

6 `export const VERIFICATION_SLA_HOURS = 24;`
Service level of the review queue: a case waiting longer is flagged (Admin Web « SLA 24 h »).

9 `export const DOCUMENT_RETENTION_MONTHS = 24;`
Documents and declared details are deleted this long after the decision (section 06).

11 `export const EXPIRY_REMINDER_DAYS = [60, 14] as const;`
Reminders before a verification expires (section 06).

13 `export const RENEWAL_WINDOW_DAYS = 60;`
A new case may replace a verified one from this many days before it expires.

15 `export const ABANDONED_CASE_DAYS = 90;`
Cases never submitted are deleted, with their files, after this many days without activity.

17 `export const REJECTIONS_BEFORE_APPEAL = 2;`
An appeal is possible after this many rejections of the same type (section 06).

20 `export const REJECTION_REASONS = [`
Reasons a reviewer can choose when rejecting.

31 `export const STEP_TYPES = [`
The documents a case can ask for.

41 `export const IDENTITY_DOCUMENT_KINDS = ['NATIONAL_ID', 'PASSPORT', 'STUDENT_CARD'] as const;`
Documents accepted to prove identity.

45 `export interface StepRule {`
One step of a case and whether it is required.

51 `const RANK: Record<ProfessionalType, number> = { STUDENT: 0, RESIDENT: 1, DOCTOR: 2 };`
DOCTOR ranks above RESIDENT above STUDENT; a status case never moves a person down.

54 `export function canTarget(current: ProfessionalType, target: ProfessionalType): boolean {`
True when moving from `current` to `target` is not a step down.

59 `export function targetOf(`
The professional type a status case proves; identity cases prove none.

72 `export function stepsFor(type: VerificationType, target: ProfessionalType | null): StepRule[] {`
The steps a case needs: identity front (+ optional back), student proof, residency attestation or Ordre proof.

```
90  const ORDER_NUMBER = /^[A-Za-z0-9][A-Za-z0-9/-]{0,29}$/;
    Ordre number: letters, digits, "/" and "-".
111 export type DeclaredDetails =
    Valid declared details of any case.
118 function schemaFor(type: VerificationType, target: ProfessionalType | null) {
    The schema of the details for a case and role.
125 export type DetailsCheck =
    Result of checkDetails: the parsed details, or the field errors.
129 export function checkDetails(
    Validates complete declared details for a case (on save and again on submission).
145 export function maskOrderNumber(orderNumber: string): string {
    The Ordre number reduced to its last three characters (what the owner sees after submission).
150 export function addMonths(date: Date, months: number): Date {
    Adds months in UTC (negative values go back).
157 export const EXTENSIONS: Record<string, 'jpg' | 'png' | 'pdf'> = {
    File types accepted for verification documents.
```

verification.service.ts · 512 lines

```
35  const TYPES: VerificationType[] = ['IDENTITY', 'PROFESSIONAL_STATUS', 'STUDENT_STATUS'];
    The three kinds of case, in the order the app lists them.
39  const UPLOAD_RATE = { name: 'verification.upload', limit: 30, windowSeconds: 3600 };
    Upload requests per person and hour (ADR-009 style limit on signed-URL issuance).
42  export interface CaseState {
    A case and whether the person may appeal its decision.
48  export interface TypeState {
    The latest case of one type and whether a new one may start.
51  canStart: boolean;
    Whether a new case of this type may be started now (for the person's current type).
56  export interface VerificationOverview {
    What the verification screen shows.
62  export interface DetailsInput {
    Declared details the person types or picks (which ones depend on the case).
73  export interface UploadRequest {
    What the app declares before uploading a file: its step, type, size and checksum.
85  export class VerificationService {
    The person's side of verification (section 06): one case per type, prepared step by step, submitted once, then decided by a reviewer. Identity and status are separate cases.
96  async overview(userId: string): Promise<VerificationOverview> {
    The person's latest case of each type, with whether they may start a new one or appeal.
122 async start(
    Starts a new case of one type (and, for a status case, the role it proves); refused while one is open or when already verified.
151 async saveDetails(userId: string, caseId: string, input: DetailsInput): Promise<CaseState> {
    Replaces the declared details of a case being prepared; they must be complete and valid.
168 async requestUpload(
    Signs a direct upload for one step (private bucket, 15 minutes, exact size and type).
195 async attach(
    Confirms an uploaded file and attaches it to its step, replacing the previous file of that step.
```

```
237 async removeDocument(userId: string, caseId: string, documentId: string): Promise<CaseState> {
```
Removes one file from a case being prepared.

```
250 async submit(userId: string, caseId: string, client: ClientInfo): Promise<CaseState> {
```
« Envoyer »: the details and every required step must be present.

```
276 async appeal(userId: string, caseId: string, client: ClientInfo): Promise<CaseState> {
```
Appeal after a second rejection; a different reviewer decides (section 06).

```
308 async eraseDocuments(userId: string, caseId: string, client: ClientInfo): Promise<CaseState> {
```
Deletion request after the decision (section 06): files and declared details go now.

```
329 private latest(userId: string, type: VerificationType): Promise<CaseRecord | null> {
```
The most recent case of one type.

```
338 private load(caseId: string): Promise<CaseRecord> {
```
Reloads a case after a change.

```
346 private async owned(userId: string, caseId: string): Promise<CaseRecord> {
```
A case of this person (another person's case is "not found").

```
356 private async preparing(userId: string, caseId: string): Promise<CaseRecord> {
```
A case of this person still being prepared (not yet sent).

```
365 private async lockPreparing(tx: Prisma.TransactionClient, caseId: string): Promise<void> {
```
Locks the case row for the transaction and checks it is still being prepared.

```
374 private assertStep(record: CaseRecord, stepType: StepType): void {
```
Refuses a step that does not belong to this case.

```
386 private async assertDetails(
```
Checks declared details: the schema of the case, a student card only for students, and that every picker id exists.

```
414 private defaultTarget(
```
The role a status case proves by default: the person's current role.

```
425 private resolveTarget(
```
The role a new status case proves: students prove STUDENT; others DOCTOR or RESIDENT, never a lower role than today.

```
453 private startRefusal(
```
Why a new case may not start now, or null when it may.

```
477 private async canAppeal(record: CaseRecord): Promise<boolean> {
```
True for the latest case of its type, rejected by a reviewer, with documents, after enough rejections, and not appealed yet.

```
491 private record(
```
Audit entry of an action of the person on their case.

apps/api/src/openapi/

Builds the OpenAPI document from the controllers and DTOs.

document.ts · 32 lines

```
5 export const OPENAPI_TITLE = 'DokTool API';
```
Title and version written into the published OpenAPI contract.

```
9 export function buildOpenApiDocument(app: INestApplication): OpenAPIObject {
```
Builds the OpenAPI document that packages/api-contract publishes to the Dart and TS clients.

apps/api/src/scripts/

Command-line tools: create an admin, generate the contract, seed demo data, Medicalcul extraction and import.

create-admin.ts · 49 lines

extract-medicalcul.ts · 73 lines

35 `const files = readdirSync(raw)`
Calculator pages: every page at the root except the indexes and the start pages.

43 `const catalogue = buildCatalogue(`
Translate every page and write the deliverables.

generate-openapi.ts · 101 lines

13 `const contractPath = fileURLToPath(`
Where the published contract lives (packages/api-contract).

18 `function sortKeys(value: unknown): unknown {`
Recursively sorts object keys so the file is byte-stable across runs and machines.

32 `const env = loadEnv({`
Preview mode resolves the module graph and controller metadata without instantiating providers, so no database connection is attempted and the values below (never secrets) are never used.

54 `function nameEnums(value: unknown): void {`
`auth.sign-up` → `authSignUp`: generated clients need identifiers for such enum values.

import-medicalcul.ts · 123 lines

apps/api/src/scripts/medicalcul/

Medicalcul extraction: parse the HTML pages, translate exact formula patterns into definitions, build the catalogue and coverage report.

catalogue.ts · 326 lines

13 `'STRUCTURED_CALCULATION_DEFINITION',`
f_Calculer translated exactly into a DokTool definition (draft, review required).

15 `'FORMULA_TEXT_AVAILABLE',`
f_Calculer exists but uses constructs without an exact engine equivalent.

17 `'METADATA_ONLY',`
A page without calculation logic.

19 `'UNRESOLVED',`
The page itself could not be read.

57 `indexed: z.boolean(),`
Listed in the app's alphabetical index (_indexalpha.html).

114 `const hash = (data: string | Buffer) => createHash('sha256').update(data).digest('hex');`
SHA-256 in hex.

117 `export const slugFor = (id: string) =>`
DokTool slug from the page file name: abcd2.html → abcd2, aclf_grades.html → aclf-grades.

128 `export function buildEntry(file: string, bytes: Buffer, indexed: boolean): CatalogueEntry {`
Turns one downloaded Medicalcul page into a catalogue entry: title, specialty, provenance and, when the translator succeeds, a DokTool draft definition.

227 `export function buildCatalogue(`
Builds the whole catalogue from every page, in a stable order.

263 `const csvCell = (value: string | number | boolean | null | undefined | string[]) => {`
One cell of coverage.csv.

273 `export function coverageCsv(catalogue: Catalogue): string {`
One row per page: the coverage matrix of the integration report.

324 `export const catalogueJsonSchema = () =>`
JSON Schema of catalogue.json, written next to it.

logic.ts · 1192 lines

30 `export class Unsupported extends Error {}`
A construct the translation refuses; the message is reported in the coverage matrix.

33 `export class NoCalculation extends Unsupported {}`
The page computes nothing (a reference table or a classification to read).

38 `const MAX_NODES = 2000;`
Larger formulas are not reviewable by a person; they stay out of the import.

42 `function treeSize(e: Expr): number {`
Node count of an expression tree (shared subtrees counted each time), capped.

58 `function bounded(e: Expr): Expr {`
Refuses formulas that grow beyond the size DokTool accepts.

64 `type Val =`
What a JavaScript value of the page becomes during translation.

67 `| { t: 'fixed'; e: Expr; int: boolean; digits: number }`
`x.toFixed(n)`: a display string in JS, accepted only as a result written to a field.

70 `| { t: 'str'; e: Expr }`
A constant text or an `if` tree of constant texts (interpretation labels).

72 `| { t: 'raw'; field: string }`
`document.Form.<field>.value` before any numeric coercion: still a string in JS.

75 `| { t: 'conflict'; what: string };`
Different kinds on two branches: an error only if something reads it afterwards.

78 `type Env = Map<string, Val>;`
Variables (or output fields) and their translated values.

80 `interface State {`
Variables and outputs at one point of the page's script.

86 `export interface Translation {`
A translated calculator and the points a reviewer must check.

89 `reviewNotes: string[];`
Points a reviewer must look at: values the source does not state and DokTool requires.

93 `interface Field {`
One form field of the page (radios of one name, a select, a checkbox or a number box).

103 `export function translate(page: ParsedPage): Translation {`
Translates a page's f_Calculer() into a DokTool definition, or throws Unsupported with the reason.

113 `function findCalculer(scripts: string[]): FunctionDeclaration {`
Finds the f_Calculer function among the page's scripts.

133 `export function keyFor(name: string, taken: Set<string>): string {`
camelCase engine key from a form field name, stable and unique within the page.

145 `function indexFields(controls: PageControl[]): Map<string, Field> {`
Groups the page controls into fields with stable camelCase keys.

168 `const choiceValue = (index: number) => `o${index + 1}`;`
Value of the n-th answer of a CHOICE input: o1, o2...

171 `function pointsOf(raw: string | null, where: string): number {`
Numeric value of a radio or option, exactly as JS `value * 1` reads it.

184 `class Context {`
Walks the JavaScript syntax tree of f_Calculer and builds the equivalent DokTool expressions.

193 `private lookup(x: string): { name: string; field: Field; control: PageControl | null } {`
`document.Form.<x>`: browsers resolve <x> as a field name, then as an element id. Returns the field and, when reached through an id, the one control it designates.

```
204 block(statements: readonly Statement[], state: State): void {
```
Translates a list of statements in order.

```
209 private statement(node: Statement, state: State): void {
```
Translates one statement (declaration, assignment, if, for, call).

```
231 private declare(node: VariableDeclaration, state: State): void {
```
var x = ...

```
242 private expressionStatement(expression: Expression, state: State): void {
```
An expression used as a statement (assignment, field write, call).

```
261 private assign(node: AssignmentExpression, state: State): void {
```
x = ..., x += ..., or a write to an output field.

```
286 private ifStatement(node: IfStatement, state: State): void {
```
if/else: both branches are translated, then merged with if() expressions.

```
297 private forStatement(node: ForStatement, state: State): void {
```
Simple counted loops over a fixed list of fields (unrolled).

```
337 private tryStatement(node: TryStatement): void {
```
Only the Android history call is wrapped in try/catch; it has no effect on results.

```
350 private read(name: string, state: State): Val {
```
The current value of a variable.

```
358 value(node: Expression | JsNode, state: State): Val {
```
Translates one expression.

```
432 private member(node: MemberExpression, state: State): Val {
```
form.field.value, Math.PI and similar member reads.

```
469 private call(node: CallExpression, state: State): Val {
```
Function calls: Math.*, parseFloat, the page's helper functions, toFixed.

```
484 const value = pointsOf(field.controls[0]!.value, `"${name.e.str}"`);
```
_scripts.js: the checked element's value × 1, else 0.

```
534 throw new Unsupported(`Math.${method}`);
```
round/floor/ceil/abs/trunc have no exact engine operator (Math.round rounds halves up, the engine rounds them away from zero).

```
555 private choicePoints(name: string, field: Field): Val {
```
The points of the selected answer of a radio group or select.

```
562 number(value: Val, node: JsNode | Expression): Extract<Val, { t: 'num' }> {
```
JS numeric coercion (`x * 1`, `-`, `*`, `/`, comparisons).

```
597 boolean(value: Val, node: JsNode | Expression): Expr {
```
JS truthiness, only where it is exact.

```
604 binary(operator: string, left: Val, right: Val, node: JsNode | Expression): Val {
```
Translates a binary operator (text concatenation, arithmetic, comparison).

```
662 private choiceComparison(field: Val, text: Val): Expr | null {
```
`select.value == "2"`: true when that option is chosen.

```
681 private arithmetic(
```
+, −, ×, ÷ on numbers, folding constants.

```
690 if (op === 'mul' && isConst(b.e) && b.e.num === 1) return a;
```
`x * 1` is JS for « read as a number »: no engine operation needed.

```
712 build(state: State): Translation {
```
Assembles the definition from the translated result and the inputs.

```
785 private inputs(notes: string[]): CalculatorInput[] {
```
The DokTool inputs of the fields the formula actually uses.

```
870 const quantityFor = (key: string) =>
```
A quantity code from an input key: bodyWeight → BODY_WEIGHT.

```
function clone(state: State): State {
```
A copy of the state, for translating one branch of an if.

```
function merge(condition: Expr, whenTrue: Env, whenFalse: Env): Env {
```
Joins the two branches of an if into one state.

```
const same = (a: Val, b: Val) => {
```
True when two translated values are identical.

```
function mergeValue(condition: Expr, a: Val, b: Val, what: string): Val {
```
Joins one variable of both branches into a single if() value.

```
interface Region {
```
A stretch of the result axis where the interpretation text is constant.

```
point: boolean;
```
A single point `lo === hi`, or the open interval between them.

```
export type ImportedBand = Omit<InterpretationBand, 'label'> & { label: string };
```
Turns text results chosen by conditions on the main result into interpretation bands `from ≤ value < to`. Conditions must compare the result with constants. When the result is provably an integer, strict and non-strict bounds convert exactly; otherwise a bound that belongs to the wrong side has no exact band form and the translation stops.

```
export function bands(
```
Turns the page's if-chains on the result into ordered interpretation bands.

```
const collect = (e: Expr) => {
```
Gathers the thresholds the conditions compare the result with.

```
const sample = (region: Region) =>
```
A value inside a region, to evaluate the conditions there.

```
const condition = (e: Expr, x: number): boolean => {
```
Evaluates a condition for a given result value.

```
const text = (e: Expr, x: number): string => {
```
The interpretation text shown for a given result value.

```
function formProperty(node: MemberExpression): { field: string; property: string } | null {
```
`document.Form.<field>.<property>` or `document.getElementById('<id>')` is not supported.

```
function mathConstant(node: MemberExpression): number | null {
```
Math.PI, Math.E… as numbers.

```
function isAndroidBridge(node: CallExpression): boolean {
```
Calls into the Android app's bridge, which do not affect the result.

```
function literalNumber(node: Expression): number | null {
```
A numeric literal, or null.

```
function compoundOperator(operator: string): string {
```
+= → +, -= → -…

```
function describe(node: JsNode | Expression): string {
```
A short text naming a syntax node, for error messages.

```
function line(node: JsNode | Expression): number {
```
The source line of a syntax node.

page.ts · 298 lines

```
type Node = DefaultTreeAdapterMap['node'];
```
parse5 node types.

```
export interface PageControl {
```
One form control of a calculator page.

```
name: string;
```
The form field name the page script reads (`document.Form.<name>`).

```
value: string | null;
```
Radio and checkbox `value` attribute, as written.

```
label: string;
```
Label text shown next to the control (radio/checkbox label, field label, result label).

```
heading: string | null;
```
Last bold heading above the control: the question of a radio group.

```
unit: string | null;
```
Text right after the field in the same cell, e.g. « kg », « mmol/l ».

```
export interface ParsedPage {
```
What is read from a calculator page.

```
specialty: string | null;
```
From the specialty icon `zzimgspe_<code>.png`.

```
specialtyLabel: string | null;
```
Specialty as worded in the page title (« … ~ Neurologie »).

```
scripts: string[];
```
Inline `<script>` bodies (shared `_scripts.js` is referenced, not inlined).

```
const isElement = (node: Node): node is Element => 'tagName' in node;
```
True for an HTML element node.

```
const attr = (element: Element, name: string) =>
```
One attribute of an element.

```
const hasClass = (element: Element, name: string) =>
```
True when an element has a CSS class.

```
export function textOf(node: Node): string {
```
Text of a subtree, whitespace (including no-break spaces) collapsed.

```
function rawText(node: Node): string {
```
The text inside a node, scripts and styles excluded.

```
function elements(root: Node, tagName: string): Element[] {
```
Every element of a tag name under a node.

```
function lines(element: Element): string[] {
```
Lines of a paragraph split on `<br>`, without its leading caption (« Références: »).

```
function trailingText(node: Element): string | null {
```
Text that follows `node` inside its parent, up to the next form control.

```
function leadingText(node: Element): string {
```
Text before `node` inside its parent (result captions: « Score : <input> »).

```
export function parsePage(html: string): ParsedPage {
```
Reads a calculator page: title, specialty, controls with their labels, scripts, interpretation and references.

```
let headingUsedBy: string | null = null;
```
The radio group that already took the current heading: a heading names one question.

```
if (node.tagName === 'td' && attr(node, 'colspan') && !hasField(node)) {
```
A full-width cell without any field is the question above the next options.

```
const cleanHeading = (text: string) =>
```
A heading without numbering and trailing colon.

```
function hasField(element: Element): boolean {
```
True when an element contains a form field.

```
function rowLabel(element: Element): string | null {
```
Caption in the cell before the field's cell, on the same table row (« Tranche d'âge : »).

```
function insideLabel(element: Element): boolean {
```
True when an element sits inside a <label>.

apps/api/src/scripts/

Command-line tools: create an admin, generate the contract, seed demo data, Medicalcul extraction and import.

seed-demo.ts · 980 lines

```
229 function random(seed: number): () => number {
```
Deterministic pseudo-random numbers: the same database each run.

```
241 const between = (min: number, max: number) => min + Math.floor(next() * (max - min + 1));
```
A pseudo-random integer in [min, max] (seeded, so the demo is reproducible).

```
243 const daysAgo = (days: number, hour = between(8, 21)) => {
```
A moment `days` ago at a plausible hour.

```
249 const slug = (text: string) =>
```
A URL-safe slug.

```
256 const sha256 = (bytes: Uint8Array) => createHash('sha256').update(bytes).digest('hex');
```
SHA-256 in hex.

```
259 async function demoPdf(title: string, lines: string[]): Promise<Uint8Array> {
```
One-page A4 PDF that says what it is: a demonstration document.

```
336 async function seed(prisma: PrismaService, storage: StorageService): Promise<void> {
```
Fills an empty database with demo content, people, payments and analytics.

```
416 const showcasePassword = randomBytes(12).toString('base64url');
```
Users ——

```
420 const lockedHash = await hasher.hash(randomBytes(32).toString('base64url'));
```
Nobody knows this one: the other users exist for lists and dashboards only.

```
423 interface Person {
```
A demo person and the state they are placed in.

```
431 engagement: number;
```
Chance of using the app on a given day: a few daily users, many occasional ones.

```
435 const history: Prisma.AuditLogEntryCreateManyInput[] = [];
```
Audit entries of what the team did, so the history reads like a running service.

```
705 payments.sort((a, b) => a.at.getTime() - b.at.getTime());
```
Payments are numbered in the order they happened, as the checkout numbers them.

```
858 const subject = (kind: 'user' | 'demo', id: string) =>
```
Analytics subject hash, computed like the API does.

```
861 const tierOf = (person: Person): EntitlementTier =>
```
The access tier a demo person ends up in.

The mobile app (apps/mobile)

apps/mobile/lib/

The app's entry point: `main.dart` starts Riverpod and the app.

main.dart · 10 lines

apps/mobile/lib/src/

The root widget: themes, languages, router, and the analytics and offline hosts around every screen.

app.dart · 39 lines

14 `class DokToolApp extends ConsumerWidget {`
Root widget: light and dark themes from the tokens, FR/EN from packages/i18n, both switched live from Settings.

apps/mobile/lib/src/core/api/

The dio client with its interceptors, `ApiError`, and the retry rule for providers (the generated client is skipped here).

api_client.dart · 44 lines

11 `class AcceptLanguageInterceptor extends Interceptor {`
Sends the interface language, so server texts (e.g. OTP messages) match it.

18 `void onRequest(RequestOptions options, RequestInterceptorHandler handler) {`
Adds the UI language to every request so the API answers in French or English.

26 `final dioProvider = Provider<Dio>((ref) {`
HTTP client for the DokTool API: interface language and session token on every call, with a single-flight refresh after a 401.

43 `final apiProvider = Provider<DokToolApi>((ref) => DokToolApi(ref.watch(dioProvider)));`
The generated client of the user API (packages/api-contract).

api_error.dart · 78 lines

6 `class ApiError implements Exception {`
A failed API call, read from the backend error envelope `{ error: { code, message, details } }`. Screens translate [code]; they never show [message], which is for developers and logs.

10 `const ApiError.network() : this(code: networkCode);`
No response reached the app (offline, timeout, DNS...).

19 `final Map<String, List<String>> fields;`
Field name → messages, for VALIDATION_FAILED.

26 `bool get isSessionEnded => code == 'UNAUTHENTICATED' || code == 'SESSION_INVALID';`
The session can no longer be used; the person must sign in again.

29 `static ApiError from(Object error) {`
Turns any failure (network, timeout, HTTP error body) into one ApiError with a stable code.

65 `static Map<String, List<String>> _fields(Object? raw) {`
Reads the per-field validation messages of a 400 answer.

76 `String toString() => 'ApiError($code, status: $statusCode)';`
Short text for logs (never shown to the user).

provider_retry.dart · 23 lines

9 `Duration? providerRetry(int retryCount, Object error) {`
How a failed provider tries again. A request that never reached the server (no network) is not retried: the screen says « Connexion requise » at once and offers « Réessayer », instead of a skeleton that waits through Riverpod's default ten attempts. A timeout or a server error (5xx) may be passing, so it gets two quick attempts. A refusal (4xx) is an answer, never retried.

22 `Duration _delay(int retryCount) => Duration(milliseconds: 400 * (retryCount + 1));`
Waits a little longer before each new attempt (0.4 s, 0.8 s, 1.2 s...).

apps/mobile/lib/src/core/auth/

The session (signed out, account, demo), the vault holding tokens, the controller the router follows, the token interceptor.

auth_interceptor.dart · 52 lines

8 `class AuthInterceptor extends Interceptor {`
Adds the session's bearer token to API calls. After a 401 it refreshes once and replays the call; if the refresh is refused, the vault ends the session.

14 `final Dio Function() _dio;`
The client this interceptor belongs to, used to replay a request after a refresh.

20 `Future<void> onRequest(RequestOptions options, RequestInterceptorHandler handler) async {`
Before each request: attaches the current access token, renewing it first when it has expired.

32 `Future<void> onError(DioException err, ErrorInterceptorHandler handler) async {`
On a 401: renews the token once and replays the request; a second 401 ends the session.

session.dart · 27 lines

4 `sealed class Session {`
Who is using the app on this device (ADR-009, ADR-010).

9 `final class SignedOut extends Session {`
Nobody signed in: Welcome, sign-in, sign-up and the demo entry only.

14 `final class UserSession extends Session {`
A real account. Its refresh token lives in the Keychain / Keystore.

21 `final class DemoSession extends Session {`
A demo access context (section 27): no account, read endpoints only.

session_controller.dart · 88 lines

22 `class SessionController extends AsyncNotifier<Session> {`
The current [Session], restored from secure storage at launch. The router follows it.

27 `Future<Session> build() async {`
Restores the saved session at launch, and listens for the vault ending it (revoked, expired).

38 `Future<UserSession> store(AuthTokensDto tokens) => _vault.saveUser(tokens);`
Stores the tokens of a confirmed sign-up or a sign-in, without switching the app yet, so the caller can finish onboarding calls first. Call [enter] afterwards.

41 `void enter() => state = AsyncData(_vault.session);`
Switches the app to the stored session.

44 `Future<void> startDemo(DemoContext context) async {`
« Essayer la démo »: asks the API for a short-lived demo token for this device, no sign-up.

56 `Future<void> signOut() async {`
Signs out on the server when reachable, and always on the device.

70 `Future<void> _forgetPersonalData() async {`
What the person leaves on the device goes with their session (shared phones).

76 `Future<DeviceDto> device() async => DeviceDto(`
The device descriptor sent at sign-in and sign-up confirmation (sessions are device-bound).

83 `Language language() =>`
The interface language, for server texts (OTP messages).

session_vault.dart · 184 lines

24 `class SessionVault {`
Holds the credentials of the current [Session]. - Refresh and renewal tokens are kept in secure storage. - Access tokens are kept in memory only. A new access token is obtained with a single-flight refresh: concurrent requests wait for the same refresh (refresh tokens are single use and rotated, ADR-009).

30 `final DokToolApi bareApi;`
Client without the auth interceptor, used for refresh and renewal calls.

39 `static const _expiryMargin = Duration(seconds: 30);`
A token is renewed this long before it expires.

47 `void Function()? onSessionEnded;`
Called when the server ends the session (revoked, expired, reused refresh token).

52 `Future<Session> restore() async {`
Reads which session (account or demo) was saved on this device; signed out when none.

66 `Future<UserSession> saveUser(AuthTokensDto tokens) async {`
After sign-in: stores the refresh token in the secure store and keeps the access token in memory.

77 `Future<DemoSession> saveDemo(DemoSessionDto demo) async {`
After « Essayer la démo »: stores the demo's renewal token and context.

89 `Future<void> clear() async {`
Forgets every token (sign-out).

98 `Future<String?> accessToken({bool forceRefresh = false}) {`
A valid access token for the current session, or null when signed out. Throws [ApiError.network] when a refresh was needed but the server could not be reached. The session is kept in that case, so the person stays signed in offline.

111 `Future<String?> _renew() async {`
Gets a new access token with the refresh (or demo renewal) token; ends the session if refused.

145 `Future<String?> _end() async {`
The session is over (revoked, expired): clears it and tells the app to show the welcome screen.

152 `void _remember(String token, DateTime expiresAt) {`
Keeps the access token and its expiry in memory only (never written to disk).

158 `Future<void> _clearStored() async {`
Deletes every stored token of both account and demo sessions.

168 `final bareDioProvider = Provider<Dio>((ref) {`
Dio without the auth interceptor: token refresh, demo renewal and signed storage uploads.

apps/mobile/lib/src/core/config/

Build-time configuration (`--dart-define`): API address and environment.

app_config.dart · 15 lines

7 `abstract final class AppConfig {`
Build-time configuration, passed with `--dart-define`. ```sh flutter run --dart-define=API_BASE_URL=http://10.0.2.2:3000 flutter build apk --dart-define=API_BASE_URL=https://api.example.dz --dart-define=APP_VERSION=1.0.0+1 ```

9 `static const apiBaseUrl = String.fromEnvironment('API_BASE_URL', defaultValue: 'http://localhost:3000');`
Base URL of the DokTool API (the Android emulator reaches the host at 10.0.2.2).

13 `static const String? appVersion = bool.hasEnvironment('APP_VERSION') ? String.fromEnvironment('APP_VERSION') : null;`
The release reported with analytics (e.g. `1.0.0+12`), set by the release build; none in development.

apps/mobile/lib/src/core/database/

The drift (SQLite) database: settings, offline tables, analytics queue; personal settings forgotten at sign-out.

app_database.dart · 165 lines

8 `class LocalSettings extends Table {`
Device-local key/value settings (appearance, language, offline preferences, queues).

18 `class OfflineDocuments extends Table {`
Reference PDFs kept for offline use (section 18). The file lives in the app's private storage; its SHA-256 was checked against the manifest before the row was written.

22 `TextColumn get type => text()();`
ORDONNANCE or CERTIFICATE.

31 `DateTimeColumn get updatedAt => dateTime()();`
When the server last changed the model (the « données du » date).

41 `class OfflineCalculators extends Table {`
Published calculator versions installed for offline use, after their test cases passed on the device (ADR-003). The JSON columns keep the API's own shapes.

51 `TextColumn get published => text()();`
PublishedVersionDto as JSON (definition, sources, review).

54 `TextColumn get testCases => text()();`
PublishedTestCaseDto list as JSON.

63 `class OfflineSubstances extends Table {`
Pregnancy · lactation profiles kept for offline reading (ADR-019): the API's opened profile as JSON, replaced when a newer version is published.

70 `TextColumn get profile => text()();`
SubstanceProfileDto as JSON.

80 `class AnalyticsQueue extends Table {`
Analytics events waiting to be sent (section 16): at most 5 000, kept with their original time, and only for the session that recorded them. Codes, counts and flags only — never free text.

84 `TextColumn get sessionKey => text()();`
The user or demo session the event belongs to; another session never sends it.

87 `TextColumn get type => text()();`
ClientAnalyticsEventType as sent to the API.

91 `TextColumn get properties => text()();`
The event's properties as JSON.

97 `class AppDatabase extends _$AppDatabase {`
The app's SQLite store (drift).

101 `factory AppDatabase.onDevice() => AppDatabase(driftDatabase(name: 'doktool'));`
The on-device database file.

119 `Future<String?> readSetting(String key) async {`
Reads one local setting (null when never written).

125 `Future<void> writeSetting(String key, String value) =>`
Writes one local setting, replacing the old value.

129 `Future<void> deleteSetting(String key) => (delete(localSettings)..where((s) => s.key.equals(key))).go();`
Removes one local setting.

135 `static const personalSettings = ['account.summary', 'activity.queue', 'search.recent'];`
Settings that belong to the person signed in, not to the device: their cached account (name, profession), unsent research history and recent searches. Removed when the session ends, so the next person on a shared phone neither sees them nor sends them under their own account — nor gets the previous person's account as an offline fallback after signing in.

139 `static const calculatorInputsPrefix = 'calculator.inputs.';`
The last values typed in each calculator (`calculator.inputs.<slug>`): a patient's values, never left for the next person either.

143 `Future<void> forgetPersonalSettings() async {`
Forgets [personalSettings] and the remembered calculator inputs (sign-out, session ended, account deleted).

153 `Future<void> forgetAccountData({required Set<String> keep}) => transaction(() async {`
After an account deletion: everything the account left on the device goes — cached account, recent searches, queued history and analytics, last calculator inputs, the remembered identifier — except the device preferences in [keep].

160 `final appDatabaseProvider = Provider<AppDatabase>((ref) {`
Overridden in tests with an in-memory database.

apps/mobile/lib/src/core/network/

Whether the phone is online, on Wi-Fi or on mobile data.

connectivity.dart · 29 lines

5 `enum NetworkKind { offline, wifi, mobile }`
How the phone is connected right now.

8 `NetworkKind _kindOf(List<ConnectivityResult> results) {`
Wi-Fi or Ethernet count as Wi-Fi; mobile or VPN as mobile data; anything else is offline.

21 `final networkProvider = StreamProvider<NetworkKind>((ref) async* {`
The device's link: offline is a product mode, not an error (Locks L16, L44).

28 `final isOnlineProvider = Provider<bool>((ref) => ref.watch(networkProvider).value != NetworkKind.offline);`
True unless the device is known to be offline (unknown counts as online).

apps/mobile/lib/src/core/storage/

Keychain / Keystore storage for secrets and the installation id.

secure_store.dart · 42 lines

7 `class SecureStore {`
Keychain / Keystore storage for secrets (audit Appendix D): the installation id and the session tokens. Never the database, never shared preferences.

16 `Future<String>? _installationId;`
The id being read or created: concurrent first calls share it, so a fresh install never creates two ids (a session bound to one would be refused from the other).

20 `Future<String> installationId() => _installationId ??= _loadOrCreateInstallationId();`
A random id created on first launch. Device sessions and demo sessions are bound to it; it carries nothing about the person or the hardware.

23 `Future<String> _loadOrCreateInstallationId() async {`
Reads the installation id, creating a random one the first time.

32 `Future<String?> read(String key) => _storage.read(key: key);`
Reads one value from the Keychain / Keystore.

35 `Future<void> write(String key, String value) => _storage.write(key: key, value: value);`
Writes one value to the Keychain / Keystore.

38 `Future<void> delete(String key) => _storage.delete(key: key);`
Deletes one value from the Keychain / Keystore.

apps/mobile/lib/src/features/access/

The AccessProfile from the API, reloaded on launch, resume, verification and payment.

access_controller.dart · 36 lines

```
13  class AccessController extends AsyncNotifier<AccessProfileDto?> {
```
The backend's AccessProfile for this viewer (section 27). The app renders locks from it and never re-derives access from the professional type (Lock L34). Refreshed on launch, on resume and after verification or purchase events.

```
16  Future<AccessProfileDto?> build() async {
```
Fetches GET /v1/me/access for the current session (null when signed out).

```
23  Future<void> reload() async {
```
Fetches the profile again (after verification, purchase, resume).

```
32  extension AccessProfileX on AccessProfileDto {
```
Helpers on the access profile.

```
34  FeatureDecisionDto? feature(FeatureKey key) => features.where((feature) => feature.key ==
key).firstOrNull;
```
The decision (available or locked, with reason) for one feature, if the profile lists it.

apps/mobile/lib/src/features/account/

The signed-in account (cached for offline start) and the public pickers (wilayas, specialties, institutions, universities, legal links).

account_controller.dart · 94 lines

```
18  class AccountSummary {
```
What the app keeps about the signed-in person, cached on the device so a signed-in device opens offline (ScreenWelcome « offline » state).

```
42  final ProfessionalType? professionalType;
```
Null until « Qui êtes-vous ? » is answered.

```
45  Map<String, Object?> toJson() => {
```
The summary as JSON, for the offline cache.

```
54  class AccountController extends AsyncNotifier<AccountSummary?> {
```
The signed-in account, or null for a demo or signed-out device.

```
59  Future<AccountSummary?> build() async {
```
Loads /v1/me and caches it; offline, falls back to the cached copy.

```
77  Future<void> setProfessionalType(ProfessionalType type) async {
```
« Qui êtes-vous ? »: records the professional type once (a later change is a re-verification).

```
87  Future<void> reload() async {
```
Loads the account again.

reference_data.dart · 29 lines

```
16  final legalProvider = FutureProvider<LegalDto>((ref) =>
ref.watch(apiProvider).account.usersControllerLegal());
```
Current terms version and legal page links for sign-up.

apps/mobile/lib/src/features/activity/

Favorites (server or demo memory), the history recorder and queue, the « Activité et favoris » screen, the ★ button.

activity_data.dart · 109 lines

```
19  class ContentRef {
```
A model or calculator the person opened, resolved to what the app shows.

```
36  final String? slug;
```
Calculators open by slug.

```
40  final bool isSubstance;
```
A « Grossesse · Allaitement » profile (ADR-019).

```
53  bool get opens => isCalculator || isSubstance;
```
Calculators and profiles « Ouvrir », models « Voir le modèle ».

```
57  final contentIndexProvider = FutureProvider<Map<String, ContentRef>>((ref) async {
```
Every published model and calculator the viewer sees, by id (from the cached library lists).

```
62  final substances = await ref.watch(substanceListProvider.future).catchError((Object _) =>
<SubstanceItemDto>[]);
```
Profiles are optional here: a failure to list them never hides models and calculators.

```
65  ContentRef reference(ReferenceItemDto item) => ContentRef(
```
A reference model as a content row.

```
74  ContentRef calculator(CalculatorSummaryDto item) =>
```
A calculator as a content row.

```
92  final activityHistoryProvider = FutureProvider.family<List<ActivityDto>, ActivityKind?>((ref, kind)
async {
```
The person's history, newest first (one page), optionally of one kind.

```
99  final recentContentProvider = FutureProvider<List<ContentRef>>((ref) async {
```
Home « Récents »: the last distinct models and calculators opened, still published.

activity_recorder.dart · 112 lines

```
19  class ActivityRecorder {
```
Records the person's own research history (searches, models consulted, calculators used) on the device and syncs it in batches, offline included. Nothing is recorded for a demo, and never a value typed in a calculator (Lock L42).

```
30  Future<void> search(String query) => _add(ActivityKind.search, queryText: query.trim());
```
Records a search (two characters at least) in the person's own history.

```
33  Future<void> referenceViewed(String referenceId) => _add(ActivityKind.referenceView, targetId:
referenceId);
```
Records that a reference model was opened.

```
36  Future<void> calculatorUsed(String calculatorId) => _add(ActivityKind.calculatorUse, targetId:
calculatorId);
```
Records that a calculator gave a result.

```
39  Future<void> substanceViewed(String substanceId) => _add(ActivityKind.pregnancyView, targetId:
substanceId);
```
Records that a pregnancy · lactation profile was opened.

```
42  Future<void> _add(ActivityKind kind, {String? queryText, String? targetId}) async {
```
Queues one history entry locally (real accounts only), then tries to send the queue.

```
58  Future<void> flush() => _flushing ??= _flush().whenComplete(() => _flushing = null);
```
Sends what is queued; kept for later when offline.

```
61  Future<void> _flush() async {
```
Sends the queue in batches; keeps it on network or server errors, drops a batch the API refuses.

```
99  Future<List<Map<String, Object?>>> _read(AppDatabase database) async {
```
Reads the waiting queue from the local database.

```
107 Future<void> _write(AppDatabase database, List<Map<String, Object?>> queue) =>
```
Saves the waiting queue to the local database.

activity_screen.dart · 357 lines

```
29  enum _Segment { favorites, activity }
```
The two tabs of the screen.

33 `class ActivityScreen extends ConsumerStatefulWidget {`
ScreenFavoritesRecents (Profil › Activité et favoris): favorites grouped by type, and the person's own research history grouped by day. Private: admins only ever see counts.

41 `class _ActivityScreenState extends ConsumerState<ActivityScreen> {`
Which tab is shown and which history filter is chosen.

57 `child: DtSegmentedControl<_Segment>(`
Switches between « Favoris » and « Activité ».

79 `void openContent(BuildContext context, ContentRef content) {`
Opens a favorite or history item: profile, calculator or reference model.

92 `void openReference(BuildContext context, String id, ReferenceDocumentType type) {`
Opens a reference model from a tapped row, counted as REFERENCE_OPENED (section 16).

101 `class _FavoritesList extends ConsumerWidget {`
« Favoris » tab: favorites grouped by type, with an empty state.

169 `trailing: TextButton(`
« Retirer » this favorite.

184 `class _History extends ConsumerWidget {`
« Activité » tab: the person's history grouped by day, with filters and « Effacer ».

191 `String _day(BuildContext context, AppLocalizations l10n, DateTime at) {`
Day heading: « Aujourd'hui », « Hier » or the date.

201 `Future<void> _clear(BuildContext context, WidgetRef ref) async {`
« Effacer l'historique »: asks to confirm in a sheet, then deletes the history on the server.

216 `DtButton(`
Confirms clearing the history.

223 `DtButton(`
« Annuler »: keeps the history.

246 `final days = <(String, List<Widget>)>[];`
One lifted card per day (ADR-020).

330 `child: DtButton(`
« Effacer l'historique ».

346 `class _GroupTitle extends StatelessWidget {`
A small heading above a group of rows.

favorite_button.dart · 95 lines

16 `class FavoriteButton extends ConsumerWidget {`
The ★ of a model or a calculator (viewer and calculator top bars, search sheet).

23 `final bool onHero;`
Placed on the hero top bar (ADR-020): white outline, white fill when starred.

31 `return IconButton(`
The star: adds or removes the favorite.

52 `class _FilledStar extends StatelessWidget {`
Lucide draws outlines only (an icon font ignores `fill`): the starred state paints the star's inside under the same outline, so it reads by shape and not by colour alone (section 20).

71 `class _StarPainter extends CustomPainter {`
Draws the filled star of a favorite.

78 `void paint(Canvas canvas, Size size) {`
Draws the five-pointed star path.

93 `bool shouldRepaint(_StarPainter oldDelegate) => oldDelegate.color != color;`
Redraws only when the colour changes.

favorites_controller.dart · 87 lines

14 `class FavoriteKey {`
A favorite's identity: what it points at.

29 `class Favorites {`
The favorites of this session: server entries and a quick lookup set.

33 `final List<FavoriteDto> entries;`
Server entries of a real account (with titles and access); empty for a demo.

36 `final Set<FavoriteKey> keys;`
Everything starred, for the star buttons and « favoris d'abord » ordering.

39 `bool contains(FavoriteTargetType type, String id) => keys.contains(FavoriteKey(type, id));`
True when this item is a favorite.

44 `class FavoritesController extends AsyncNotifier<Favorites> {`
Favorites: on the server for a real account; kept in memory only for a demo (session-local, nothing a demo does is written to user tables — section 27).

47 `Future<Favorites> build() async {`
Loads the favorites from the server (real account) or starts empty (demo).

58 `Future<void> toggle(FavoriteTargetType type, String id) async {`
Adds or removes a favorite at once on screen, then on the server; undone if the server refuses.

apps/mobile/lib/src/features/analytics/

Analytics events, the recorder with its offline queue, and the host that records screen views.

analytics_event.dart · 170 lines

8 `enum DemoModule { home, search, library, reference, calculator, favorites, offline, subscription, profile }`
Screens a demo can open or be stopped at (DEMO_FEATURE_VIEWED `module`, API `DEMO_MODULES`).

11 `enum PregnancySection { pregnancy, lactation }`
The two blocks of a pregnancy · lactation profile (PREGNANCY_VIEWED `section`).

14 `enum SubscriptionScreen { plans, subscription, review }`
The plan screens SUBSCRIPTION_VIEWED reports.

20 `final class AnalyticsEvent {`
One analytics event of the V1 vocabulary (section 16). Only the named constructors build one, each with exactly the properties the API accepts for its type: counts, codes and flags — never free text, a query, a calculator input or a PDF's content (Lock L42). The server adds the envelope (who, tier, platform) itself.

24 `factory AnalyticsEvent.searchPerformed({required int resultCount, required Iterable<SearchType> groups}) {`
A search returned (debounced): how many results and which kinds — not the words.

39 `factory AnalyticsEvent.referenceSearched({required ReferenceDocumentType referenceType, required int resultCount}) =>`
A search inside one library section.

46 `factory AnalyticsEvent.referenceOpened({required String id, required ReferenceDocumentType referenceType}) =>`
A reference row was tapped.

50 `factory AnalyticsEvent.referenceViewed({required String id, required int fileVersion, required bool offline}) =>`
The PDF was rendered, from the network or from the offline copy.

57 `factory AnalyticsEvent.calculatorStarted({required String id, required int version}) =>`
A calculator was opened on a published version.

62 `factory AnalyticsEvent.calculatorCompleted({`
A valid result was shown: the version, the interpretation band code and the engine — never the values entered.

74 `factory AnalyticsEvent.favorite({required String id, required FavoriteTargetType targetType, required bool added}) =>`
The star was set or cleared.

80 `factory AnalyticsEvent.pregnancyViewed({`
A pregnancy · lactation profile was read on one of its two blocks (ADR-019).

90 `factory AnalyticsEvent.subscriptionViewed(SubscriptionScreen screen) =>`
A plan screen was shown.

94 `factory AnalyticsEvent.demoStarted(DemoContext demoContext) =>`
A demo session was created (demo sessions only).

98 `factory AnalyticsEvent.demoFeatureViewed(DemoModule module, {required bool locked}) =>`
The demo opened a module, or met its limit there (demo sessions only).

110 `static const demoOnly = {ClientAnalyticsEventType.demoStarted, ClientAnalyticsEventType.demoFeatureViewed};`
Types only a demo session may send.

114 `bool get isValid {`
Whether the API will accept it: the same rules as `CLIENT_EVENTS` (analytics.policy.ts), so a value the app could not express (an unknown enum, a negative count) is dropped on the device.

analytics_host.dart · 125 lines

17 `SubscriptionScreen? subscriptionScreenFor(String location) {`
The plan screen a location shows, for SUBSCRIPTION_VIEWED.

26 `DemoModule? demoModuleFor(String location) {`
The module a location belongs to, for DEMO_FEATURE_VIEWED.

40 `String? currentLocation(GoRouter router) => router.routerDelegate.currentConfiguration.lastOrNull?.matchedLocation;`
The location on screen, pushed routes included.

43 `void recordDemoLimit(BuildContext context) {`
A demo met its limit (the lock sheet of a demo-locked item), counted against the module shown.

56 `class AnalyticsHost extends ConsumerStatefulWidget {`
Sends what the device holds — analytics and the research history — when the app comes back, goes to the background or finds the network again, and reports screen views that need no widget of their own: the plan screens and, in a demo, each module opened.

66 `class _AnalyticsHostState extends ConsumerState<AnalyticsHost> with WidgetsBindingObserver {`
Watches the route and the app lifecycle to record screen views and send events.

72 `void initState() {`
Starts listening to app lifecycle and route changes.

80 `void dispose() {`
Stops listening when the app widget goes away.

88 `void didChangeAppLifecycleState(AppLifecycleState state) {`
Sends what is waiting when the app comes back to the foreground or goes to the background.

93 `void _send() {`
Sends the waiting analytics events and history entries.

99 `void _routeChanged() {`
A new screen is shown: records a screen view (and the subscription funnel steps).

analytics_recorder.dart · 212 lines

22 `typedef AnalyticsSender = Future<void> Function(AnalyticsBatchDto batch);`
Sends one batch; the API answers with what it accepted.

27 `class AnalyticsRecorder {`
Records analytics events on the device and sends them in batches (section 16): every session, demo included, buffered offline up to 5 000 events with their original time, sent on the next connection. An event belongs to the session that recorded it — a later account never sends it.

29 `AnalyticsRecorder({`
Everything is injected so tests can run it without a phone or server.

51 `static const maxQueued = 5000;`
Device buffer (section 16, audit Appendix C).

54 `static const batchSize = 100;`
Events per upload (API `MAX_BATCH`).

57 `static const maxAge = Duration(days: 30);`
The API drops older events (`MAX_EVENT_AGE_DAYS`), so the device does too.

60 `static const sendThreshold = 20;`
Sent at once from this many waiting events; otherwise after the send delay.

69 `Future<void> track(AnalyticsEvent event) async {`
Queues one event for the current session; nothing is recorded while signed out.

98 `Future<void> flush() => _flushing ??= _flush().whenComplete(() => _flushing = null);`
Sends what the current session has waiting. Network errors keep the events for later; a batch the API refuses outright is dropped so it cannot block the queue.

101 `Future<int> pending() async {`
Events of the current session still on the device.

107 `void dispose() => _timer?.cancel();`
Stops the pending send timer.

110 `Future<void> _flush() async {`
Sends the queued events of the current session in batches; keeps them when offline.

142 `AnalyticsBatchDto _batchOf(List<QueuedAnalyticsEvent> rows) {`
One batch as the API expects it: platform, app version, data version and the events.

162 `Future<void> _trim() async {`
Keeps the newest [maxQueued] events.

177 `Future<int> _waiting(String key) async {`
How many events of this session are still waiting.

187 `static String? _keyOf(Session? session) => switch (session) {`
Which session the events belong to (an account, a demo, or none when signed out).

208 `extension AnalyticsTracking on AnalyticsRecorder {`
Records an event without waiting; analytics never interrupt what the person is doing.

210 `void record(AnalyticsEvent event) => unawaited(track(event).catchError((Object _) {}));`
Records an event without waiting and without ever failing the screen that called it.

apps/mobile/lib/src/features/calculators/

The calculator runtime (online or offline, self-tested) and the calculator screen.

calculator_runtime.dart · 110 lines

18 `enum RuntimeState { ready, locked, unsupported, selfTestFailed }`
Whether the calculator can be used: ready, locked, needing a newer app, or failed its self-test.

22 `class CalculatorRuntime {`
A calculator ready for the device engine: the published version, parsed and self-tested. Inputs are computed on the device only; they never leave it (Lock L42).

30 `final bool offline;`
Opened from the installed offline version (« hors ligne »).

34 `List<ReferenceCase> referenceCasesOf(List<PublishedTestCaseDto> testCases) => [`
The API's test cases as the device self-test reads them.

50 `CalculatorRuntime runtimeOf(CalculatorDetailDto detail, {bool offline = false}) {`
Parses and self-tests a published version.

73 `final calculatorRuntimeProvider = FutureProvider.autoDispose.family<CalculatorRuntime, String>((ref, slug) async {`
Online: the current published version. Without a connection: the version installed by the offline sync, computed on the device (section 18).

85 `class LastInputs {`
The last inputs of a calculator, remembered on this device only (ScreenCalculatorMobile).

91 `static String _key(String slug) => 'calculator.inputs.$slug';`
Local setting key of a calculator's last inputs.

94 `Future<Map<String, InputValue?>> read(String slug) async {`
The last values typed in this calculator on this device (empty if none).

100 `Future<void> write(String slug, Map<String, InputValue?> inputs) => _database.writeSetting(`
Saves the values typed in this calculator, on this device only.

calculator_screen.dart · 1022 lines

39 `class CalculatorScreen extends ConsumerWidget {`
ScreenCalculatorMobile: select tool → enter validated inputs → calculate → review result + source → done (section 28). Computed on the device with the published, self-tested version.

68 `DtButton(`
« Réessayer ».

103 `class _Message extends StatelessWidget {`
A full-screen message instead of the form (locked, unsupported, self-test failed).

126 `class _Field {`
What the person typed in one field, before it is committed to the engine.

128 `_Field(this.input, String languageCode)`
One input on screen: its text controller, focus, chosen unit (the default of the UI language) and choice.

138 `bool unreadable = false;`
Text that could not be read as a number (shown as « Nombre attendu »).

142 `class _Calculator extends ConsumerStatefulWidget {`
The calculator form with its result, for a ready runtime.

152 `class _CalculatorState extends ConsumerState<_Calculator> {`
Fields, committed values, the latest evaluation and the idle timer.

157 `final Map<String, InputValue?> _committed = {};`
Values committed to the engine (field commit: next/done, focus change, or 400 ms idle).

163 `bool _useRecorded = false;`
A use is recorded once per opening, at the first result (never the values).

166 `bool _completionRecorded = false;`
CALCULATOR_COMPLETED is counted at the first result, and again after « Réinitialiser ».

174 `void initState() {`
Records « calculator started » for analytics (no inputs are ever sent).

183 `void didChangeDependencies() {`
Creates one field per input (once), then restores the last values typed.

198 `Future<void> _restore() async {`
Puts back the values last typed in this calculator on this device.

221 `void dispose() {`
Frees the timer, text controllers and focus nodes.

```
231 String _formatInput(num value) {
```
A number as typed in the fields (decimal comma in French).

```
238 void _typed(_Field field) {
```
A key was typed: commits the field after 400 ms without typing.

```
244 void _commit(_Field field) {
```
Reads the field's value (number in its unit, or the choice), stores it and re-evaluates.

```
271 void _evaluate() {
```
Runs the device engine on the committed values and saves them locally.

```
298 void _reset() {
```
« Réinitialiser »: clears every field, the result and the saved values.

```
319 String _number(num value, {int? decimals}) {
```
A number in the UI language, with a fixed number of decimals when given.

```
332 String _range(AppLocalizations l10n, Range? range, String unit) {
```
A range as text: « entre 1 et 10 kg », « ≥ 1 kg » or « ≤ 10 kg ».

```
342 String _label(CalculatorInput input) => input.label.of(Localizations.localeOf(context).languageCode);
```
An input's label in the UI language.

```
345 String? _fieldError(AppLocalizations l10n, _Field field) {
```
The error under a field, only once the person has committed it (never per keystroke).

```
363 String? _fieldNote(AppLocalizations l10n, _Field field) {
```
The note under a field: « vérifiez la valeur » outside the plausible range, or the unit conversion.

```
415 IconButton(
```
« Source »: version, sources, reviewers and limitations.

```
546 String _describeIssue(AppLocalizations l10n, Issue issue, String language) {
```
Why no result is shown, as a sentence (outside the validated range, cross-field rule, missing input…).

```
563 String _citation(SourceDto source) => [
```
A source as one line: authors, title, publisher, edition, year, where the formula is.

```
571 void _showSourceSheet(BuildContext context) {
```
« Source »: a sheet with the version, its sources, reviewers and limitations.

```
644 class _InputField extends StatelessWidget {
```
One input: a number with its unit picker, a list of answers, or a yes/no switch, with help and error.

```
690 control = TextField(
```
A number field: commits on « suivant », on leaving it, or after a pause.

```
720 child: DropdownButton<String>(
```
The unit picker of a number field (converted exactly to the canonical unit).

```
759 control = LayoutBuilder(
```
Short answers (up to four) stay a segmented control; long ones, or more than four, become a list of full rows so no answer is ever cut (ADR-026).

```
823 class _ResultBlock extends StatelessWidget {
```
The result: value, unit, interpretation band, formula with the values, and the version.

```
928 class _Detail extends StatelessWidget {
```
A titled block under the result.

```
959 class _ResultBar extends StatelessWidget {
```
Sticky glass result bar: value + unit, Copier, Réinitialiser.

```
996 DtButton(
```
« Copier le résultat » (value, unit and band; never the inputs).

```
1006 DtButton(
```
« Réinitialiser ».

apps/mobile/lib/src/features/calculators/engine/

The Dart engine: definition parsing, evaluation, formula text, self-test of published cases.

definition.dart · 395 lines

6 `class DefinitionFormatException extends FormatException {`
A published definition the device cannot read (the calculator is then not offered).

13 `Json _map(Object? value, String path) {`
The value as a JSON object, or a format error naming where in the definition.

19 `List<Object?> _list(Object? value, String path) {`
The value as a list, or a format error.

25 `String _string(Object? value, String path) {`
The value as a string, or a format error.

31 `double _number(Object? value, String path) {`
The value as a finite number, or a format error.

37 `double? _optionalNumber(Object? value, String path) => value == null ? null : _number(value, path);`
The value as a number, or null when absent.

40 `int _int(Object? value, String path, {required int fallback}) {`
The value as a whole number (fallback when absent), or a format error.

48 `class LocalizedText {`
A text in French and English.

60 `String of(String languageCode) => languageCode == 'en' ? en : fr;`
The text in the given language (English for "en", French otherwise).

65 `class Range {`
A range with an optional minimum and maximum (inclusive).

69 `static Range? parse(Object? raw, String path) {`
Reads a range from the definition (null when absent).

79 `bool contains(double value) => (min == null || value >= min!) && (max == null || value <= max!);`
True when the value is inside the range.

92 `static Expr parse(Object? raw, String path) {`
Reads a formula node and its children; any operator outside the closed set is refused.

188 `class UnitConversion {`
How to convert a unit to the input's canonical unit, and where the factor comes from.

198 `class Choice {`
One answer of a choice input, with its points if the score uses them.

207 `enum InputKind { number, integer, choice, boolean }`
What an input asks for.

211 `class CalculatorInput {`
One input of a calculator: kind, label, units, the three ranges (hard, validated, plausible), help.

300 `List<String> get unitCodes => [canonicalUnit, for (final unit in units) unit.code];`
Units the field offers: the canonical unit first, then the declared conversions.

304 `enum RuleOutcome { invalid, outOfValidatedRange }`
What a cross-field rule does when it matches.

308 `class CrossFieldRule {`
A rule over several inputs (e.g. diastolic < systolic) that blocks the result.

319 `class InterpretationBand {`
A band of the result (from / to) with its severity and label.

331 `class CalculatorDefinition {`
A whole calculator version: inputs, rules, formula, result unit and decimals, bands, limitations.

evaluate.dart · 374 lines

13 `enum Outcome { result, invalid, outOfValidatedRange }`
What an evaluation gives: a result, invalid inputs, or values outside the validated range.

17 `class InputValue {`
A value typed by the person; [unit] defaults to the input's canonical unit.

21 `final Object? value;`
num, String (choice) or bool; null means « not given ».

27 `class Issue {`
Why there is no result, with the input, rule or range concerned.

32 `final String code;`
UNKNOWN_INPUT, REQUIRED, WRONG_TYPE, NOT_AN_INTEGER, UNKNOWN_UNIT, UNKNOWN_CHOICE, HARD_RANGE, CROSS_FIELD, VALIDATED_RANGE, MISSING_INPUT or NON_FINITE_RESULT.

37 `final Range? range;`
The range that was not met, in [rangeUnit] (the input's canonical unit).

42 `String get key => [code, ?(ruleId ?? input)].join(':');`
`CODE` or `CODE:subject`, as the vectors compare them.

47 `class Conversion {`
A unit conversion applied to an input, shown to the person.

65 `class PlausibilityNote {`
An input outside its plausible range: the result is shown with « vérifiez la valeur ».

75 `class Evaluation {`
The engine's answer: a result (value, band, notes, conversions) or the issues.

104 `class _MissingInput implements Exception {`
A formula reached an optional input that was not given.

110 `Evaluation evaluate(CalculatorDefinition definition, Map<String, InputValue?> raw) {`
Evaluates [definition] on [raw] inputs. Pure and deterministic.

221 `Object _accept(CalculatorInput input, InputValue given, List<Conversion> conversions) {`
The accepted canonical value (double, String or bool), or the [Issue] refusing it.

256 `Object _run(Expr node, Map<String, Object> values, CalculatorDefinition definition) {`
Computes one formula node (a number or a boolean) from the converted input values.

258 `double numOf(Expr expr) => _run(expr, values, definition) as double;`
Helpers: a child node's value as a number or a boolean.

337 `double jsPow(double base, double exponent) {`
JavaScript `Math.pow` semantics (IEEE pow differs for a NaN exponent and |base| = 1 with an infinite exponent, where JavaScript answers NaN).

344 `double log10(double value) {`
Base-10 logarithm, exact on powers of ten like JavaScript's `Math.log10`.

354 `double roundHalfAwayFromZero(double value, int digits) {`
Rounds to [digits] decimals, halves away from zero, on the shortest decimal representation (1.005 → 1.01), as the TypeScript engine does.

362 `double _shiftDecimal(double value, int places) {`
Multiplies by a power of ten exactly (through the decimal text), for correct rounding.

370 `String _shortest(double value) {`
The shortest round-trip decimal text, like JavaScript's `String(number)` for finite values.

formula_text.dart · 58 lines

5 `String formulaText(Expr node, {required Map<String, String> values, required String Function(String key) labelOf}) {`
The formula as mono text with the entered values substituted (ScreenCalculatorMobile), e.g. `(70 × (140 − 60)) ÷ (72 × 1.1)`. Inputs without a value keep their label.

7 `String text(Expr expr) => formulaText(expr, values: values, labelOf: labelOf);`
Writes a child node.

54 `String _number(double value) {`
A number without a useless ".0".

self_test.dart · 46 lines

5 `class ReferenceCase {`
One published reference case, as the API sends it.

16 `Map<String, InputValue?> inputsFromJson(Object? raw) {`
Reads the `{ key: { value, unit } }` map of a test case.

28 `bool passesReferenceCases(CalculatorDefinition definition, List<ReferenceCase> cases) {`
Runs the published version's reference cases on this device before first use (section 28). A version that fails any case is not used: the device engine would not be trustworthy for it.

apps/mobile/lib/src/features/home/

The Home screen.

home_screen.dart · 478 lines

36 `class HomeScreen extends ConsumerWidget {`
ScreenHomeMobile / ScreenHomeStudent: identity line, the glass search field first, the three V1 resources on one strip, then « Commencer ». What can be opened comes from the AccessProfile, never from the professional type (Lock L34); the type only orders the quick actions.

56 `if (studentTier) quick.insert(0, quick.removeAt(2));`
Students lead with the calculators, the content open to them (ADR-006 Q2).

61 `body: AnnotatedRegion<SystemUiOverlayStyle>(`
Light status bar icons over the deep top of the hero (ADR-020).

96 `_GlassSearchField(label: l10n.homeSearchHint, onTap: () => context.push(Routes.search)),`
The search bar: opens the search screen.

99 `_GlassSearchField(`
Fastest way to a drug in pregnancy or lactation: straight into typing (ADR-025).

139 `static String _roleName(AppLocalizations l10n, ProfessionalType type) => switch (type) {`
The profession's name in the UI language.

146 `static String _accessName(AppLocalizations l10n, AccessProfileDto access) => switch (access.tier) {`
The access tier's name (Pro, sample, student, demo, restricted).

156 `class _IdentityLine extends ConsumerWidget {`
The top line: the person's name, role and access, with the notifications bell.

201 `DtGlassIconButton(`
The bell: opens the notifications.

212 `class _GlassSearchField extends StatelessWidget {`
The search bar of Home; tapping it opens the search screen.

267 `class _QuickActions extends StatelessWidget {`
Shortcuts to each library section.

283 `child: Semantics(`
A shortcut tile: opens that library section.

325 `class _VerificationLine extends ConsumerWidget {`
One neutral line when verification needs attention (never red on Home).

351 `action: InkWell(`
« Voir la vérification ».

360 `class _DemoBar extends StatelessWidget {`
« Mode démo · contenu d'exemple réel · Voir les offres » (ScreenDemo): thin, neutral.

376 `TextButton(`
« Voir les offres ».

395 `class _RecentsAndFavorites extends ConsumerWidget {`
Récents (models and calculators opened) and Favoris; on first use, a short « Commencer » list.

435 `ListTile(`
A row of the « Raccourcis » list.

apps/mobile/lib/src/features/library/

Library sections, their lists (online or offline) and the library screen.

library_data.dart · 77 lines

13 `enum LibrarySection {`
The library sections: the three V1 types and « Grossesse · Allaitement » (ADR-019).

24 `static LibrarySection fromSlug(String? slug) =>`
The section of a URL segment (ordonnances when unknown).

35 `enum ListSource { live, offline }`
Where a list came from: the server, or the offline store when there is no connection.

39 `final referenceListProvider = FutureProvider.family<List<ReferenceItemDto>, ReferenceDocumentType> ((ref, type) async {`
A–Z reference models of one type, each with the server's access decision; without a connection, the models downloaded for offline use.

52 `final calculatorListProvider = FutureProvider<List<CalculatorSummaryDto>>((ref) async {`
Published calculators; without a connection, the versions installed for offline use.

66 `final substanceListProvider = FutureProvider<List<SubstanceItemDto>>((ref) async {`
« Grossesse · Allaitement »: substances with a published profile, each with the server's access decision; without a connection, the profiles kept for offline reading (ADR-019).

library_screen.dart · 330 lines

33 `class LibraryScreen extends ConsumerStatefulWidget {`
ScreenReferenceLibrary: segmented Ordonnances · Certificats · Calculateurs, a search field scoped to the section, an A–Z list. A row has one action; a locked row keeps its place with the lock and its reason word. No « new », editor or floating button (Lock L31); no categories (Q8).

39 `final bool focusSearch;`
Opened from a search shortcut: the keyboard comes up at once.

46 `class _LibraryScreenState extends ConsumerState<LibraryScreen> {`
The filter text of the section, and the timer that reports a library search once typing pauses.

50 `Timer? _report;`
A library search counts once the typing has paused, not per keystroke.

54 `void didUpdateWidget(LibraryScreen oldWidget) {`
Another section was chosen: clears the filter.

64 `void dispose() {`
Frees the timer and the text controller.

71 `void _changed(String value) {`
The filter changed: redraws, and reports the search after a pause (two characters at least).

110 `child: DtPillTabs<LibrarySection>(`
Section tabs: ordonnances, certificates, calculators, pregnancy.

133 `child: TextField(`
The filter field of the section.

166 `class _ReferenceList extends ConsumerWidget {`
The ordonnances or certificates of the section, filtered, with locks and loading / error states.

209 `class _CalculatorList extends ConsumerWidget {`
The calculators, filtered by name.

251 `class _ListState extends StatelessWidget {`
Loading, error (« Réessayer ») or empty state of a list, with pull-to-refresh.

285 `DtButton(label: l10n.commonRetry, variant: DtButtonVariant.secondary, onPressed: onRetry),`
« Réessayer ».

apps/mobile/lib/src/features/notifications/

The notifications list and its actions.

notifications_screen.dart · 202 lines

23 `class NotificationsController extends AsyncNotifier<NotificationPageDto?> {`
The person's notifications (newest first). Demo sessions have none.

26 `Future<NotificationPageDto?> build() async {`
Loads the notifications of a real account (none for a demo).

33 `Future<void> markAllRead() async {`
« Tout marquer comme lu ».

39 `Future<void> markRead(String id) async {`
Marks one notification read.

50 `String? notificationText(AppLocalizations l10n, NotificationDto notification) {`
Renders a notification from its catalogue key (section 19); unknown keys are skipped.

85 `String _date(String locale, String? iso) {`
An ISO date parameter in the person's language (« 12 octobre 2026 »).

92 `class NotificationsScreen extends ConsumerStatefulWidget {`
ScreenNotifications: « Nouveau » then « Plus tôt », with « Tout lire ». The list is fetched again each time the screen opens (the bell's count may be from earlier) and on pull.

100 `class _NotificationsScreenState extends ConsumerState<NotificationsScreen> {`
Reloads the list each time the screen opens.

103 `void initState() {`
Fetches the list again right after the screen opens.

121 `TextButton(`
« Tout marquer comme lu ».

140 `DtButton(`
« Réessayer ».

183 `class _NotificationTile extends ConsumerWidget {`
One notification: its text, date, unread dot; tapping marks it read and opens its link.

194 `return ListTile(`
A notification: marks it read and opens its link.

apps/mobile/lib/src/features/offline/

Offline: the store (drift + files), the sync controller, the status bar and the settings screen.

offline_bar.dart · 157 lines

17 `enum OfflineBarKind { offline, syncing, failed, outdated }`
The bars the shell can show: offline, syncing, sync failed, data outdated.

20 `final offlineBarKindProvider = Provider<OfflineBarKind?>((ref) {`
Which bar shows, if any (the shell reserves its place under the status bar).

34 `class OfflineBar extends ConsumerWidget {`
OfflineBar (section 18): neutral when offline, progress while syncing, warning when the data is older than 30 days, critical with « Réessayer » when a sync failed. Nothing when all is well. Tapping it opens « Données et hors ligne ».

75 `TextButton(`
« Réessayer » the sync.

91 `child: InkWell(`
The bar: opens « Données et hors ligne ».

113 `class OfflineSyncHost extends ConsumerStatefulWidget {`
Runs the offline sync at launch, on resume after 6 hours and when the connection returns.

123 `class _OfflineSyncHostState extends ConsumerState<OfflineSyncHost> with WidgetsBindingObserver {`
Syncs again when the app returns to the foreground and the data is stale.

126 `void initState() {`
Starts listening to the app lifecycle.

133 `void dispose() {`
Stops listening to the app lifecycle.

140 `void didChangeAppLifecycleState(AppLifecycleState state) {`
Back in the foreground: syncs if the data is stale.

offline_settings_screen.dart · 151 lines

23 `class OfflineSettingsScreen extends ConsumerWidget {`
ScreenOfflinePacks « Données et hors ligne »: last sync, data version, storage, what downloads (all models or favorites only), mobile data, update now and removal. V1 content only.

92 `ExcludeSemantics(`
Read once, as the label of the choice below.

97 `DtSegmentedControl<OfflineScope>(`
What to keep offline: everything or only favorites.

108 `SwitchListTile(`
Allows syncing over mobile data.

118 `DtButton(`
« Mettre à jour maintenant ».

126 `DtButton(`
« Supprimer les données hors ligne », after a confirmation.

136 `TextButton(onPressed: () => Navigator.of(dialog).pop(false), child: Text(l10n.commonCancel)),`
« Annuler ».

138 `TextButton(onPressed: () => Navigator.of(dialog).pop(true), child: Text(l10n.offlineRemove)),`
Confirms removing the offline data.

offline_store.dart · 237 lines

27 `PeriodSummariesDto? periodSummaries(ProfileContentDto? content) {`
The per-period summary the search endpoint gives, rebuilt from a stored profile.

48 `class OfflineStore {`
Read side of the offline store: what the device can open without a connection (section 18). Everything here was entitled at the last sync; a lost entitlement purges it at the next online launch.

50 `OfflineStore(this._database, {Future<Directory> Function()? directory})`
The folder is injectable for tests; on a phone it is the app's private support folder.

57 `Future<Directory> filesDirectory() async {`
The private folder where offline PDFs are written (created if missing).

65 `Future<List<OfflineDocument>> documents({ReferenceDocumentType? type}) {`
The reference models kept offline, optionally of one type.

72 `Future<OfflineDocument?> document(String id) =>`
One reference model kept offline, if any.

76 `Future<Uint8List?> bytesOf(OfflineDocument document) async {`
The PDF bytes, checked again against their SHA-256 (a damaged file is never shown).

84 `Future<List<OfflineCalculator>> calculators() => _database.select(_database.offlineCalculators).get();`
The calculator versions kept offline.

87 `Future<OfflineCalculator?> calculatorBySlug(String slug) =>`
One calculator kept offline, by its slug.

91 `Future<List<OfflineSubstance>> substances() => _database.select(_database.offlineSubstances).get();`
The pregnancy · lactation profiles kept offline.

94 `Future<SubstanceProfileDto?> substance(String id) async {`
A profile kept for offline reading, as the API gave it.

102 `Future<List<SubstanceItemDto>> substanceItems() async => [`
« Grossesse » rows built from the store.

117 `Future<List<SubstanceHitDto>> substanceHits() async => [`
Profiles kept offline as search results, each with its periods (ADR-025 offline fallback).

133 `Future<void> saveSubstance(SubstanceProfileDto profile) => _database`
Keeps a profile for offline reading (replacing an older copy).

147 `Future<void> deleteSubstance(String id) =>`
Removes a profile from offline storage.

151 `Future<List<ReferenceItemDto>> referenceItems(ReferenceDocumentType type) async => [`
Library rows built from the store, with the access they had at sync time.

167 `Future<List<CalculatorSummaryDto>> calculatorItems() async => [`
Calculator rows built from the store, for the library when offline.

183 `CalculatorDetailDto detailOf(OfflineCalculator calculator) => CalculatorDetailDto(`
The installed version as the calculator screen expects it (computed on the device).

201 `Future<void> purge({bool calculators = true}) async {`
Removes every protected copy (lost entitlement, sign-out, or « Supprimer les modèles »).

214 `Future<int> storageBytes() async => (await documents()).fold<int>(0, (total, document) => total + document.sizeBytes);`
Space used by the offline PDFs, shown in « Données et hors ligne ».

217 `Future<void> saveDocument(OfflineDocumentsCompanion row) =>`
Records a downloaded PDF.

221 `Future<void> deleteDocument(OfflineDocument document) async {`
Deletes a PDF file and its record.

228 `Future<void> saveCalculator(OfflineCalculatorsCompanion row) =>`
Records a calculator version for offline use.

232 `Future<void> deleteCalculator(String id) =>`
Removes a calculator version from offline storage.

offline_sync.dart · 331 lines

30 `enum OfflineScope { all, favorites }`
What to keep offline: everything the person may open, or only their favorites.

33 `enum SyncPhase { idle, syncing, failed }`
Where the sync is.

37 `class OfflineStatus {`
What the offline indicators show (OfflineBar, DataVersionChip, Données et hors ligne).

56 `final bool enabled;`
Offline is open for this account (never for a demo).

66 `final int substances;`
Pregnancy · lactation profiles kept for offline reading.

70 `final int pendingDocuments;`
Models waiting for Wi-Fi (or for « Tous les modèles »): « Mise à jour disponible ».

75 `final bool lostEntitlement;`
Protected copies were removed because the account no longer opens them.

78 `bool get outdated => syncedAt != null && DateTime.now().difference(syncedAt!) > const Duration(days: 30);`
Section 18: data older than 30 days shows a warning.

81 `OfflineStatus copyWith({`
A copy with some fields changed (clear... flags remove a value).

108 `class OfflineSyncController extends AsyncNotifier<OfflineStatus> {`
Keeps the device's offline store in line with the manifest of the account (section 18): new or changed models are downloaded and checksum-verified, calculator versions are installed after their test cases pass on the device, and anything no longer listed is removed.

117 `static const staleAfter = Duration(hours: 6);`
A sync runs on launch, then again on resume after this long.

125 `Future<OfflineStatus> build() async {`
Real accounts only: loads the status and starts a sync; a demo or sign-out forgets the offline data.

135 `Future<void> _forget() async {`
Signed out or demo: the store and its markers are removed.

143 `Future<OfflineStatus> _load({required bool enabled}) async {`
Reads the status from local settings: data version, last sync, scope, mobile data, space used.

163 `Future<void> sync({bool ifStale = false}) {`
Syncs now unless one is already running; [ifStale] skips a recent sync (resume).

173 `Future<void> _sync() async {`
Downloads the offline manifest, then each missing or changed PDF, calculator and profile; removes the rest.

219 `Future<int> _syncDocuments(List<ManifestReferenceDto> items, bool Function(ManifestReferenceDto) wanted) async {`
Returns how many wanted models are waiting (Wi-Fi only, favorites only).

264 `Future<void> _syncCalculators(List<ManifestCalculatorDto> items) async {`
Installs each new calculator version only if its reference cases pass on this device.

297 `Future<void> setScope(OfflineScope scope) async {`
« Tout » or « Favoris seulement »: saves the choice and syncs again.

304 `Future<void> _syncSubstances(List<SubstanceProfileDto> items) async {`
Keeps the listed profiles (they are small: always all of them), replaced on a new version.

316 `Future<void> setMobileData(bool allowed) async {`
Allows or forbids syncing over mobile data.

323 `Future<void> removeDocuments() async {`
« Supprimer les modèles téléchargés »: calculators stay installed (they are tiny).

apps/mobile/lib/src/features/onboarding/

Welcome, demo start, role choice, sign-up, code, sign-in, password reset, and their helpers (draft, validators, code boxes, resend timer).

auth_memory.dart · 18 lines

9 `final lastIdentifierProvider = FutureProvider<String?>(`
The identifier last used to sign in on this device (ScreenWelcome « returning device »: sign-in becomes the primary action and the field is prefilled). No password is ever stored.

14 `Future<void> rememberIdentifier(WidgetRef ref, String identifier) async {`
Remembers the e-mail or phone last used, to pre-fill sign-in.

demo_start_screen.dart · 110 lines

21 `class DemoStartScreen extends ConsumerStatefulWidget {`
ScreenDemo step 2 « Contexte professionnel »: the demo is an access context, never a professional type. No sign-up, no personal data, no time limit (ADR-006 Q5).

29 `class _DemoStartScreenState extends ConsumerState<DemoStartScreen> {`
The demo context chosen (doctor, resident, student) and the start progress.

35 `Future<void> _start() async {`
« Commencer la démo »: starts a demo session for the chosen context; errors are shown inline.

67 `DtButton(label: l10n.demoStart, loading: _starting, onPressed: _start),`
« Commencer la démo ».

71 `ExcludeSemantics(`
Read once, as the label of the choice below.

76 `DtSegmentedControl<DemoContext>(`
The demo context: doctor, resident or student.

onboarding_draft.dart · 63 lines

8 `enum OtpPurpose { signUp, passwordReset }`
Why a code was sent: confirming a sign-up or resetting a password.

13 `class OnboardingDraft {`
What onboarding remembers between its screens, in memory only (never persisted): the role chosen before the account exists, and the pending OTP challenge.

17 `final ProfessionalType? role;`
« Qui êtes-vous ? » answered before sign-up; recorded once the account is confirmed.

23 `final String? afterSignIn;`
Where the router sends a newly signed-in person (the verification intro after sign-up).

26 `OnboardingDraft copyWith({`
A copy with some fields changed.

42 `class OnboardingDraftController extends Notifier<OnboardingDraft> {`
Holds the draft while the person moves between onboarding screens.

47 `void chooseRole(ProfessionalType role) => state = state.copyWith(role: role);`
Remembers the role picked before the account exists.

50 `void awaitCode(OtpChallengeDto challenge, OtpPurpose purpose, String identifier) =>`
A code was sent: remembers the challenge, its purpose and where it went.

54 `void signedUp(String destination) => state = OnboardingDraft(role: state.role, afterSignIn: destination);`
Sign-up confirmed: keeps the role and where to go after signing in.

57 `void reset() => state = const OnboardingDraft();`
Forgets everything once onboarding is over or abandoned.

otp_code_input.dart · 121 lines

10 `class OtpCodeInput extends StatefulWidget {`
Six boxes over one hidden field, so paste and SMS / e-mail code autofill work (ScreenAccountCreate OTP state). [onCompleted] fires when the sixth digit is entered.

31 `class _OtpCodeInputState extends State<OtpCodeInput> {`
The six boxes redraw when the hidden field gains or loses focus.

36 `void initState() {`
Redraws the boxes on focus changes.

43 `void dispose() {`
Frees the focus node.

otp_screen.dart · 173 lines

25 `class OtpScreen extends ConsumerStatefulWidget {`
Sign-up confirmation (step 3 of 3): 6 boxes, resend after 60 s, 3 tries then a 10-minute lock (ADR-009). On success the role chosen earlier is recorded and verification starts.

33 `class _OtpScreenState extends ConsumerState<OtpScreen> {`
The code typed, progress flags and the last error.

42 `void dispose() {`
Frees the text controller.

48 `Future<void> _verify(String code) async {`
Checks the six-digit code; a confirmed sign-up signs in and continues onboarding.

89 `Future<void> _resend() async {`
« Renvoyer le code »: asks for a new code (the timer prevents asking too often).

136 `DtButton(`
« Vérifier »: possible once the six digits are typed (not while locked).

161 `String otpErrorText(BuildContext context, ApiError error) {`
Messages of the OTP errors, shared with the password reset.

password_field.dart · 75 lines

10 `class PasswordField extends StatefulWidget {`
Password input with a show/hide control and, for new passwords, the strength line.

37 `class _PasswordFieldState extends State<PasswordField> {`
Whether the password is shown in clear (eye button).

67 `trailing: IconButton(`
The eye: shows or hides the password.

password_reset_screens.dart · 216 lines

27 `class ResetRequestScreen extends ConsumerStatefulWidget {`
« Mot de passe oublié » step 1: the API always answers with a challenge, whether or not the account exists, so this screen never reveals who has an account.

35 `class _ResetRequestScreenState extends ConsumerState<ResetRequestScreen> {`
The e-mail or phone typed, progress and error.

42 `void dispose() {`
Frees the text controller.

48 `Future<void> _send() async {`
« Envoyer le code »: asks for a reset code; the answer is the same whether or not the account exists.

83 `DtButton(label: l10n.resetSend, loading: _sending, onPressed: valid ? _send : null),`
« Envoyer le code »: needs a valid e-mail or phone.

100 `class ResetConfirmScreen extends ConsumerStatefulWidget {`
Step 2: the code received and the new password, sent together. Every session of the account is revoked by the API afterwards.

108 `class _ResetConfirmScreenState extends ConsumerState<ResetConfirmScreen> {`
The code and new password typed, progress and error.

118 `void dispose() {`
Frees the text controllers.

128 `Future<void> _confirm() async {`
Sets the new password with the code; every session of the account ends.

157 `Future<void> _resend() async {`
« Renvoyer le code ».

187 `actions: [DtButton(label: l10n.signInSubmit, onPressed: () => context.go(Routes.signIn))],`
« Se connecter » with the new password.

198 `DtButton(label: l10n.resetConfirm, loading: _saving, onPressed: _valid ? _confirm : null),`
« Réinitialiser »: needs the code and a long enough password.

resend_timer.dart · 54 lines

9 `class ResendCodeButton extends StatefulWidget {`
« Renvoyer dans 42 s », then « Renvoyer le code » once the server allows it (ADR-009: 60 s).

21 `class _ResendCodeButtonState extends State<ResendCodeButton> {`
Redraws every second so the countdown moves.

26 `void initState() {`
Starts the one-second tick.

35 `void dispose() {`
Stops the tick.

46 `return DtButton(`
« Renvoyer le code », disabled with a countdown until it is allowed.

role_select_screen.dart · 189 lines

24 `class RoleSelectScreen extends ConsumerStatefulWidget {`
ScreenRoleSelect « Qui êtes-vous ? »: sets `professionalType` and nothing else. Before sign-up the choice waits in memory; for a signed-in account without a profile it is recorded at once.

32 `class _RoleSelectScreenState extends ConsumerState<RoleSelectScreen> {`
The role selected, progress and error.

39 `void initState() {`
Pre-selects a role chosen earlier in onboarding.

47 `Future<void> _continue() async {`
« Continuer »: before an account, remembers the role and opens sign-up; signed in, saves it on the account.

93 `DtButton(label: l10n.commonContinue, loading: _saving, onPressed: _selected == null ? null : _continue),`
« Continuer »: possible once a role is chosen.

109 `_RoleRow(`
A role row: selects it.

126 `class _RoleRow extends StatelessWidget {`
One selectable role: title, line, badge and check mark.

sign_in_screen.dart · 160 lines

23 `class SignInScreen extends ConsumerStatefulWidget {`
« Se connecter »: e-mail or phone and password; 5 wrong passwords lock the account 15 minutes.

31 `class _SignInScreenState extends ConsumerState<SignInScreen> {`
The identifier and password typed, progress and error.

39 `void initState() {`
Pre-fills the e-mail or phone last used on this device.

47 `void dispose() {`
Frees the text controllers.

56 `Future<void> _submit() async {`
« Se connecter »: signs in; an unconfirmed sign-up goes to the code screen.

87 `String _errorText(AppLocalizations l10n, ApiError error) => switch (error.code) {`
The message for each sign-in refusal (wrong password, locked until…, not confirmed, suspended).

103 `DtButton(label: l10n.signInSubmit, loading: _submitting, onPressed: _valid ? _submit : null),`
« Se connecter »: needs an identifier and a password.

105 `DtButton(`
« Pas encore de compte ? »: role choice, then sign-up.

139 `DtButton(`
« Accès perdu ? »: the access help screen.

147 `DtButton(`
« Mot de passe oublié ? ».

sign_up_screen.dart · 320 lines

31 `class SignUpScreen extends ConsumerStatefulWidget {`
ScreenAccountCreate: name, e-mail or phone, password with strength, wilaya, consent. Fields validate on blur; the primary action stays disabled until everything is valid.

39 `enum _Field { firstName, lastName, identifier, password }`
The fields of the form.

42 `class _SignUpScreenState extends ConsumerState<SignUpScreen> {`
Text controllers, focus nodes, fields already left (errors show only then), terms and progress.

53 `void initState() {`
A field shows its error only after the person has left it once.

66 `void dispose() {`
Frees the text controllers and focus nodes.

77 `String _text(_Field field) => _controllers[field]!.text.trim();`
The trimmed text of a field.

80 `String? _errorOf(_Field field, AppLocalizations l10n) {`
The error under a field (only once left): names required, valid e-mail or phone, strong enough password.

105 `Future<void> _submit() async {`
« Créer mon compte »: sends the form with the terms version; the API answers with a code to confirm.

145 `String _errorText(AppLocalizations l10n, ApiError error) {`
The message for each sign-up refusal (account exists, terms changed, code not sendable, too many codes…).

155 `Future<void> _open(String? url) async {`
Opens the terms or privacy page in the browser.

161 `String _roleName(AppLocalizations l10n, ProfessionalType? role) => switch (role) {`
The chosen role's name, shown at the top of the form.

188 `? DtButton(`
The account exists: « Se connecter ».

198 `DtButton(label: l10n.signUpSubmit, loading: _submitting, onPressed: _valid && legal != null ? _submit : null),`
« Créer mon compte »: needs every field valid and the terms accepted.

284 `Checkbox(`
Accepts the terms and the privacy policy.

validators.dart · 42 lines

3 `library;`
Client-side checks that mirror the API (ADR-009), so errors show on blur. The server stays the judge; these only spare a round trip.

10 `bool isValidIdentifier(String raw) {`
E-mail, E.164 phone, or an Algerian mobile written locally (05…, 06…, 07…).

23 `const passwordMinLength = 8;`
Minimum length accepted by the API.

26 `enum PasswordStrength { tooShort, weak, fair, strong }`
How strong a password looks (guidance shown under the field).

29 `PasswordStrength passwordStrength(String password) {`
Guidance only: the API requires 8 characters; « Solide » asks for 10 with digits and symbols.

41 `final orderNumberPattern = RegExp(r'^[A-Za-z0-9][A-Za-z0-9/-]{0,29}$');`
Ordre registration number as the API accepts it.

welcome_screen.dart · 95 lines

18 `class WelcomeScreen extends ConsumerWidget {`
ScreenWelcome: vertical lockup, the promise in one line, three actions in order of commitment. A returning device makes « Se connecter » the primary action.

28 `DtButton(`
« Créer un compte ».

38 `DtButton(`
« Se connecter ».

45 `DtButton(`
« Essayer la démo »: no sign-up.

62 `child: DtHeroBackground(`
ADR-020: the brand on the deep part of the hero, the choices in a lifted white card.

apps/mobile/lib/src/features/pregnancy/

Pregnancy and lactation: search with period filter, the substance profile, the error report sheet.

pregnancy_search.dart · 290 lines

31 `enum SearchPeriod { t1, t2, t3, peripartum, lactation }`
The period a « Grossesse » search answers for; null = no period chosen.

34 `PeriodSummaryDto? summaryFor(PeriodSummariesDto? periods, SearchPeriod period) => switch (period) {`
A hit's summary for the chosen period (trimester, peripartum or lactation).

43 `String searchPeriodLabel(AppLocalizations l10n, SearchPeriod period) => switch (period) {`
The name of a period filter.

52 `typedef SubstanceResults = ({List<SubstanceHitDto> items, int total, bool offline});`
One page of results; `offline` when it came from the profiles kept on the device.

58 `final substanceSearchProvider = FutureProvider.autoDispose.family<SubstanceResults, ({String q, int limit})>((`
Server-side search (ADR-025): partial, accent-insensitive words, bounded pages. Without a connection it searches only the profiles kept offline, and says so.

77 `class SubstanceSearch extends ConsumerStatefulWidget {`
« Grossesse · Allaitement » in the library: type a few letters of the drug, pick the period, read its conduct on the row. Locked rows keep their place with their reason.

87 `class _SubstanceSearchState extends ConsumerState<SubstanceSearch> {`
The query after the pause, how many hits are shown, and the period filter.

95 `void initState() {`
Starts with the query already typed.

102 `void didUpdateWidget(SubstanceSearch oldWidget) {`
The query changed: searches again after a 300 ms pause.

107 `_timer = Timer(const Duration(milliseconds: 300), () {`
A request once typing pauses, not per keystroke.

118 `void dispose() {`
Stops the pause timer.

133 `Padding(`
Period filter: T1, T2, T3, peripartum, lactation.

160 `DtButton(`
« Réessayer ».

216 `child: DtButton(`
« Afficher N de plus ».

232 `class _HitRow extends StatelessWidget {`
One substance found: name with the matched letters highlighted, and its conduct for the chosen period.

252 `detailLabel = l10n.pregnancySearchNoData(periodName);`
Nothing approved for this period: say so, never imply safe or unsafe.

report_sheet.dart · 172 lines

21 `Future<void> showReportSheet(`
« Signaler une erreur » on the block being read: a reason and, if useful, a short comment for the clinical team — never patient data (Lock L42).

43 `class _ReportSheet extends ConsumerStatefulWidget {`
« Signaler une erreur » sheet for one section of a profile.

54 `class _ReportSheetState extends ConsumerState<_ReportSheet> {`
The reason chosen, the optional comment, progress and error.

62 `void dispose() {`
Frees the text controller.

68 `Future<void> _send() async {`
« Envoyer »: sends the report to the review team; closes the sheet with a thank-you message.

136 `child: ListTile(`
One reason of the report.

166 `DtButton(label: l10n.pregnancyReportSend, loading: _sending, onPressed: _reason == null ? null : _send),`
« Envoyer »: possible once a reason is chosen.

substance_screen.dart · 791 lines

46 `final substanceProvider = FutureProvider.autoDispose.family<SubstanceProfileDto, String>((ref, id) async {`
One substance's published profile, with the server's access decision (ADR-019). One profile; without a connection, the copy kept for offline reading (ADR-019).

58 `enum _Period { t1, t2, t3, peripartum }`
The pregnancy periods, as tabs.

63 `class SubstanceScreen extends ConsumerStatefulWidget {`
« Grossesse · Allaitement » for one substance (ScreenPregnancyDesktop, ScreenLactationMobile, EvidenceBlock): conduct and evidence level side by side but never merged, no safe/unsafe verdict, « Absence de données » as a full block, the evidence footer under every block.

73 `class _SubstanceScreenState extends ConsumerState<SubstanceScreen> {`
The section shown (pregnancy or lactation) and what has already been recorded.

80 `void _record(SubstanceProfileDto profile) {`
Records the visit once in the person's history and each section seen once for analytics.

145 `class _Failure extends StatelessWidget {`
Error state with « Réessayer » (or « introuvable » when the profile no longer exists).

172 `DtButton(label: l10n.commonRetry, variant: DtButtonVariant.secondary, onPressed: onRetry),`
« Réessayer ».

181 `class _Profile extends StatelessWidget {`
The whole profile: header, section switch, the chosen block, sources and « Signaler une erreur ».

203 `DtButton(`
« Débloquer »: explains the lock and its path.

220 `DtSegmentedControl<PregnancySection>(`
Switches between « Grossesse » and « Allaitement ».

```
245 String? _text(BuildContext context, ProfileTextDto? text) {
```
The author's text in the interface language, French when no English was written.

```
252 String evidenceWord(AppLocalizations l10n, num level) => switch (level.toInt()) {
```
The evidence level as its word (« Données limitées »…); the word carries the meaning.

```
261 String conductWord(AppLocalizations l10n, Conduct conduct) => switch (conduct) {
```
A conduct as its word, for screen readers and plain text.

```
268 class ConductBadge extends StatelessWidget {
```
A clinical instruction as icon + word; never a « safe » verdict.

```
293 class EvidenceMeter extends StatelessWidget {
```
Four neutral segments, the level word and the population — the word carries the meaning.

```
303 if (evidence.level == null) return Text('—', style: DtType.body.copyWith(color: colors.inkSecondary));
```
A published profile always has a level (review requires it); never read null as « no data ».

```
340 class _Labelled extends StatelessWidget {
```
A label above a text (nothing when the text is empty).

```
368 class _Section extends StatelessWidget {
```
A card around a part of the profile.

```
391 class _EvidenceLine extends StatelessWidget {
```
The evidence footer: level, source and date, always under a block.

```
415 class _PeriodSources extends StatelessWidget {
```
The sources supporting this period specifically (ADR-025), each opening in the browser.

```
438 child: InkWell(
```
A source with a link: opens it in the browser.

```
470 class _Alternatives extends StatelessWidget {
```
« Alternatives »: other substances to consider, each opening its own profile.

```
495 ActionChip(
```
An alternative substance: opens its profile.

```
513 class _NoData extends StatelessWidget {
```
« Absence de données — ce n'est pas une preuve d'innocuité »: a full block with alternatives.

```
548 class _ConflictAlert extends StatelessWidget {
```
Shown when sources disagree: what the conflict is and which sources say what.

```
571 class _PregnancyBlock extends StatefulWidget {
```
The pregnancy block: one tab per period with its conduct and evidence.

```
582 class _PregnancyBlockState extends State<_PregnancyBlock> {
```
The period tab chosen.

```
586 PregnancyPeriodDto? _of(_Period period) => switch (period) {
```
The content of one period.

```
594 String _label(AppLocalizations l10n, _Period period) => switch (period) {
```
The name of a period tab.

```
612 DtPillTabs<_Period>(
```
Period tabs: T1, T2, T3, peripartum.

```
682 class _LactationBlock extends StatelessWidget {
```
The lactation block: conduct, evidence and notes.

```
753 class _EvidenceFooter extends StatelessWidget {
```
Sources (title · edition), reviewer and date, version, « Signaler une erreur » (section 25).

apps/mobile/lib/src/features/profile/

The profile screen, its edit screen and the full account data.

edit_profile_screen.dart · 187 lines

21 `class EditProfileScreen extends ConsumerStatefulWidget {`
« Modifier le profil »: name, photo and a short bio (≤ 280 characters). Needs a connection.

29 `class _EditProfileScreenState extends ConsumerState<EditProfileScreen> {`
The names and bio being edited, and progress flags.

40 `void dispose() {`
Frees the text controllers.

50 `Future<void> _save() async {`
« Enregistrer »: saves the names and bio.

70 `Future<void> _changePhoto() async {`
« Changer la photo »: picks a JPEG or PNG and uploads it.

93 `Future<void> _removePhoto() async {`
« Supprimer la photo ».

120 `DtButton(label: l10n.profileSave, loading: _saving, onPressed: me != null && _valid ? _save : null),`
« Enregistrer »: needs valid names.

136 `DtButton(`
« Changer la photo ».

146 `TextButton(`
« Supprimer la photo ».

profile_data.dart · 73 lines

17 `class MeController extends AsyncNotifier<MeDto?> {`
The full account (profile header, edit screen); null for a demo.

20 `Future<MeDto?> build() async {`
Loads /v1/me for a real account (null for a demo).

27 `Future<void> save({required String firstName, required String lastName, required String bio}) async {`
An empty [bio] clears it (requests never carry null fields).

39 `Future<void> uploadPhoto(File file, String contentType) async {`
Uploads a JPEG or PNG; the API re-encodes it without metadata.

67 `Future<void> removePhoto() async {`
Removes the profile photo.

profile_screen.dart · 299 lines

25 `class ProfileScreen extends ConsumerWidget {`
ScreenProfileMobile: header (photo, name, type, verification and access badges, « Modifier »), then Activité et favoris, Abonnement (students: « Accès étudiant — gratuit »), Vérification, Notifications, Paramètres, Se déconnecter. A demo sees Paramètres and « Quitter la démo » only.

82 `DtHeroBackground(`
ADR-020: the identity sits on the hero, the menu in lifted white cards below.

102 `ExcludeSemantics(`
The initials repeat the name read just after.

130 `Semantics(`
Its own button: otherwise the whole identity block reads as one control.

141 `child: InkWell(`
« Modifier le profil ».

219 `static String _initials(String first, String last) =>`
The initials shown when there is no photo.

223 `static String _roleName(AppLocalizations l10n, ProfessionalType type) => switch (type) {`
The profession's name in the UI language.

231 `class _Badges extends ConsumerWidget {`
Verification and access, as words with a reinforcing colour (never colour alone).

259 `class ProfileAvatar extends StatelessWidget {`
The profile photo, or the initials when there is none.

apps/mobile/lib/src/features/recovery/

« Accès perdu ? » and the assisted recovery steps.

access_help_screen.dart · 111 lines

15 `class AccessHelpScreen extends StatelessWidget {`
« Accès perdu ? » (ADR-023): three situations, each to the shortest way back in. The heavy path (identity check) is offered only when nothing simpler can work.

26 `_Choice(`
« Mot de passe oublié » → reset with a code.

34 `_Choice(`
« J'ai changé de contact » → sign in with the other contact.

42 `_Choice(`
« J'ai perdu l'accès aux deux » → assisted recovery.

56 `class _Choice extends StatelessWidget {`
One way out: an icon, a title, a line, and where it leads.

recovery_screen.dart · 437 lines

38 `enum _Step { form, code, photos, done }`
The steps of recovery: the form, the code, the two photos, done.

41 `enum _PhotoState { empty, uploading, sent, failed }`
Where each photo is.

44 `final _recoveryUploadsProvider = Provider<Dio>((ref) {`
Signed PUTs of the two photos to private storage (outside the API client: no auth header).

59 `class RecoveryScreen extends ConsumerStatefulWidget {`
« J'ai tout perdu » (ADR-023): lost contact + names + a new contact proven by a code, then an identity photo and a selfie, then a review by the team. The recovery token stays in memory only; leaving the screen means starting again (it expires after an hour anyway).

67 `class _RecoveryScreenState extends ConsumerState<RecoveryScreen> {`
Everything the recovery steps need: the texts typed, the request, the code challenge, the token and photos.

89 `void dispose() {`
Frees the text controllers.

103 `Future<void> _run(Future<void> Function() action) async {`
Runs one step with the busy flag and shows its error if it fails.

118 `Future<void> _start() => _run(() async {`
Step 1: the lost contact, names and a new contact; a code is sent to the new contact.

139 `Future<void> _confirm() => _run(() async {`
Step 2: the code proves the new contact; the API gives a token for the photos.

160 `Future<void> _resend() async {`
« Renvoyer le code ».

183 `Future<void> _takePhoto(RecoveryPhotoSlot slot, String title) async {`
Step 3: takes the identity photo or the selfie and uploads it to private storage.

227 `await File(path).delete().catchError((Object _) => File(path));`
The photo lives only until it is sent: never kept on the phone.

232 `Future<void> _submit() => _run(() async {`
« Envoyer la demande »: the team reviews it; once approved it applies after 24 hours.

```
245  String _errorText(AppLocalizations l10n, ApiError error) => switch (error.code) {
```
The message for each refusal of the recovery steps.

```
270  DtButton(label: l10n.recoverySendCode, loading: _busy, onPressed: _formValid ? _start : null),
```
« Envoyer le code »: needs the lost contact, the names and a new contact.

```
315  DtButton(
```
« Vérifier »: possible once the six digits are typed.

```
336  DtButton(
```
« Envoyer la demande »: possible once both photos are sent.

```
364  actions: [DtButton(label: l10n.recoveryBackToSignIn, onPressed: () => context.go(Routes.signIn))],
```
« Retour à la connexion ».

```
374  class _PhotoTile extends StatelessWidget {
```
One photo slot: what to photograph, and its state (to take, sending, sent, failed).

```
423  DtButton(
```
« Prendre la photo » / « Reprendre ».

apps/mobile/lib/src/features/reference/

Downloading and checking a reference PDF, and the read-only viewer.

reference_files.dart · 51 lines

```
12  typedef ReferenceFile = ({Uint8List bytes, String sha256, int fileVersion});
```
A reference PDF as downloaded and verified.

```
16  class ReferenceFiles {
```
Fetches reference PDFs through short-lived signed URLs (10 minutes, ADR-011) and checks each against its published SHA-256 before it is shown or stored.

```
20  static const integrityCode = 'FILE_INTEGRITY';
```
Error code of a file whose bytes do not match the published SHA-256.

```
26  Future<ReferenceFile> download(String referenceId) async {
```
Gets a signed URL from the API, downloads the PDF bytes and checks their checksum.

reference_viewer_screen.dart · 338 lines

```
33  class OpenedReference {
```
A reference model and its PDF, checked against the published SHA-256.

```
40  final DateTime? cachedAt;
```
Set when the copy comes from the offline store without a connection (« Cache · date »).

```
45  final openedReferenceProvider = FutureProvider.autoDispose.family<OpenedReference, String>((ref, id)
async {
```
Online: the current file, from the offline copy when it is the same version. Without a connection: the offline copy, or « Non disponible hors ligne » (section 18).

```
71  class ReferenceViewerScreen extends ConsumerStatefulWidget {
```
ScreenReferenceViewer: full-screen and read-only. The reference notice is pinned and never dismissible. There is no export, print, share, download, generate, edit or patient-field control (Lock L31), and text selection is off so nothing leaves the viewer.

```
81  class _ReferenceViewerScreenState extends ConsumerState<ReferenceViewerScreen> {
```
The PDF controller, the page shown and whether the view was counted.

```
87  bool _viewRecorded = false;
```
REFERENCE_VIEWED is counted once, when the PDF has rendered.

```
91  void initState() {
```
Records the visit in the person's history.

```
221  class _ZoomControl extends StatelessWidget {
```
The floating control: page « 2 / 5 », zoom out and zoom in.

```
255 IconButton(tooltip: l10n.viewerZoomOut, icon: const Icon(LucideIcons.zoomOut), onPressed: onZoomOut),
```
Zoom out.

```
257 IconButton(tooltip: l10n.viewerFit, icon: const Icon(LucideIcons.scan), onPressed: onFit),
```
Fit the page to the width.

```
259 IconButton(tooltip: l10n.viewerZoomIn, icon: const Icon(LucideIcons.zoomIn), onPressed: onZoomIn),
```
Zoom in.

```
263 child: Semantics(
```
« page n sur N » for screen readers (section 20), « n / N » on screen.

```
281 class _PaperSkeleton extends StatelessWidget {
```
A grey page while the PDF loads.

```
302 class _Failure extends StatelessWidget {
```
Error state with « Réessayer ».

```
331 DtButton(label: l10n.commonRetry, variant: DtButtonVariant.secondary, onPressed: onRetry),
```
« Réessayer ».

apps/mobile/lib/src/features/search/

Global search and the recent searches kept on the phone.

recent_searches.dart · 44 lines

```
9  class RecentSearches extends AsyncNotifier<List<String>> {
```
The person's own last 10 searches, on this device only. They are never sent to analytics (Lock L42, section 16).

```
15 Future<List<String>> build() async {
```
Reads the recent searches kept on this device.

```
23 Future<void> add(String query) async {
```
Puts a search first in the list (no duplicates, at most a few).

```
31 Future<void> remove(String query) async => _save((await future).where((item) => item != query).toList());
```
Removes one recent search.

```
34 Future<void> clear() => _save(const []);
```
« Effacer »: removes every recent search.

```
37 Future<void> _save(List<String> items) async {
```
Shows the new list at once and saves it.

search_screen.dart · 481 lines

```
35 class SearchScreen extends ConsumerStatefulWidget {
```
ScreenSearchMobile: full-screen overlay, keyboard up. Grouped results (Ordonnances, Certificats, Calculateurs — the only V1 types), 3 rows per group, chips with counts filtering in place; locked results stay in place with their lock (never hidden).

```
43 class _SearchScreenState extends ConsumerState<SearchScreen> {
```
The query, the timers, the results and totals per type, and the type filter.

```
48 Timer? _report;
```
A search counts once its results have stayed on screen briefly, not per keystroke.

```
53 Map<SearchType, int> _totals = const {};
```
Totals of the unfiltered search, so the chips keep their counts while filtering.

```
58 List<ContentRef> _local = const [];
```
Local results shown when the full search is unavailable.

```
63 void dispose() {
```
Frees the timers and the text controller.

```
71 void _changed(String value) {
```
The query changed: clears below two characters, otherwise searches after a short pause.

86 `Future<void> _run() async {`
Searches the API (or the offline copy); an older answer arriving late is ignored.

124 `List<ContentRef> local = const [];`
Without the full search, look through what the device holds (offline store included).

144 `void _setFilter(SearchType? type) {`
A type chip was tapped: searches again with that filter.

150 `void _useQuery(String query) {`
A recent search was tapped: puts it in the field and searches.

157 `void _open(SearchHitDto hit) {`
Opens a result and remembers the query in recent searches and history.

174 `String _typeLabel(AppLocalizations l10n, SearchType type) => switch (type) {`
The name of a result type.

198 `DtGlassIconButton(`
Closes the search.

206 `child: TextField(`
The search field: searches as the person types; « Entrée » keeps the query in recent searches.

222 `: IconButton(`
Clears the field.

252 `_Chip(label: l10n.searchAll, selected: _filter == null, onTap: () => _setFilter(null)),`
« Tout »: removes the type filter.

261 `_Chip(`
A type chip with its count: keeps only that type.

277 `Widget _body(BuildContext context, AppLocalizations l10n, String query) {`
What is under the field: recent searches, offline results, loading, error, nothing found, or results.

311 `DtButton(label: l10n.commonRetry, variant: DtButtonVariant.secondary, onPressed: _run),`
« Réessayer ».

328 `ListTile(`
A spelling suggestion: searches it.

373 `child: TextButton(`
« Voir les N »: filters on this type.

386 `class _Chip extends StatelessWidget {`
One filter chip.

413 `class _EmptyQuery extends ConsumerWidget {`
Empty query: recent searches (removable) and the browse chips of the three V1 types.

433 `TextButton(`
« Effacer » the recent searches.

441 `ListTile(`
A recent search: searches it again.

447 `trailing: IconButton(`
Removes this recent search.

468 `ActionChip(`
An example query: searches it.

apps/mobile/lib/src/features/settings/

Settings: appearance, language, account security (contacts, password, recovery), account deletion.

account_security_screen.dart · 602 lines

37 `final accountContactsProvider = FutureProvider.autoDispose<AccountContactsDto>(`
The e-mail and phone of the account, whether both are confirmed, and any recovery in progress.

43 `class AccountSecurityScreen extends ConsumerStatefulWidget {`
Settings › « Coordonnées et sécurité » (ADR-023): both contacts (so losing one never locks the person out), the password, and « Ce n'est pas moi » when someone asks to recover the account.

55 `Future<void> _cancelRecovery() async {`
« Annuler la récupération »: the owner stops a recovery during its 24-hour hold.

72 `Future<void> _remove(MessageChannel channel) async {`
« Retirer »: opens the password sheet that removes a contact.

90 `Future<void> _edit(MessageChannel channel, bool exists) async {`
« Modifier » / « Ajouter »: opens the contact screen, then confirms the change.

132 `action: DtButton(`
« Ce n'est pas moi »: cancels the recovery in progress during its 24-hour hold.

185 `child: ListTile(`
« Changer le mot de passe ».

210 `action: DtButton(`
« Réessayer » loading the contacts.

225 `class _ContactRow extends StatelessWidget {`
One contact (e-mail or phone) with its buttons.

267 `DtButton(`
« Retirer » this contact (only when the other one exists).

275 `DtButton(`
« Ajouter » / « Modifier » this contact.

289 `class _RemoveContactSheet extends ConsumerStatefulWidget {`
« Retirer »: confirmed with the password; the other contact stays.

299 `class _RemoveContactSheetState extends ConsumerState<_RemoveContactSheet> {`
The password typed, progress and error.

306 `void dispose() {`
Frees the text controller.

312 `Future<void> _remove() async {`
Removes the contact after checking the password; the other contact is told.

362 `DtButton(`
« Retirer »: possible once the password is typed.

375 `String _contactErrorText(BuildContext context, ApiError error, {bool removing = false}) {`
The message for each refusal of a contact or password change.

394 `class ContactEditScreen extends ConsumerStatefulWidget {`
Add or change the e-mail or the phone: new value + current password → code → confirmed.

405 `class _ContactEditScreenState extends ConsumerState<ContactEditScreen> {`
The new contact, the password, the code, the challenge, progress and error.

415 `void dispose() {`
Frees the text controllers.

428 `Future<void> _run(Future<void> Function() action) async {`
Runs one step with the busy flag and shows its error if it fails.

443 `Future<void> _send() => _run(() async {`
Step 1: the new contact and the password; a code is sent to the new contact.

458 `Future<void> _confirm() => _run(() async {`
Step 2: the code confirms the new contact; the old one is told.

490 `DtButton(`
« Confirmer »: possible once the six digits are typed.

505 `DtButton(`
« Envoyer le code »: needs a valid contact and the password.

```
528 class PasswordChangeScreen extends ConsumerStatefulWidget {
```
« Changer le mot de passe »: current + new; other devices are signed out by the API.

```
536 class _PasswordChangeScreenState extends ConsumerState<PasswordChangeScreen> {
```
The current and new passwords, progress and error.

```
544 void dispose() {
```
Frees the text controllers.

```
551 Future<void> _save() async {
```
« Enregistrer »: changes the password; the other sessions end and the contacts are told.

```
580 DtButton(
```
« Enregistrer »: needs the current password and a long enough new one.

app_settings.dart · 47 lines

```
4 enum AppearancePreference {
```
« Apparence »: Clair · Sombre · Appareil (section ScreenSettings).

```
17 enum AppLanguage {
```
Interface language (FR/EN only, no RTL — section 19). Reference PDFs keep their own language.

```
24 static AppLanguage fromDevice(Locale deviceLocale) =>
```
French unless the device is set to English (DokTool is French-first).

```
30 class AppSettings {
```
The device settings: appearance (light, dark, system) and language.

```
37 AppSettings copyWith({AppearancePreference? appearance, AppLanguage? language}) =>
```
A copy with some settings changed.

delete_account_screen.dart · 126 lines

```
22 class DeleteAccountScreen extends ConsumerStatefulWidget {
```
Settings › Compte › « Supprimer mon compte » (section 08): what is erased, what the law keeps, the end of a current subscription without refund, then the password to confirm. The API erases the account; the device then forgets everything the account left on it and returns to Welcome.

```
30 class _DeleteAccountScreenState extends ConsumerState<DeleteAccountScreen> {
```
The password typed, progress and error.

```
38 void dispose() {
```
Frees the text controller.

```
44 Future<void> _delete() async {
```
« Supprimer mon compte »: deletes the account with the password, then forgets everything on the device.

```
99 DtButton(
```
« Supprimer mon compte »: possible once the password is typed.

settings_controller.dart · 56 lines

```
9 final deviceLocaleProvider = Provider((ref) => PlatformDispatcher.instance.locale);
```
The device locale; overridden in tests.

```
13 class SettingsController extends AsyncNotifier<AppSettings> {
```
Appearance and language, stored on the device and applied instantly without restart (ScreenSettings, LanguageSwitch). A failed save reverts the value and rethrows.

```
18 static const deviceKeys = {_appearanceKey, _languageKey};
```
Preferences of the device, not of an account: they survive an account deletion.

```
24 Future<AppSettings> build() async {
```
Reads appearance and language from the device (system and French by default).

```
35 Future<void> setAppearance(AppearancePreference appearance) =>
```
Changes the appearance.

39 `Future<void> setLanguage(AppLanguage language) =>`
Changes the language of the app.

43 `Future<void> _update(AppSettings Function(AppSettings) change, String key, String value) async {`
Applies a change at once, then saves it; goes back if saving fails.

settings_screen.dart · 159 lines

19 `class SettingsScreen extends ConsumerWidget {`
« Paramètres » (ScreenSettings): appearance and language applied instantly, data and offline, and for a real account « Supprimer mon compte » (section 08).

38 `_Section(`
The section heading names each control; a group label would read it twice.

41 `child: DtSegmentedControl<AppearancePreference>(`
Appearance: light, dark or like the system.

56 `child: DtSegmentedControl<AppLanguage>(`
Language of the app: French or English.

69 `child: Semantics(`
Its own node: a single row would otherwise merge into the section heading.

74 `child: ListTile(`
« Données et hors ligne ».

93 `ListTile(`
« Sécurité du compte »: contacts, password, recovery.

101 `ListTile(`
« Supprimer mon compte ».

123 `class _Section extends StatelessWidget {`
A titled group of settings with an optional helper line.

apps/mobile/lib/src/features/shell/

The tab bar and the tab stacks.

app_shell.dart · 218 lines

16 `class AppShell extends ConsumerWidget {`
The tab scaffold (BottomNav): Accueil · Recherche (raised disc) · Bibliothèque · Profil on a Clinical Glass bar. Each tab keeps its stack; re-tapping the active tab pops to its root; the disc opens the search overlay. The bar hides while the keyboard is up.

22 `final bool profileAttention;`
Dot on « Profil » when verification needs an action.

25 `void _go(int branch) => navigationShell.goBranch(branch, initialLocation: branch == navigationShell.currentIndex);`
Opens a tab; tapping the current tab again goes back to its first screen.

56 `class _GlassBar extends StatelessWidget {`
The floating tab bar: Accueil, Bibliothèque, the search button, Activité, Profil.

74 `return SafeArea(`
A floating pill of frosted glass, lifted off the content (ADR-020).

93 `_Destination(`
« Accueil » tab.

105 `child: InkResponse(`
The round search button in the middle of the bar.

128 `_Destination(`
« Bibliothèque » tab.

135 `_Destination(`
« Profil » tab (with a dot when something needs attention).

153 `class _Destination extends StatelessWidget {`
One tab of the bar: icon, label and selected state.

apps/mobile/lib/src/features/subscription/

Plans, review and payment, payment result, the current subscription.

payment_result_screen.dart · 152 lines

24 `class PaymentResultScreen extends ConsumerStatefulWidget {`
« Résultat du paiement »: the API asks the payment gateway what happened (never trusting the browser), so this screen polls the transaction until it is final, and again whenever the app comes back.

34 `class _PaymentResultScreenState extends ConsumerState<PaymentResultScreen> with WidgetsBindingObserver {`
Checks the payment every 4 seconds and on return to the app, until it is no longer pending.

42 `void initState() {`
Checks at once, then every 4 seconds.

51 `void dispose() {`
Stops the checks.

59 `void didChangeAppLifecycleState(AppLifecycleState state) {`
Back from the payment page: checks at once.

64 `Future<void> _check() async {`
Asks the API for the payment's status; success refreshes the access profile.

112 `DtButton(label: l10n.paymentSeeSubscription, onPressed: () => context.go(Routes.subscription)),`
« Voir mon abonnement ».

116 `DtButton(label: l10n.paymentRetry, onPressed: () => context.pushReplacement(Routes.plans)),`
« Réessayer »: back to the plans.

118 `DtButton(`
« Autre moyen de paiement »: back to the review of the same plan.

128 `DtButton(label: l10n.paymentCheck, loading: _checking, onPressed: _check),`
« Vérifier »: asks the payment status now.

131 `DtButton(`
« Rouvrir la page de paiement ».

payment_state.dart · 31 lines

6 `class PendingCheckout {`
The checkout being paid on the provider's page (in memory only).

15 `class PendingCheckoutController extends Notifier<PendingCheckout?> {`
The payment started in the browser, in memory until its result is known.

18 `PendingCheckout? build() => null;`
No payment pending at launch.

21 `void start(String transactionId, String redirectUrl, String planId) =>`
A payment was started: remembers it while the person is on the payment page.

25 `void clear() => state = null;`
The payment is over.

plans_screen.dart · 171 lines

24 `class PlansScreen extends ConsumerWidget {`
ScreenPlans « Offres »: for each plan what it includes and does not include, duration, price with taxes and access level. Calm: no ribbon, countdown or fake scarcity (Lock L39). Choosing a plan is possible only once professional verification is VERIFIED (ADR-006 Q6).

41 `DtButton(`
« Réessayer ».

80 `class _PlanCard extends ConsumerWidget {`
One plan: includes, not included, duration, price and its button (or why it is not open yet).

110 `CheckoutEligibility.open => DtButton(`
« Choisir »: opens the review of this plan.

117 `CheckoutEligibility.roleRequired => DtButton(`
Not verified yet: opens the verification.

123 `CheckoutEligibility.accountRequired => DtButton(`
In the demo: « Créer un compte ».

review_screen.dart · 175 lines

27 `class ReviewScreen extends ConsumerStatefulWidget {`
« Récapitulatif & confirmation » (section 06): plan, price, period, payment method, access, renewal mode and one acknowledgement line; « Payer » stays disabled until it is ticked.

37 `class _ReviewScreenState extends ConsumerState<ReviewScreen> {`
The card chosen, the « non remboursable » acknowledgement, progress and error.

39 `PaymentMethod? _choice;`
The card chosen; null until the person picks one of those the API accepts right now.

45 `Future<void> _pay(String acknowledgementVersion, PaymentMethod method) async {`
« Payer »: starts the payment on the API and opens the payment page in the browser.

89 `final methods = plans.paymentMethods.where((method) => method != PaymentMethod.$unknown).toList();`
Only the cards the API accepts now (ADR-021); a card unknown to this app version is skipped.

114 `DtButton(`
« Payer »: needs the plan open, a card chosen and the acknowledgement ticked.

134 `ExcludeSemantics(`
Read once, as the label of the choice below.

142 `DtSegmentedControl<PaymentMethod>(`
The card: CIB or Edahabia (those the API accepts now).

152 `child: CheckboxListTile(`
« J'ai compris que l'abonnement n'est pas remboursable ».

166 `String _message(BuildContext context, ApiError error) {`
The message for each refusal (a payment already in progress, method paused...).

subscription_data.dart · 50 lines

13 `final plansProvider = FutureProvider<PlansDto>((ref) {`
Offers and whether checkout is open (also for the demo, with ACCOUNT_REQUIRED).

19 `final subscriptionOverviewProvider = FutureProvider<SubscriptionOverviewDto>((ref) {`
The subscription screen: current subscription, eligibility and payments.

25 `String moneyText(BuildContext context, num minor, String currency) {`
« 1 500 DA » / « 1,500 DZD », from centimes.

38 `String durationText(AppLocalizations l10n, num days) => switch (days) {`
A plan duration as text: one month, one year, or N days.

45 `String methodText(AppLocalizations l10n, PaymentMethod method) => switch (method) {`
The card's name; which gateway took it is not the person's concern (ADR-021).

subscription_screen.dart · 207 lines

28 `class SubscriptionScreen extends ConsumerWidget {`
ScreenSubscriptionMobile: plan, status as a neutral or teal badge, period, access, renewal information and the payments. Calm by design: no red, no countdown, no promotion, and no refund request anywhere (ADR-005).

43 `DtButton(`
« Réessayer ».

57 `DtButton(`
« Voir les offres » / « Renouveler ».

69 `DtButton(`
« Arrêter le renouvellement ».

89 `Future<void> _stopRenewal(BuildContext context, WidgetRef ref) async {`
« Arrêter le renouvellement »: asks to confirm; the access stays until the end date.

97 `TextButton(onPressed: () => Navigator.of(dialog).pop(false), child: Text(l10n.commonCancel)),`
« Annuler ».

99 `TextButton(onPressed: () => Navigator.of(dialog).pop(true), child: Text(l10n.subscriptionStopRenewal)),`
Confirms stopping the renewal.

110 `class _Current extends StatelessWidget {`
The current subscription: plan, dates and renewal state.

175 `class _TransactionRow extends StatelessWidget {`
One payment of the history: date, amount, method and status.

apps/mobile/lib/src/features/verification/

Verification: intro, identity and status steps, camera, on-device document check, document slots, summary, status screen, repository and controller.

case_draft_builder.dart · 68 lines

16 `class CaseDraftBuilder extends ConsumerStatefulWidget {`
Opens (or resumes) the draft case of [type], then builds the step with it. The case shown is always the latest one held by the controller, so uploads elsewhere refresh this step.

27 `class _CaseDraftBuilderState extends ConsumerState<CaseDraftBuilder> {`
Opens (or creates) the draft once and shows its loading or error state.

31 `Future<VerificationCaseDto> _open() => ref.read(verificationControllerProvider.notifier).draftFor(widget.type);`
The draft verification case of this type.

53 `DtButton(label: l10n.commonRetry, onPressed: () => setState(() => _opening = _open())),`
« Réessayer » opening the case.

document_camera_screen.dart · 255 lines

15 `class DocumentCameraScreen extends StatefulWidget {`
Full-screen capture with corner guides (FileUpload `camera-first`), then a review step « Utiliser cette photo / Reprendre ». Returns the photo path, or null. The photo is shown only here, before sending; afterwards the app shows the file name and size, never a thumbnail.

21 `final bool selfie;`
The front camera, for the selfie of an account recovery (ADR-023).

24 `static Future<String?> capture(BuildContext context, String title, {bool selfie = false}) => Navigator.of(context)`
Opens the camera full screen and returns the photo's path (null when closed).

37 `class _DocumentCameraScreenState extends State<DocumentCameraScreen> {`
The camera, whether it is unavailable, and the photo taken.

45 `void initState() {`
Opens the camera.

51 `Future<void> _open() async {`
Opens the back camera (front for a selfie); without one, shows « caméra indisponible ».

79 `void dispose() {`
Releases the camera.

85 `Future<void> _shoot() async {`
Takes the photo and shows it for « Utiliser » or « Reprendre ».

100 `Future<void> _retake() async {`
« Reprendre »: deletes the photo and shows the camera again.

119 `leading: IconButton(`
Closes the camera without a photo.

169 `DtButton(`
« Utiliser la photo ».

175 `DtButton(`
« Reprendre ».

191 `DtButton(`
The shutter button.

208 `class _Guides extends StatelessWidget {`
Corner guides of an ID-1 card (85.6 × 54 mm) centred in the preview.

225 `class _CornerPainter extends CustomPainter {`
Draws the four corners of the framing guide.

232 `void paint(Canvas canvas, Size size) {`
Draws the corner strokes.

253 `bool shouldRepaint(_CornerPainter oldDelegate) => oldDelegate.color != color;`
Redraws only when the colour changes.

document_check.dart · 76 lines

7 `class DocumentCheck {`
Result of the on-device check of a photographed document (FileUpload « processing » state).

11 `final bool legible;`
Enough text was read to consider the photo usable.

14 `final bool nameFound;`
The account's last name (and first name when present) appear in the text read.

20 `abstract class DocumentChecker {`
Reads a document photo on the device with ML Kit (ADR-013: nothing it reads leaves the device) to catch blurred, cut or glaring photos and a name that differs from the account before upload. The reviewer remains the judge; this only saves a rejection.

25 `class MlKitDocumentChecker implements DocumentChecker {`
Checks on the phone (ML Kit text recognition) that a document is readable and bears the person's name.

27 `static const _minCharacters = 25;`
Below this many letters and digits, the photo is treated as unreadable.

32 `Future<DocumentCheck> check(String imagePath, {required String firstName, required String lastName}) async {`
Reads the photo text: too little text is unreadable; the name must appear in it.

47 `String foldForMatch(String value) {`
Upper case without accents, every separator (including the MRZ filler `<`) as one space.

62 `bool namePresent(String text, String firstName, String lastName) {`
Every word of the last name must appear; the first name's first word too when it has one.

document_slot.dart · 453 lines

26 `enum _Phase { idle, checking, uploading, quality, done, failed }`
Where the slot is: empty, checking, uploading, quality warning, done, failed.

29 `enum _Quality { illegible, nameMismatch }`
Why a photo looks wrong.

32 `class _Picked {`
A file picked or photographed, waiting to be sent.

43 `class DocumentSlot extends ConsumerStatefulWidget {`
One document step of a verification case (component FileUpload, variant `document`): camera first, file import second, on-device legibility and name check for photos, then upload to private storage. After upload it shows what was read and the size — never a thumbnail.

60 `final VerificationDocumentDto? document;`
The file already attached to this step, if any.

68 `class _DocumentSlotState extends ConsumerState<DocumentSlot> {`
The phase, the warning, the pending file, the upload progress and error.

76 `String? _nameRead;`
What the on-device check read for the file uploaded in this session (the server never gets it).

80 `Future<void> _fromCamera() async {`
« Prendre une photo ».

86 `Future<void> _fromFiles() async {`
« Choisir un fichier »: a PDF, JPG or PNG.

103 `Future<void> _accept(_Picked picked) async {`
Checks size and (for photos) readability and name; warns before sending a doubtful photo.

159 `Future<void> _upload(_Picked picked) async {`
Uploads the file with a signed URL, then registers it on the case.

196 `Future<void> _discard(_Picked picked) async {`
Camera photos are temporary files of this app; they are deleted once sent or refused.

206 `Future<void> _remove(VerificationDocumentDto document) async {`
« Retirer » a document sent to the case.

251 `DtButton(`
« Reprendre »: discards the doubtful photo and opens the camera again.

263 `DtButton(`
« Envoyer quand même »: the reviewer will judge.

291 `DtButton(`
« Remplacer » the document sent.

300 `DtButton(`
« Retirer » the document sent.

331 `DtButton(`
« Prendre une photo ».

339 `DtButton(`
« Importer un fichier ».

379 `: DtButton(`
« Réessayer » the upload.

392 `String _typeLabel(String contentType) => switch (contentType) {`
A file type as PDF, PNG or JPG.

400 `class _Row extends StatelessWidget {`
One row of the slot: icon, title, subtitle and action.

identity_step_screen.dart · 168 lines

29 `class IdentityStepScreen extends StatelessWidget {`
ScreenVerifyIdentity: step 1 of 3. Biometric ID card or passport (or a student card for medical students), camera first, with the on-device legibility and name check.

40 `class _IdentityStep extends ConsumerStatefulWidget {`
The identity step: document kind, then front (and back for a card).

```
50  class _IdentityStepState extends ConsumerState<_IdentityStep> {
```
The document kind chosen, progress and error.

```
60  bool _provided(VerificationStepType step) => widget.draft.documents.any((document) => document.stepType
== step);
```
True when a document was sent for this step.

```
63  Future<void> _changeKind(VerificationDetailsDtoDocumentKind kind) async {
```
Changes the document kind; a passport removes a back side sent earlier.

```
74  Future<void> _continue() async {
```
« Continuer »: saves the document kind and goes to the next step.

```
111 trailing: TextButton(
```
« Plus tard »: back to Home, the draft is kept.

```
118 DtButton(
```
« Continuer »: possible once the front (or passport page) is sent.

```
126 DtSegmentedControl<VerificationDetailsDtoDocumentKind>(
```
Document kind: national ID card, passport or student card.

status_step_screen.dart · 267 lines

```
33  class StatusStepScreen extends ConsumerWidget {
```
ScreenVerifyProfessional: step 2 of 3, the status case. Doctors give their Ordre number, an attestation or diploma and their specialty; residents their residency year, institution and specialty in progress; students their university and year. Pickers only, never free text.

```
49  class _StatusStep extends ConsumerStatefulWidget {
```
The professional status step: order number, specialty, institution or university, year.

```
60  class _StatusStepState extends ConsumerState<_StatusStep> {
```
The details being typed, progress and error.

```
73  void dispose() {
```
Frees the text controller.

```
105 Future<void> _continue() async {
```
« Continuer »: saves the details and goes to the summary.

```
237 trailing: TextButton(
```
« Plus tard »: back to Home, the draft is kept.

```
244 DtButton(
```
« Continuer »: needs the details and the proof document.

summary_screen.dart · 172 lines

```
23  class SummaryScreen extends ConsumerStatefulWidget {
```
« Récapitulatif » (step 3 of 3): step checkmarks and file descriptions only, then « Envoyer ».

```
31  class _SummaryScreenState extends ConsumerState<SummaryScreen> {
```
Progress and error of sending.

```
36  Future<void> _send() async {
```
« Envoyer pour vérification »: submits each complete case; the status screen follows.

```
70  DtButton(label: l10n.verificationSummarySend, loading: _sending, onPressed: complete ? _send : null),
```
« Envoyer pour vérification »: possible once every case is complete.

```
88  bool _complete(VerificationCaseDto draft) {
```
True when a case has its details and every required document.

```
97  class _SummaryCard extends StatelessWidget {
```
One case in the summary: its documents, details and « Modifier ».

```
159 child: DtButton(
```
« Modifier » this case.

verification_controller.dart · 121 lines

16 `VerificationType statusCaseFor(ProfessionalType type) =>`
The status case a professional type proves: students their enrolment, doctors and residents their professional status.

29 `extension VerificationOverviewX on VerificationOverviewDto {`
Helpers on the verification overview.

31 `VerificationTypeStateDto stateOf(VerificationType type) => types.firstWhere(`
The state of one case type (cannot start, when the API does not list it).

37 `List<VerificationType> get relevantTypes => [`
The cases this person goes through: identity, then their status case.

45 `class VerificationController extends AsyncNotifier<VerificationOverviewDto> {`
The person's verification overview (latest case of each type). Every change made through this controller replaces the case it returns, so screens stay consistent without refetching.

50 `Future<VerificationOverviewDto> build() {`
Loads the verification overview of the signed-in account.

57 `Future<void> reload() async {`
Loads the overview again.

62 `Future<VerificationCaseDto> draftFor(VerificationType type) async {`
The case being prepared for [type]: the open draft if there is one, otherwise a new case.

78 `void replace(VerificationCaseDto updated) {`
Replaces [updated] in the overview (it is the latest case of its type).

95 `Future<void> submitDrafts() async {`
« Envoyer »: sends every relevant case still being prepared, identity first.

107 `Future<void> appeal(String caseId) async {`
« Faire appel » of a refusal.

113 `Future<void> erase(String caseId) async {`
« Effacer mes documents » of a decided case.

verification_intro_screen.dart · 88 lines

19 `class VerificationIntroScreen extends ConsumerStatefulWidget {`
Verification intro (section 06): why, what we check, where documents go, how long it takes. One primary action; « Plus tard » is always available.

27 `class _VerificationIntroScreenState extends ConsumerState<VerificationIntroScreen> {`
Ends onboarding when verification is reached.

30 `void initState() {`
Clears the onboarding draft.

66 `trailing: TextButton(`
« Plus tard »: back to Home.

72 `DtButton(`
« Commencer »: identity first, then the status case.

verification_labels.dart · 54 lines

12 `String caseName(AppLocalizations l10n, VerificationType type) => switch (type) {`
The name of a case type.

19 `String stepName(AppLocalizations l10n, VerificationCaseDto draft, VerificationStepType step) => switch (step) {`
The name of a document step (« Page d'identité » for a passport).

32 `String documentDescription(BuildContext context, VerificationCaseDto draft, VerificationStepType step) {`
« Recto · JPG · 1,1 Mo » — the file description shown instead of any image of the document.

46 `String rejectionReason(AppLocalizations l10n, VerificationRejectionReason reason) => switch (reason) {`
A refusal reason in plain words.

verification_repository.dart · 112 lines

20 `const maxVerificationFileBytes = 10 * 1024 * 1024;`
Largest verification file the API accepts (ADR-011).

23 `class VerificationRepository {`
Verification calls of the user API, and the direct upload of files to private storage.

29 `final Dio _uploads;`
Plain client for signed storage URLs: no API headers, no bearer token.

32 `Future<VerificationOverviewDto> overview() => _api.verification.verificationControllerOverview();`
Every case type with its latest case and what can be done.

35 `Future<VerificationCaseDto> start(VerificationType type, {StartVerificationDtoProfessionalType? target}) =>`
Opens a new case of this type (for a status case, the profession targeted).

41 `Future<VerificationCaseDto> saveDetails(String caseId, VerificationDetailsDto details) =>`
Saves the declared details (document kind, order number, specialty...).

46 `Future<VerificationCaseDto> upload({`
Signs, uploads and attaches one step's file. The SHA-256 is declared up front; the signed PUT fixes the type and exact size.

84 `Future<VerificationCaseDto> removeDocument(String caseId, String documentId) =>`
Removes a document from a draft case.

88 `Future<VerificationCaseDto> submit(String caseId) => _api.verification.verificationControllerSubmit(id: caseId);`
Sends a case for review.

91 `Future<VerificationCaseDto> appeal(String caseId) => _api.verification.verificationControllerAppeal(id: caseId);`
Appeals a refusal.

94 `Future<VerificationCaseDto> erase(String caseId) => _api.verification.verificationControllerErase(id: caseId);`
Erases the documents of a decided case.

verification_status_screen.dart · 356 lines

26 `const _expiringWindow = Duration(days: 60);`
A verified case expiring within this many days shows « Expire bientôt » (section 06).

29 `enum _Overall { toDo, inReview, verified, toRedo }`
The overall state shown at the top.

34 `class VerificationStatusScreen extends ConsumerWidget {`
ScreenVerifyOutcomes « Statut »: identity and status as separate cases, each with its timeline (Reçu → En cours de revue → Décision), the reason of a refusal in plain words and one action. `critical` is kept for a definitive refusal only; a fixable step is `warning`.

47 `actions: [DtButton(label: l10n.commonRetry, onPressed: () => ref.invalidate(verificationControllerProvider))],`
« Réessayer ».

71 `DtButton(label: l10n.verificationActionSeePlans, onPressed: () => context.push(Routes.plans)),`
« Voir les offres » once verified.

81 `_Overall _overallOf(VerificationOverviewDto overview) {`
To redo if a case is refused or expired; verified if all are; in review or to do otherwise.

92 `Widget _overallBadge(AppLocalizations l10n, _Overall overall) => switch (overall) {`
The badge of the overall state.

113 `class _CaseCard extends ConsumerStatefulWidget {`
One case: its timeline, the refusal reason and its one action.

124 `class _CaseCardState extends ConsumerState<_CaseCard> {`
Progress and message of the card's action.

132 `Future<void> _run(Future<void> Function() action, {String? done}) async {`
Runs the action with the busy flag; shows a message when done or failed.

148 `Future<void> _confirmErase(VerificationCaseDto latest) async {`
Asks to confirm, then erases the case's documents.

156 `TextButton(onPressed: () => Navigator.of(context).pop(false), child: Text(l10n.commonCancel)),`
« Annuler ».

158 `TextButton(onPressed: () => Navigator.of(context).pop(true), child: Text(l10n.verificationActionErase)),`
Confirms erasing the documents.

223 `DtButton(`
« Commencer » / « Continuer » this case.

229 `DtButton(label: l10n.verificationActionCorrect, onPressed: _busy ? null : () => context.push(_stepRoute)),`
« Corriger »: back to the step to redo.

232 `DtButton(`
« Faire appel » of the refusal.

240 `DtButton(label: l10n.verificationActionRenew, onPressed: _busy ? null : () => context.push(_stepRoute)),`
« Renouveler » an expired verification.

265 `PopupMenuButton<void>(`
The « ⋮ » menu: « Effacer mes documents ».

308 `class _Timeline extends StatelessWidget {`
Reçu → En cours de revue → Décision, with the current step marked.

apps/mobile/lib/src/router/

Every route and the redirect rules.

app_router.dart · 265 lines

48 `abstract final class Routes {`
Route map of audit §11. Guards read the session and the account; locked content is never hidden by a guard — it renders its locked state (section 27).

80 `static String library(LibrarySection section, {bool focus = false}) =>`
Addresses with a parameter (focus=1 puts the cursor in the library filter).

90 `static const public = {`
Reachable while signed out.

104 `static const accountOnly = [`
Only for real accounts (never a demo token, section 27).

115 `class _RouterRefresh extends ChangeNotifier {`
Re-runs the redirect when the session or the account changes.

117 `_RouterRefresh(Ref ref) {`
Re-checks the redirects whenever the session or the account changes.

126 `String? redirectFor({`
Where the router sends a viewer, or null to stay.

256 `class _LaunchScreen extends StatelessWidget {`
Shown while the session is restored from secure storage.

apps/mobile/lib/src/ui/

The widget kit and token access (`context.colors`): buttons, alerts, badges, fields, pickers, rows, locks, glass and hero, tabs, skeletons.

choice_list.dart · 130 lines

11 `class DtChoiceList<T> extends StatelessWidget {`
Exclusive choices as full-width rows, one per answer, the whole text visible (ADR-026). Used when answers are too long for a [DtSegmentedControl] or more than four: a score's answers are read in full and chosen in one tap. Selection is an icon and weight, never colour alone.

59 `class _ChoiceRow extends StatelessWidget {`
One answer with its radio mark.

112 `bool segmentsFit(BuildContext context, List<String> labels, double width) {`
Whether every label fits on one line of a segmented control of [width] (measured with the person's text size); otherwise the answers belong in a [DtChoiceList].

content_row.dart · 163 lines

12 `String localizedTitle(BuildContext context, String fr, String? en) =>`
Title in the interface language, falling back to French (section 13).

18 `class ContentRow extends StatelessWidget {`
A list row of the library, search and home: title, optional one line, and either the single action (« Voir le modèle » / « Ouvrir ») or the lock with its reason word. A locked row keeps its place and opens the lock sheet.

40 `final String? highlight;`
Query words to emphasise by weight, never by colour (section 12).

43 `final Widget? detail;`
Extra line under the title (e.g. a period's conduct badge in « Grossesse » search).

46 `final String? detailLabel;`
What [detail] says, read with the title (the row's semantics are merged into one label).

111 `class _Highlighted extends StatelessWidget {`
A title with the searched letters in bold.

119 `static String _fold(String value) => value`
Lower case without accents, so « hepar » finds « Héparine ».

dt_alert.dart · 59 lines

8 `enum DtAlertSeverity { critical, warning, info, success, offline, neutral }`
The kinds of alert, each with its colour and icon.

12 `class DtAlert extends StatelessWidget {`
Alert · Banner: icon + optional title + body. Critical is for blocking validation and real failures only — never for subscriptions or locked content.

dt_badge.dart · 44 lines

7 `enum DtBadgeVariant { neutral, brand, teal, success, warning, critical, info }`
The colours a badge can have.

11 `class DtBadge extends StatelessWidget {`
Badge (22 px): always a word, the colour only reinforces it. `critical` is for a definitive rejection only; subscription states never use critical or warning (Lock L39).

dt_button.dart · 101 lines

7 `enum DtButtonVariant { primary, secondary, ghost, danger, teal }`
The looks of a button.

10 `enum DtButtonSize { sm, md, lg }`
The heights of a button.

15 `class DtButton extends StatelessWidget {`
The single action control (component Button): one primary per screen region, labels in the infinitive and under three words. While [loading], a spinner replaces the icon and the button ignores taps; a null [onPressed] renders the disabled state.

dt_logo.dart · 23 lines

5 `class DtLogo extends StatelessWidget {`
The vertical DokTool lockup (design/system/assets/Logos), light or dark with the theme.

11 `final bool onHero;`
On the hero gradient (ADR-020): the light-on-dark lockup whatever the theme.

dt_page.dart · 130 lines

12 `class DtPage extends StatelessWidget {`
A mobile flow page in the glass style (ADR-020): the hero gradient behind the status bar and a toolbar with a frosted back button and an optional trailing action (e.g. « Plus tard »); the eyebrow (« Étape 1 sur 3 · Identité »), title and intro in white on the hero, fading into the page; scrollable content with the 16 px gutter; the actions pinned at the bottom.

31 `final List<Widget> actions;`
Bottom actions, primary first.

46 `ColoredBox(`
The toolbar stays on the deep part of the hero; the title block scrolls under it.

96 `class DtSection extends StatelessWidget {`
A titled block on a lifted white card (settings sections, verification intro paragraphs, summaries) — ADR-020.

dt_picker.dart · 261 lines

9 `class DtPickerOption<T> {`
One option of a [DtPickerField].

18 `class DtPickerField<T> extends StatelessWidget {`
A picker (never typed free text, section 06): a field that opens a searchable bottom sheet.

71 `ExcludeSemantics(`
Read as part of the field below (« label, value »), not twice.

142 `class _PickerSheet<T> extends StatefulWidget {`
The bottom sheet of a picker, with a search field for long lists.

162 `class _PickerSheetState<T> extends State<_PickerSheet<T>> {`
The search text typed in the sheet.

166 `static String _fold(String value) => value`
Lower case without accents, for the search.

198 `IconButton(`
Closes the picker without changing the choice.

210 `child: TextField(`
Filters the options as the person types.

243 `return ListTile(`
One option: tapping it chooses it and closes the sheet.

dt_text_field.dart · 143 lines

11 `class DtTextField extends StatelessWidget {`
Text field of the Input component: label above, 48 px well on mobile, helper line that becomes the error message (with an icon, never colour alone). Validation is shown on blur or submit — the caller passes [error] only then.

40 `final String? requiredMark;`
Words shown after the label, e.g. « requis ».

67 `ExcludeSemantics(`
Read as the field's own label below, not as a separate line.

error_text.dart · 43 lines

9 `String commonErrorText(BuildContext context, ApiError error) {`
The message for errors every screen can meet. Screens handle their specific codes first (e.g. OTP_INVALID) and fall back to this.

18 `String timeText(BuildContext context, DateTime? at) {`
A local time such as « 14:05 » for « Réessayez à … » messages.

24 `String dateText(BuildContext context, DateTime? at) {`
A local date such as « 21/09/2026 ».

30 `String sizeText(BuildContext context, num bytes) {`
« 1,1 Mo » / « 1.1 MB ».

glass.dart · 191 lines

10 `class DtHeroBackground extends StatelessWidget {`
The hero gradient (ADR-020): the brand at its deepest at the top, fading into the page ground. It sits behind the status bar and the first block of Home, Welcome and profile-like headers.

34 `class DtGlass extends StatelessWidget {`
A frosted surface (Clinical Glass): blur behind, a translucent fill, a hairline edge that catches the light, and an optional soft shadow. Use [DtGlass.onHero] over the hero gradient.

47 `const DtGlass.onHero({required this.child, this.radius = DtRadius.lg, this.padding, super.key})`
White frost over the hero gradient (icon buttons, quick actions).

69 `decoration: BoxDecoration(borderRadius: shape, boxShadow: shadow ?? const []),`
The shadow sits outside the clip so it is not cut by it.

92 `class DtGlassIconButton extends StatelessWidget {`
A round frosted icon button on the hero (search, notifications), 44 pt as every touch target.

109 `onTap: onPressed,`
The InkWell below is excluded with its tap action: the action is given here, so TalkBack and VoiceOver can activate the button.

131 `class DtLiftedCard extends StatelessWidget {`
A white card lifted over the hero or the page: large radius, soft brand-tinted shadow.

156 `class DtCardGroup extends StatelessWidget {`
Rows grouped in one lifted white card, separated by hairlines (ADR-020): search results, favorites, history, notifications — the same look as the library sections.

hero.dart · 246 lines

12 `const double kHeroFade = DtSpace.s6;`
Height of the soft fade from the hero's middle shade into the page ground (ADR-020).

15 `class DtOnHero extends StatelessWidget {`
Text and icons placed on the hero (trailing actions such as « Plus tard ») take its colours.

44 `class DtHeroToolbar extends StatelessWidget {`
The row at the top of a hero: a frosted back button (when there is somewhere to go back to), an optional title, and trailing actions, all in the hero's colours.

57 `final Widget? titleWidget;`
Replaces [title] when the title is not plain text (two lines, a badge beside it).

75 `DtGlassIconButton(`
Back arrow of the header.

116 `class DtHeroAppBar extends StatelessWidget implements PreferredSizeWidget {`
A screen's top bar in the glass style (ADR-020): the hero gradient behind the status bar and the toolbar, fading into the page ground, with the title in white and a frosted back button. For screens that build their own [Scaffold]; flow pages get the same top through [DtPage].

122 `final Widget? titleWidget;`
Replaces [title] when the title is not plain text (a loading placeholder, two lines).

156 `class DtHeroTitle extends StatelessWidget {`
The hero block that carries a page's eyebrow, title and intro in white, starting where the fixed toolbar ends and fading into the page (scrolls with the content).

219 `class DtHeroBand extends StatelessWidget {`
A free-form block on the hero (e.g. the search field), from behind the status bar down to a fade into the page ground.

locks.dart · 122 lines

16 `String lockWord(AppLocalizations l10n, LockReason? reason) => switch (reason) {`
The reason word shown next to the lock on a row (ScreenLockedStates). Never red, never hidden.

26 `String lockExplanation(AppLocalizations l10n, LockReason? reason) => switch (reason) {`
Why something is locked, in plain words.

38 `class LockLabel extends StatelessWidget {`
The lock word as a small inline label with the lock icon.

60 `Future<void> showLockSheet(BuildContext context, ItemAccessDto access, {required String title}) {`
The calm sheet a locked item opens (section 27): what the boundary is and, when there is one, the single unlock path. Students get an explanation only — no purchase action (Lock L38).

102 `DtButton(`
The unlock path the API gives (verify, see plans, create an account): closes the sheet and goes there.

112 `DtButton(`
« Continuer la démo » / « Fermer ».

pill_tabs.dart · 91 lines

8 `class DtPillTab<T> {`
One choice of a [DtPillTabs].

19 `class DtPillTabs<T> extends StatelessWidget {`
A scrolling row of pill chips (ADR-020): icon + full label, the selected one filled with the brand, the others white with a hairline. For section choices that no longer fit a segmented control (the library's four sections), never truncated.

segmented_control.dart · 126 lines

8 `class DtSegment<T> {`
One option of a [DtSegmentedControl].

18 `class DtSegmentedControl<T> extends StatelessWidget {`
The design system's segmented control (`.dt-seg`): finite exclusive choices of up to four options (Settings appearance and language, calculator choices). Full width, pill shaped, with a 44 px touch height (section 20).

48 `child: IntrinsicHeight(`
Each segment is tappable over the control's full height; the inset is drawn inside it.

77 `class _SegmentButton extends StatelessWidget {`
One segment: its label and selected state.

90 `return Semantics(`
One node per segment, with its own action: merged into the control, the segments could not be chosen with TalkBack or VoiceOver.

skeleton.dart · 51 lines

7 `class SkeletonRows extends StatelessWidget {`
Loading rows (section 22: skeleton rows, never a spinner, for lists).

text_match.dart · 23 lines

2 `String foldText(String value) {`
Lower case without accents, for accent- and case-insensitive matching (section 12).

```
15  bool matchesAllWords(String query, Iterable<String?> texts) {
```
True when every word of [query] appears in one of [texts], in any order (same rule as the API).

tokens.dart · 11 lines

```
7   extension DtContext on BuildContext {
```
Token access from widgets: `context.colors.brand`. Widgets never use raw colours, radii, blur or durations (Lock L35).